



MBISTArchitect™ Process Guide

Software Version 2015.4

December 2015

**© 2004-2015 Mentor Graphics Corporation
All rights reserved.**

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

U.S. GOVERNMENT LICENSE RIGHTS: The software and documentation were developed entirely at private expense and are commercial computer software and commercial computer software documentation within the meaning of the applicable acquisition regulations. Accordingly, pursuant to FAR 48 CFR 12.212 and DFARS 48 CFR 227.7202, use, duplication and disclosure by or for the U.S. Government or a U.S. Government subcontractor is subject solely to the terms and conditions set forth in the license agreement provided with the software, except for provisions which are contrary to applicable mandatory federal laws.

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the owner of the Mark, as applicable. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/trademarks.

The registered trademark Linux[®] is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: supportnet.mentor.com/

Send Feedback on Documentation: supportnet.mentor.com/doc_feedback_form

Table of Contents

Chapter 1

Getting Started	15
MBISTArchitect Overview	15
High Test Quality	15
Easy Application	15
Versatility	16
Invoking MBISTArchitect	16
Using the Tool from the Command Line	17
Loading Libraries	18
Resetting MBISTArchitect	18
Exiting MBISTArchitect	19
MBISTArchitect Architecture	19
MBISTArchitect Usage Flow	19
BIST Insertion Phase (Includes Generation Activity)	20
BIST Insertion Phase Modes	21
Switching Between Modes	21
Report Memory Instances	21
Default Dofile Contents	22
BIST Generation Phase	23
Tool Flows in the Insertion Phase	24
Top-Down Insertion Flow	24
Bottom-Up Insertion Flow	28
Block-Based Insertion Flow	31
Important Considerations in Selecting a Tool Flow	36

Chapter 2

Input and Output Files	37
Input Files	37
Chip Design Files	37
MBISTArchitect Library Model	37
MBISTArchitect Dofile	37
User-Defined Algorithms File	37
ROM Content File	38
Output Files	39
MBISTArchitect Output File Naming	39
Regarding 'include Directives	40
HDL BIST Circuitry File	40
VHDL Only - BIST Controller Configuration Format	41
HDL BIST Connection File and Test Bench	42
Pattern Files and Controller Test Description Files	42
Synthesis Driver File for BIST Circuitry	44
Insertion Utility File and Synthesis Driver File	44

BSDArchitect Dofile.	44
Output Files for Diagnosis.	44
Creating the Diagnostic Configuration File	45
Creating the Controller Mapping File.	45
Chapter 3	
Memory Modeling.	49
Memory Modeling Syntax.	49
Defining the Memory Model.	50
Header.	50
BIST Definition.	50
BIST Testing of Output Enables.	76
Fast Column Addressing.	76
Fast Row Addressing.	76
BIST-Ready Memory Support.	76
Pin Declarations for a BIST-Ready Memory.	77
Clock Signal.	79
Example - Mixed BIST-Ready and Non-Paired Signals.	79
Example - All Input Signals Are Paired.	80
Bypass-Ready Memories.	83
Memory Modeling Examples.	85
Example - RAM4x8.	85
Example - RAM Model for the Memory Device.	85
RAM4x8 Model Description.	86
Defining the Memory Model and I/O Models with Specparams.	104
Verilog Memory Model.	105
IO Pad Model.	107
Input Pad.	107
Output Pad.	107
Bidirectional Pad.	108
BIST Controller and Memory Blocks.	108
Chapter 4	
Algorithms.	111
Fault Types.	111
Coupling Faults.	112
Stuck-at Faults.	113
Transition Faults.	114
Pre-Defined Algorithms.	114
march1.	115
march2.	116
march3.	118
col_march1.	118
unique.	120
checkerBoard (topChecker).	122
retentionCB.	123
rom1 (rom).	124
rom2.	124

Table of Contents

addressdecoder_bg0 and addressdecoder_bg1	125
Port Interaction Testing Algorithm	126
Faults Targeted by the Port Interaction Algorithm	126
Port Isolation Testing Algorithm	128
Types of Multiport Memories	128
Types of Multiport Problems	129
Port Isolation Testing Process	129
Interactions and Limitations with BISA	130
Port Isolation Testing with Diagnostic Monitor	131
Port Isolation Terminology (Keywords)	131
Retention Testing	133
Retention Test Scheme for Multiple BIST Controllers	133
Waiting Period	134
Controlling the Retention Test Delay Time	134
Retention Testing at SoC Level	134
Sequential BIST Scheme	135
Parallel BIST Scheme	135
Online Algorithm Selection Capability	135
Online Algorithm Selection Protocol	135
Hardware Impact	136
Required Skip States	136
Required Shift Register	137
Required Mux Modules	137
Additional Hardware	137
Listing the Names of the Algorithms	137
Algorithm Selection Register	137
Multiport Memories	139
Controller Interface Signals	140
Comparator Test	141
Reporting Algorithms	142
Algorithm Clock Cycles - Determining Test Time	142
Reporting Algorithm Steps	142

Chapter 5

User-Defined Algorithms	143
Adding UDAs to the Design	143
User-Defined Algorithm Language	144
Test Definition	145
Repetition Definition	147
Step Definition	148
UDAs with Data Registers	158
Write Enable Mask Algorithm	164
Write Enable Mask Algorithm Detail	164
Setting Up The Write Enable Mask UDA Part	165
Port Isolation User-Defined Algorithm	170

Chapter 6**BIST, Memory, and the System..... 173**

The Default Scenario	173
BIST Circuitry Interface Ports	174
Comparators Versus Compressors	176
Comparators for RAMs	177
Compressors for ROMs	177
Controlling RAMs Separately From ROMs	178
The Compressor in the Chip	179
Clocking Schemes	180
Primary Versus Secondary Clocks	180
Asynchronous Memories	181
Modifying the Memory Clock	181
The Original System Memory Clock	182
Muxing the Memory Clock	182
Muxing and Inverting the Memory Clock	183
Inverting the BIST Controller Clock	184
Special Controls	184
Hold	184
Reset	184
Concurrent Versus Sequential Testing of Memories	184
Concurrent Memory Test	184
Sequential-Contiguous Memory Test	185
Sequential-Interleaved Memory Test	186
Top-Level Insertion Circuitry	187
Top-Level Pin Mapping	187
Controller Pin Sharing	188
Collar Pin Sharing	188
Adding Output Logic	189
Sharing Top-Level Bidirectional Ports	190
Bidirectional Enable Signal Behavior	191

Chapter 7**BIST and Diagnostics..... 193**

BIST Diagnostic Scheme	193
Diagnostic Clock Control	193
Interface to Diagnostic Block	193
Bit Sequence of Diagnostic Scan Out Data	195
Selection of Scan Out Data	195
Scanning Out Diagnostic Data	196
Field Size (Diagnostic Register Width)	197
Generating a BIST Controller with Diagnostic Capabilities	197
Diagnostics with BIST Full-Speed	199
Setting the Diagnostic Mode in MBISTArchitect	200
Diagnostic Clock Domains	200
Synchronization Between MBIST Clock and Diagnostic Clock	200
Setting Recovery and Hold Recovery States	202
Avoiding Timing Problems	203

Table of Contents

Fail_h Behavior	204
Chapter 8	
BISA for Repair	207
BISA Rules and Limitations	207
Mixing Memories With and Without Redundancy	208
BISA Block Diagram	209
Activating BISA With a Column Repair Strategy	210
Activating BISA With a Row Repair Strategy	213
BISA Report Format	214
Example - BISA Report for Column Bits and Column Index	216
Example - BISA Report with Row Repair	217
Example - BISA Report with Memid Field Omitted	218
Example - BISA Report with RR and NR Memories	219
BISA Timing Diagram	220
Chapter 9	
Controller Test Description Language	223
CTDL Syntax Conventions	223
Controller Test Description File Contents	224
Controller Declaration Block	225
Timeplate Definitions	226
Controller Test Procedure	228
Changing the Default Time Scale	230
Controller Test Access File Contents	232
Controller Instance Declaration	233
Controller Access Timeplate Definition	234
Controller Access Procedure	235
Timeplates for Test Patterns	236
Controller Timeplates	237
SoC Timeplates	237
Map_timeplate Statement	237
Automatic Timeplate Mapping	238
Timeplate Optimization	238
Chapter 10	
Full-Speed BIST	239
Current Memory BIST Status	239
Understanding At-Speed Testing	242
Understanding Full-Speed Testing	243
Pipelined Read/Write Operations	244
Problems To Consider with Full-Speed	247
Practical Considerations for Full-Speed	247
Pipelining	251
Pipelining the Expect Process	251
Comparator Result Pipelining	252
Memory I/O Pipelining	255

Chapter 11**Diagnosing Memory Failures..... 261**

The Memory Diagnosis Process	261
Memory Diagnosis Requirements and Input Files.	263
Preparing the ATE Failure Log for Memory Diagnosis	264
Rules	264
Keywords	264
ASCII Failure File Example	265
Running Memory Diagnosis	265
Interpreting Memory Diagnosis Results	267
Diagnosis Report Example	267
Verbose Diagnosis Report Example	267
Command Reference	269
Add Controller Mapping	270
Delete Controller Mapping	271
Diagnose Failures	272
Echo	273
Exit	274
Help	275
History	276
Report Controller Mapping	277
Report Diagnostic Monitor	278
Set Diagnosis	280
Set Dofile Abort	281
System	282

Appendix A**Getting Help 283**

Documentation	283
Mentor Graphics Support	284

Appendix B**Design Rules Checking..... 285**

The Design Rules	285
CTDF Rule Checking	285
Integration Rule Checking (I Rules)	286
Pad Rule Checking	289

Appendix C**MBISTArchitect Flow with BSDArchitect..... 295**

Generating TAP Compliant Hardware	295
Creating the BSDArchitect Dofile	295
Inserting Boundary Scan	296
Examples	296
Simple Configuration	296
Diagnostics	301
Retention	306
MISRs	309

Table of Contents

Repair	314
Online Algorithm Selection	317
Appendix D	
Pre-Defined Algorithm File Contents	325
Index	
Third-Party Information	
End-User License Agreement	

List of Figures

Figure 1-1. Memory BIST Architecture	19
Figure 1-2. MBISTArchitect Usage Flow	20
Figure 1-3. Top-Down Insertion Flow	25
Figure 1-4. Before Top-Down Insertion Flow	26
Figure 1-5. After Top-Down Insertion Flow	26
Figure 1-6. Bottom Up Insertion Flow	29
Figure 1-7. Block-Based Insertion Flow	32
Figure 3-1. Pin Connections Example	53
Figure 3-2. The don't_touch Signal	55
Figure 3-3. The Meaning of top_word	59
Figure 3-4. Address Descrambling	60
Figure 3-5. Data Descrambling	61
Figure 3-6. Address and Data Descrambling	63
Figure 3-7. Deriving the Address Descrambling Information.....	64
Figure 3-8. Change Event Example	66
Figure 3-9. Assert Event Examples	66
Figure 3-10. Expect Event Examples	68
Figure 3-11. Wait Event Example	69
Figure 3-12. SRAM for Read/Write Cycle Optimization Example	71
Figure 3-13. SRAM Read and Write Respective Timing Example	72
Figure 3-14. SRAM Read/Write/Read Optimized Timing Example	72
Figure 3-15. SRAM Shortened Timing Sequence Example	73
Figure 3-16. SRAM for Fix Modifier Example	74
Figure 3-17. Fix Modifier Timing Example	75
Figure 3-18. BIST-Ready Memory	77
Figure 3-19. RAM4X8 Example	85
Figure 3-20. Read Cycle Timing and Events.....	92
Figure 3-21. Write Cycle Timing and Events	93
Figure 3-22. Single Port Synchronous RAM8X2 Example	93
Figure 3-23. SRAM Read Cycle Timing Diagram	94
Figure 3-24. SRAM Write Cycle Timing Diagram	95
Figure 3-25. Test Timing for a March2 Algorithm SRAM Example	97
Figure 3-26. Dual Port Synchronous RAM8X2 Example	99
Figure 3-27. Test Timing for a March2 Algorithm Example	101
Figure 3-28. SRAM with Two Control Enables Example	102
Figure 3-29. Controller-To-Collar Instances Connected By Specparam	109
Figure 4-1. Inversion Coupling Fault	112
Figure 4-2. Idempotent Coupling Fault	112
Figure 4-3. Stuck-at Fault State Diagram.....	113
Figure 4-4. Transition Fault	114
Figure 4-5. Transition Fault State Diagram	114

List of Figures

Figure 4-6. Modifying the MarchC Algorithm	116
Figure 4-7. Modified MarchC Algorithm	117
Figure 4-8. MarchC+ (March2) Algorithm	117
Figure 4-9. March2 Algorithm with Varied Background	118
Figure 4-10. March3 Algorithm.	118
Figure 4-11. Col_March1 Algorithm	119
Figure 4-12. Unique Address Algorithm	121
Figure 4-13. Checkerboard Algorithm (TopChecker)	122
Figure 4-14. Synchronized Retention Testing Across Multiple Controllers	134
Figure 4-15. Online Algorithm Selection Protocol	136
Figure 4-16. Online Algorithm Selection Registers	138
Figure 4-17. Algorithm Selection JTAG Interface	138
Figure 5-1. UDA Language Syntax	144
Figure 5-2. Test Definition Usage	145
Figure 5-3. Repetition Definition Usage	147
Figure 5-4. Step Definition Usage	148
Figure 5-5. Address Sequence Examples	152
Figure 6-1. Comparator for Two RAMs	177
Figure 6-2. Compressor for a ROM	178
Figure 6-3. Compressor (MISR) Clock Gating	179
Figure 6-4. Asynchronous Memory Write Cycle	181
Figure 6-5. Setup Memory Clock -System	182
Figure 6-6. Setup Memory Clock -Test Noinvert	183
Figure 6-7. Setup Memory Clock -Test Invert	183
Figure 6-8. Concurrent Memory Test	185
Figure 6-9. Sequential-Contiguous Algorithm Ordering	186
Figure 6-10. Interleaved Sequential Algorithm Ordering	187
Figure 6-11. Top-Level Pin Mapping Example	188
Figure 6-12. Controller Pin Sharing Example	188
Figure 6-13. Collar Pin Sharing Example	189
Figure 6-14. Adding Output Logic Example	189
Figure 6-15. Bidi Port Configured as an Input Port	190
Figure 6-16. Bidi Port Configured as an Output Port	190
Figure 6-17. Bidi Port Controlled by a Primary Input (PI)	191
Figure 6-18. Bidi Port Controlled by a Primary Input (PI) with Fan Out	191
Figure 7-1. Diagnostic Output Data	195
Figure 7-2. BIST Architecture Using Diagnostic Functionality	198
Figure 7-3. Diagnostic Control Process in MBIST Clock Domain	201
Figure 7-4. Diagnostic Scan Process in Diagnostic Clock Domain	202
Figure 8-1. BISA Block Diagram	209
Figure 8-2. BISA Clock Gating	210
Figure 8-3. Redundant Column Repair Example	212
Figure 8-4. Redundant Row Repair Example	214
Figure 8-5. BISA Combined Report	214
Figure 8-6. BISA Unit Report	215

Figure 8-7. BISA Report Block	215
Figure 8-8. Combined Report for Column Repair Example	216
Figure 8-9. Combined Report for Row Repair Example	218
Figure 8-10. Combined Report for Memid Omitted Example	219
Figure 8-11. Combined Report for RR and NR Example	219
Figure 8-12. BISA Timing Diagram	220
Figure 10-1. Memory BIST Controller	240
Figure 10-2. Read Operation	241
Figure 10-3. Write Operation	241
Figure 10-4. Consecutive Read/Write Operations	242
Figure 10-5. MBIST Full-Speed Pipelined Read Operations	245
Figure 10-6. MBIST Full-Speed Pipelined BIST Controller	246
Figure 10-7. MBIST Full-Speed Pipelined Write Operation	246
Figure 10-8. Pipelined Consecutive Read/Write Operations	247
Figure 10-9. Pipelined Read/Write with Negative-edge BIST Controller	249
Figure 10-10. Pipelining Read/Write Operations with Negative-edge	250
Figure 10-11. No Expect Indexing	251
Figure 10-12. Expect Indexing and Pipelining	252
Figure 10-13. Restart Diagnostics	253
Figure 10-14. Comparator Result Pipelining (Restart)	253
Figure 10-15. Hold / Nohold Diagnostics	254
Figure 10-16. Comparator Result Pipelining (Hold / Nohold)	254
Figure 10-17. No Output Pipeline Instance	256
Figure 10-18. Output Pipeline Instance In Collar	256
Figure 10-19. Output Pipeline Instance In Controller	257
Figure 10-20. No Input Pipeline Instance	258
Figure 10-21. Input Pipeline Instance In Collar	259
Figure 10-22. Input Pipeline Instance In Controller	259
Figure C-1. Memory BIST to Boundary Scan Process	295
Figure C-2. Default BSDArchitect Configuration	301
Figure C-3. Boundary Scan Configuration with Diagnostics	305
Figure C-4. Boundary Scan Configuration with Retention	309
Figure C-5. Boundary Scan Configuration with MISRs	313
Figure C-6. Boundary Scan Configuration with Repair	317
Figure C-7. Boundary Scan Configuration with Serial Algorithm Selection	321
Figure C-8. Boundary Scan Configuration with Parallel Algorithm Selection	324

List of Tables

Table 2-1. Output File Names	39
Table 2-2. WGL Files Generated	43
Table 3-1. Write Enable Mapping of the Given Example	56
Table 4-1. Pre-Defined Algorithm Summary	115
Table 4-2. RetentionCB Algorithm	124
Table 4-3. Online Algorithm Interface Signals	140
Table 5-1. Operation Data for “my_test” Algorithm	156
Table 5-2. Shifted Address Locations	157
Table 5-3. Data Registers Used for Operations	159
Table 5-4. Steps for a Proposed Algorithm for the Example	165
Table 5-5. Sequence of Steps for Concurrent Controller	166
Table 5-6. Sequence of Steps for Sequential Non-Interleaved Controller	167
Table 5-7. Sequence of Steps for Sequential Interleaved Controller	167
Table 6-1. BIST Controller Input and Output Ports	174
Table 6-2. BIST Collar Input and Output Ports	176
Table 6-3. test_control_signal for Different Pins	192
Table 7-1. Diagnostic Block Interface	193
Table 7-2. Diagnostic Data Explanation	194
Table 8-1. Example Column Repair Controls	212
Table 11-1. Memory Diagnosis Process	263
Table 11-2. ASCII Failure File Keywords	264
Table 11-3. Memory Diagnosis Command Summary	269

MBISTArchitect Overview

The Mentor Graphics MBISTArchitect™ tool provides all the features required for testing embedded memories by applying Built-In Self-Test (BIST). The tool is used to generate and insert complete Register Transfer-Level (RTL) test logic that can be applied to an unlimited number of memories, with varying sizes and configurations.

The rich feature set of MBISTArchitect efficiently addresses three key areas to ensure all embedded SRAMs and ROMs are thoroughly tested: high test quality, easy application, and versatility.

High Test Quality

The fine geometries typical of an embedded memory make it susceptible to subtle defects. Testing it requires a thorough set of patterns strategically chosen to expose manufacturing defects. The MBISTArchitect tool creates test circuitry that applies, reads, and compares test patterns to expose these defects. It allows you to choose from a large set of memory test algorithms. These algorithms include the common march and checkerboard algorithms, varied pattern backgrounds, and many others.

For memories requiring the application of proprietary algorithms, the MBISTArchitect tool offers a user-definable algorithm feature. Online algorithm selection enables optimum balancing between defect coverage and test time during production.

Since memories rely on small amounts of charge transfer for proper operation, many faults are only observable when memories are run at their maximum operating speed. The MBISTArchitect tool can apply patterns “at-speed” to ensure higher coverage of speed-related defects. The MBISTArchitect Full-Speed™ feature can accelerate at-speed test by up to a factor of three. Using a patent-pending pipeline technique, MBIST Full-Speed simultaneously applies patterns, reads them back, and compares the results. MBIST Full-Speed has been applied to memories running in excess of 800MHz.

Easy Application

Memory test concerns should not consume valuable design time. The MBISTArchitect tool automates the entire process by simply reading a model of the memory, creating the entire BIST circuitry, and automatically inserting it into your design. You can test memories concurrently or sequentially as well as determine which memories share controllers. The memory BIST circuits

are automatically inserted into your RTL design, allowing control of which memories share controllers and the sequence of testing. To quickly verify BIST insertion, the MBISTArchitect tool automatically creates a simulation testbench.

One of the tool's most important features is award-winning customer support. Mentor Graphics employs an expert staff of support engineers who specialize in design-for-test (DFT) and can assist in the memory BIST process if necessary.

Versatility

The rapidly changing world of memory technology requires a tool that can adapt to many different technologies and test configurations. The MBISTArchitect tool supports a wide variety of memory configurations, including those with multiple ports, data scrambling, and pre configured BIST access. The MBISTArchitect tool is used by a broad range of customers, from large semiconductor companies, to small fabless operations, and Mentor Graphics works closely with these customers to develop ongoing tool enhancements that keep pace with changing technology and needs. This development process insures you have access to the latest and most comprehensive memory testing technology available.

When BIST logic identifies faults in a memory, optional on-chip diagnostics can help you pinpoint its exact location. Since the diagnostic report is serial, you can access it via boundary scan using BSDArchitect™, Mentor's 1149.1-compliant boundary scan tool. The MBISTArchitect tool also offers on-chip BISA (Built-In Self Analysis), which tracks defective memory locations found during test, analyzes them, and produces a serial report which you can use to activate a redundant memory resources as part of a memory repair strategy.

The MBISTArchitect tool is supported by many of the leading memory vendors, including Artisan Components, Virage Logic, and MoSys. These companies provide the model libraries needed to get you up and running.

The MBISTArchitect tool is part of the Mentor Graphics technology-leading tool suite, which includes integrated solutions for scan, ATPG, embedded deterministic test, advanced memory test, logic BIST, boundary scan, and a variety of DFT-related flows.

Invoking MBISTArchitect

MBISTArchitect can be invoked in three phases:

- **BIST Insertion Phase**
In the BIST Insertion phase you can create BIST logic and insert that logic into your design, or you can insert already generated BIST into your design.
- **BIST Generation Phase**
In the BIST Generation phase you only generate BIST logic. You would use the BIST insertion phase to insert the logic you generated in this phase.

To invoke the tool, enter the **mbistarchitect** invocation command at the command line of a terminal window, followed optionally by the name of the design and design type, and the libraries you want loaded during the invocation. When specifying the libraries to use, you can specify a directory of library files by specifying the path.

Using the Tool from the Command Line

The tool defaults to command-line mode at invocation. The following text shows the command line arguments for the different phases of the tool. The following sections of this chapter show some ways the tool can be invoked using these arguments. For more detail on a particular command-line argument, see the “[Command Dictionary](#)” in the *MBISTArchitect Reference Manual*.

```
mbistarchitect
  {<design_name> [<design_type>] [-INCdir <search_path>...]}
  [-LICense_wait {minutes | NONE | UNLlimited}]
  {
    {-Insertion -TOp <name> {-LVERilog <library> | -Hlerarchical}}
  }
  [-LOgfile <name>]
  [-Replace]
  [-Dofile <name>]
  [-HIStory]
} |
  {-Bistgen [-Llibrary <filename>] [-LICense_wait {minutes | NONE | UNLlimited}]}
  [-LOgfile <name>]
  [-Replace]
  [-Dofile <name>]
  [-HIStory]
} |
  [-Help | -Usage | -MANual | -VERSion]
```

Loading Libraries

You can load libraries by specifying the library in the invocation script as shown in [Using the Tool from the Command Line](#).

Resetting MBISTArchitect

At times, you might find it necessary to discard all your entered commands and start over from the beginning. This typically happens when you make more than one customization to the BIST implementation.

Reset State Command

The Reset State command lets you effectively reset all the command arguments and values to their default values, which is equivalent to exiting and re-invoking on the same design. However, unless you use the -All switch, any loaded libraries will remain loaded.

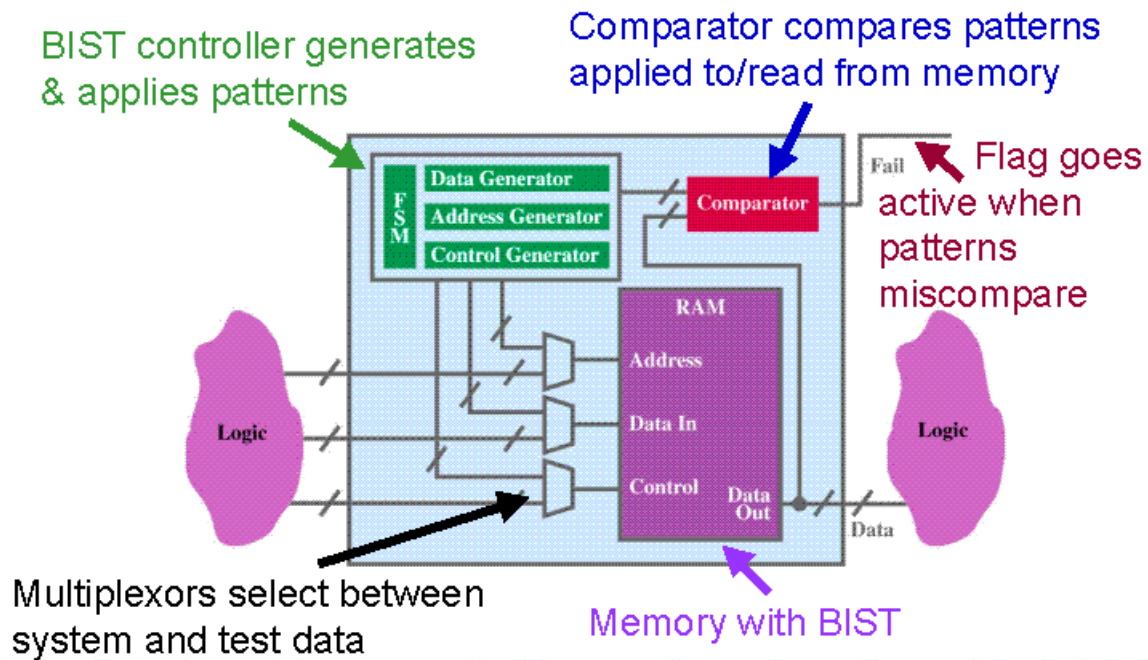
Exiting MBISTArchitect

You can exit the tool by clicking Exit in the Control Panel button pane. You can also exit the tool by entering exit on the command line.

MBISTArchitect Architecture

Figure 1-1 shows the MBISTArchitect tool's memory BIST architecture.

Figure 1-1. Memory BIST Architecture

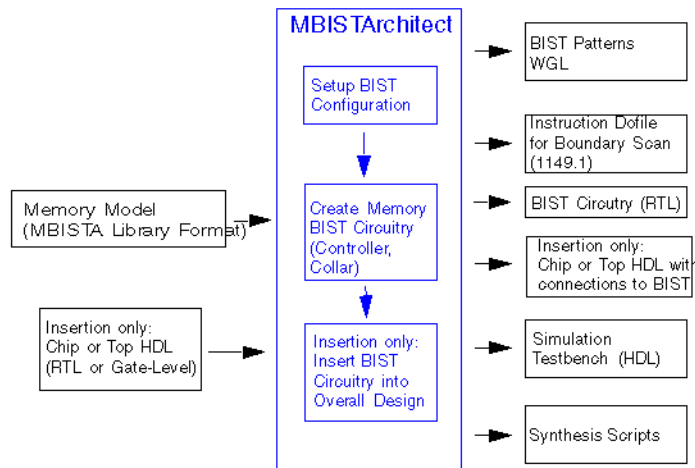


The embedded logic runs the full memory test on chip. The finite state machine (FSM) generates and applies patterns, also known as test algorithms. The comparator checks data read from memory.

MBISTArchitect Usage Flow

Figure 1-2 shows the top-down insertion phase flow for MBISTArchitect. If you ignore the “Insertion only” boxes, then this is also the generation phase flow.

Figure 1-2. MBISTArchitect Usage Flow



The MBISTArchitect insertion phase has additional flow options, as shown later in this chapter.

BIST Insertion Phase (Includes Generation Activity)

When you invoke the tool using ***mbistarchitect -insertion*** you enter the BIST insertion phase. In this phase you can perform the generation activity and the insertion activity in one invocation of the tool.

Note



While MBISTArchitect can create output files formatted in VHDL, it cannot perform insertion with models formatted in VHDL. If you want to use MBISTArchitect for insertion, you must use Verilog.

To invoke the tool in the BIST insertion phase, use the following at the command line:

mbistarchitect

```
{<design_name> [<design_type>] [-INCdir <search_path>...]  
[-LICense_wait {minutes | NONE | UNlimited}]  
    {-Insertion -TOp <name> {-LVerilog <library> | -Hierarchical}}  
    [-Logfile <name>]  
    [-Replace]  
    [-Dofile <name>]  
    [-HISTORY]
```

BIST Insertion Phase Modes

The BIST insertion phase has the following modes.

SETUP Mode

In the Setup mode the screen prompt is SETUP> and all SETUP legal mode commands (commands like Load Library, and Load Design Object) are available, and can be used.

BIST Mode

In the BIST mode the screen prompt is BIST> and all BIST legal mode commands (commands like Add New Controller, and Add Existing Controller) are available, and can be used.

INT Mode

In the Integration mode the screen prompt is INT> and all INT legal mode commands (commands like Add Pattern Translation, and Delete Patterns) are available.

The *MBISTArchitect Reference Manual* lists which modes are legal for every command.

Switching Between Modes

You can switch between modes using the Set System Mode command.

RTL Design Rules Check (DRC) are executed every time you switch from Setup mode to BIST mode. There are two CTDF rules checked. The CTDF rules verify the correctness of the controllers description.

The tool will enable you to use the command Report Memory Instances to list all of the memory instances in the design. For each memory instance, the memory model name and the library file are reported. If the memory is already BISTed, the instance and module name of the BIST controller will be listed.

Report Memory Instances

The following is an example of the Report Memory Instances command when Add Existing Controller and Add New Controller commands are executed.

Example - Reporting Memory Instances

Consider the following example dofile with the following commands:

```
load library mbist_lib
report memory instances
```

```
set system mode bist
add new controller mbistc1 -dofile mbist1.do /U1/regs_picdram
/U3/regs_picdram
add new controller /U2/mbistc2 -dofile mbist2.do /U2/regs_picdram
report memory instances
```

The result of first report memory instances command is shown in the following example.

```
// -----
//      Memory Instance  Memory Model  Model File      Controller Instance  Controller Module  Controller File
// -----
//      /U3/regs_picdram  picdram3    mbist_lib        NO BIST             NO BIST            NO BIST
//      /U2/regs_picdram  picdram2    mbist_lib        NO BIST             NO BIST            NO BIST
//      /U1/regs_picdram  picdram1    mbist_lib        NO BIST             NO BIST            NO BIST
// -----
```

The result of second report memory instances command is shown in the following example.

```
// -----
//      Memory Instance  Memory Model  Model File      Controller Instance  Controller Module  Controller File
// -----
//      /U3/regs_picdram  picdram3    mbist_lib        /mbistc1            picdram1_bist_0    picdram1_bist_0.v
//      /U2/regs_picdram  picdram2    mbist_lib        /U2/mbistc2          picdram2_bist_1    picdram2_bist_1.v
//      /U1/regs_picdram  picdram1    mbist_lib        /mbistc1            picdram1_bist_0    picdram1_bist_0.v
// -----
```

Once you identify the embedded memories in the design to be BISTed with the Report Memory Instances command, for each group of memories, you need to issue one of the following commands:

- Add Existing Controller command to associate the already generated controller and collars with memory instances in the design (bottom-up flow).

This command will only schedule the controller and associated memories for insertion. The memory collar associated with each memory has to be specified. There will be no BIST generation performed.

- Add New Controller command to trigger BIST generation to take place for the specified memory instances.

You can overwrite the default BIST generation dofile by specifying a dofile using the -dofile <mbist_dofile> switch option of the command.

A default dofile with a set of MBIST commands will be executed if no dofile is explicitly specified.

Default Dofile Contents

The default dofile has the following commands.

```
reset state
add memory model
setup mbist algorithms March2
set bist insertion -on
set bsda -on
add signal synchronization test_h
set design name controller -module
set file naming -bist_model
set file naming -connected_model
set file naming -testbench
set file naming -script
set file naming -ctdl
set file naming -wgl
run
save bist -verilog -script -replace
exit -force
```

Upon the completion of the Add New Controller command, the tool will automatically import the pathname of the generated BIST controller name, file pathname, as well as the generated controller description file. The result will be reflected the next time a Report Memory Instances command is executed.

Example - Add New Controller

```
add new controller contr1 -dofile mbist.do mem0 mem1 /a/a/mem2
add new controller /blk1/contr2 mem3 /a/mem4
add new controller contr3 mem5 -collar mycollar mem6 /a/mem1
```

BIST Generation Phase

To invoke the tool in the BIST generation-only phase, use the following at the command line. Because the -Bistgen switch is the tool default, you do not have to use it.

mbistarchitect

-Bistgen [-Llibrary <filename>] [-LICense_wait {minutes | NONE | UNLimited}]

[-Logfile <name>]

[-Replace]

[-Dofile <name>]

[-HIStory]

The BIST Generation Phase has just one mode, Bistgen, marked by the screen prompt BISTGEN>. See the *MBISTArchitect Reference Manual* for commands which are legal in the BISTGEN mode.

The generation phase is simpler than the insertion phase because here the tool requires your memory model (MBISTArchitect library format) but not the HDL of your chip or top design or

memories. The output of this phase includes, among other things, the HDL of the BIST circuitry (the BIST controller and memory collars). In the [Bottom-Up Insertion Flow](#) you pass some of these outputs back into the tool for a second invocation of the tool.

Note

 While MBISTArchitect can create output files formatted in VHDL, it cannot perform insertion with models formatted in VHDL. If you want to use MBISTArchitect for insertion, you must use Verilog.

Tool Flows in the Insertion Phase

The tool offers the following flows related to the insertion phase:

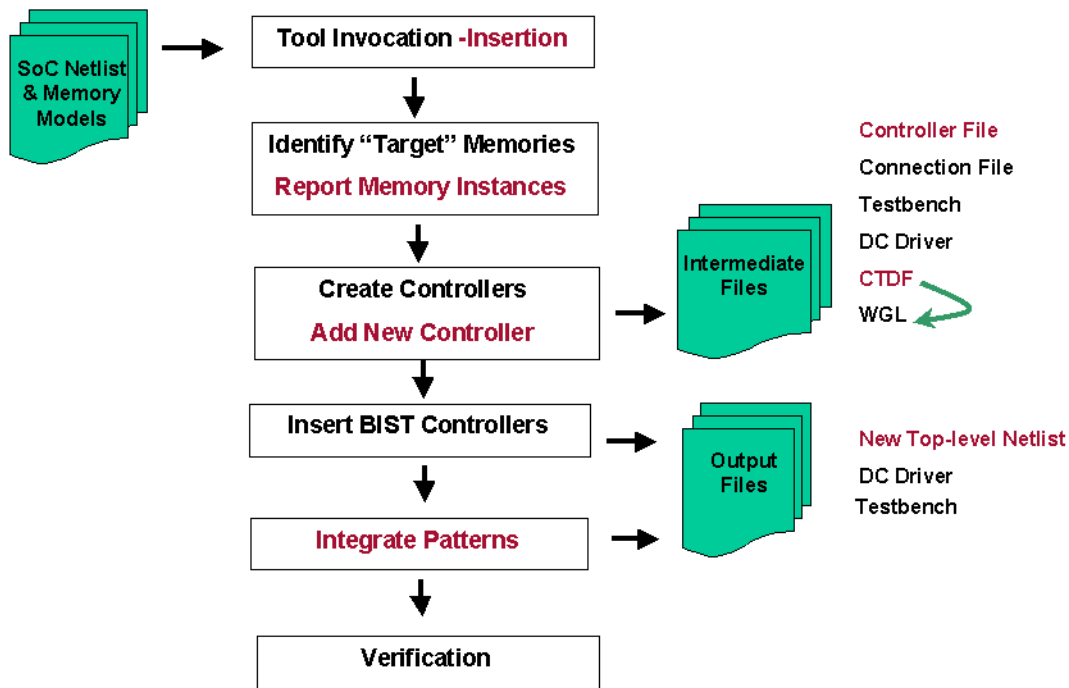
- [Top-Down Insertion Flow](#)
- [Bottom-Up Insertion Flow](#)
- [Block-Based Insertion Flow](#)

Top-Down Insertion Flow

In the top-down insertion flow you perform both BIST generation and insertion activities in one invocation of the tool. First, you generate BIST controllers, then you insert them into the netlist and hook them up to the top-level design. The generation activity results in saving BIST circuitry to intermediate files, including a controller test bench which drives just those files. The insertion activity results in saving a revised version of your input chip or top HDL in which the BIST circuitry has been inserted and connected. The file structure of your input chip or top HDL is maintained in the BIST-inserted output netlist and none of your original design or signal names are changed. Finally, in this flow you can perform the integration activity, which results in an a top-level test bench which drives the BIST-inserted output netlist using your integration patterns. Afterwards you can verify the generated/inserted outputs by simulating the test benches.

[Figure 1-3](#) shows a flow chart of the top-down insertion flow.

Figure 1-3. Top-Down Insertion Flow



The example in [Figure 1-4](#) shows a circuit before it is BISTed using the top-down insertion flow.

The example in [Figure 1-5](#) shows the same circuit after the top-down flow. In [Figure 1-5](#) “Cont” stands for “BIST Controller”. The BIST-inserted memory collars are not shown, but they surround the “Mem” (Memory) blocks after insertion.

The following sections show the example scripts you might use to perform the top-down insertion flow.

Figure 1-4. Before Top-Down Insertion Flow

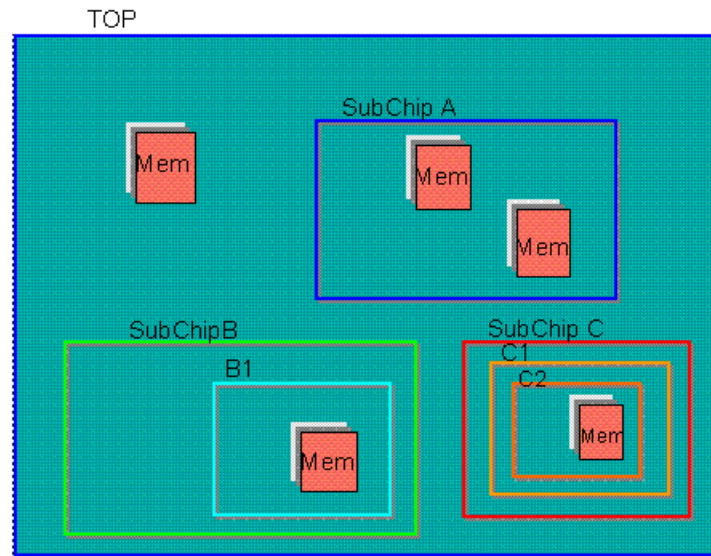
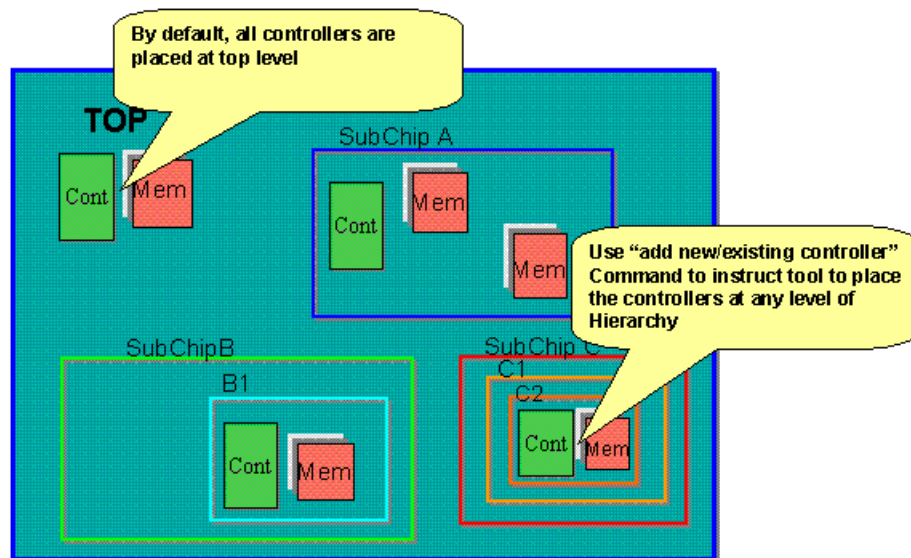


Figure 1-5. After Top-Down Insertion Flow



Example - Top-Down Insertion Flow Invocation Script

The following example script invokes MBISTArchitect in the top-down insertion flow.

```
$HOME_NAME/bin/mbistarchitect \  
  ./netlists -verilog \  
  -insertion \  
  -top top \  
  -lverilog ./libs/Verilog \  
  -logfile ./transcripts/mbist.log -replace \  
  -
```

```
-dofile ./run_mbist.do \
```

Example - MBISTArchitect Insertion Dofile

The following example insertion dofile is the -dofile argument in argument in the previous top-down insertion flow command line invocation script.

```
// Load the library of memory models.
load library memory_models.lib
// Get a report of the memories in the netlist.
report memory instances
// Change to the BIST mode.
set system mode bist
// Perform BIST generation.
add new controller ram8x8_ctlr \
    -dofile ram8x8.do /U2/C2_M3 /U2/C1_C3/M3
add new controller sp1024x8_ctlr \
    -dofile sp1024.do /U1/C1_ /U1/C1_C3/C3_M1
add new controller rad5a804_BIST_READY_ctlr \
    -dofile rad5a804.do /U2/C1/BISTREADY_1
// Insert a controller and connect them to the top-level.
insert bist logic
report controllers

save design all_RTL.v -suffix bisted -replace
// Perform pattern conversion and create top-level test bench.
set system mode integration
add pattern translation -all
report pattern translation
integrate patterns
report pattern translation
save pattern ./ramtest1_tb_all.v -verilog -replace
exit -force
```

Example - MBIST Generation Dofile

The following example generation dofile is the nested -dofile argument of the Add New Controller command in the insertion dofile shown in the preceding section.

```
reset state
add memory model ram8x8
add memory model ram8x8
setup mbist algorithms march2
set bist insertion -on
set bsda -on
set design name controller -module ram8x8_bist_0
set file naming -bist_model ram8x8_bist_0.v
set file naming -connected_model ram8x8_bist_0_con.v
set file naming -testbench ram8x8_bist_0_tb.v
set file naming -script ram8x8_bist_0.v_dcscript
set file naming -ctdl ram8x8_bist_0.v.ctdf
set file naming -wgl ram8x8_bist_0.wgl
```

```
run
save bist -verilog -script -replace
exit -force
```

Notice that in the insertion phase you have one dofile (mainly for insertion) invoking another dofile (mainly for generation). Alternatively, you could use this example generation dofile by itself as the `-dofile` argument in a generation phase command line invocation, either for manual connection to your chip, or as a part of the bottom-up insertion flow.

Example - Test Bench and Netlist Compile Script

The following is an example of a script that can be used to compile the integration mode testbench and the BIST-inserted output netlist for simulation. The script uses ModelSim™ vlib and vlog, which are part of the Mentor Graphics suite of design simulation tools.

```
#!/usr/bin/csh -f

$HOME_NAME/bin/vlib ./work
$HOME_NAME/bin/vlog ./ramtest1_tb_all_RTL.v \
    ./netlists/blk1_bisted.v \
    ./netlists/blk2_bisted.v \
    ./netlists/soc_bisted.v
    ./libs/Verilog/mlrotrom_specparam.v \
    ./libs/Verilog/rad5a804_BIST_READY_specparam.v \
    ./libs/Verilog/ram8x8_specparam.v \
    ./libs/Verilog/sp1024x8_specparam.v \
-work ./work
```

Example - Simulation Script

After performing the compilation shown in the preceding section, you can simulate the compiled testbench using ModelSim vsim. The following example script will run your simulation.

```
#!/usr/bin/csh -f

$HOME_NAME/bin/vsim top_ramtest1_tb_all_v_ctl -do "run -all" -c \
    -l ./transcripts/sim_mbist.log
```

Bottom-Up Insertion Flow

In the bottom-up insertion flow, the activities of BIST generation and BIST insertion are performed in two separate invocations of the tool. The invocation steps are as follows:

1. BIST generation (accomplished using `-bistgen` on the command line).
2. BIST insertion (accomplished using `-insertion` on the command line).

Figure 1-6 shows the bottom-up insertion flow.

Example - Bottom-Up Insertion Flow Invocation Script 1

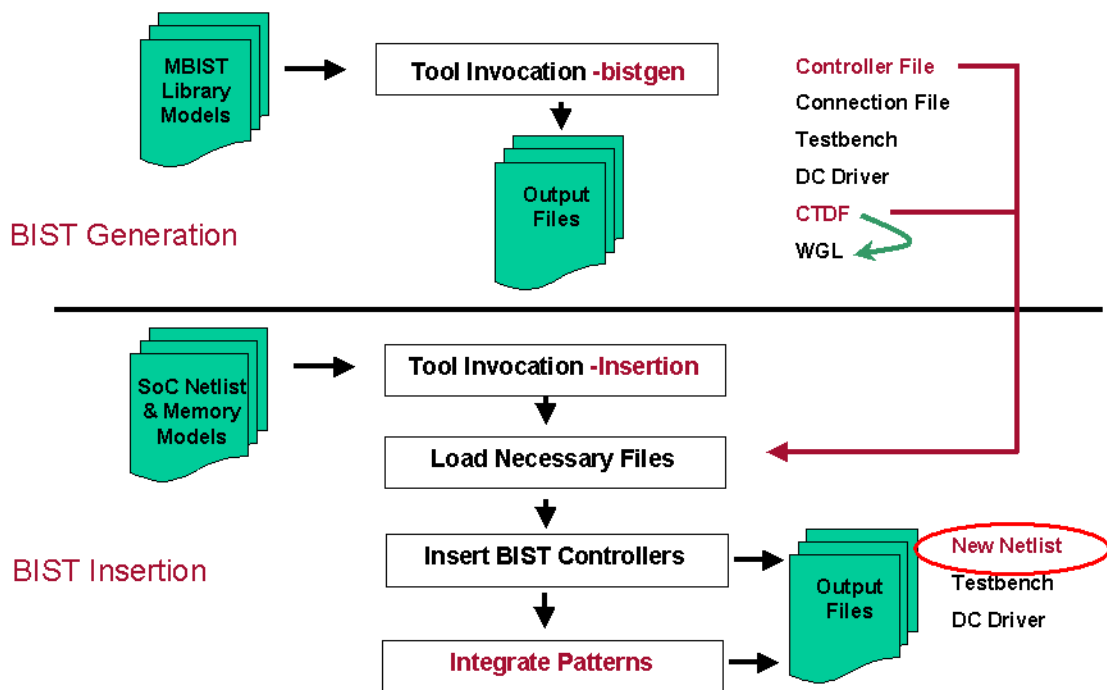
The following example script invokes MBISTArchitect in bistgen mode, which is the first part of the bottom-up insertion flow.

```
$HOME_NAME/bin/mbistarchitect -bistgen \  
-lib ./libs/MBIST/ram8x8.lib \  
-logfile ./transcripts/bgen.log -replace \  
-dofile ./run_bgen.do \  

```

The -lib argument is a memory model description in MBISTArchitect file format.

Figure 1-6. Bottom Up Insertion Flow



Example - BISTGEN Dofile

The following bistgen dofile is the -dofile argument of the bistgen command line invocation in the previous section.

```
add memory model ram4x4 ram8x8
set bist insertion -on
set design name Controller -module bist_for_2ram
set design name Collar -module mem_block
set file naming -bist_model RTL_bist.v
run
save bist -verilog -replace
exit
```

After executing this dofile, the tool has created a controller module named *bist_for_2ram* and a collar module named *mem_block*. These will be used this in the second invocation of the tool, shown in the next two sections.

Example - Bottom-Up Insertion Flow Invocation Script 2

The following example script invokes MBISTArchitect in insertion mode, which is the second part of the bottom-up insertion flow.

```
$HOME_NAME/bin/mbistarchitect core_top.v -insertion \  
-lverilog libs/vlog/ram8x8.v libs/vlog/ram8x8.v \  
-top core_top \  
-dofile run_bins.do \  
-logfile ./transcripts/bins.log -verilog -replace
```

The `-lverilog` arguments are the memory descriptions in Verilog. The first argument, `core_top.v`, and the `-top` argument specify the top-level Verilog file and top-level module, whose hierarchy will be BIST-inserted after this invocation of the tool. In this case module `core_top` is at instance path `"/`.

Example - Insertion Dofile

The following insertion dofile is the `-dofile` argument of the insertion invocation of the previous section.

```
1  load design objects RTL_bist.v  
2  load controller description RTL_bist.v.ctdf  
3  report memory instances  
4  set system mode bist  
5  add existing controller /U1/cntr_under_hier bist_for_2ram \  
6  /mem3 mem_block \  
7  /mem4 mem_block  
8  report memory instances  
9  insert bist logic  
10 save design -replace  
11 set system mode integration  
12 integrate patterns
```

The Load commands read in the Verilog and CTFD files which describe the BIST circuitry that was generated during the first invocation of the tool. The BIST controller module was named *bist_for_2ram*, and the Add Existing Controller command instantiated it as instance path */U1/cntr_under_hier*. The collar module was named *mem_block*, and the same Add Existing Controller command is rebinding the memory instances at */mem3* and */mem4* to instances of *mem_block*. Note that on line 10, you cannot specify a full output name as a target to the Save Design command. You can name a suffix or output directory, but not the entire filename.

Example - More Complicated Insertion Dofile

Adding the following “extra” command to the previous insertion dofile. This command can be placed right before or right after the Add Existing Controller command.

```
add new controller /cntr_on_top -dofile run_bgen2.do \
/U1/mem1 /U2/mem2
```

This combines the bottom-up and top-down insertion flows because the “original” Add Existing Controller command is using previously generated BIST circuitry, and this Add New Controller command is generating some additional BIST circuitry.

The file *run_bgen2.do* is not shown; however, *bgen2.do* specifies that the new BIST controller module is named *other_bist* and the netlist is *other_bist.v*.

The result of first report memory instances command is as follows.

```
// command: report memory instances
// -----
// Memory Instance  Memory Model  Model File  Controller Instance  Controller Module  Controller File
// -----
// /U1/mem1         ram4x4      ram4x4.v    NO BIST              NO BIST            NO BIST
// /U2/mem2         ram8x8      ram8x8.v    NO BIST              NO BIST            NO BIST
// /mem3            ram4x4      ram4x4.v    NO BIST              NO BIST            NO BIST
// /mem4            ram8x8      ram8x8.v    NO BIST              NO BIST            NO BIST
// /U3/U4/mem5      ram16x256   ram16x256.v NO BIST              NO BIST            NO BIST
// -----
```

The result of a second report memory instances command is as follows.

```
// command: report memory instances
// -----
// Memory Instance  Memory Model  Model File  Controller Instance  Controller Module  Controller File
// -----
// /U1/mem1         ram4x4      ram4x4.v    /cntr_on_top        other_bist         other_bist.v
// /U2/mem2         ram8x8      ram8x8.v    /cntr_on_top        other_bist         other_bist.v
// /mem3            ram4x4      ram4x4.v    /U1/cntr_under_heir bist_for_2ram      RTL_bist.v
// /mem4            ram8x8      ram8x8.v    /U1/cntr_under_heir bist_for_2ram      RTL_bist.v
// /U3/U4/mem5      ram16x256   ram16x256.v NO BIST              NO BIST            NO BIST
// -----
```

In this example, assume that the insertion activity found a fifth memory at /U3/U4/mem5 which was not targeted by either the Add Existing Controller or the Add New Controller commands.

Block-Based Insertion Flow

One reason to use a block-based flow would be where two or more design teams are in different locations (cities or even countries) Another reason might be where two or more teams are

designing the chip in blocks. Using a block-based flow each team can perform their BIST activities separately.

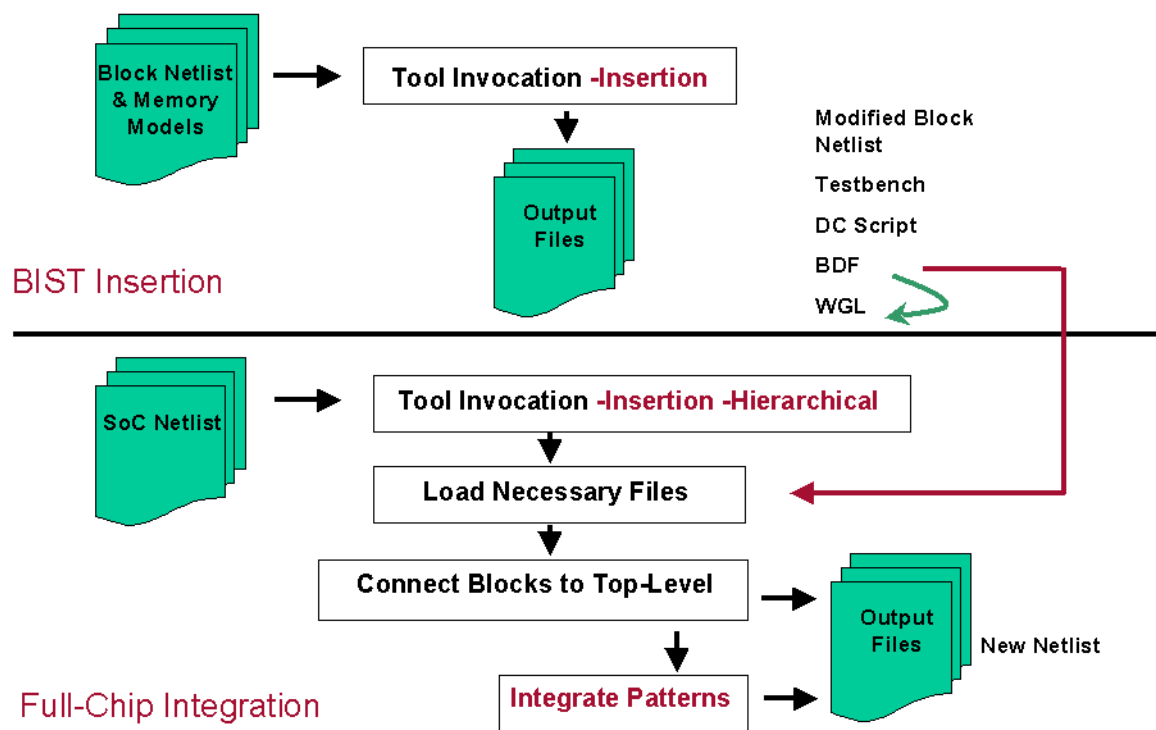
Also, when performing the insertion activity on extremely large chip designs you will need a powerful workstation to run the tool. If your CPU and workstation memory resources are limited, you can simplify the insertion activity by using the Block-Based Insertion Flow, which involves multiple invocations of the tool and breaks the large task of insertion into several smaller subtasks.

The invocation steps are as follows:

1. BIST generation and insertion activity on a single design block (accomplished using `-insertion` on the command-line). Repeat this step for all major memory related blocks in your design.
2. BIST insertion activity on the top-level design, unifying the various design blocks of the previous step (accomplished using `-insertion -hierarchical` on the command-line).

Figure 1-7 shows the block-based insertion flow.

Figure 1-7. Block-Based Insertion Flow



During the `-insertion -hierarchical` invocation, the tool uses intermediate files from the previous step to determine the interface of blocks which must be stitched together in the top-level design.

Example - Block-Based Insertion Flow Input HDL

Assume your design is represented by the following four main Verilog files:

```
// system.v
module top (clk, we, adr, i1, i2, i3, o1, o2, o3);
input clk, we;
input [6:0] adr;
input [7:0] i1, i2, i3;
block1 u1 (clk, we, adr, i1, o1);
block2 u2 (clk, we, adr, i2, o2);
block3 u3 (clk, we, adr, i3, o3);
endmodule

// block1.v
module block1(clk, we, adr, di, do);
input clk, we; input [6:0] adr; input [7:0] di; output [7:0] do;
ram r1 (clk, adr, we, di, do);
endmodule

// block2.v
module block2(clk, we, adr, di, do);
input clk, we; input [6:0] adr; input [7:0] di; output [7:0] do;
ram r2 (clk, adr, we, di, do);
endmodule

// block3.v
module block3(clk, we, adr, di, do);
input clk, we; input [6:0] adr; input [7:0] di; output [7:0] do;
ram r3 (clk, adr, we, di, do);
endmodule
```

For simplicity this example has just one type of memory, called *ram*. Assume that the memory is described in a separate Verilog file *ram.v*, and an MBISTArchitect format memory model description file *mbist.lib*.

Example - Block-Based Insertion Flow Invocation Script 1

The following example script invokes MBISTArchitect in insertion mode for the three major memory-related blocks of the design, which is the first part of the block-based insertion flow.

```
$HOME_NAME/bin/mbistarchitect block1.v -insertion \
-lverilog ram.v -top block1 -dofile bbins1.do \
$HOME_NAME/bin/mbistarchitect block2.v -insertion \
-lverilog ram.v -top block2 -dofile bbins2.do \
$HOME_NAME/bin/mbistarchitect block3.v -insertion \
-lverilog ram.v -top block3 -dofile bbins3.do \
```

Example - Insertion Dofiles Number 1

The following insertion dofiles are the –dofile arguments of the invocation script in the previous section.

```
//bbins1.do
load library mbist.lib
set system mode bist
add new controller mbist_ctrl1 -dofile bbgen.do /r1
insert bist logic
save design -blackbox -replace
set system mode integration
add pattern translation -all
integrate pattern
save patterns pat1.v -verilog -replace
save patterns pat1.wgl -wgl -replace
write block description block1.ctdf -replace
exit

//bbins2.do
load design object ram_bist.v
load controller description ram_bist.v.ctdf
set system mode bist
add existing controller mbist_ctrl2 ram_bist \
                               /r2 ram_bist_ram_block

insert bist logic
save design -blackbox -replace
set system mode integration
add pattern translation -all
integrate pattern
save patterns pat2.v -verilog -replace
save patterns pat2.wgl -wgl -replace
write block description block2.ctdf -replace
exit

//bbins3.do
load design object ram_bist.v
load controller description ram_bist.v.ctdf
set system mode bist
add existing controller mbist_ctrl3 ram_bist \
                               /r3 ram_bist_ram_block

insert bist logic
save design -blackbox -replace
set system mode integration
add pattern translation -all
integrate pattern
save patterns pat3.v -verilog -replace
save patterns pat3.wgl -wgl -replace
write block description block3.ctdf -replace
exit
```

Notice that in the Add New/Existing Controller commands, the leading “/” of an instance path is optional: the tool adds it internally when you omit it. Also, during the invocation of a this first-level dofile, the top-level design appears to be the memory currently specified as –top, for example, block1.

Example - BISTGEN Dofile Number 1

The BIST generation activity in the preceding section is performed by the following simple script:

```
//bbgen.do
add memory model ram
set bist insertion -on
run
save bist -verilog -replace
exit
```

In this example you are generating the RAMs BIST circuitry for block1 and will reuse it for block2 and block3, so only one bistgen dofile is needed.

Example - Block-Based Insertion Flow Invocation Script 2

After executing the first invocation script shown, the tool has created BIST circuitry and has inserted it into the three design blocks. Now you are ready to stitch them together into the total chip or top hierarchy.

The following example script invokes MBISTArchitect in hierarchical insertion mode, which is the second part of the block-based insertion flow.

```
$HOME_NAME/bin/mbistarchitect \
block1_black_box.v block2_black_box.v \
block3_black_box.v system.v \
-insertion -hierarchical \
-top top -dofile bbins_hier.do \
```

After this script invocation, system_mbist.v is the top level of your BIST-inserted netlist, and the BIST-inserted block designs are in block1_mbist.v, block2_mbist.v, and block3_mbist.v.

Example - Insertion Dofile Number 2

The following example script is the `-dofile` argument of the invocation in the previous section:

```
load controller description block1.ctdf
load controller description block2.ctdf
load controller description block3.ctdf
set system mode bist
insert bist logic
save design -include none -replace
set system mode integration
add pattern translation -all
integrate patterns
save patterns mapped_hier.v -verilog -replace
exit
```

Important Considerations in Selecting a Tool Flow

The important considerations in selecting a tool flow are:

- The quickest and easiest way to get your job done (top-down flow).
- The need to batch jobs (bottom-up flow).
- The design team is in different locations, or the size of your design (block-based flow).

Chapter 2

Input and Output Files

Input Files

MBISTArchitect works from a library model, as well as a design netlist to do the insertion. The library model, along with a small set of application commands, is all the tool requires to generate the appropriate BIST circuitry. The following sections describe each MBISTArchitect inputs.

Chip Design Files

When using the insertion phase, supply the design files for the “chip” or “top” in which you are inserting BIST circuitry. This is RTL. The memory designs can be annotated with Verilog specparams as described in “[Defining the Memory Model and I/O Models with Specparams](#)” on page 104.

MBISTArchitect Library Model

The MBISTArchitect library format is used to describe your memory models for the BIST generation activity. The syntax specific to the MBISTArchitect tool is described in “[Memory Modeling](#)” on page 49; however, the library format can also be used to model objects used in other Mentor Graphics tools.

If you have an existing memory model in the MBISTArchitect library format, you can enhance it for use with the MBISTArchitect tool by adding a “bist_definition” section. The tool reads the bist_definition, the model header, and input and output declarations. The tool ignores other library constructs.

MBISTArchitect Dofile

A file that contains a list of valid MBISTArchitect commands. You can execute a dofile using either the -Dofile invocation switch or the [Dofile](#) command.

User-Defined Algorithms File

This optional file contains the definitions for any user-defined algorithms (UDAs). You load a UDA file using the [Load Algorithms](#) command during BIST generation. The defined algorithms can then be added to the BIST design. For more information, see “[User-Defined Algorithms](#)” on page 143.

ROM Content File

This optional input, specified as the argument to “Add Memory Models -Filename”, specifies the hex values stored in each row of a read-only memory. It is used to calculate a compressor signature for the ROM.

If you use the ROM1 test algorithm by issuing the Add Mbist Algorithm command, you will need to supply a ROM content file in the Mentor Graphics Modelfile format.

A Mentor Graphics Modelfile contains addresses and data. You present the addresses in hexadecimal format. You can specify a range of addresses such as 0-1f. An address range can contain an asterisk (*) wildcard character. For example, to specify that you want all addresses set to hexadecimal F, use “*/f;”. You cannot use an X in an address.

You can present the data in either binary or hexadecimal format; the default format is hexadecimal. To specify data in binary format, add a ‘%’ to the beginning of the data values. If you use an X within hexadecimal data, all four bits that the X represents are all X’s; therefore, to set a single bit to X, you must use binary format. See the following examples.

The following two examples are equivalent. The first example shows both an address and its associated data in hexadecimal. The second example shows the same address and data, but the data is now shown in binary.

```
ABCD / 123X;  
ABCD / %000100100011XXXX;
```

The following is an example of what an initialization file might look like (range 0-1f).

```
0 / a;  
1-f / 5;  
10 / 1a;  
11-1f / a;
```

You can use an asterisk (*) for an address range. For example, you could rewrite the previous initialization file as in the following example. The first line assigns the data value “a” to the full address range, 0-1f. The subsequent lines overwrite the “a” value with the new data values for the specified addresses.

```
* / a;  
1-f / 5;  
10 / 1a;
```

Pin order is positive-dependent. Any order is acceptable as long as the pins match up in position-dependent fashion.

Output Files

The generation phase always produces three kinds of HDL output files: the BIST circuitry file, a connection file, and a testbench.

The connection file and the testbench are used to verify the functionality of the BIST circuitry. These files are described in greater detail in the following sections.

Optionally, the tool can produce various other outputs: a pattern file, which contains either the input values from the BIST controller to the memory or the output values from the memory model, a CTDF file, which describes the controller I/O and test modes, and a synthesis driver script, which you can use as a template for synthesizing the BIST models from RTL with a synthesis tool of your choice.

MBISTArchitect Output File Naming

MBISTArchitect generates files that use a specific set of naming conventions. The tool saves the generated output files with the default file name `model_name_suffix.extension`, where the model's HDL format determines the extension and the type of output determines the suffix.

If you added multiple memory models during the setup phase of running the tool, the generated default file names include the term `multi` in addition to `model_name_suffix`. For example, the default Verilog file name for a BIST session that includes multiple memories is `model_name_multi_bist.v`.

The following table lists all possible MBISTArchitect outputs files and their default prefixes and suffixes.

Table 2-1. Output File Names

Verilog Files	VHDL Files
<code>model_name_bist.v</code>	<code>model_name_bist.vhd</code>
<code>model_name_bist_con.v</code>	<code>model_name_bist_con.vhd</code>
<code>model_name_tb.v</code>	<code>model_name_tb.vhd</code>
<code>model_name_v.dcscrip</code>	<code>model_name_vhd.dcscrip</code>

Note



While MBISTArchitect can create output files formatted in VHDL, it cannot perform insertion with models formatted in VHDL. If you want to use MBISTArchitect for insertion, you must use Verilog.

Regarding ‘include Directives

See also the [Add Verilog Include](#) command located in the *MBISTArchitect Reference Manual*.

Note



If your chip-level Verilog input file has an ‘include directive (statement) **between** modules (outside a module), then during BIST insertion the tool will preserve the ‘include as-is when writing the output netlist. If the ‘include is placed **inside** a module, the tool will replace it with the contents of the included file (the HDL specified within it).

HDL BIST Circuitry File

The BIST circuitry output file always includes the generated RTL for the BIST controller(s) specified in the dofile, and if you are in the insertion phase (or if you use the command Set Bist Insertion -on), it additionally includes the generated RTL of the memory collars for all relevant memory models that need collars. The BIST controller contains a finite state machine to control the operation of memory test algorithms you have selected. It contains the address generator, write data generator, expect data generator, and control signal generator. The controller usually contains a comparator to determine whether test results are good. The controller may also contain logic for optional features like diagnostics, BISA, and pipelining. The BIST collar instantiates the memory from your original design in addition to test muxes and optional logic such as scan bypass logic and scrambling. The collar contains a compressor when you are not using comparators in the controller.

In BIST generation, use the Save Bist command to write the BIST circuitry. In the insertion phase, use the Save Design command. By default, the controller for a memory model is named `<model_name>_bist` and the BIST circuitry file is named `<model_name>_bist.v` (or `.vhd`). If the controller is testing more than one kind of memory model, the controller is named `<model_name>_multi_bist` and the circuitry file is named `<model_name>_multi_bist.v` (or `.vhd`), based on the name of the first model encountered.

The collar name is based on the controller name:

```
<controllername>_<modelname>
```

For example, if you have added some memory models of type MYRAM1 and MYRAM2, by default the tool creates the following BIST collars:

```
MYRAM1_multi_bist_MYRAM1_block  
MYRAM1_multi_bist_MYRAM2_block
```

If you use the command Set Design Name Controller ram_bist, these two collars would be named as follows:

```
ram_bist_MYRAM1_block  
ram_bist_MYRAM2_block
```


The Set Design Name command offers flexibility to rename the collars and other modules which are part of the BIST circuitry.

VHDL Only - BIST Controller Configuration Format

The format for the configuration statements that the tool places in the BIST controller file is determined by whether the declaration is for the controller or for one of the other components. The following paragraphs describe each format. Notice that in the format descriptions all bold text is literal (including the underscores) and that the italic text varies with the design declarations. The italic text is defined at the end of the format descriptions.

The BIST controller configuration format is as follows:

```
configuration entity_name_behavior_cfg of entity_name is  
  for behavior  
    .  
    .  
    .  
  end for;  
end entity_name_behavior_cfg
```

All other components' configuration format:

```
configuration entity_name_ext_behavior_cfg of entity_name_ext is  
  for behavior  
    .  
    .  
    .  
  end for;  
end entity_name_ext_behavior_cfg
```

- *entity*

The name of the first memory being tested.

- *name*

- **bist** — for single memory controller.
- **multi_bist** — for multiple memory controller.

- *ext*

- **mux_MUXnumber** — for muxes.
- **register_widths** — for a scalar pipeline register.
- **register_widthv** — for a vector pipeline register.
- **address_dsc_memnumber** — for address descramblers.
- **data_dsc_memnumber** — for data descramblers.

- *MUXnumber*
The number of the muxes being declared.
- *width*
The width, in bits, of the pipeline register.
- *memnumber*
The integer number of the memory descrambler (the value is 0 for a single memory controller).

HDL BIST Connection File and Test Bench

The connection file, named `<model_name>_bist_con.v` (or `.vhd`), and the test bench, named `<model_name>_tb.v` (or `.vhd`), are used to validate the BIST circuitry. The connection file instantiates the BIST circuitry and connects all the ports together. The test bench instantiates the connection model and provides stimulus to start the BIST circuitry's algorithmic test activities and report test status when done.

In the testbench, BIST is proceeding while `test_h=1`. The signal `tst_done=1` indicates that BIST ran to completion. The `fail_h` signal is initialized to 0 but goes high at the first occurrence of a memory error. Normally the `fail_h` signal is “sticky,” staying high until the completion of BIST.

If you use the optional Diagnostics feature, `fail_h` is “momentary”; when `fail_h` goes high, the diagnostic failure information is scanned out, and then `fail_h` goes low until either the next error or the end of test. In the case of multiple failflags, `fail_h` goes high at the end of test to show that at least one error occurred during BIST. In all cases, if `tst_done=1` and `fail_h=0` then the test completed successfully with no detected failures, but if `tst_done=1` and `fail_h=1` then BIST detected errors in the memory.

The Save Bist command writes the connection file and the testbench at the same time as it writes the BIST circuitry file.

Pattern Files and Controller Test Description Files

The tool optionally produces output pattern files as inputs to testing equipment, using one of several standard or proprietary pattern formats: WGL (Waveform Generation Language), STIL (Standard Test Interface Language), FJTDL (Fujitsu Test Description Language), as well as Verilog and VHDL. In WGL, for example, a pattern file defines a vector of input/output ports, their rise/fall timeplate, and a sequence of values to drive or expect to receive for a given number of cycles.

The tool can also produce output CTDF (Controller Test Description Files) which refer to WGL and describe the overall test scheduling. CTDF files use the CTDL language to specify the port interfaces of the BIST circuitry, define edge timing rules (timeplates), and define test

procedures for the tester to drive and receive port data on the chip interface. For more information on how the tool uses CTDL, see “[Controller Test Description Language](#)” on page 223.

During BIST generation, the tool writes WGL and CTDF pertaining to your BIST controller when you use the **Set Bist Insertion -On** command and switch. In insertion phase you can additionally write a top-level integrated pattern file in WGL, STIL, FJTDL, Verilog, or VHDL using the command Save Patterns.

The WGL file written during BIST generation is named based on the controller name, for example: *<controllername>.wgl*. The CTDF file name is based on the BIST circuitry filename, for example: *<circuitryfilename>.ctdf*. For example, if your controller is named *ram_bist*, then the WGL file is *ram_bist.wgl*, and when controller is in Verilog the CTDF file will be named *ram_bist.v.ctdf*.

Depending on what kinds of algorithms and BIST features you are using, you may have one or more CTDF procedure modes and pattern blocks. [Table 2-2](#) shows the WGL patterns and CTDF procedure modes generated by the BIST generation tool for different configurations of the BIST controller.

Table 2-2. WGL Files Generated

Controller Configuration	No. of WGL Pattern Files Generated	Patterns Generated	Modes Defined in CTDF
Simple controller	1	Reset + BIST operation	run_bist
1 RetentionCB algorithm	3	Reset + BIST operation	run_bist_0
		Retention cycles	run_bist_1 and run_bist_2
1 RetentionCB algorithm + BISA	4	Reset + BIST operation	run_bist_0
		Retention cycles	run_bist_1 and run_bist_2
		BISA patterns	run_bisa
1 RetentionCB algorithm + BISA + Algorithm selection	5	Reset + BIST operation	run_bist_0
		Retention cycles	run_bist_1 and run_bist_2
		BISA patterns	run_bisa
		Algorithm selection patterns	run_algsel

Table 2-2. WGL Files Generated (cont.)

Controller Configuration	No. of WGL Pattern Files Generated	Patterns Generated	Modes Defined in CTDF
1 RetentionCB algorithm + 1 MISR	4	Reset + BIST operation	run_bist_0
		Retention cycles	run_bist_1 and run_bist_2
		MISR	run_misr

Synthesis Driver File for BIST Circuitry

MBISTArchitect can optionally write a basic synthesis script, targeted for Synopsys Design Compiler. You can use this script as a template for synthesizing and optimizing the MBIST models generated by the tool.

To write this output file in the generation phase, use the **Save Bist -Script** command and switch to write the script simultaneously with writing the BIST circuitry.

Insertion Utility File and Synthesis Driver File

If you use the Add Pin Sharing or Add Pin Mapping command during the insertion phase, the tool writes a small file called `mgc_utility.v` that contains the behavioral description basic logic gates which are inserted.

If you use **Save Driver Files -Logic_synthesis** command and switch, then the tool writes an additional output file which is a synthesis driver script to synthesize the utility file into gates.

BSDArchitect Dofile

If you are using BSDArchitect to access the BIST controller by using the JTAG TAP, you can choose to have the MBISTArchitect tool write a dofile for the later BSDArchitect invocation. To write this optional dofile, use the **Save Driver Files -Bsda** command and switch in the insertion phase.

Output Files for Diagnosis

The MBISTArchitect tool produces the following two output files that are required as input files by the Diagnosis tool for diagnostics. For information on using the Diagnosis tool to diagnose memory failures, see [“Diagnosing Memory Failures.”](#)

- **Diagnostics Configuration File** — The diagnostics configuration file is produced during BIST generation only if the Set Controller Debug command is set to “-On” and

the “-Write_diag_config” switch is used with the Save Bist command. The default name of this file is <file_name>.diagcrgb.

- **Controller Mapping File** — The controller mapping file contains the mapping between the diagnostic configuration file and the associated chip-level BIST controller instance and memory collar instances. MBISTArchitect generates this file with the default name of cntl.map during BIST insertion.

Creating the Diagnostic Configuration File

The diagnostic configuration file contains the BIST controller state and memory configuration information that is required for diagnosis. The diagnostic configuration is a model that can be used to restore all the necessary controller state information and memory configuration for the purpose of diagnosis. You must run BIST generation with the MBISTArchitect tool to create this configuration file.

Prerequisites

- A valid design.
- Your generation dofile must contain the “Set Controller Debug -On” command and switch.
- Configured your controller (i.e. entered all commands and dofiles).

Procedure

1. Issue the “Run” command to generate the BIST controller.
2. Issue the “Save Bist -Write_diag_config” command and switch to create the output files.

Creating the Controller Mapping File

The controller mapping file is a mapping between a BIST controller instance (at the chip level) and the corresponding controller configuration file. For each controller instance, the associated memory collar instances are stored in the order as seen by the controller configuration. You must know the path names of the BIST controllers, the associated memory collar instances, and the relationships to one another and the diagnostic configuration file.

Prerequisites

- A valid design.
- Your insertion vs. generation dofile must contain the “[Add Existing Controller -Diagnostic_configuration](#)” command and switch or the “[Add New Controller](#)” command.

- Previously executed all other commands for BIST insertion (Run and Save are the last steps).

Procedure

1. Issue the “Run” command to generate the BIST controller.
2. Issue the “Save Bist -Write_diag_config” command and switch to create the output files.

Example

The following example uses the MBISTArchitect commands and switches to create the controller mapping file. In this example, the mapping of three BIST controllers is created. The first two controllers are almost the same, but instantiated twice to test different memories on a chip. The controller “/U1/cntl1” tests three memories /U1/mem0, /U1/mem1, and /U1/mem2. The order as seen by the controller is 1, 2, and 0, respectively.

In BIST generation, issue the following in the dofile of BIST controller synthesis, run BISTGEN and generate the controller model.

```
set controller debug -on
run
..
save bist -verilog -rep
Perform controller synthesis as usual.
$Tessent_Tree_Path/bin/mbistarchitect -dofile bistgen.do \
-log bistgen.log -rep
```

Note



If BIST insertion is not performed at the top level, then the instance names for each controller and its associated memories have to be edited to reflect its exact location at design hierarchy.

If the command “Add New Controller” is used, verify that the command and switch “Set Controller Debug -On” is issued in your BIST generation dofile as shown in the following example.

```
add existing controller /U1/cntl1 ram8x8_bist -diagnostic_config
cntl1.diagcfg ...
add existing controller /U1/cntl2 ram8x8_bist -diagnostic_config
cntl1.diagcfg ...
add existing controller /U1/U12/cntl3 ram4x4_bist -diagnostic_co
cntl2.diagcfg ...
...
insert bist logic
save controllers mapping cntl.map -rep
```

The following is an example of the contents of a controller mapping file.

```
// for BIST controller /U1/cntl1
```

```
add controller mapping cntl1.diagcfgb /U1/cntl1 /U1/mem2 /U1/mem0 /U1/mem1
// for BIST controller /U1/cntl2
add controller mapping cntl1.diagcfgb /U1/cntl2 /U1/mem3 /U1/mem4 /U1/mem5
// for BIST controller /U1/U12/cntl3
add controller mapping cntl2.diagcfgb /U1/U12/cntl3 /U1/U12/mem7
/U1/U12/mem8
```


Chapter 3

Memory Modeling

Memory model creation is a very important step in the creation of memory BIST (Built-In Self-Test). The memory model provides necessary information to the tool that is used to create the BIST controller. Memory models are usually provided by the memory vendor; however, if the memory vendor does not provide these models, you can create them using the syntax outlined in this chapter.

Memory Modeling Syntax

The following list specifies basic information you must know before writing model descriptions for the MBISTArchitect tool.

1. Legal characters within model descriptions include letters, numbers, and the underscore character, “_”. If you use any other characters in the model description, you must enclose the text string in double quotes. However, if you do use quoted names, you should be aware that this can potentially cause problems downstream in the VHDL or Verilog models produced.
2. All keywords are case sensitive and must appear in the case shown. User-supplied model names are case insensitive.
3. If your library has multiple memory models with the same name, and they have the same vendor and technology, the tool issues a warning and overwrites the existing model with the subsequent, identically named model. If the identically named models have different vendors or technologies, the tool keeps only the first model description and issues a warning message upon encountering each subsequent, identically-named model.
4. When you individually specify the pins of a multi-bit signal, the tool assumes MSB to LSB bit ordering from left to right. The tool uses this pin ordering when it connects the BIST controller to the RAM model. Thus, mismatches between the specified library pin ordering and the HDL model pin ordering can result in an improperly connected BIST model.

The usage examples in this chapter use the following syntax conventions:

- **Bold** — Indicates a keyword. Enter the entire keyword exactly as shown.
- *Italic* — Indicates a user-supplied argument. Replace the italicized string with the appropriate value.
- **Double Slashes //** — Indicates a comment. The tool ignores any text between the double slashes and the end of line character.

Defining the Memory Model

The MBISTArchitect memory model has the following required sections:

- [Header](#) — Defines the memory name and interface pins.
- [BIST Definition](#) — Defines the characteristics of the memory.

There may be additional memory model requirements if you are using other tools, such as Tessent® FastScan™ or Tessent DFTAdvisor (hereafter referred to as DFTAdvisor). You can place your MBISTArchitect-specific models in a separate library file.

Header

A model defines the name of a single cell in the technology library. The model statement defines the library cell. The model statement uses the following syntax:

```
model model_name (list_of_pins) (  
    ...  
)
```

- *Model_name*

The *model_name* equates to the cell name you use in your design data.

- *List_of_pins*

The *list_of_pins* describes the interface pins on the cell boundary. These include input, output, and bidirectional pins. Because the tool connects BIST circuitry to the memory models based on port names, the *list_of_pins* must exactly match, both in name and case, the port names specified in the associated Verilog or VHDL model.

For example, the following model header statement describes an 8x3, 2-port RAM named RAM2 and its interface pins:

```
model RAM2 (R1, W1, A1[2], A1[1], A1[0], D1[2], D1[1], D1[0],  
            R2, W2, A2[2], A2[1], A2[0], D2[2], D2[1], D2[0]) (  
    ...  
)
```

BIST Definition

The *bist_definition* section contains the information requires to set up BIST for the memory. It has the following sections:

- [Pin Declarations](#) — Defines the signal and pin types and properties.
- [Parameters](#) — Defines the physical and vendor-specific properties of the memory, such as size and addressing.

- [Address and Data Descrambling](#) — Defined the descrambling logic for memories with address or data scrambling.
- [Port and Cycle Definitions](#) — Defines the read and write behavior of the memory.

The following example shows the basic structure of memory model:

```

model model_name (list_of_pins) (

    bist_definition (

        // Pin declarations
        address          name (list_of_pins);
        data_in          name (list_of_pins);
        data_out         name (list_of_pins);
        data_inout       name (list_of_pins);
        write_enable     pin  assert_state;
        read_enable      pin  assert_state;
        output_enable    pin  assert_state;
        chip_enable      pin  assert_state;
        clock           pin  assert_state;
        control          pin  assert_state;

        // Parameters
        tech = technology_name;
        vendor = vendor_name;
        version = "number";
        message = "message_text";
        address_size = number;
        min_address = lowest_address;
        max_address = highest_address;
        data_size = data_bus_bits;
        addr_inc = number;

        //Descrambling
        descrambling_definition (
            address (...)
            data_in (...)
        ) /end of descrambling definition

        // Port and cycle definitions
        read_write_port (
            write_cycle (...)
            read_cycle (...)
        ) //end read_write_port definition

    ) //end of bist definition
) //end of model description

```

Pin Declarations

The pin declaration section is at the beginning of the `bist_definition`. The pin declaration section includes statements for:

- [Address and Data Pin Types](#)
- [Control Pin Types](#)
- [Dont_touch Pin Types](#)
- [Write Enable Mapping](#)

The pin declaration statements can be in any order. However, the `write_enable_map` statements must be after any referenced signal statements.

Address and Data Pin Types

Address and data pin types define the multi-bit address and data lines. Each signal name uses one statement, terminated with a semicolon. The address and data pin type statements use the following syntax:

```
pin_type name (list_of_pins);
```

- *pin_type*

The following address and data pin types are available:

- **address** — Defines an address bus.
- **data_in** — Defines an input data bus.
- **data_out** — Defines an output data bus.
- **data_inout** — Defines a bidirectional data bus.

- *name*

Required argument that specifies the signal you are defining. If you are using bit notation for the `list_of_pins`, the name can be any string. If you are using array notation for the `list_of_pins`, the name must match prefix of the pins listed in the model header.

- *list_of_pins*

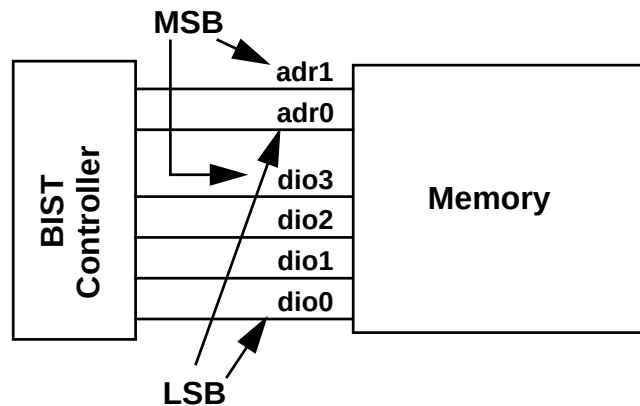
For buses, you can use either bit notation or array notation for the *list_of_pins*. You can have a mix of notation across different pin types. However, you cannot mix bit and array notation for a single pin type. For example, you can use bit notation for the address type and array notation for data type, but you cannot use both bit and array notation for the address type.

The following example shows the address and data pin declarations using bit notation:

```
address adr (a1,a0);  
data_inout dio (dio3,dio2,dio1,dio0);
```

When you specify the individual bits of a multi-bit signal, MBISTArchitect assumes that the left-most pin is the most significant bit (MSB), and uses this information when it connects the BIST controller to the RAM. [Figure 3-1](#) shows the address and data connections.

Figure 3-1. Pin Connections Example



The following example specifies the same signals using array notation:

```
address adr (array = 1:0;);
data_inout dio (array = 3:0;);
```

Control Pin Types

For control signals, each statement uses the following syntax:

```
pin_type name assert_state data_control_state;
```

- *pin_type*

The following control pin types are available:

- **write_enable** — Defines the write enable control signal.
- **read_enable** — Defines the read enable control signal.
- **output_enable** — Defines the output enable control signal.
- **chip_enable** — Defines the chip enable control signal.
- **clock** — Defines the clock signal.
- **control** — Defines a generic control signal.
- **bist_mode** — Defines the mode select signal for BIST-ready memories. For more information, see [“BIST-Ready Memory Support”](#) on page 76.
- **atpg_mode** — Defines the mode select signal for bypass-ready memories. For more information, see [“Bypass-Ready Memories”](#) on page 83.
- **scan_clk** — Defines the clock signal for existing scan cells.
- **scan_bypass_enable** — Defines the bypass scan enable.
- **scan_in** — Defines the bypass scan input.
- **scan_out** — Defines the bypass scan output.

- *name*
Required argument that specifies the signal you are defining. The name must match one of the pins listed in the model header.
- *assert_state*
Optional argument that defines the signal's active state. During the read and write cycles, the control signal always remains at the value opposite this state except when explicitly asserted. Assert has no effect for memory clock signals.
 - **high** — The signal is active high. For clock signals, this indicates the rising edge. If you do not specify an *assert_state*, the signal is assumed active high.
 - **low** — The signal is active low. For clock signals, this indicates the falling edge.
- *data_control_state*
Optional argument that defines the tri-state control state for control signals that operate a bidirectional data bus. If you define a model with a bidirectional data bus (**data_inout** keyword), you must specify a tri-state output control state for at least one of the defined control signals.
 - **tri_h** — Signal must be high to enable tri-state data buffers.
 - **tri_l** — Signal must be low to enable tri-state data buffers.

For example, the following model definition defines a 4-bit bidirectional data bus, and an active low write enable signal whose value must be high to enable the tri-state output buffers.

```
data_inout dio(dio3,dio2,dio1,dio0);  
address addr(a1,a0);  
write_enable wen low tri_h;
```

Note

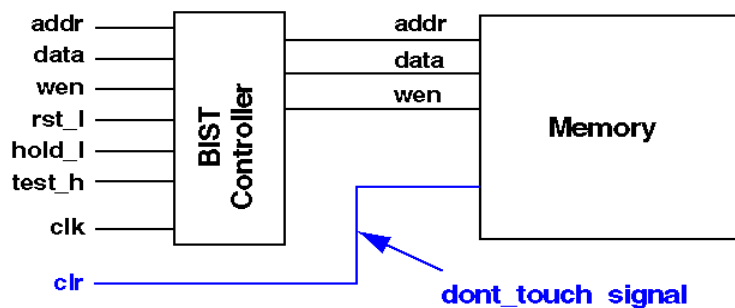


If you add a write-masked algorithm to a BIST controller which tests memories with **and** without the `write_enable_map` construct, the BIST controller will apply the write-masked algorithm to all memories. This can cause the failflag to go high during RTL simulation.

Dont_touch Pin Types

The `dont_touch` keyword allows you to specify signals (ports) that do not need to be controlled by the BIST controller. The tool connects these ports to the top level.

Figure 3-2. The don't_touch Signal



The syntax for specifying `dont_touch` ports is as follows:

```
dont_touch name assert_state direction;
```

- *name*
Required argument that specifies the signal you are defining. The name must match one of the pins listed in the model header.
- *assert_state*
Optional argument that defines the active state of the signal. You can specify high (default) or low. The `dont_touch` ports always remain at the value opposite their activated state.
- *direction*
Optional argument that defines the direction of the port. You can specify input (default) or output.

The following example declares two `dont_touch` ports, an active low input port named “`clr`” and an active high output port named “`refcntso`”:

```
dont_touch clr low;
dont_touch refcntso output;
```

The `dont_touch` attribute can be arrayed in the same way as address and data. The following example shows the array notation for the `dont_touch` attribute.

```
dont_touch in_vec (array = 3:0;) input;
dont_touch out_vec (array = 3:0;) output;
```

Write Enable Mapping

You can relate write enable signals to bits on the data bus. This can be specified using the **write_enable_map** keyword in the MBISTArchitect library definition of the memory. The specified `write_enable` and data input signals must be defined prior to the mapping statements. Include one statement for each bit of the `write_enable` signal.

Note



If you are including write enable mapping statements for a multi-ported memory, you must define write enable mapping for every port. You cannot define write enable mapping for a single port of a multi-ported memory.

For write enable mapping, each statement uses the following syntax:

```
write_enable_map write_enable_signal_bit data_signal_bits...;
```

- *write_enable_signal_bit*
Specifies a defined write_enable signal. Include the bit number in parenthesis.
- *data_signal_bits...*
Specifies one or more input data bus signals. Include the bit number or range in parenthesis.

The following example maps the control of the first and second input data bits to the first bit of the write_enable signal:

```
write_enable_map WEN(0) DBUS(0) DBUS(1);
```

If there is exactly a bit-by-bit correspondence then the following syntax can be used.

```
write_enable_map WEN DBUS;
```

For example WEN(0) => DBUS(0), WEN(1) => DBUS(1).... WEN(12) => DBUS(12).....

Or the mapping can be defined as follows, and illustrated in [Table 3-1](#).

```
write_enable_map WEN(0) DBUS(1:0) DBUS(15:14);
write_enable_map WEN(1) DBUS(3:2) DBUS(13:12);
write_enable_map WEN(2) DBUS(5:4) DBUS(11:10);
write_enable_map WEN(3) DBUS(7:6) DBUS(9:8);
```

In this case the wen mapping table will be as follows.

Table 3-1. Write Enable Mapping of the Given Example

WEN	DBUS			
0	0	1	14	15
1	2	3	12	13
2	4	5	10	11
3	6	7	8	9

This means that WEN(0) controls the “writing” of bits 0,1,14,15 of vector DBUS.

So if you specify a write enable mask of 1001 and you are writing FFFF to the memory then the data written in the memory will be 1100 0011 1100 0011= C3C3

So the expect value from the memory will be C3C3 and not FFFF.

Example - Pin Declaration with One Read and One Write Port

The following example shows a RAM model with one write port and one read port using bit notation:

```
model RAM1(W1, A1[2], A1[1], A1[0], D1[2], D1[1], D1[0],
           R2, A2[2], A2[1], A2[0], D2[2], D2[1], D2[0]) (
    bist_definition (
        write_enable W1 low;
        address addr1 (A1[2], A1[1], A1[0]);
        data_in d_in (D1[2], D1[1], D1[0]);
        read_enable R2 low;
        address addr2 (A2[2], A2[1], A2[0]);
        data_out dout (D2[2], D2[1], D2[0]);
        ...
    )
)
```

The following example shows the same RAM model using array notation

```
model RAM1(W1, A1, D1, R2, A2, D2) (
    bist_definition (
        write_enable W1 low;
        address A1 (array = 2:0;);
        data_in D1 (array = 2:0;);
        read_enable R2 low;
        address A2 (array = 2:0;);
        data_out D2 (array = 2:0;);
        ...
    )
)
```

Parameters

The parameter statements are placed after the pin declarations. Each parameter statement is terminated by a semicolon. The following list describes each parameter statement:

- **tech** = *technology_name*;
Specifies the technology name. If you specify a technology, the tool places this information in the synthesis script it generates.
- **vendor** = *vendor_name*;
Specifies the vendor name. If present in the description, the tool can use this information if it encounters multiple models with the same name in the libraries it reads.
- **version** = "*number*";
Specifies the model version. You must use quotes around the version string.

- **message** = *“message text”*;
Specifies any additional model information. You must place quotes around the message string.
- **address_size** = *integer*;
The width of the address bus as measured in bits. The memory has $2^{\text{address_size}}$ individually addressable elements, called words. If you do not specify the address size, the tool will derive the address_size from the “address” pin declaration.

The address_size cannot be larger than *sizeof(int)* on the host platform on which you execute MBISTArchitect. On most platforms this maximum size is either 32 or 64 bits.
- **min_address** = *lowest address*;
The smallest address value. The min_address must be 0, if specified.
- **max_address** = *highest address*;
Specify a maximum address if the highest valid memory address is less than $2^{\text{address_size}} - 1$.
- **data_size** = *data_bus_bits*;
The total number of bits in the data bus. A word is data_size bits wide. If you do not specify the data size, MBISTArchitect derives the data size from the first “data_*” pin declaration.
- **addr_inc** = *number*;
The address increment value for the Col_March1 algorithm, which must be greater than 2 and cannot exceed the memory’s size. It is normally a power of 2.

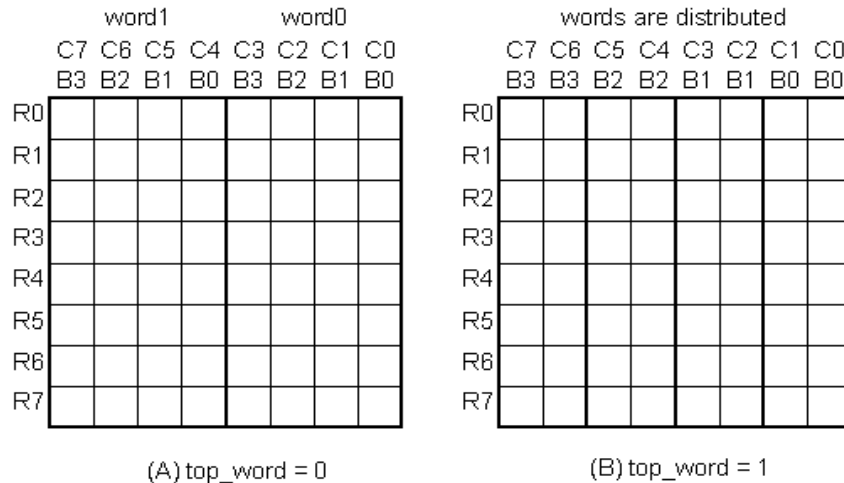
Addr_inc is also used inside BISA to specify the physical number of words per row in the memory. The number of rows is equal to $2^{\text{address_size}} / \text{addr_inc}$, and the number of columns in each row is equal to $\text{data_size} * \text{addr_inc}$. If addr_inc is omitted from the library file, for the purpose of BISA its value is 1 by default.

This parameter is only required for the Col_March1 algorithm.
- **top_column** = *number*;
Specifies the physical number of words per row in the memory, and must be greater than zero. The checkerboard algorithm uses the value of top_column along with top_word. This parameter is only required for algorithms using the checkerboard data pattern.

See also the description for top_word which follows, and the discussion of top_column and top_word in “[checkerBoard \(topChecker\)](#)” on page 122.
- **top_word** = 0 | 1;
Specifies the organization of bits on a row.
 - **0** — A row physically consists of a linear sequence of words.
 - **1** — A row physically consists of all the 0th bits of the words on that row, followed by all the 1st bits of the words on that row, and so on up to the (N-1)th bits.

To illustrate, consider an example memory model with `address_size` 4, `data_size` 4, `addr_inc` 2. It has 16 addressable words, each 4 bits wide. The memory model has 8 rows and 8 columns. [Figure 3-3](#) shows what this memory looks like with both values of `top_word`. R is row, C is column, and B is bit.

Figure 3-3. The Meaning of `top_word`



The value of `top_word` affects the checkerboard algorithm. With `top_word = 0`, checkerboard alternates between writing 0101 to all the words on a row and writing 1010 to all the words on a row. With `top_word = 1`, checkerboard alternates between writing 0000 to word 0, then 1111 to word 1, then 0000 again for the remainder of the row; and then on the next row writes 1111, then 0000, then 1111, ...; and on the third row goes back to 0000, 1111, 0000, ..., etc. With the two different `top_word` settings, the checkerboard writes different logical values to obtain the same physical checkerboard pattern.

This parameter is only required for algorithms using a checkerboard data pattern.

The use of Address Scrambling and/or Data Scrambling will not affect your definitions of `top_column` and `top_word`. See [“Address and Data Descrambling”](#) on page 59.

Address and Data Descrambling

Frequently in memory designs, physically adjacent cells do not correspond to consecutive external addresses. That is, the memory translates the supplied external address (logical address) to some internal address (topological address) that it uses to access a specific memory cell. This translation is also known as address scrambling.

Memory data is also communicated by a sequence of bits in an external data word (logical data) that might differ from the sequence of bits in the data words that physically exist in the memory (topological data). The translation between these bit sequences is known as data scrambling.

To test interactions between physically adjacent memory cells, you must provide the detailed description of the address translation, also known as the address descrambling information. This is done with the descrambling_definition section.

Note



Not all memories require address or data scrambling. Contact your memory vendor to identify which memories require address or data scrambling in the memory model.

Figure 3-4 illustrates the relationship of the address descrambler to the overall BIST structure.

Figure 3-4. Address Descrambling

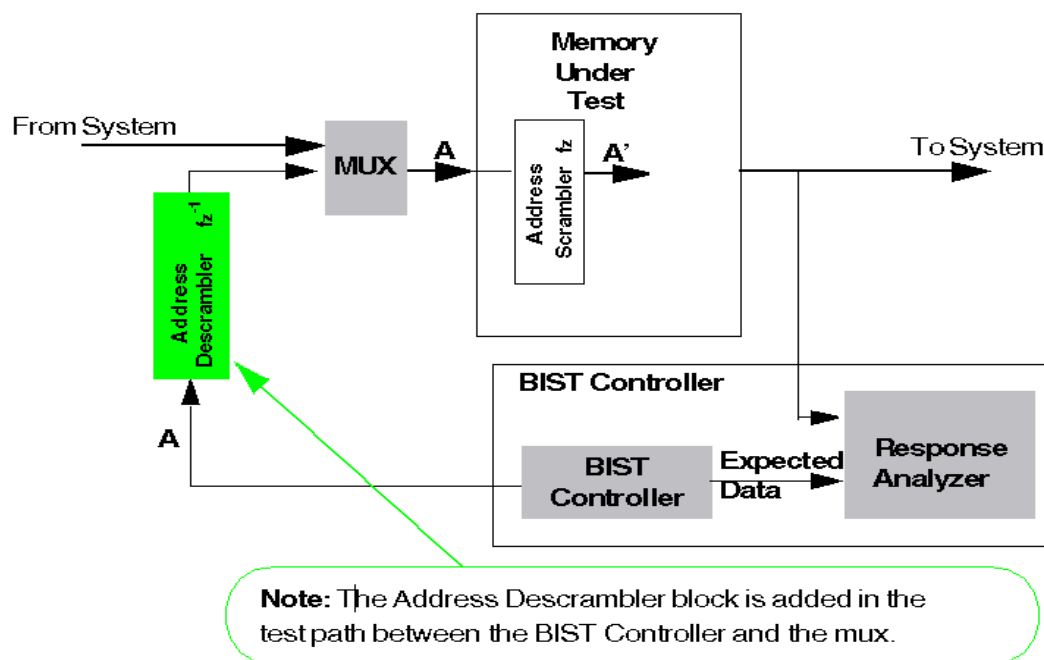
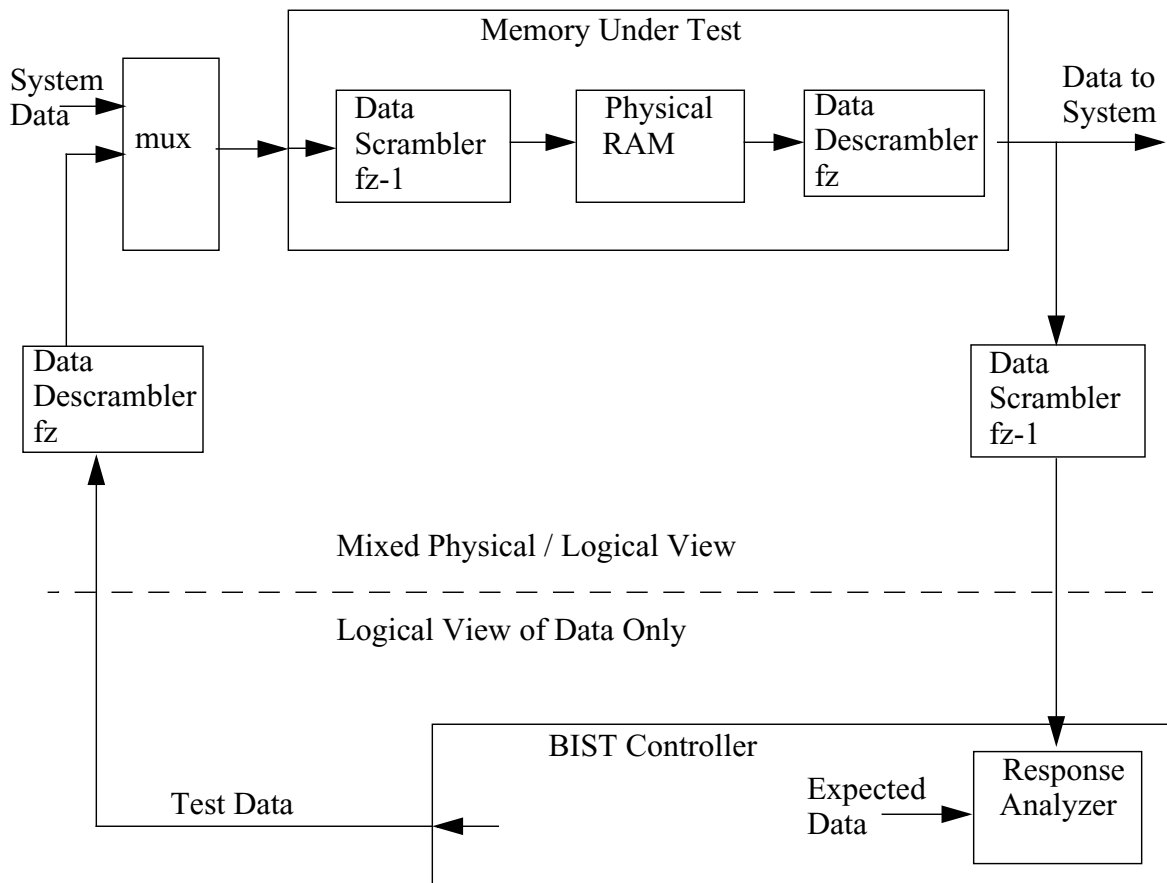


Figure 3-5 is an example of the relationship of the data descramblers to the overall BIST structure.

Figure 3-5. Data Descrambling

It is important to take into account these address and data scrambling effects when testing memories using the Checkerboard algorithm which is dependent upon the following:

- Topological address/data information.
- Consistency between logical data values and electrical data values.

To perform the memory cell interaction tests, you must provide both the address and data descrambler descriptions. To do so, use the `descrambling_definition` in the memory model description.

Descrambling Definition Syntax

The `descrambling_definition` subsection of the memory model description is nested within the `bist_definition` section. The BIST controller assumes that all the descrambling information is located here and takes the information into consideration, if it exists, when accessing the memory device.

The tool uses the same address and data descrambling scheme for all ports on a multi-port memory.

The descrambling definition is nested within the `bist_definition`, following the parameter statements, but before the port and cycle definitions. The descrambling definition has two subsections, `address` and `data_in`, as shown in the following example.

```
model model_name (list_of_pins) (  
    bist_definition (  
        ...  
        descrambling_definition (  
            address (  
                descrambled_LSB_name = boolean_statement;  
                ...  
                descrambled_MSB_name = boolean_statement;  
            ) // end address bus section  
            data_in (  
                descrambled_LSB_name = boolean_statement;  
                ...  
                descrambled_MSB_name = boolean_statement;  
            ) // end data input bus section  
        ) // end descrambling definition  
        ...  
    ) // end BIST definition  
    ) // end model description
```

The `address` subsection defines the descrambling for the address bus, and the `data_in` subsection defines the descrambling for the data input bus. For each address/data line of the memory, there must be a line in the corresponding subsection. For example, if the width of the address bus is four, there must be four lines in the `address` subsection of the descrambling definition section of the memory model. Similarly, if the width of the data bus is eight, there must be eight lines in the `data_in` subsection of the descrambling definition section of the memory model.

Note



The names of the descrambled address/data lines are arbitrary, but the order of the statements in each section is important. The descrambler statements must be listed from LSB to MSB.

The supported Boolean operators for address descrambling are BUF, INV, AND, NAND, OR, NOR, XOR, and XNOR.

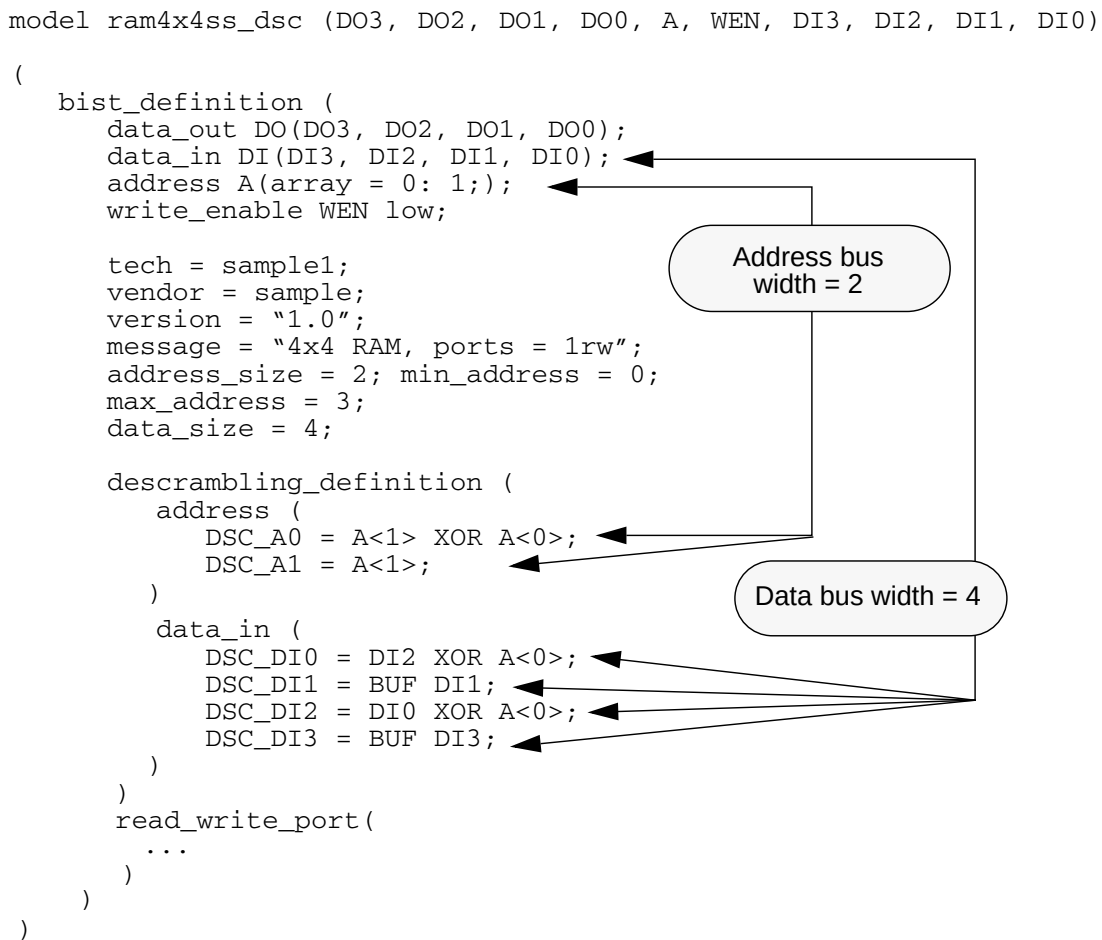
The supported Boolean operators for data descrambling are BUF, INV, and XOR.

You must define both the `address` and `data_in` subsections, regardless of whether or not scrambling information exists for both. In other words, the descrambling definition must contain statements for both `address` and `data_in` subsections regardless of the descrambling information.

Only one address and one data descrambler can be defined. This is also true in the case of multi-port memories. The tool creates multiple instances of the same descrambler and applies them to each port. The tool does not allow different descrambler functions for different ports.

Figure 3-6 illustrates the address and data descrambling definition sections within the bist definition:

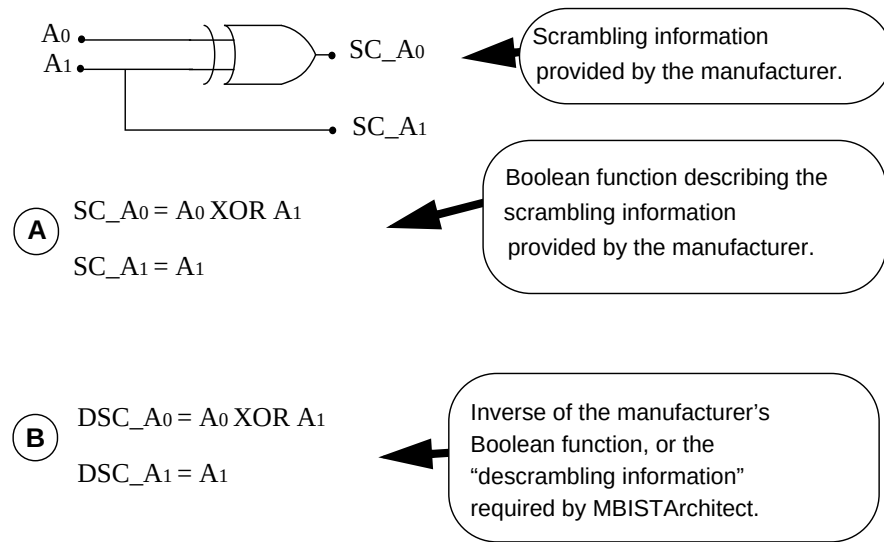
Figure 3-6. Address and Data Descrambling



Deriving the Address Descrambling Information

Address scrambling information is often presented differently from one memory manufacturer to another. In the following example, Figure 3-7 illustrates how to derive the descrambling information for the previous example based upon the scrambling information presented by the manufacturer.

Figure 3-7. Deriving the Address Descrambling Information



The inverse functions A and B in [Figure 3-7](#) happen to be identical, but this may not be the case for your descrambling function.

The address descrambling syntax allows you to specify the addressing order for writing and reading during the memory BIST test.

Example - Descrambling for Address Order of 0, 1, 3, 2...

The following example shows that if the required address order is 0x000, 0x001, 0x003, 0x002, 0x004, and so on, the following example address scrambling definition would be required.

```
Descrambling_definition(
  Address (
    Dsc_a0 = a<1> xor a<0>;
    Dsc_a1 = a<1>;
    Dsc_a2 = a<2>;
    Dsc_a3 = a<3>;
  )
  Data_in(
    Dsc_d0 = d<0>;
    Dsc_d1 = d<1>;
  )
)
```

Port and Cycle Definitions

Each port requires its own port definition section. Port definitions consist of a port type, that is either **write_port**, **read_port**, or **read_write_port**, followed by parentheses.

Note

To model correct MBISTArchitect memory library definitions, you must clearly define the `read_write_port` definitions after all previous declarations have been completed.

The **read_port** has a **read_cycle**.

The **write_port** has a **write_cycle**.

The **read_write_port** has a **read_cycle** and a **write_cycle** (that can be in either order).

If you have N `read_write_port` blocks, by definition the memory has N ports, numbered 0...N-1 in diagnostics hardware, but numbered as 1...N in the dofile when using the command `Add Mbist Algorithms`.

When you use `read_write_port`, you cannot also define a `read_port` or `write_port` in the same memory.

When using `read_port` and `write_port`, you can have at most 1 of each. If you use both by definition the memory is a register file with 2 ports (one R, one W), but both are numbered 0 in diagnostics hardware and numbered 1 in the dofile when using the command `Add Mbist Algorithms`.

The following is an example of a port definition.

```
<port_type> (  
...  
) // end port definition
```

Cycle definitions nest within the port definition parentheses. Cycle definitions consist of a cycle type (either `read_cycle` or `write_cycle`) followed by a series of event statements enclosed within parentheses.

The `_cycle` definitions can apply `change/assert/expect` keywords only to pins defined earlier in the `bist_definition`.

The following is an example of a cycle definition.

```
<port_type> (  
  <cycle_type> (  
    <event statements> ...  
  ) // end cycle definition  
) // end port definition
```

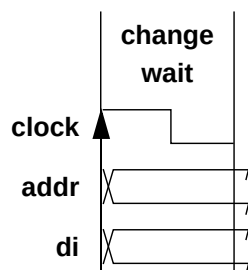
Multiple event statements describe the event sequence that defines the port's operation. The event statements utilize the following keywords:

- **change**

Indicates that the specified input signals (address or data) accept a new value or new scheduled value. For example, the following series of statements generate the timing sequence shown in [Figure 3-8](#).

```
change addr;  
change di;  
wait;
```

Figure 3-8. Change Event Example



- **assert**

Indicates that the specified control signal goes to its active state. (The active state, whether high or low, is declared in the bist_definition.) Asserted (activated) signals return to their inactive states in the next test clock cycle (“wait” statement) unless another assert statement keeps them active. For example, the following two series of statements generate the respective timing sequences shown in [Example 3-9](#) (note that ‘oe’ is active high):

Example 1

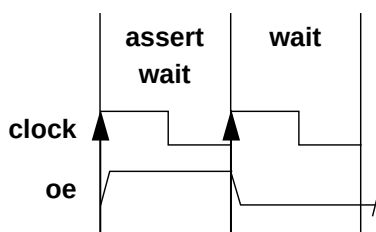
```
assert oe;  
wait;  
wait;
```

Example 2

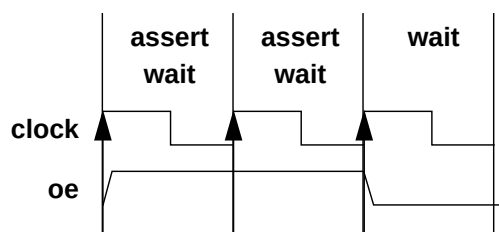
```
assert oe;  
wait;  
assert oe;  
wait;  
wait;
```

Figure 3-9. Assert Event Examples

Example 1



Example 2



An assert statement can optionally include a fix modifier. Normally, this indicates that the associated signal must be activated during this cycle because the address is being changed. Use the fix modifier to indicate that all operations before the next wait can be skipped if the signals changed within that cycle do not actually change. For example, a read-write-read March algorithm has three operations done at the same address before moving to the next address. The assert operation associated with the fix might only need to be done for the first read. They might allow reducing the number of cycles used for a read-write-read process by two or more.

Assert has no effect for memory clock signals.

- **expect**

Indicates that you can anticipate a change on the specified (data) output signals due to the occurrence of a previous event. In effect, this schedules the controller to strobe the data outputs of the memory.

An expect statement can include an optional move modifier that specifies when an event executes. The move modifier means the tool can move this event to a later clock cycle when optimizing the BIST structure. The move option applies to data outputs. The tool uses the move option only when it is trying to optimize circuitry while combining read and write cycles together to form read/write/read cycles or other large cycles.

Minimum Cycle Requirements

The cycle definitions you create must abide by the following minimum requirements:

- The minimum read cycle definition must contain three events: an address change, a wait, and data expected on an output or bidirectional data signal.
- The minimum write cycle definition must contain two events: an address change and an input or bidirectional data signal change.
- Normally there will be a control signal activated during only the read cycle or during the write cycle to distinguish between the read and write operations.
- Either all write or all read cycles must activate at least one control signal: (read_enable, write_enable, chip_enable, or output_enable).

Toggling chip_enable

A port defined as chip_enable may not be toggled during read and write cycles. You must define either the read_enable, or the write_enable to make the chip_enable toggle during the read or write cycle.

Note



ROMs have only read cycles, which have no requirements for activating control signals.

Example: Timing Sequence

The following two series of statements generate the respective timing sequences shown in [Figure 3-10](#) (note that 'oe' is active high).

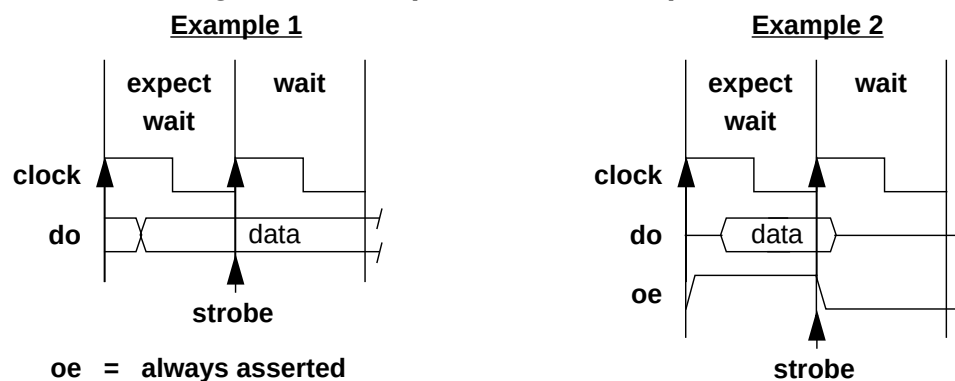
Example 1

```
expect do move;  
wait;
```

Example 2

```
assert oe;  
wait;  
expect do move;  
wait;
```

Figure 3-10. Expect Event Examples



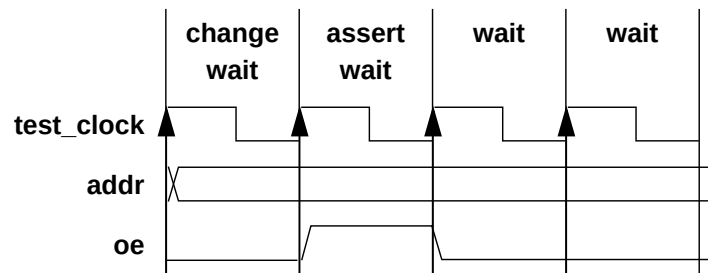
In Example 1 of [Figure 3-10](#), data out (do) is expected some time (specified per the memory specification) after a clocked memory has received the clock edge. This is because the output enable (oe) is tied activated. You might want to consider a wait state, as in this example, to allow the output data to become stable. Note that the BIST logic strobbs the data at the following rising clock edge (after the expect move statement).

In Example 2 of [Figure 3-10](#), we have added the output enable (oe). The data out (do) is now expected some time (specified per the memory specification) after 'oe' has been activated.

- **wait**

Inserts a one test clock cycle wait period. That is, all events prior to a wait, execute in a single test clock cycle. For example, the following series of statements generates the timing sequences shown in [Figure 3-11](#).

```
change addr;  
wait;  
assert oe;  
wait;  
wait;  
wait;
```

Figure 3-11. Wait Event Example

Example - Port and Cycle Definition

The following example illustrates the port and cycle definition sections within a complete model description:

```
model ram4x4 (DO3, DO2, DO1, DO0, A1, A0, WEN, DI3, DI2, DI1, DI0)
(
  bist_definition (
    data_out d_o(DO3, DO2, DO1, DO0);
    data_in di(DI3, DI2, DI1, DI0);
    address addr(A1, A0);
    write_enable WEN low;

    tech = sample1;
    vendor = sample;
    version = "1.0";
    message = "4x4 RAM, ports = 1rw";
    address_size = 2;
    min_address = 0;
    max_address = 3;
    data_size = 4;

    read_write_port(
      read_cycle(
        change addr;
        wait;
        expect d_o move;
      )
      write_cycle(
        change addr;
        change di;
        wait;
        assert WEN;
        wait;
      ) // end write cycle
    ) // end port definition
  ) // end bist definition
) // end model description
```

Example - Port and Cycle Definition Using Array Notation

The following example shows the same model description as the previous example but using array notation:

```
// 4X4 RAM model
model ram4x4(do,a,wen,di) (
    bist_definition (
        data_out do (array=3:0;);
        data_in di(array=3:0;);
        address a (array=1:0;);
        write_enable wen low;

        tech = sample1;
        vendor = sample;
        version = "1.0";
        message = "4x4 RAM, ports = 1rw";
        address_size = 2;
        min_address = 0;
        max_address = 3;
        data_size = 4;

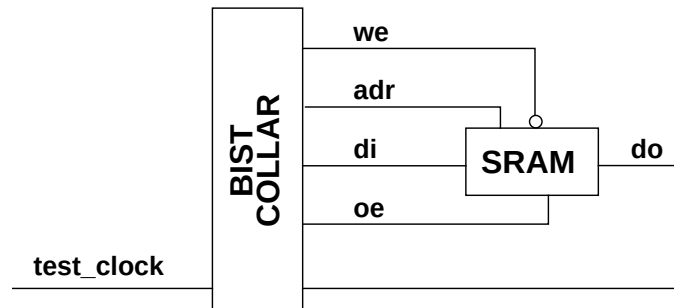
        read_write_port(
            read_cycle(
                change addr;
                wait;
                expect do move;
            ) // end read cycle

            write_cycle(
                change a;
                change di;
                wait;
                assert wen;
                wait;
            ) // end write cycle
        ) // end port definition
    ) // end bist definition
) // end model description
```

Example - Read/Write Cycle Optimization

If you choose an algorithm such as the March C+ (March2), a read/write/read operation is required. Under these conditions, the tool can optimize the events that are specified in both the read and write port definitions. That is, the tool combines common event statements such as the change (for address and data) and sequences the read and write port event statements (such as activates and waits) for the read/write/read operation. This allows for the operations to be done in fewer cycles and with less logic.

To emphasize this, we will use the SRAM shown in [Figure 3-12](#) with an active low write enable (we) and an active high output enable (oe).

Figure 3-12. SRAM for Read/Write Cycle Optimization Example

Example - bist_definition with read_cycle and write_cycle

The following is an example `bist_definition` with the `read_cycle` and `write_cycle` definitions for the SRAM shown in [Figure 3-12](#):

```
bist_definition (
    write_enable we low;
    output_enable oe high;
    data_in di (di1, di0);
    address adr (adr1, adr0);
    data_out do (do1, do0);
    address_size = 2;
    data_size = 2;

    read_write_port(
        read_cycle(
            change adr;
            wait;
            assert oe;
            wait;
            expect do move;
            wait;
        ) // end read cycle
        write_cycle(
            change adr;
            change di;
            assert we;
            wait;
            wait;
        ) // end write cycle
    ) // end port definition
) // end bist definition
```

The respective timing sequences for such read and write cycles could individually be seen as those shown in [Figure 3-13](#). However, the tool takes the two definitions and combines them to produce the optimized timing sequence shown in [Figure 3-14](#).

Note how the tool accounts for the extra read operation for the read/write/read operation. That is, the events from the read port definition are reused.

Figure 3-13. SRAM Read and Write Respective Timing Example

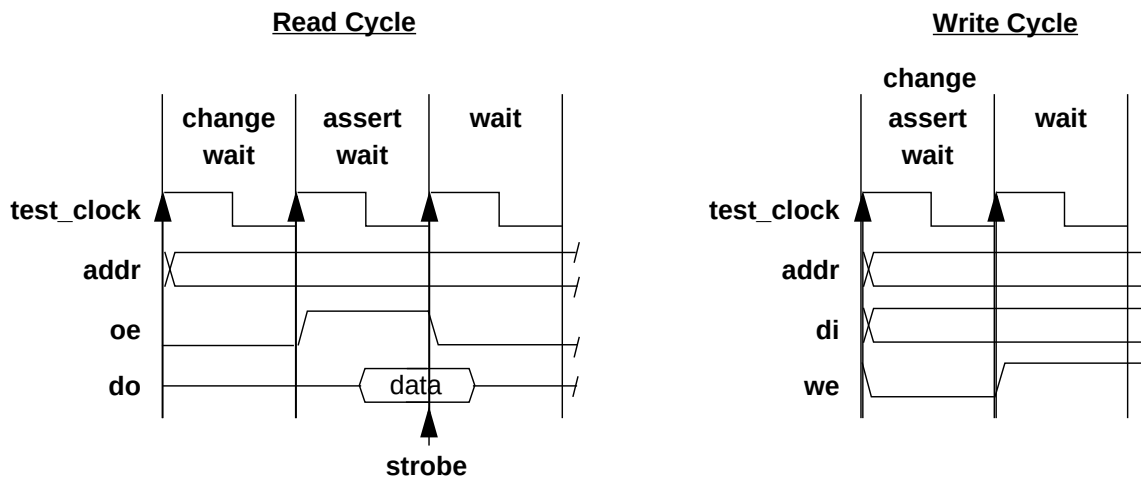
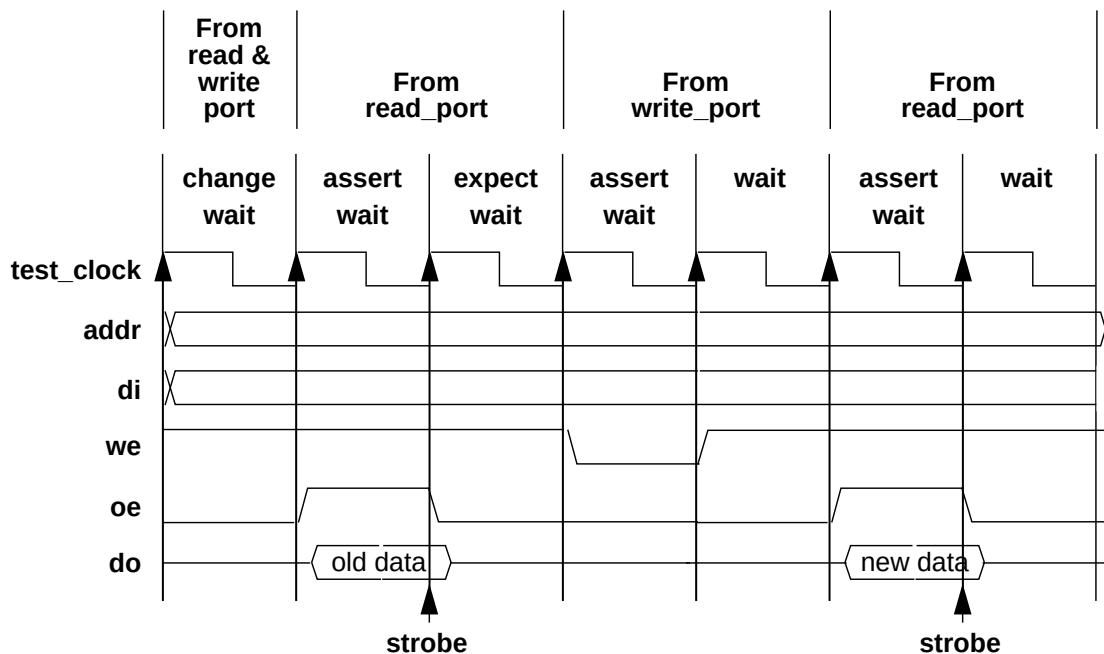


Figure 3-14. SRAM Read/Write/Read Optimized Timing Example



With careful attention, the read/write/read operation can be further optimized by modifying the read and write port definitions to eliminate wait statements (clock cycles).

Example - Shortened Timing Sequences

The following read and write cycle definitions generate the shortened timing sequences shown in Figure 3-15:

```
read_write_port (
  read_cycle(
    change adr;
    wait;
```



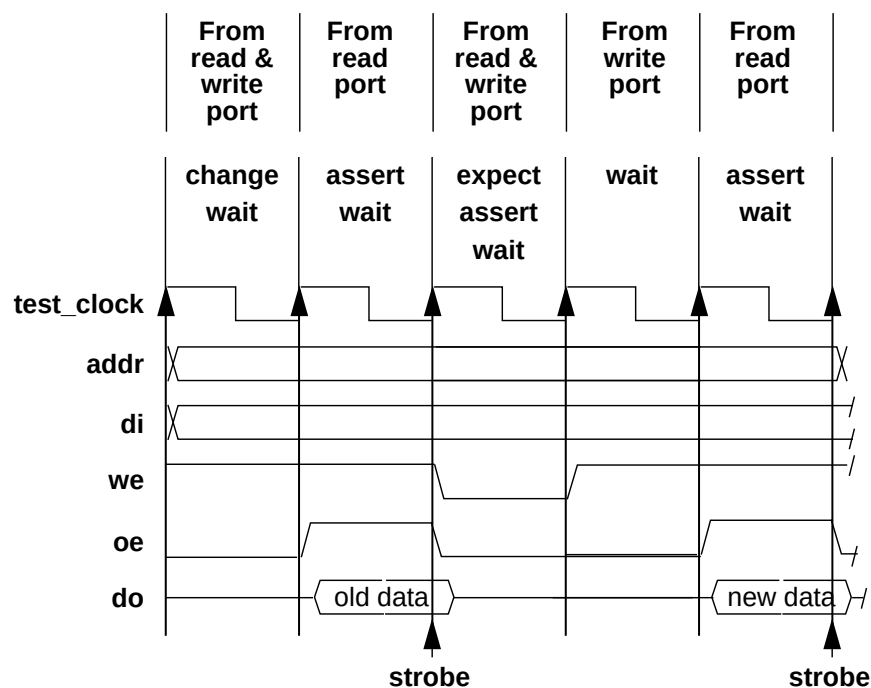
```

    assert oe;
    wait;
    expect do move;
  ) // end read cycle

write_cycle(
  change adr;
  change di;
  assert we;
  wait;
) // end write cycle
) // end port definition

```

Figure 3-15. SRAM Shortened Timing Sequence Example



When comparing the timing in [Figure 3-14](#) and [Figure 3-15](#), notice that we have eliminated two clock cycles from the operation. The final wait statement in the read port and the final wait statement in the write port were removed. Care should be taken to assure that output data settles properly before the actual strobe occurs.

Note



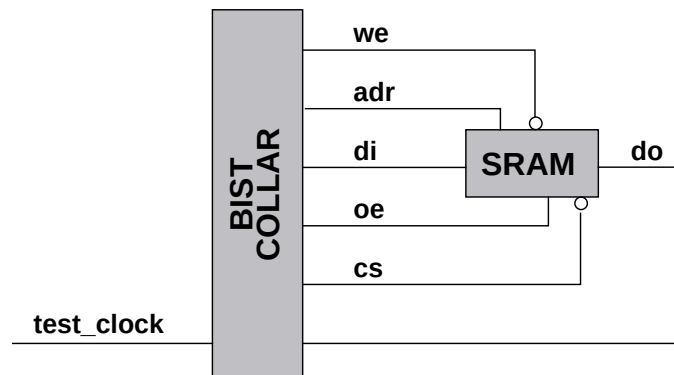
Cycle optimization as specified above might cause a problem in diagnostic mode.

Example - Assert Statement Fix Modifier for Optimization

An assert statement can optionally include a fix modifier for read/write/read cycle timing. When the read/write/read cycle is derived (as in a March C+ algorithm), it activates the specified signal with the fix modifier once, and only once, in the entire cycle.

To exemplify the fix modifier, we will use the same example from [Figure 3-12](#), but add an active low chip select as shown in [Figure 3-16](#).

Figure 3-16. SRAM for Fix Modifier Example



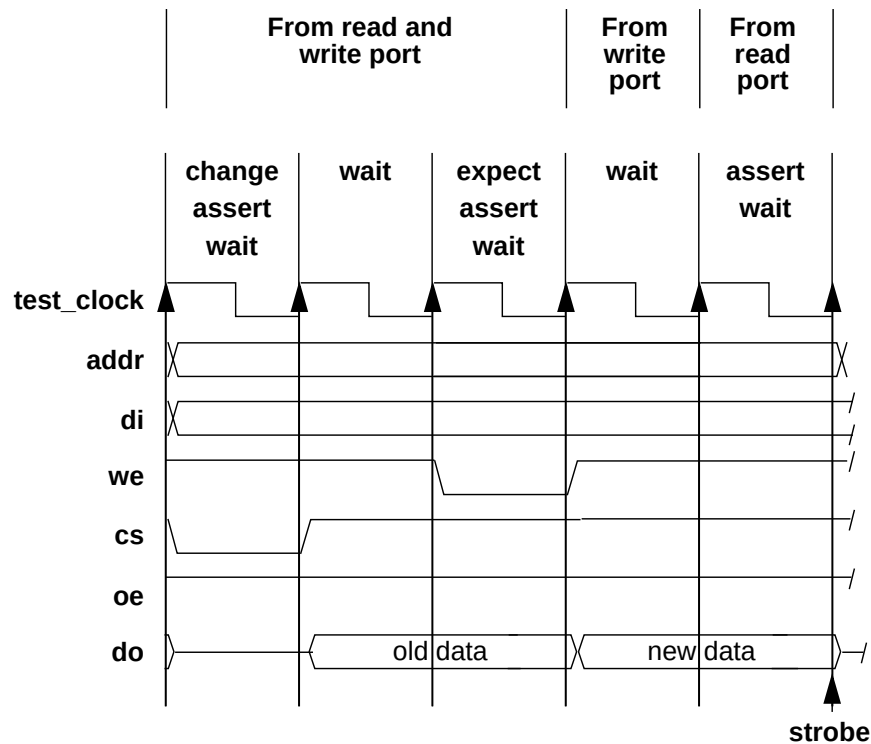
The following bist_definition illustrates how you can use the fix modifier within a read_cycle definition for the SRAM shown in [Figure 3-17](#):

```
bist_definition (  
  write_enable we low;  
  output_enable oe low;  
  chip_enable cs high;  
  data_in di (din, di0);  
  address adr (adr1, adr0);  
  data_out do (don, do0);  
  address_size = 2;  
  data_size = 2;  
  
  read_write_port  
  read_cycle(  
    change adr;  
    assert cs fix;  
    wait;  
    wait;  
    expect do move;  
  ) // end read cycle  
  write_cycle(  
    change adr;  
    change di;  
    assert we;  
    wait;  
  ) // end write cycle  
  ) // end port definition  
) // end bist definition
```

In the `bist_definition` the chip select has been defined as active high, when (in reality) the SRAM requires an active low. Similarly, the output enable has been defined as active low when in reality it is active high. The default settings for the test will have `oe` set to high, and the chip select set to low. Therefore, the RAM will always be selected with the outputs enabled except when the chip select is activated. The `read_cycle` specifies an activation of `cs` at the beginning of the operation.

The timing sequence generated by such read and write cycles is shown in the following figure.

Figure 3-17. Fix Modifier Timing Example



Fix Modifier Uses

The intent is to disable the RAM for only one clock cycle with the remaining test sequence to always be enabled. Thus, always remember that the fix modifier option is used for the following:

- Only when you want the tool to optimize circuitry while combining read and write cycles together to form read/write/read cycles or other large cycles.
- To indicate that the action executes only once in a specific test clock cycle (relative to the address change) within a read or write cycle.

Fix is used when you want the memory to be active, except when the address is changed. Often there is logic in the RTL model of a memory that issues error messages if the address is changed

while the output is enabled. With the fix modifier, the enable signal is changed only for the first read of a read-write-read. Note that the address does not change during a read-write-read.

BIST Testing of Output Enables

The tool can create BIST circuitry to test a RAM's output enables. However, due to the potentially destructive result when enabling the output enable signals, you should exercise caution when deciding whether the BIST circuitry should test the output enables. In many cases, generating external test patterns at the chip-level might be a safer way to test output enable signals for tri-state buses.

Some memories only drive their output signals when specific control signals are driven; the tool supports this process. Often when something is defined as an `output_enable` it implies that the memory does not drive the outputs unless requested. This is described at tri-stated outputs. Implicit in that there might be other devices: memories, CPU, DMA controllers, that might be able to drive the wires to which the memory outputs are attached. You need to make sure that other possible drivers are inactive during memory testing. In general, memory testing will drive each output bit to 1 and to 0 for extended periods. If another device is driving the signals it is likely that the chip will be damaged.

Fast Column Addressing

MBISTArchitect supports fast column addressing through the address scrambling statements inside the memory model.

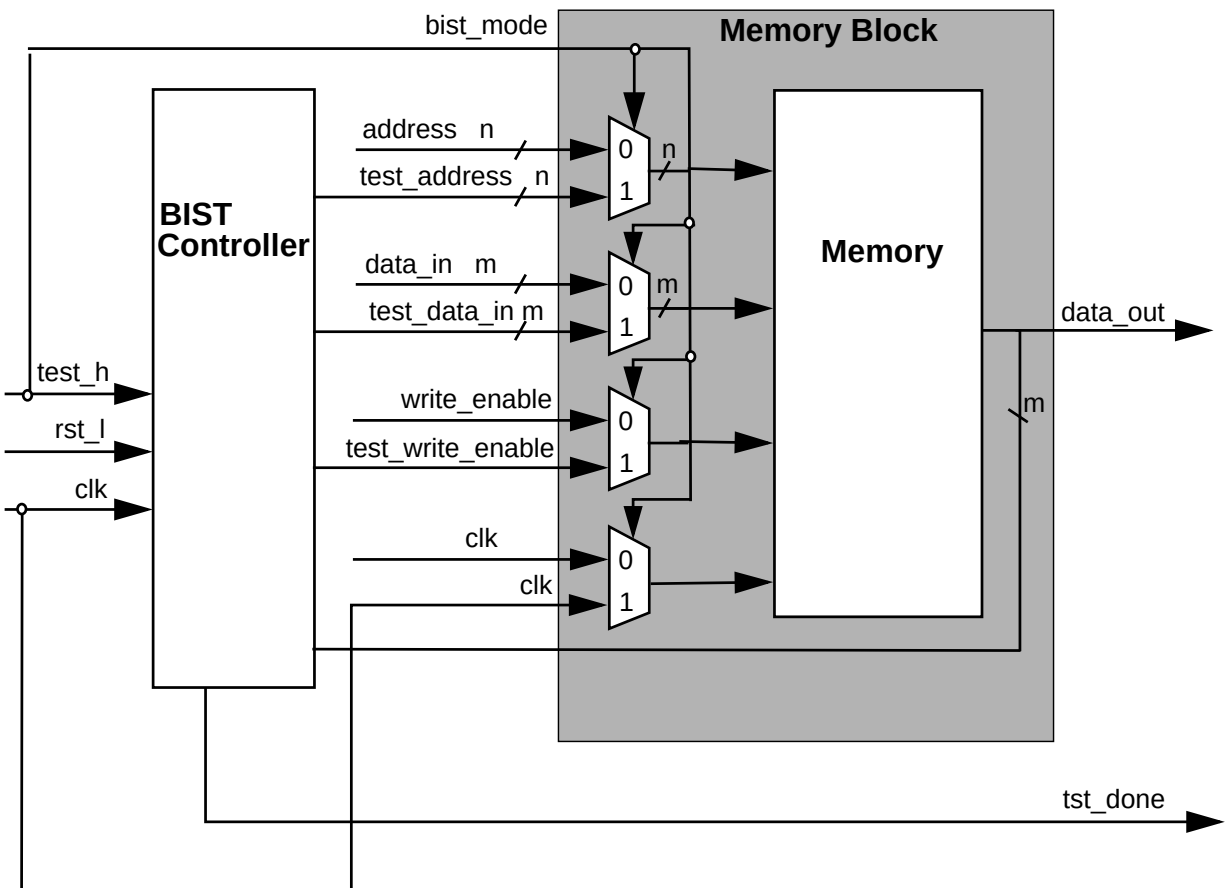
Fast Row Addressing

When fast row addressing is required, the `addr_inc` statement can be added to the BIST definition portion of the MBISTArchitect memory model. The number associated with the `addr_inc` is used to calculate what the next address is when a “jump” statement is used in a user-defined algorithm.

BIST-Ready Memory Support

Your memory design might already have embedded muxes for the purpose of enabling test mode. Memories with embedded muxes for BIST are called BIST-ready memories. [Figure 3-18](#) depicts a BIST-ready memory after insertion. Prior to insertion, the `bist_mode` signal is not connected to `test_h`. It is driven by a temporary BIST enable signal in the generation test bench.

Figure 3-18. BIST-Ready Memory



The MBISTArchitect library format has syntax to indicate which ports in your memory are BIST-ready.

Pin Declarations for a BIST-Ready Memory

You can use both paired and unpaired pin declarations in your BIST definition, according to which memory ports are BIST-ready. The tool will add muxes into the memory collar only for the unpaired declarations which are not covered by a `dont_touch` attribute.

The BIST test bench drives control signals to their deactivated state when they are not actively used. It is assumed that your system is normally responsible for driving control signals in system mode regardless of whether your memory is BIST-ready or not.

Unpaired Signals

In the BIST definition, normally the address/data pins are declared in the following “unpaired signal” format:

```
port_type name ( list-or-array )
```

Control signals are declared in the following format:

```
port_type name asserted_state
```

Paired Signals

For BIST-ready address/data pins, specify the chip mux input and test mux input using the following “paired signal” format:

```
port_type name:test_name ( list-or-array )
```

For BIST-ready control signals, use this “paired signal” format:

```
port_type name:test_name asserted_state
```

For multi-port memories, repeat your pin declarations as necessary. For example a dualport BIST-ready memory with a 9-bit address could have these declarations:

```
address ADRA:TADRA ( array = 8:0; );  
address ADRB:TADRB ( array = 8:0; );
```

BIST Mode Signals

Because a mux needs a select input to choose between its system input and test input, BIST-ready memories have this additional mandatory control signal for mux selection:

```
bist_mode name active_state
```

A multi-port BIST-ready memory might have separate bist_mode control signals per port. Make sure that you declare them all.

A single-port BIST-ready memory which activates the test inputs when the mux select signal is high could have this declaration:

```
bist_mode TestMode high
```

If you have an active low bist_mode signal, an inverter is needed on the test_h signal from the controller. MBISTArchitect will automatically add this inverter in the memory collar when the collar is generated. To generate the collar for a BIST-ready memory, use the [Set Bist Insertion -On](#) command. You can also generate the collar when inserting bypass logic with the [Set Scan Logic](#) command. If you do not generate the collar, the inverter will be automatically added in the connection file.

Clock Signal

The following cases describe the different connections for the clock:

1. The memory may be asynchronous. In which case, there is no clock defined in the library model and there will be no clock passed to the memory RTL module.
2. For a variety of reasons, the clock is sent directly to the memory. This is the normal case and is the default for the tool. It can be explicitly specified with the Setup Memory Clock -System command and switch.

For cases 1 and 2, there is no mux, or anything else, placed in the path of the clock signal. It is assumed that for testing, the clock signal sent to the BIST controller and the signal sent to the memory are either the same or synchronized.

3. The clock may have a mux inserted by the tool. This is done using the Setup Memory Clock -Test command and switch. Care will be required to make sure that the delay introduced by the mux is compensated, as needed, with clock-tree adjustments especially for the system path.
4. The memory may have an embedded mux for the clock signal. This is rare and is not directly supported. You need to define each signal as a clock and use the default state using the Setup Memory Clock -System command and switch. Both clocks will be driven in sync by the test bench. So, simulation will work correctly. Care must be taken later to make sure each signal is connected to the correct source.

Example - Mixed BIST-Ready and Non-Paired Signals

```
model BIST_Ready_rad5a804 (  
    bist_mode,  
    addr,  
    D,  
    wen, test_wen,  
    oen, test_oen,  
    chip_mode0,  
    chip_model,  
    sleep_mode,  
    chip_is_happy,  
    clk,  
    Q  
)  
(  
    bist_definition(  
        bist_mode      bist_mode      high;  
        address         addr (array = 5:0;);  
        data_in         D (array = 7:0;);  
        write_enable    wen:test_wen low;  
        output_enable    oen:test_oen low;  
        control         chip_mode0 high;
```

```
control      chip_model low;
dont_touch   sleep_mode    high input;
dont_touch   chip_is_happy  output;

clock clk;
data_out Q (array = 7:0;);

tech = really_deep_submicron;
vendor = Startup_1002;
version = "1.0";
message = "ReallyFast BistReady"
address_size = 5;
min_address = 0;
max_address = 63;
data_size = 8;

read_write_port (
  read_cycle (
    change addr;
    assert test_oen;
    wait;
    expect Q; // Output ready one clock later.
  )
  write_cycle (
    change addr;
    change D;
    wait; // set up address and data_in
    assert test_wen;
    wait; // Strobe the write one clock later.
  )
)
```

Example - All Input Signals Are Paired

```
model BIST_Ready_rad5a804 (
  bist_mode,
  addr, test_addr,
  D, test_D,
  wen, test_wen,
  oen, test_oen,
  clk,
  Q
)
(
  bist_definition(
    bist_mode    bist_mode    high;
    address      addr:test_addr (array = 5:0;);
    data_in      D:test_D (array = 7:0;);
    write_enable wen:test_wen low;
    output_enable oen:test_oen low;
    clock        clk;
    data_out      Q (array = 7:0;);
```



```
tech = deep_submicron1;
vendor = Startup_1001;
version = "1.0";
message = "HighSpeed BistReady"
address_size = 6;
min_address = 0;
max_address = 63;
data_size = 8;

read_write_port (
  read_cycle (
    change test_addr;
    assert test_oen;
    wait;
    expect Q;// Output ready one clock later.
  )

  write_cycle (
    change test_addr;
    change test_D;
    wait;// set up address and data_in
    assert test_wen;
    wait;// Strobe the write one clock later.
  )
)
```

Note the following items regarding the previous example.

- Only the input signals are paired.
- The output signal is not paired.
- The order for a pair is:
 system signal : test mode signal
- The test mode signal is referenced in the read_cycle and write_cycle definitions.
- Typical values were used for the active state: bist_mode is often activated high. Enables are often activated low. The clock state is not defined explicitly, but takes the default of high; the low-high transition is the clock edge.
- In the test bench generated by the tool, the system mode signals will be driven as specified to accomplish writes and reads with the bist_mode signal deactivated.

Notes Regarding the Previous Examples

- The example has a variety of signals for explanatory purposes, so it does represent a real design.
- The basic change is to assume that only the write_enable and output_enable signals have critical timing. The design is such that the mux-circuitry was added for these only.

- In the read and write cycles the following is true.
 - For paired signals: the test entity named is used.
 - For unpaired signals that need to be changed or activated, the unpaired signal name is used.
- For the non-paired input signals, except for dont_touch inputs and the clock, the tool will generate the following:
 - A mux whose output will drive the unpaired signal.
 - A new signal name of the form *Test_<original_name>*. This is the signal that will be driven during test mode.
- The example defines two control signals, chip_mode0 and chip_mode1 along with their active states.

The following list describes some characteristics about how the MBISTArchitect tool handles these signals.

- There will be muxes generated for these signals.
 - Since they are not mentioned in the read_cycle or write_cycle, they will be deactivated by the BIST controller during BIST testing using the test inputs to the added muxes. During the system mode testing by the test bench, the system inputs to the muxes will be driven by deactivated values.
 - The signal chip_mode0 will be driven with a 0 and chip_mode1 will be driven high (with a 1); that is, they are deactivated.
 - These signals were defined to illustrate a way to handle these kinds of global mode signals. Depending on your design, this simplistic approach might not be sufficient. In such cases, you might need to modify the test bench. An alternative is to define the signals as dont_touch.
- The previous example includes a dont_touch input signal called sleep_mode. The important distinction here is that no mux will be added for this signal. The tool will bring it out to the collar of the block module. For simulation and verification purposes, it will be brought out of any enclosing modules and be controlled by the test bench. The signal will be driven as deactivated. In this case, it will be driven with a 0.

You need to be aware of the following issues:

- For proper testing on an actual tester, it will be necessary for these signals to be driven properly. This is the responsibility of the person doing the final netlist. The example here is only meant to suggest that this signal puts the memory into a power reduced mode.
- Typically, on the actual chip the signal would be driven from some source to many sub-circuits of the device. If the design actually has such a sleep mode, it might be necessary that the chip not be in sleep mode for the BIST test to work. The person

that is setting up the tester would be responsible for making sure the chip was not in sleep mode.

- Since there is no mux inserted, the BIST controller cannot control this signal. The `dont_touch` signals cannot be used in `read_cycles` and `write_cycles`.
- The example contains an output `dont_touch` signal called `chip_is_happy`. This is meant only for illustration. Other than bringing these to the collar of any block module, the tool does no processing of these types of signals.
- Although the clock signal is an input signal, the decision about whether it gets a mux is made based on the Setup Memory Clock command. The default switch for this command is `-System`, which will not add a mux. This choice is made for the vast majority of designs, which allows you the easiest control of getting the timing correct on this most critical signal. If this choice is made, the tester must be set up to drive both the BIST clock and the chip clock in synchronization. The most frequent alternative is to specify the following example command.

setup memory clock -test

This command causes the insertion of a mux that drives the memory's clock signal. The test input to this mux will be driven with the same clock signal as the BIST. Adding such a mux is convenient for BIST testing since only one clock is involved, but it may complicate clock-tree generation. Some attention also needs to be paid that the mux does not delay the clock edge seen by the memory enough to cause timing problems. If this becomes an issue, it may be necessary to instantiate pipeline registers on the test inputs.

Bypass-Ready Memories

The following section contains specific information regarding memories that have embedded bypass circuitry. The existing mux on the output can choose between normal memory data output and a bypass value that is computed from the bypassed memory inputs inside of the bypass block. You add the bypass block with the [Set Scan Logic](#) command.

If your design has embedded bypass circuitry, the following example extracted from a library model of a bypass-ready memory shows you the syntax for defining your bypass circuitry:

Note



This is an example only; your memory model may differ.

```
chip_enable CEN:TCEN:CENY low;  
address A:TA:AY (array=9:0;);  
data_in D:TD:DY (array=7:0;);  
control WEN:TWEN:WENY (array=7:0;);  
write_enable GWEN:TGWEN:GWENY low;  
data_out Q:TQ (array=7:0;);  
bist_mode TEN low;  
atpg_mode BEN low;
```

As shown in **boldface** in the example, the presence of a third signal name following a second colon signifies a bypass-ready input, and on the output side of memory, the presence of a second signal name following a first colon signifies a bypass-ready output. These signals are the bypassed version of the interface pins.

Note



You must define the `atpg_mode` signal for a bypass-ready memory.

In addition to the required `atpg_mode` pin type, a bypass-ready memory also can have these pin types:

```
scan_clk, scan_bypass_enable, scan_in, scan_out
```

The syntax and usage are as follows:

```
atpg_mode name asserted_state
```

Scan bypass clock:

```
scan_clk name
```

Scan bypass enable:

```
scan_bypass_enable name asserted_state
```

Scan input:

```
scan_in name;
```

Scan output:

```
scan_out name;
```

See the Set Scan Logic command for more information about some possible cells that can be generated by the tool. Note that not all of these keywords need to be used, depending on your design.

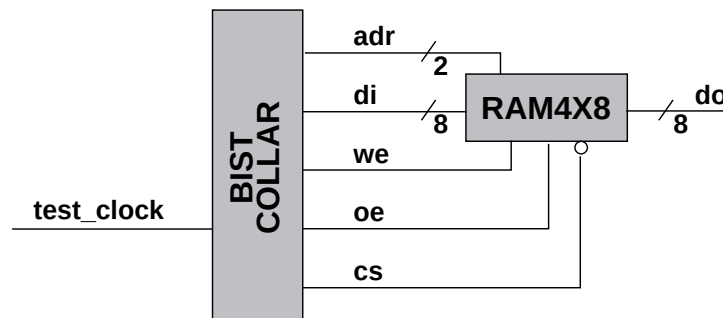
Memory Modeling Examples

The purpose of this subsection is to highlight more details of memory modeling through examples.

Example - RAM4x8

In this example, we will model the 4x8 RAM shown in [Figure 3-19](#). For the example, the RAM will have an active high write enable, an active low chip select, and an active high output enable for the tool.

Figure 3-19. RAM4X8 Example



The RAM model for the memory device shown in [Figure 3-19](#) is as follows.

Example - RAM Model for the Memory Device

```

model RAM4x8 (we, cs, oe,
              di7, di6, di5, di4, di3, di2, di1, di0,
              adr1, adr0,
              do7, do6, do5, do4, do3, do2, do1, do0)
// 1. Start of model definition
(
// 2. Start of bist definition
  bist_definition (
    "
      write_enable we high;
      output_enable oe low;
      chip_enable cs high;
      address_size = 2;
      data_in di (di7,di6,di5,di4,di3,di2,di1,di0); // 2a. data_in bits
      address adr (adr1,adr0); // 2b. address bits
      data_size = 8;
      data_out do (do7,do6,do5,do4,do3,do2,do1,do0); // 2c. data_out bits

      tech = sample_tech;
      vendor = sample-vendor;
      version = "1.0";
      message = "1-Port RAM with 4 words by 8 bits";
      min_address = 0;
      max_address = 3;
    "
  )
)
  
```

```
read_write_port
(
    read_cycle(                // 3. Start read/write port
        read_cycle(            // 3a. Start read cycle

            change adr;
            assert oe fix;
            wait;
            wait;
            assert cs fix;
            wait;
            wait;
            expect do move;
        )                      // 3a. End read cycle
    write_cycle(                // 3b. Start write cycle
        change adr;
        change di;
        wait;
        assert we;
        wait;
    )                          // 3b. End write cycle
)                              // 3. End read/write port
)                              // 2. End bist definition
)                              // 1. End model description
```

RAM4x8 Model Description

The RAM model description can be broken down into two main parts:

- Header
- BIST Definition

The tool uses the existing model header format of the Mentor Graphics DFT library; however, it requires a completely new section, the `bist_definition` section, to specify additional memory information that the BIST process requires. That is, for RAM models that exist in the DFT library, the tool can share the same header information, but must have the additional `bist_definition` section.

The other option is just having a library of RAM models for MBISTArchitect only, each consisting of a header and a `bist_definition`. Either library approach is acceptable.

Examining the Header

The header consists of the following portion of the model description:

```
model RAM4x8 (we, cs, oe,
    di7, di6, di5, di4, di3, di2, di1, di0,
    adr1, adr0,
    do7, do6, do5, do4, do3, do2, do1, do0)
(
```

The header contains the keyword `model` followed by the model name, in this case `RAM4x8`. Following the model name, parenthesis surround the list of interface pins. You can list these pins in any order in the header, although it is good practice to specify the signals using the same order found in the RAM data sheet, or the RAM RTL, that the model is describing.

Examining the `bist_definition` Section

Along with the header information, the tool obtains all the remaining information it needs from the `bist_definition` section. The `bist_definition` section consists of the following parts:

Examining the Pin Declarations

The first part of the `bist_definition` section contains the signal or pin declarations:

```
(  
  bist_definition  
    write_enable we high;  
    output_enable oe low;  
    chip_enable cs high;  
    data_in di (di7,di6,di5,di4,di3,di2,di1,di0);  
    address adr (adr1,adr0);  
    data_out do (do7,do6,do5,do4,do3,do2,do1,do0);  
)
```

The following rules apply to the pin declaration syntax:

- Each statement describes either a bit, or bits for control pin, or for the address, data input, or data output ports.
- Each statement begins with the pin type. You specify the signal type with one of the following keywords: `write_enable`, `read_enable`, `output_enable`, `chip_enable`, `clock`, `address`, `data_in`, `data_out` and `dont_touch`.
- The pin name follows the pin type keyword. You use this signal name when defining the read and write cycle events later in the `bist_definition` section.
- For single-bit control signals, the signal name you specify must match the name in the model header's pin list.
- For multi-bit address or data ports, you can assign a logical name that represents the specified bits. If you use array notation, the pin name must match that in the model header.
- The order in which you specify the individual bits of address and data ports should match the corresponding HDL model, to ensure the tool properly connects the BIST controller to the RAM model. However, the tool will normally instantiate memory models using the named parameter approach.
- For control signals, the active state follows the pin name. The active state, which is either high (default) or low, defines the signal's active state (also called the "On state")

by ATPG tools). During the read and write cycles, the control signal always remains at the value opposite this state except when explicitly activated.

- For address and data ports, a list of individual bits (separated by commas and enclosed in parentheses) follows the signal name. Instead of the individual bits, you can specify a signal name and use array notation to specify the width. For example, instead of specifying:

```
data_out do (do7,do6,do5,do4,do3,do2,do1,do0);
```

You could specify the following:

```
data_out do (array = 7:0;);
```

- Each statement ends with a semicolon (;).

Examining the Memory Information

After the pin declarations section, the model description might contain a set of optional information statements similar to the following:

```
tech = sample_tech;  
vendor = sample-vendor;  
version = "1.0";  
message = "1-Port RAM with 4 words by 8 bits";  
address_size = 2;  
data_size = 8;  
min_address = 0;  
max_address = 3;
```

The following rules apply to the information statements:

- If you specify a technology, the tool places this information in the synthesis script it generates.
- The tool can utilize the tech and vendor information if it encounters multiple models with the same name.
- The Report Memory Models command can display this optional information within the MBISTArchitect session.
- Min_address must always be 0.
- Max_address cannot exceed $2^{(\text{address_size})} - 1$.

Examining the Port Definitions

After the optional information statements, the bist_definition section contains the port definitions. This RAM has only one read/write port defined as follows:

```
read_write_port  
(
```



```
read_cycle(  
    change adr;  
    assert oe fix;  
    wait;  
    wait;  
    assert cs fix;  
    wait;  
    wait;  
    expect do move;  
) // End read cycle  
write_cycle(  
    change adr;  
    change di;  
    wait;  
    assert we;  
    wait;  
) // End write cycle  
) // End read/write port
```

The following rules apply to port definitions:

- Valid port types include `read_write_port`, `read_port`, and `write_port`.
- For each `read_write_port`, you must specify both a `read_cycle` and `write_cycle` definition.
- For each `read_port`, you must specify a `read_cycle` definition.
- For each `write_port`, you must specify a `write_cycle` definition.
- The tool numbers read/write and write ports in the order they appear within the model description. It uses this port number when you assign algorithms with the Add Mbist Algorithm command.

Each port definition contains read and/or write cycle definitions. The following rules apply to `read_cycle` and `write_cycle` definitions:

- Each statement defines an event.
- The following keywords define the events: `change`, `assert`, `wait`, and `expect`.

Read Cycle Definition

The `read_cycle` definition contains a number of events that comprise the read cycle. You define the cycle events primarily from looking at the timing diagrams of the memory component. Starting with the address change, you determine the dependencies, or required timing, between events.

To begin the read cycle, you change the values on “`adr`”. Because no timing dependencies exist between the address change and the output enable, you can activate (`assert`) `oe` at the same time. Therefore, the next event activates or rather “deactivates”, the output enable as follows:

```
read_cycle(  
    change adr;  
    assert oe fix;
```

The oe Signal Declaration

The oe signal declaration and read_cycle events take a little explanation. First, this model definition assumes you want the BIST circuitry to exercise the oe signal as part of the test process. If you wanted to leave the job of testing that signal up to some other test process you could, and typically should. In that case, you would define the oe signal to be active low, and then not exercise it at all within the port definition section.

Note



When the port definition section does not exercise control signals, they remain at the opposite value of their defined active state.

This is the key to understanding this model's description of the oe signal. This model description declares oe as active low. When you do not exercise this signal, the tool keeps this signal high. Because high is the true active state of this signal, as defined in the data sheet, this ensures enabled outputs at all times.

In this case, the model description does exercise the oe signal within the read cycle definition. Because the signal declaration defines oe as active low, the tool ensures it is always at the opposite value until an active event activates it. The event activates (asserts), or makes the signal "active" (according to the defined active state), for one clock cycle. When an active event activates a control signal, it puts it at its defined active state until the next wait statement, which equates to one tester clock cycle. Thus, the second event pulses the oe signal low, actually inactivating it briefly, at a time in which you do not care about the data on the outputs.

Based on the model's read_cycle and write_cycle definitions, the tool combines the events into one larger read/write/read cycle when implementing circuitry for March algorithm testing. In order to generate the most efficient circuitry to perform this algorithm's larger read/write/read operation, the tool combines and moves events for efficiency, while still meeting the requirements specified in the read_cycle and write_cycle definitions. The fix modifier ensures that the oe activation occurs only once during the March algorithm's expanded read/write/read cycle.

Wait Statements

The third and fourth events specify waits as follows:

```
wait;  
wait;
```

A wait forces events that follow to begin in the next tester clock cycle. You place wait statements in the cycle definitions when an action cannot occur until some time after a previous action.

The first wait causes the oe signal to go back to its defined inactive state, which is high. The second wait allows one test cycle to pass to eliminate possible timing glitches before the next action occurs, which is disabling the chip select.

Chip Enable Signal

The fifth event activates the chip enable signal, cs:

```
assert cs fix;
```

This signal has the same type of declaration as oe; that is, while truly an active low signal, the declaration defines it as active low to keep it at the low state for all cycles except that in which it activates. This event actually pulses the cs signal high for one clock cycle, at a time in which it does not affect the RAM operation.

Note



Because the chip enable signal should not be asserted in any read or write cycle, the BIST controller considers it always deasserted and drives the signal to its off state value throughout BIST. To make the signal active during BIST, define the inverse active state in the memory model.

The sixth and seventh events specify two more waits. The first wait starts a new cycle, at which point the cs signal returns to its active value. The second wait allows the chip select to meet the memory specified access time so the output data stabilizes before the read. After this wait, the seventh event specifies that the do outputs can expect valid data:

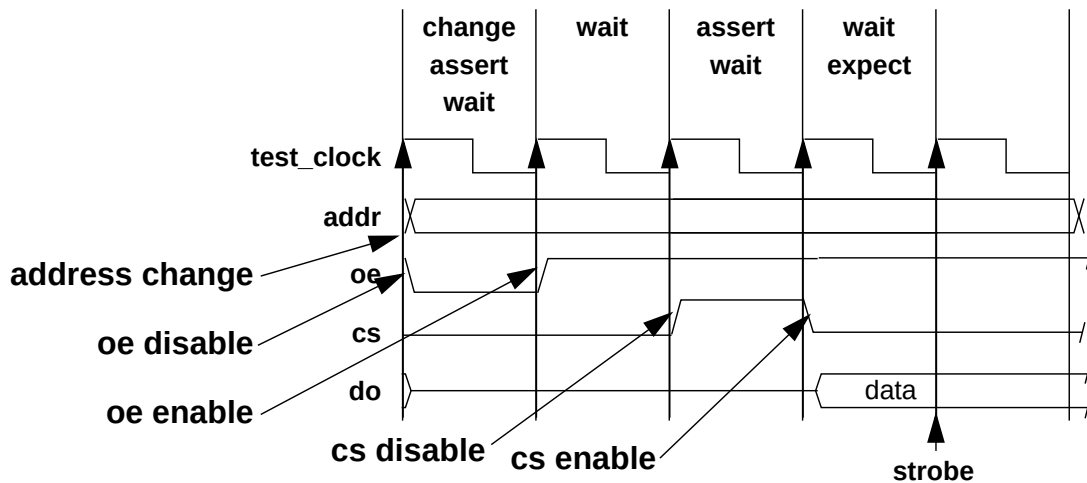
```
wait;  
wait;  
expect do move;
```

Move Modifier

The move modifier, which only applies to data output signals, specifies that you can test the data outputs (do) at this cycle--or later. The move modifier allows the tool to move this event to some later time in the algorithm's larger read-write-read cycle, if it proves more efficient.

The basic timing and events that comprise the read cycle, as defined in this cycle_definition, is shown in [Figure 3-20](#).

Figure 3-20. Read Cycle Timing and Events



Write Cycle Definition

The write_cycle definition contains a number of events that comprise the write cycle. You define the cycle events primarily from looking at the memory timing diagrams. Starting with the address change, you determine the dependencies, or required timing between events. This write_cycle definition covers the events required in the write cycle.

To begin the write cycle, you change the values on “adr”. Because no dependencies exist between the address and data inputs, you can specify a second event to change the data inputs (di) at the same time, as follows:

```
write_cycle(  
    change adr;  
    change di;
```

The third event defines a wait to ensure that enough time passes after the address changes before the fourth event activates (pulses for one clock cycle) the write enable (we) signal:

```
    wait;  
    assert we;
```

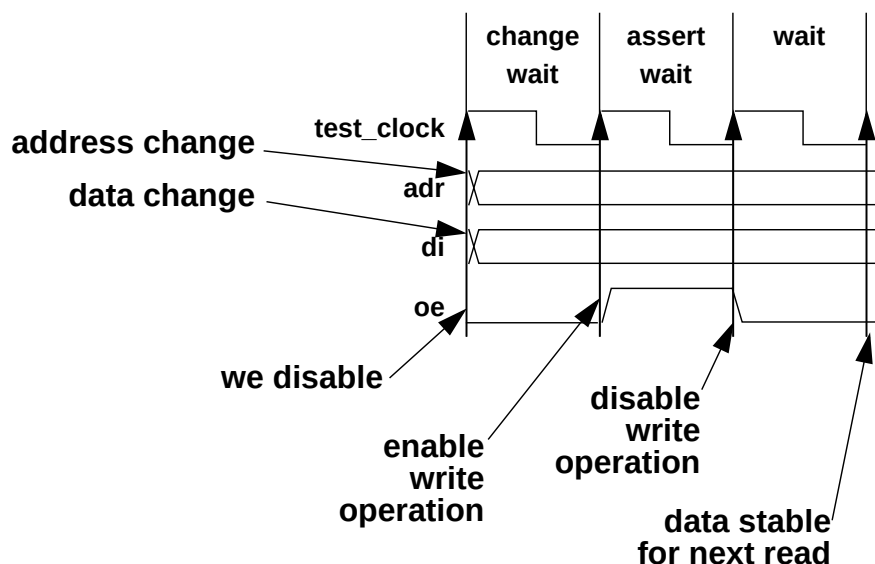
The cycle definition includes a fifth event that specifies another wait.

```
    wait;  
    ) // End write cycle
```

This wait is not necessary for a simple write operation. However, if you consider optimizing the circuitry for the March algorithm’s read/write/read cycle, this final wait allows the data to settle so the final read operation can read valid data.

The basic timing and events that comprise the write cycle, as defined in this cycle_definition, are shown in [Figure 3-21](#).

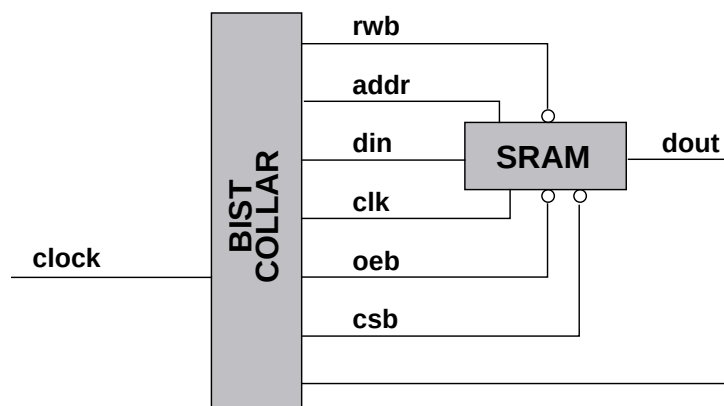
Figure 3-21. Write Cycle Timing and Events



Example - Single Port Synchronous RAM8X2

This example will model a synchronous single port RAM with tri-state outputs. This example will activate (assert) and deactivate (deassert) the control lines (oeb and csb as an exercise) to emphasize the handling of such signals in the model. [Figure 3-22](#) shows the block diagram that describes the RAM and memory BIST controller.

Figure 3-22. Single Port Synchronous RAM8X2 Example



All the control signals shown in [Figure 3-22](#) are active low including the write enable (**rwb**) is also active low (read is when **rwb** is high).

[Figure 3-23](#) and [Figure 3-24](#) provide the read and write cycle timing diagrams, respectively, for the memory as a guide for the memory modeling. In each respective cycle, numbered notes with

their description are given on the bottom of each diagram. The intent is to emphasize that you must be aware of the setup and hold time criteria to which both the memory device, and the BIST controller, must adhere.

Figure 3-23. SRAM Read Cycle Timing Diagram

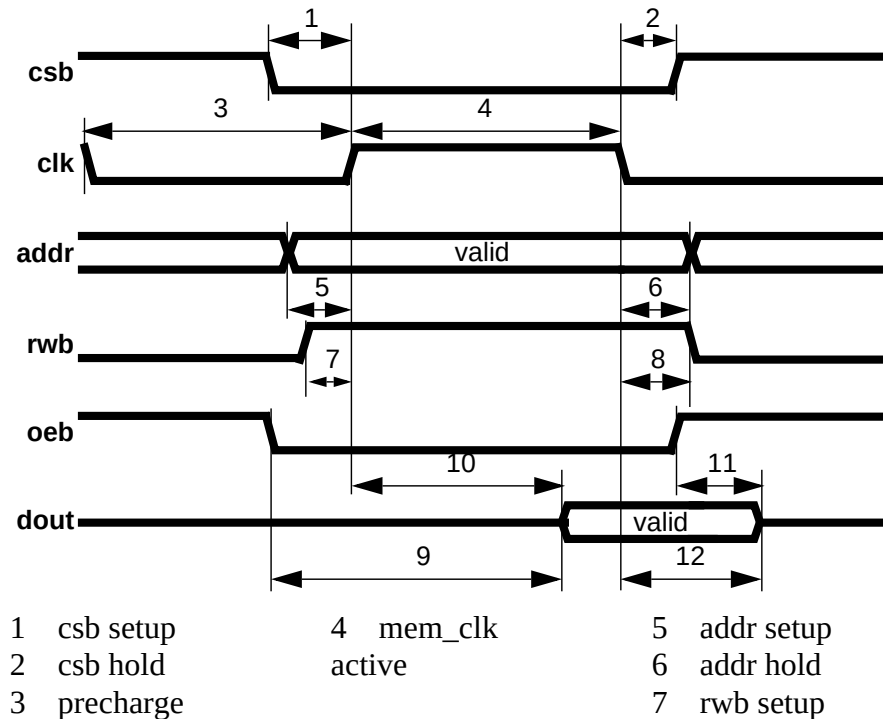
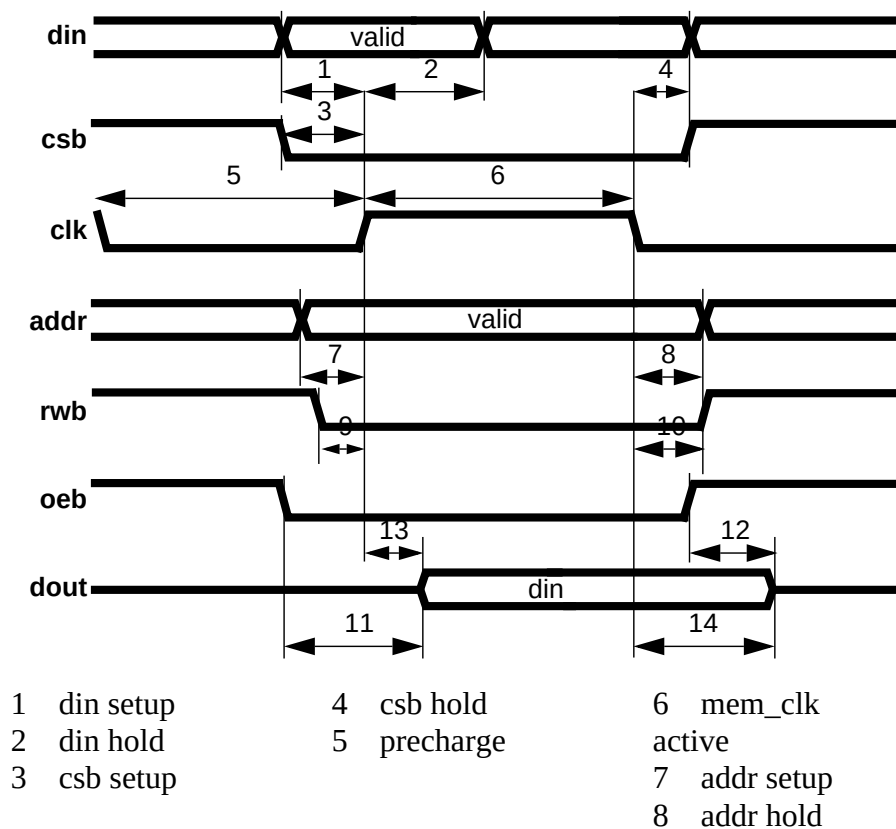


Figure 3-24. SRAM Write Cycle Timing Diagram

Example - SRAM Modeling

The SRAM modeling is as follows.

```
model sp8x2(addr, clk, csb, rwb, oeb, din, dout)

(
  bist_definition (
    address addr(array=2:0);
    data_in din(array=1:0);
    write_enable rwb low;
    clock clk high;
    output_enable oeb low;
    data_out dout(array=1:0);
    chip_enable csb low;

    tech = real fast;
    vendor = generic;
    version = "1.0";
    message = "synchronous 8x2 single port RAM";
    address_size = 3;
    min_address = 0;
    max_address = 7;
    data_size = 2;
```

```
read_write_port (
  read_cycle (

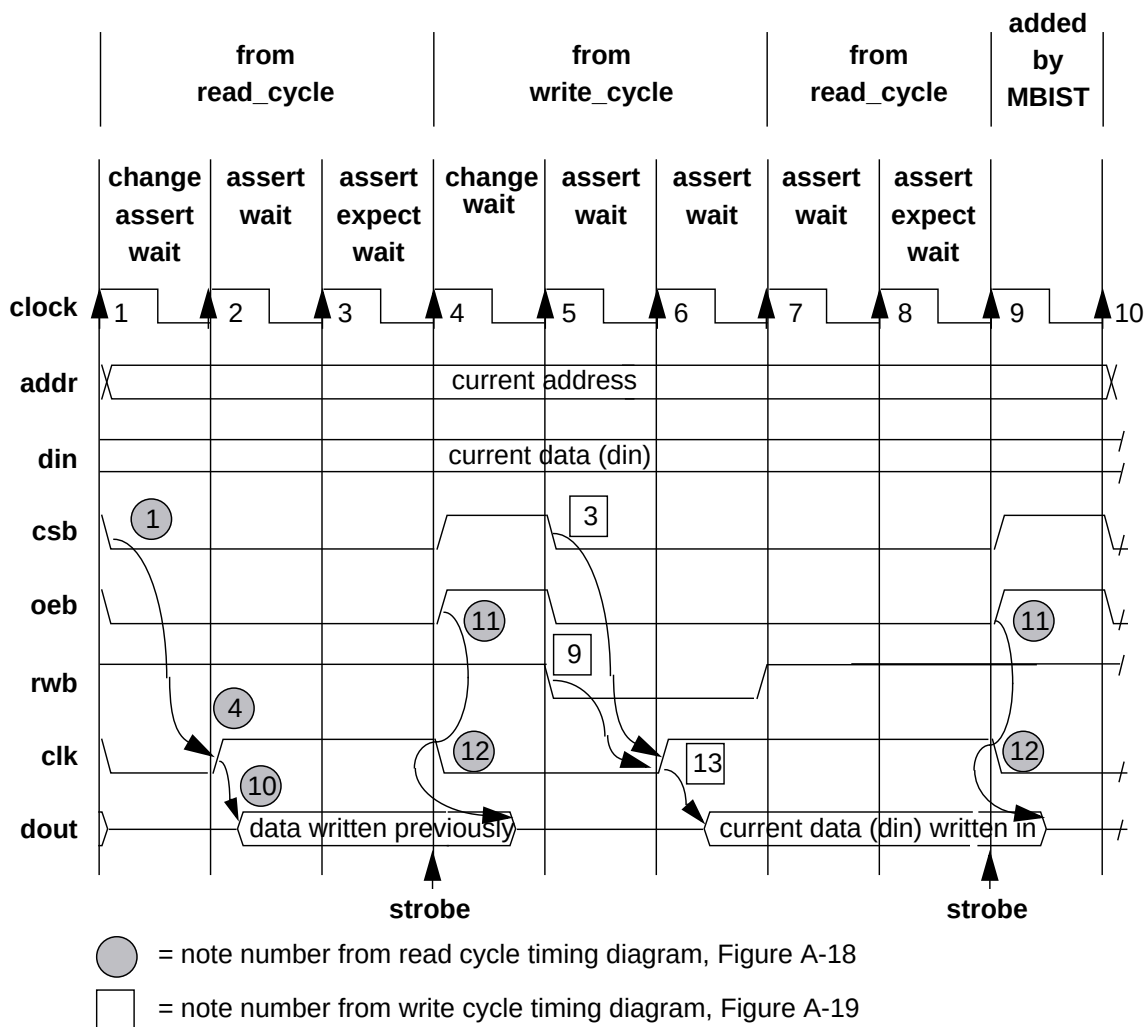
    change addr;
    assert csb;
    wait; // CSB setup (1)

    assert csb;
    assert oeb;
    wait; // Begin read access (10)

    assert csb;
    assert oeb;

    expect dout move;
    wait; // Allow read Deaccess (12)
  )
  write_cycle(
    change addr;
    change din;
    wait; // Allow addr in setup
    assert rwb;
    assert csb;
    wait; // Address setup (7), rwb setup (9)
    assert rwb;
    assert csb;
    wait;
  )
)
)
```

[Figure 3-25](#) shows the test timing of the BISTed memory, when implementing a March2 algorithm.

Figure 3-25. Test Timing for a March2 Algorithm SRAM Example

In Figure 3-25, the first read cycle in the address changes and the chip select (csb) and output enable (oeb) are activated (following the clock 1). The memory's clock (clk) is activated on the clock 2 edge. The reason being that the model must adhere to the csb setup before the clock is activated. Also note that in this memory, the clock active-level high contributes to the output enabling of dout. (csb and oeb are activated during clock 2 to keep the multiple clock read cycle active.)

An expect move on dout is not given until clock 3. The compare occurs in clock 3, but the latching of the pass/fail does not occur until clock 4. Thus, the clk must be held active for another clock.

The write cycle begins with clock 4 as well. The clk is deactivated. The first wait state of the write cycle is taken advantage by the tool to allow dout to get into the tri-state condition, and end the read access (notes 11 and 12 of the read cycle timing).

The clock 5 activates csb, oeb, and the write enable (rwb). The clock 6 activates the clk to actually write into the memory. Though rwb is activated in clock 5, there was a rwb criteria (setup) to follow before the clk could be activated. The dout also gets enabled with the new dout value, due to the active-level of clk after the edge of clk wrote into the memory.

Then clock 7 goes back into the read cycle once more for the second read. The second read does not occur clock 8 (the compare) with the latching of the pass/fail line with clock 9. Then clock 10 starts the next address sequence. The clock 9 was added by the tool to allow for the stabilization of dout to transition back into the tri-state condition.

Example - Dual Port Synchronous RAM8X2

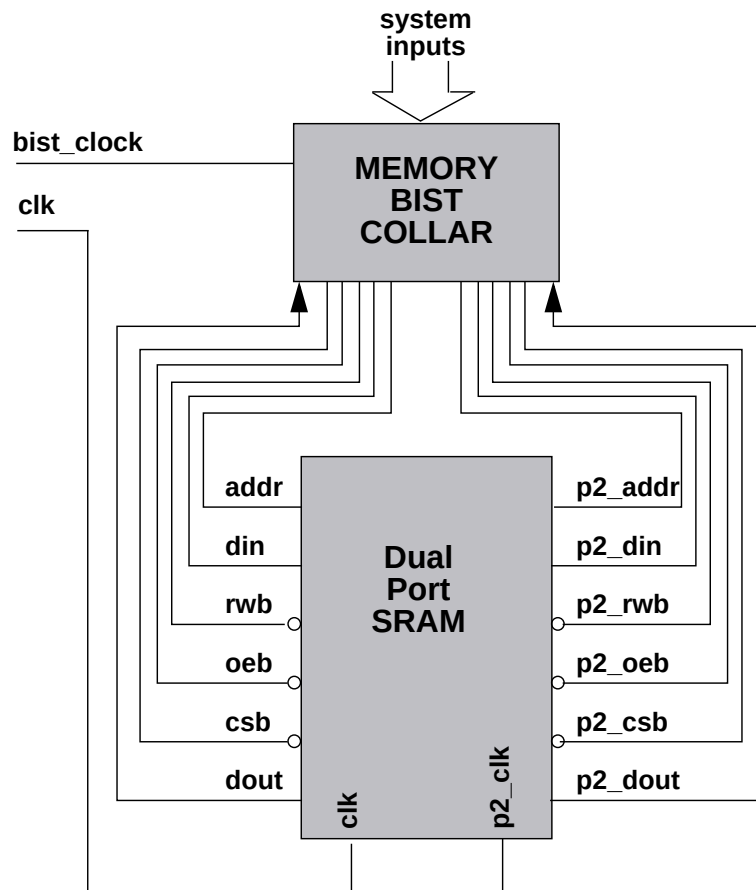
The example of the dual port synchronous memory follows the timing criteria of [Figure 3-25](#) (the single port synchronous memory). Port 1 will implement the March2 algorithm, and Port 2 will implement the Unique Address algorithm. [Figure 3-26](#) illustrates the BIST collar and memory.

Note



When separate clocks are used for the BIST and memory, they must have the same period.

Figure 3-26. Dual Port Synchronous RAM8X2 Example



The memory model is as follows:

```
model dpr8x2 (addr, clk, csb, rwb, oeb, din, dout,
  p2_addr, p2_clk, p2_csb, p2_rwb, p2_oeb, p2_din, p2_dout)
(
  bist_definition (
    address addr(array=2:0);
    address p2_addr(array=2:0);
    data_in din(array=1:0);
    data_in p2_din(array=1:0);
    write_enable rwb low;
    write_enable p2_rwb low;
    clock clk high;
    clock p2_clk high;
    output_enable oeb high; //really active low
    output_enable p2_oeb high; //really active low
    data_out dout(array=1:0);
    data_out p2_dout(array=1:0);
    chip_enable csb high; //really active low
    chip_enable p2_csb high; //really active low

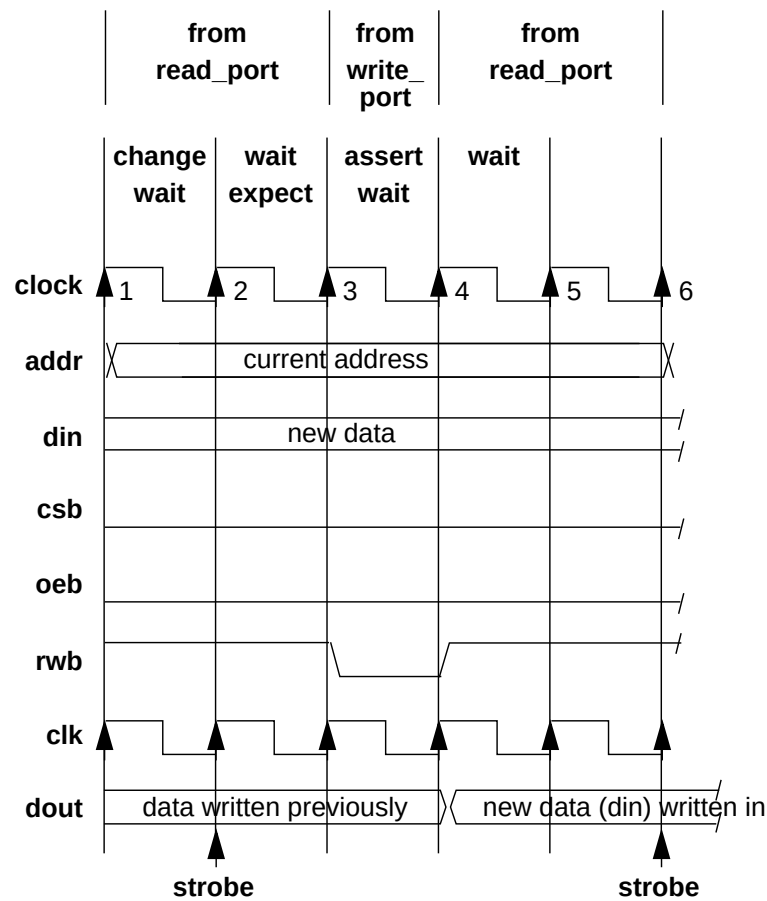
    tech = real fast;
    vendor = generic;
    version = "1.0";
```

```
message = "synchronous 8x2 dual port RAM";
address_size = 3;
min_address = 0;
max_address = 7;
data_size = 2;

read_write_port (
  read_cycle (
    change addr;
    wait;
    wait;
    expect dout move;
  )
  write_cycle (
    change addr;
    change din;
    assert rwb;
    wait;
  )
)
read_write_port (
  read_cycle (
    change p2_addr;
    wait;
    wait;
    expect p2_dout move;
  )
  write_cycle (
    change p2_addr;
    change p2_din;
    assert p2_rwb;
    wait;
  )
)
)
```

Figure 3-27 shows the test timing for Port 1, which implements the March2 algorithm.

Figure 3-27. Test Timing for a March2 Algorithm Example



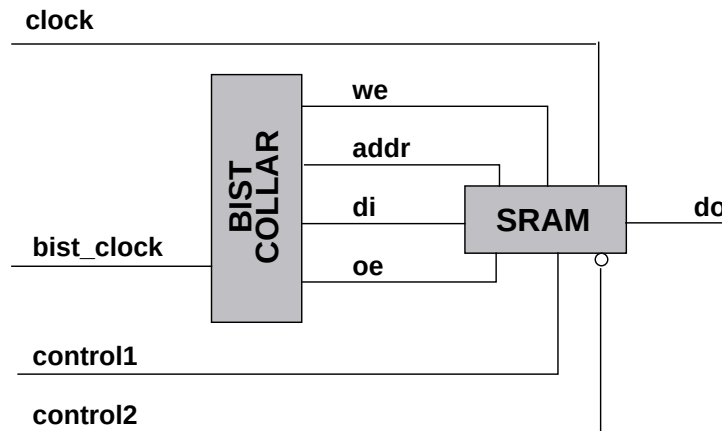
The chip select (csb) and oeb are held at a constant “active” value throughout the test. By doing so, it shortens the read-write-read action of the March2 algorithm to five clock cycles. The clk signal is constantly active to the memory.

Note When separate clocks are used for the BIST and memory, they must have the same period.

Example - Multiple Control Enables and Dont_touch

Figure 3-28 shows an example SRAM configuration with two control enables.

Figure 3-28. SRAM with Two Control Enables Example



The following is an example memory model for the SRAM shown in [Figure 3-28](#):

```

model RAM_256X32 ( do, we, oe, di, addr, clock, control1, control2 )
(
  bist_definition (
    write_enable we (array=31:0;) high;
    output_enable oe ( array=31:0;) high;
    address addr (array=7:0;);
    data_out do ( array=31:0;);
    data_in di( array=31:0;);
    dont_touch control1 low;
    dont_touch control2 high;
    clock clock;

    tech = Mult_Cntl;
    vendor = "Brand X";
    version = "1.0";
    message = "Asynchronous SRAM - 256 x 32 bit enable";

    address_size = 8;
    min_address = 0;
    max_address = 255;
    data_size = 32;

    write_port(
      write_cycle(
        change ADDR;
        change DI;
        wait;
        assert WE;
        wait;
      )
    )
    read_port(
      read_cycle(
        change ADDR;
        wait;
        assert OE;
        wait;
        expect DO;
      )
    )
  )
)
  
```

```

        wait;
    )
)
) // End of bist definition
) // End of model definition

```

The output_enable (OE) and the write_enable (WE) can be bused. In this case, each of the bits in the RAM have individual write and output enables. The busing of these control signals allows the tool to control the RAM as one common memory. The same technique was applied to both the address and data; the numbering is part of the signal name (for example, DO31, DO30, and so on), as opposed to DO(31), DO(32), where an actual bus is implied.

The dont_touch for control1 and control2 signals are assigned with their inactive states so that the implementation will result in respective active states in the test bench. When defining the active state, the default condition will be the inactive state. In this example, dont_touch control1 low results in a constant high (1) from the test bench for simulation. In the actual design, it is up to you to assure its correct sense in the design.

Example - RAM with Vector Write-Enable Signals

The tool can be used to model a RAM with vector write-enable signals. The following example contains a RAM with multiple write bits, and array control signals that are write_enable based. The busing of the control signals allows the tool to control the RAM as one common memory. The tool only generates patterns that apply all of the buses at the same time; bit-wise writes are not generated.

```

model EXAMPLERAM (Q,D,A,CLK,CEN,WEN,OEN) (
    bist_definition (
        write_enable WEN ( array = 7 : 0; ) low;
        output_enable OEN;
        chip_enable CEN;
        clock CLK high;
        address A ( array = 4 : 0;);
        data_in D ( array = 7 : 0;);
        data_out Q ( array = 7 : 0;);
        data_size = 8;
        address_size = 5;
        min_address = 0;
        max_address = 31;
        tech = cmos;
        vendor = xyz;
        version = "1.0";
        message = "Ram with multiple write bits";
        read_write_port (
            read_cycle (
                change A;
                wait;
                assert CEN;
                assert OEN;
                wait;
                assert CEN;
                assert OEN;
            )
        )
    )
)

```

```
        expect Q move;
        wait;
    )
    write_cycle (
        change A;
        change D;
        wait;
        assert CEN;
        assert OEN;
        assert WEN;
        wait;
    )
)
)
```

Defining the Memory Model and I/O Models with Specparams

In the insertion phase, the tool reads your input chip or top netlist and locates memory instances based on the identifying information you provide. In the insertion phase you have a choice: you can either define the memory models in a separate file using MBISTArchitect library format, and load it using Load Library, or you can annotate your memory models in HDL using Verilog specparams, and they will be identified automatically during the initial insertion phase HDL parsing activity. The insertion phase also seeks to locate instances of I/O pads in your chip or top netlist, and since there is no MBISTArchitect library syntax for I/O pads, these can only be identified using Verilog specparams.

The following section explains how the Verilog specparam construct is interpreted in MBISTArchitect input HDL files during the insertion phase.

Mentor Graphics Tessent tools use the following convention for Verilog specparams:

```
module mem1 ...
specparam name$mem1 = value;
```

The “\$” character is legal in Verilog identifier names, so the tool uses it as a delimiter. The model definition name is given to the left of the “\$”, and the syntactic construct to which it applies (either the enclosing module or a port in the module) is given to the right of the “\$”. In the tool, the specparam value is always a quoted or concatenated string.

As per Verilog 1995 (IEEE-1364) a specparam must be contained within a specify block:

```
module mem1 ...
specify
specparam name$mem1 = value;
endspecify
```


Verilog strings can be concatenated using the {} brace operators, such that the concatenated string {"hello", " ", "world"} is equivalent to the single quoted string "hello world".

You can embed the '\n' newline character inside Verilog strings, for example "line1\nline2". You should actually type "\n" as two characters instead of hitting your ENTER key.

Verilog Memory Model

The specparam named "bist_definition" identifies a Verilog module which is to be treated as a memory. You edit your HDL to add this specparam. This specparam applies to the enclosing module, so it should be fully named "bist_definition\$modulename".

The following is an example of a memory model identified by a "bist_definition specparam". The specparam section is in bold. Notice that the specparam value is a long concatenated string of the form {"string1", "string2", ... }.

```
module ram1024x8(a, di, dout, as, bs, we);
input as, we, bs;
input [7:0] di;
output [7:0] dout;
input [9:0] a;
specify
specparam bist_definition$ram1024x8 =
{"bist_definition\n",
"( data_out dout(array = 7:0;);\n",
"data_in di (array = 7:0;);\n",
"address a (array = 9:0;);\n",
"write_enable we high;\n",
"chip_enable bs low;\n",
"clock as high;\n",
"top_column = 8;\n",
"top_word = 1;\n",
"addr_inc = 4;\n",
"version = \"1.0\";\n",
"tech = generic;\n",
"vendor = MentorGraphics;\n",
"min_address = 0;\n",
"max_address = 1023;\n",
"data_size = 8;\n",
"read_write_port(\n",
"read_cycle(\n",
"change a;\n",
"wait ;\n",
"assert as ;\n",
"wait;\n",
"expect dout ;\n",
"wait;\n",
" )\n",
"write_cycle(\n",
"change a;\n",
"change di;\n",
"wait;\n",
"assert we ;\n",
"}\n";
endmodule
```

```
"wait ;\n",  
"assert as ;\n",  
"assert we ;\n",  
"wait;\n",  
"wait ; ) ) )\n"  
};  
endspecify  
endmodule
```

For comparison, this is how the same information might appear in MBISTArchitect library format:

```
model ram1024x8 (a, as, bs, di, dout, we)  
(  
  bist_definition  
  ( data_out dout(array = 7:0;);  
    data_in di (array = 7:0;);  
    address a (array = 9:0;);  
    write_enable we high;  
    chip_enable bs low;  
    clock as high;  
    top_column = 8;  
    top_word = 1;  
    addr_inc = 4;  
    version = "1.0";  
    tech = generic;  
    vendor = MentorGraphics;  
    min_address = 0;  
    max_address = 1023;  
    data_size = 8;  
    read_write_port(  
      read_cycle(  
        change a;  
        wait ;  
        assert as ;  
        wait;  
        expect dout ;  
        wait;  
      )  
      write_cycle(  
        change a;  
        change di;  
        wait;  
        assert we ;  
        wait ;  
        assert as ;  
        assert we ;  
        wait;  
        wait ; ) ) )  
    )  
)
```

IO Pad Model

The top design in your chip or top netlist has Verilog ports. However, these may or may not correspond to the top-level Input/Output Pads which will be the chip or top interface of your manufactured chip. If your design includes special cell instances which represent IO Pads, then you must identify them during the insertion phase using a specparam as shown in the following sections. You must edit the HDL library cell models to add these specparams.

There are two types of specparams for IO Pads:

`mgc_dft_cell_type`

Identifies the enclosing IO Pad module.

`mgc_dft_pin_type`

Identifies a port of the enclosing module.

The tool supports the following types of pads:

- Input
- Output
- Bidirectional (bidi)

Input Pad

The following is an example of a specparam that was added to an input pad model.

```
module PDIDGZ (PAD, C);
    input PAD;
    output C;
    specify
        specparam mgc_dft_cell_type$PDIDGZ = "input_pad";
        specparam mgc_dft_pin_type$C = "data_input";
        specparam mgc_dft_pin_type$PAD = "pin";
    endspecify

    // Place your HDL code here.

endmodule
```

Output Pad

The following is an example of a specparam that was added to an output pad model.

```
module PDT08DGZ (I, PAD);
    input I;
    output PAD;
```

```
specify
specparam mgc_dft_cell_type$PDT08DGZ = "output_pad";
specparam mgc_dft_pin_type$I = "data_output";
specparam mgc_dft_pin_type$PAD = "pin";
endspecify

// Place your HDL code here.

endmodule
```

Bidirectional Pad

The following example code shows the needed specify-endspecify block in module IOpad.

```
module IOpad (din, dout, ien, oen, iopin);

input dout, ien, oen;
output din;
inout iopin;

specify
specparam mgc_dft_cell_type$IOpad = "bidi_pad";
specparam mgc_dft_pin_type$din = "data_input";
specparam mgc_dft_pin_type$ien = "input_enable_h";
specparam mgc_dft_pin_type$oen = "output_enable_h";
specparam mgc_dft_pin_type$dout = "data_output";
specparam mgc_dft_pin_type$iopin = "pin";
endspecify

// Place your HDL code here.

endmodule
```

BIST Controller and Memory Blocks

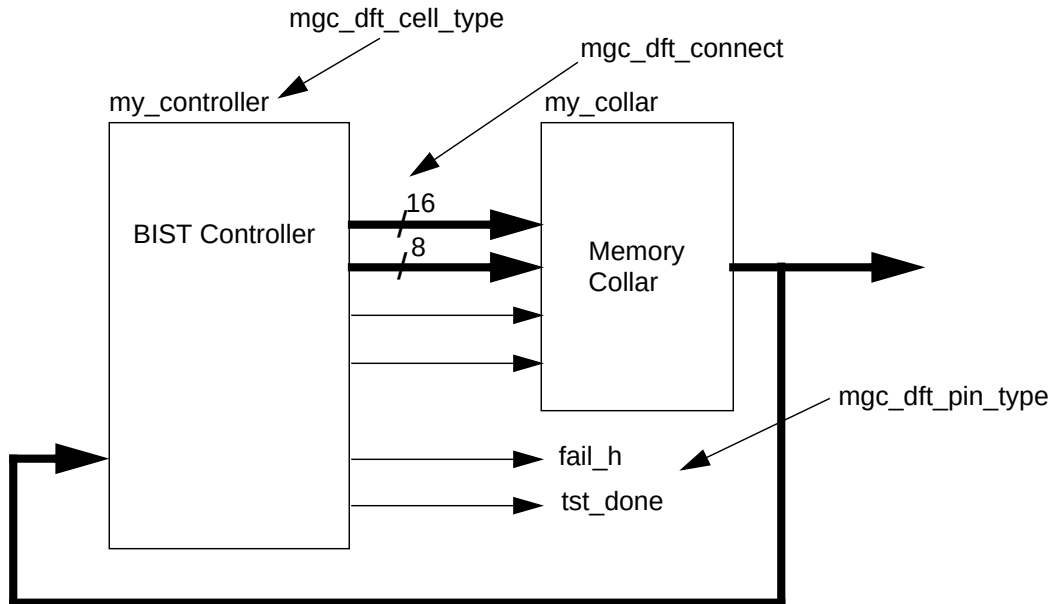
The following specparam types are defined to identify controller-to-collar and controller-to-top connections:

The following types are defined:

- **mgc_dft_cell_type**
Identifies the enclosing controller or the collar module.
- **mgc_dft_connect**
The “\$name” part of the specparam identifies a port on the enclosing module, and the “value” identifies the remote port to which it will be connected.
- **mgc_dft_pin_type**
Identifies the function a port on the enclosing controller module.

Figure 3-29 is a block diagram of a controller and collar whose connection can be specified with specparams.

Figure 3-29. Controller-To-Collar Instances Connected By Specparam



Example - Controller-To-Collar Instances Connected By Specparam

```
specify
  specparam mgc_dft_connect$aa2_0 = "BLK_8x4/<0>/test_aa2";
  specparam mgc_dft_connect$DI0_5 = "BLK_4x4/<5>/test_DI0";
  specparam mgc_dft_pin_type$sig_test="test_h";
  ...
endspecify
```

Example - Specparams Within Memory Collar

```
specify
  specparam mgc_dft_cell_type$mem_a="mbist_memory:cntrl1";
  specparam mgc_dft_pin_type$we="write_enable";
  specparam mgc_dft_pin_type$test_h="tselect";
  specparam mgc_dft_connect$addra="cntrl/0/Test_addra_0";
  specparam mgc_dft_connect$di_1="cntrl1/1/Test_di_1_0";
  ...
endspecify
```


There are memory test algorithms known to detect the majority of commonly occurring faults in memories. Many of these algorithms lend themselves well to built-in self-test (BIST) because the hardware to generate the patterns is relatively small and can serve multiple on-chip memories.

The algorithms in most common use are the March tests. March tests generate patterns that “march” up and down the memory addresses, writing values to, and reading values from known locations. These algorithms can retrieve the proper parameters from the memory model, automatically determining the memory size and word length.

The default algorithm for testing RAMs is [march2](#). The default algorithm for testing ROMs is the [rom1 \(rom\)](#) algorithm. To change the default algorithm for all ports, use the [Setup Mbist Algorithms](#) command. To change the algorithm applied to a specific memory port, use the [Add Mbist Algorithms](#) command.

Fault Types

The following fault types are discussed in this section:

- AF — Address Faults
- ADOF — Address Decoder Open Faults
- CF — [Coupling Faults](#)
 - CFin — [Inversion Coupling Faults](#)
 - CFid — [Idempotent Coupling Faults](#)
 - BF — [Bridge Coupling Faults](#)
 - CFst — [State Coupling Faults](#)
- DRF — Data Retention Faults
- SAF — [Stuck-at Faults](#)
- SOF — Stuck Open Faults
- TF — [Transition Faults](#)

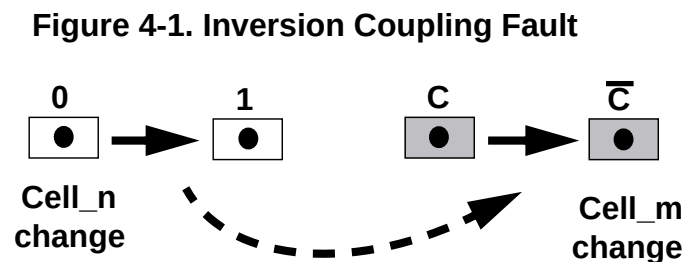
Coupling Faults

Memories also fail when a write operation in one cell influences the value in another cell. Coupling faults model this behavior and fall into four categories: inversion, idempotent, bridging, and state.

The basic method to test for coupling faults is to scan (write/read) all memory cells in ascending order, followed by a scan of all memory cells in descending order.

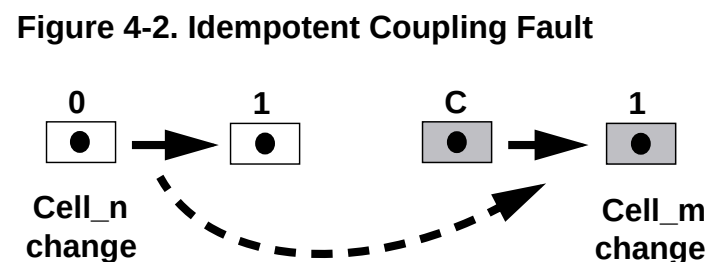
Inversion Coupling Faults

Figure 4-1 shows that inversion coupling faults, commonly referred to as CFin, occur when one cell's transition causes inversion of another cell's value. For example, a 0->1 transition in cell_n causes the value in cell_m to invert its state.



Idempotent Coupling Faults

Figure 4-2 shows that idempotent coupling faults, commonly referred to as CFid, occur when one cell's transition forces a particular value onto another cell. For example, a 0->1 transition in cell_n causes the value of cell_m to change to 1 if the previous value was 0. However, if the previous value was 1, the cell remains a 1.



Bridge Coupling Faults

Bridge coupling faults (BF) occur when a short, or bridge, exists between two or more cells or signals. In this case, a particular logic value triggers the faulty behavior, rather than a transition.

Bridging faults fall into either the AND bridging fault (ABF) or OR bridging fault (OBF) subcategories. ABFs exhibit AND gate behavior; that is, the bridge has a 1 value only when all

the connected cells or signals have a 1 value. OBFs exhibit OR gate behavior; that is, the bridge has a 1 value when any of the connected cells or signals have a 1 value.

State Coupling Faults

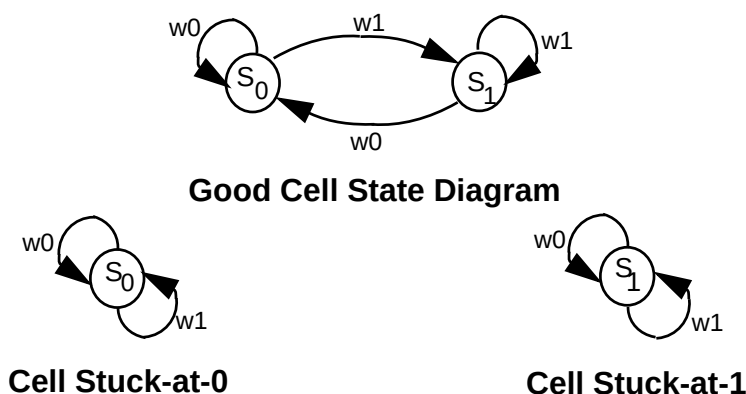
State coupling faults occur when a certain state in one cell causes another specific state in another cell. For example, a 0 value in cell i causes a 1 value in cell j.

Coupling faults involve cells affecting adjacent cells. Therefore, to sensitize and detect coupling faults, the march tests perform a write operation on one cell (j), and later read cell (i). The write/read operation performed in ascending order detects a coupling fault of the lower addresses. Likewise, the write/read operation performed in descending order detects a coupling fault of the higher addresses.

Stuck-at Faults

A memory fails if one of its control signals or memory cells remains stuck at a particular value. Stuck-at faults model this behavior, where a signal or cell appears to be tied to power (stuck-at-1) or ground (stuck-at-0). [Figure 4-3](#) shows the state diagram for a Stuck-at fault.

Figure 4-3. Stuck-at Fault State Diagram.



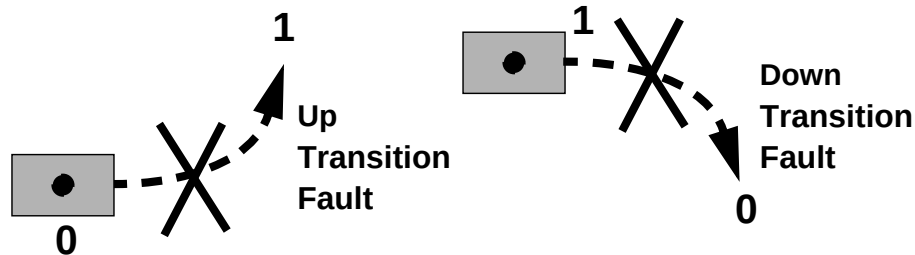
To detect stuck-at faults, you must place the value opposite to that of the stuck-at fault at the fault location. For example, to detect all stuck-at-1 faults, you must place 0s at all fault locations. To detect all stuck-at-0 faults, you must place 1s at all fault locations.

The basic method to test for stuck-at faults is writing 0's to all cells, reading all cells, writing all 1's to all cells, and then reading all cells again.

Transition Faults

A memory fails if one of its control signals or memory cells cannot make the transition from either 0 to 1, or from 1 to 0. [Figure 4-4](#) shows an up transition fault, the inability to change from 0 to 1, and a down transition fault, the inability to change from a 1 to a 0.

Figure 4-4. Transition Fault



[Figure 4-5](#) shows a cell that might behave normally when a test writes and then reads a 1. It might even transition properly from 1 to 0. However, when undergoing a 0->1 transition, the cell could remain at 0—exhibiting stuck-at-0 behavior from that point on. However, a stuck-at-0 test might not detect this fault if the cell was originally at 1.

Figure 4-5. Transition Fault State Diagram



To detect all transition faults in the memory array, a test must write a 0, followed by a 1, and then read (detects up transition). The test must then write a 1, followed by a 0, and then read (detects down transition).

The basic method to test for transition faults is to write (1->0) and immediately reading 0's at each address. Repeat this process to write (0->1) and read 1's at each address.

Pre-Defined Algorithms

A set of pre-defined algorithms is automatically loaded into MBISTArchitect. This section describes each of these algorithms including the targeted faults. Refer to "[Pre-Defined Algorithm File Contents](#)" on page 325 for the exact algorithm contents.

Table 4-1 summarizes the pre-defined algorithms that are automatically available in MBISTArchitect. For more information on any of the fault types listed in this table, see “Fault Types” on page 111.

Table 4-1. Pre-Defined Algorithm Summary

Algorithm Name	Complexity	Targeted Faults
march1 (MarchC-)	10n	AF, SAF, TF, CFin, CFid, and CFst
march2 (MarchC+)	14n	AF, SAF, TF, SOF, CFin, and CFid
march3	10n	AF, SAF, SOF, and TF
col_march1 (MarchC-)	10n	AF, SAF, TF, CFin, CFid, and CFst
unique	5n	SAF
checkerBoard (topChecker)	4n	BF
retentionCB	4n	BF and DRF
rom1 (rom)	1n	SAF
rom2	3n	SAF
addressdecoder_bg0 and addressdecoder_bg1	$n + 2n(1 + \log_2 n)$	ADOF

march1

Targeted Faults: AF, SAF, TF, CFin, CFid, and CFst

Complexity: 10n

Algorithm Steps:

```

1  up   - write 0
2  up   - read 0, write 1
3  up   - read 1, write 0
4  down - read 0, write 1
5  down - read 1, write 0
6  down - read 0

```

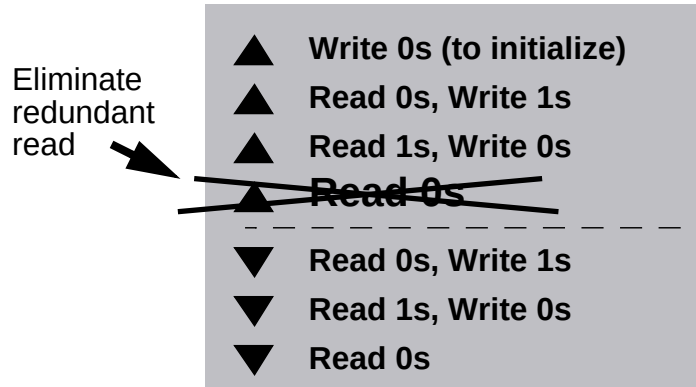
The MBISTArchitect March1 algorithm is equivalent to the standard MarchC⁻¹ algorithm, which is an optimized version of the standard MarchC² algorithm.

Figure 4-6 illustrates the modification of the MarchC algorithm to create the March1(MarchC-) algorithm. The redundant Read 0 operation, between the ascending and descending address

1. A.J. Van de Goor, *Testing Semiconductor Memories, Theory and Practice*, John Wiley & Sons, 1991.
2. M. Marinescu, *Simple and Efficient Algorithms for Functional RAM Testing*, IEEE International Test Conference, 1982, pp. 236-239.

operations, is removed. Removing this operation reduces the algorithm complexity from $11n$ to $10n$, without sacrificing any fault coverage.

Figure 4-6. Modifying the MarchC Algorithm



march2

Targeted Faults: AF, SAF, TF, SOF, CFin, and CFid

Complexity: $14n$

Algorithm Steps:

- 1 up - write 0
- 2 up - read 0, write 1, read 1
- 3 up - read 1, write 0, read 0
- 4 down - read 0, write 1, read 1
- 5 down - read 1, write 0, read 0
- 6 down - read 0

The MBISTArchitect March2 algorithm is a modified version of standard MarchC+¹ algorithm, which is derived from the standard MarchC algorithm. [Figure 4-7](#) shows the modified MarchC algorithm, having an extra read operation after each step. While increasing the algorithm from $10n$ to $13n$, this extra read allows additional fault detection.

[Figure 4-8](#) shows the MarchC+ algorithm, which further modifies the modified MarchC algorithm by adding one more read operation at the end of the final stage. This increases the algorithm from $13n$ to $14n$.

1. R. Dekker, F. Beenker, L. Thijssen, Fault Modeling and Test Algorithm Development for Static Random Access Memories, IEEE International Test Conference, 1988, pp. 343-352.

Figure 4-7. Modified MarchC Algorithm

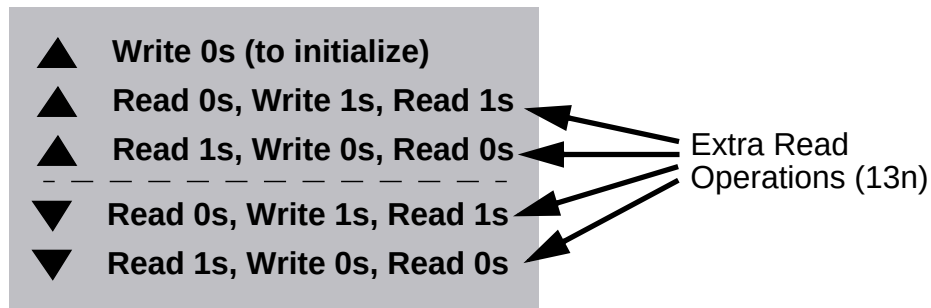
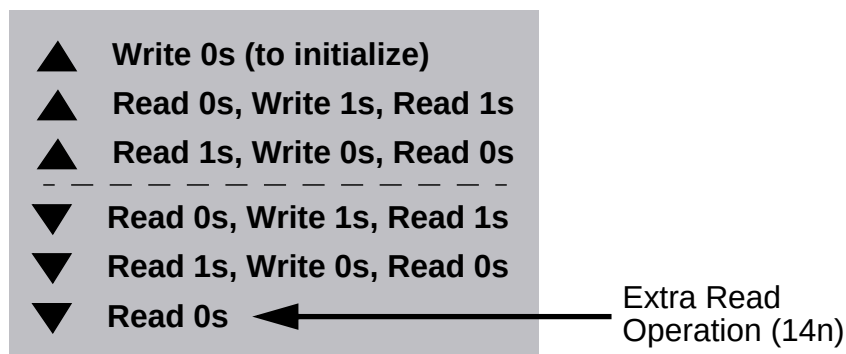


Figure 4-8. MarchC+ (March2) Algorithm



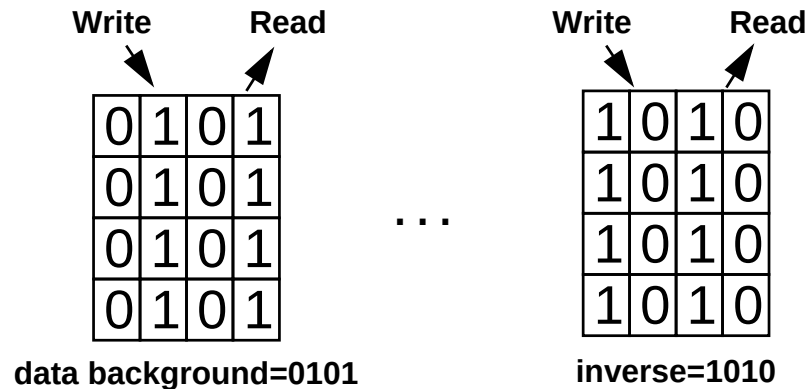
Varying Data Backgrounds

The March2 algorithm normally writes and reads words of either all 0s or all 1s. However, you can vary the value the March2 test uses for each write/read operation. By varying the data values, or data backgrounds, you can increase the fault detection. For example, by carefully choosing the data background from inductive fault analysis of the memory, the enhanced algorithm can detect state coupling faults between two cells of the same address, for which the March2 algorithm cannot normally prove detection.

For each specific data background, the March2 test uses this pattern instead of all 0s, and then uses the pattern inverse instead of all 1s. For example, assume your target memory is a 4X4 RAM with data background 0101 (see [Figure 4-9](#)). The March2 algorithm with a varied background becomes:

1. Write 0101 to all locations starting at address 0 up to address 3.
2. Read 0101 at address 0, write 1010 at address 0, read 1010 at address 0, repeating this series of operations in addresses 1, 2, and 3.
3. Read 1010 at address 0, write 0101 at address 0, read 0101 at address 0, repeating this series of operations in addresses 1, 2, and 3.
4. Repeat steps 2 and 3, but this time begin at address 3 working down to address 0.

Figure 4-9. March2 Algorithm with Varied Background



march3

Targeted Faults: AF, SAF, SOF, and TF

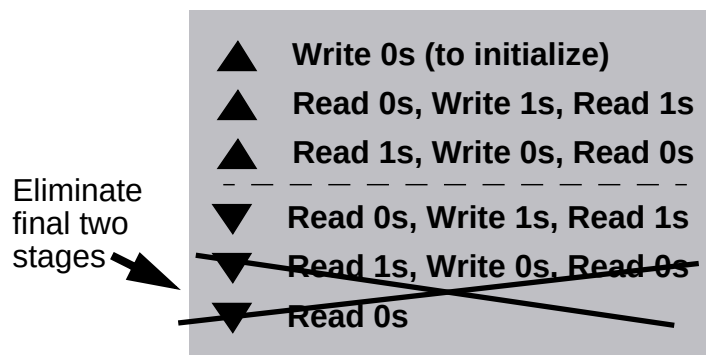
Complexity: $10n$

Algorithm Steps:

- 1 up - write 0
- 2 up - read 0, write 1, read 1
- 3 up - read 1, write 0, read 0
- 4 down - read 0, write 1, read 1

The March3 algorithm is a modified version of the March2 algorithm, where the final two steps are removed. Figure 4-10 shows the modifications to the March2 algorithm. This decreases the algorithm from $14n$ (for the March2) to $10n$.

Figure 4-10. March3 Algorithm



col_march1

Targeted Faults: AF, SAF, TF, CFin, CFid, and CFst

Complexity: $10n$

Algorithm Steps:

```

1  up   - write 0
2  up   - read 0, write 1
3  up   - read 1, write 0
4  down - read 0, write 1
5  down - read 1, write 0
6  down - read 0

```

The MBISTArchitect Col_March1 algorithm uses the same standard MarchC- algorithm as March1, but increments the address by the *addr_inc* value specified in the memory model. You change the incrementation value by including the following line in your memory model file:

```
addr_inc=<value>;
```

Where <value> is less than the address size and greater than or equal to 2. If you do not define *addr_inc*, the <value> defaults to 1.

Note



If *addr_inc* is set to 1, the Col_March1 algorithm is equivalent to the March1 algorithm.

Figure 4-11 illustrates incrementing with an *addr_inc* value of 4. The controller reads and writes to and from addresses 1, 5, 9, and 13, then goes back to the last starting address + 1 and begins again (addresses 2, 6, 10, 14). This continues until all memory locations have been accessed.

The Col_March1 algorithm gets its name from the ability to adjust the address increment to perform a column March through memory, as shown in Figure 4-11.

Figure 4-11. Col_March1 Algorithm

Memory Model

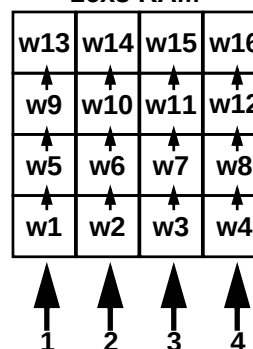
```

model ram16x8 (DO15, DO
(
    bist_definition (
        data_out d_o(DO15, DO
        data_in di(DI15, DI14
        address addr(A7, A6,
        write_enable WEN low;

    addr_inc = 4;
    version = "1.0";
    message = "16x8 RAM,
    min_address = 0;
    max_address = 15;

```

16x8 RAM



It takes 4 passes to complete each stage of the Col_March1 on a 16x8 RAM with the *addr_inc*=4.

unique

Targeted Faults: SAF

Complexity: $5n$

Algorithm Steps:

```
1  up  - write 0
2  up  - write address unique
3  up  - read address unique
4  up  - write inverse address unique
5  up  - read inverse address unique
```

Simple march type algorithm, using data for each address that is based on the address.

For memories where the address bus is larger than the data bus the data value is computed by using addition to reduce the address bus down to the data bus size; the address bus is sliced, using the data bus width, and these are added together to give the data value.

Conversely, for other memories, the data value is computed by concatenating copies of the address value enough times to fill the data bus. If necessary the most significant copy of the address value will be sliced to exactly fit the data bus width.

The Unique address algorithm provides the most benefit when testing multiple-port memories, providing you apply an algorithm (such as March1 or March2) to one of the ports to test the cell array. Following completion of the cell array tests, the unique address algorithm tests the control signals and decoder circuitry of the remaining ports.

The Unique address algorithm tests the control signals and the decoder circuitry.

The control signals and decoder circuitry of the remaining ports is tested by ensuring that each block of data (determined by the size of the data bus) is unique. This further ensures the uniqueness among additional most-significant-bits (beyond the data bus width) of the address decoder.

The steps involved in the unique address algorithm, and the tool behavior, are as follows:

1. Write all 0's to all addresses starting from minimum address to maximum address. This is the initialization process.
2. Write the address to memory.
3. Read the address from memory.
4. Write the complement of the address to memory.
5. Read the complement of the address from memory.

For example, using a 4-bit wide data bus, [Figure 4-12](#) shows the algorithm writing (and then reading) value 0000 into address 0000, value 0001 into address 0001, and so on until it reaches the beginning of data block 2 at address 16. At this point the algorithm would normally repeat

the value 0000. But, by writing the address value of address 17 (0001 = address 16 + ending block number 1), the contents of the first address in data block 2 is different from that of the first address in data block 1. This kind of circular buffer data generation continues such that at the beginning of data block 3 (address 32), the algorithm writes the value of address 34 (0010 = address 32 + ending block number 2).

If the word size exceeds the number of address bits, the algorithm adds to the address value to make the address the size of the data word. For example: assume the address bus is 3 bits and the data bus is 4 bits. In this case, the algorithm appends the LSB of the address value to the MSB bit of data value. For address 1 (001), the algorithm appends LSB (1) to the MSB position of data word, creating the data word 1001. Likewise, for address 4 (100) it appends LSB (0) to the MSB position, creating the data word 0100. And again, for address 5 (101) it appends LSB (1) to the MSB position of data word, creating the data word 1101.

Figure 4-12. Unique Address Algorithm

<u>Address</u>		<u>Data</u>		
0		0 0 0 0	= 0	
1		0 0 0 1	= 1	
2		0 0 1 0	= 2	
.		.	.	
.		.	.	Block 1
14		1 1 1 0	= E	
15		1 1 1 1	= F	
First Address	16	0 0 0 1	= 1	
+ Previous Block (1) =	17	0 0 1 0	= 2	
.		.	.	
.		.	.	Block 2
30		1 1 1 1	= F	
31		0 0 0 0	= 0	
First Address	32	0 0 1 0	= 2	
+ Previous Block (2) =	33	0 0 1 1	= 3	
	34	0 1 0 0	= 4	
.		.	.	
.		.	.	Block 3
46		0 0 0 0	= 0	
47		0 0 0 1	= 1	
First Address	48	0 0 1 1	= 3	
	49	0 1 0 0	= 4	
+ Previous Block (3) =	50	0 1 0 1	= 5	
	51	0 1 1 0	= 6	
.		.	.	
.		.	.	Block 4
62		0 0 0 1	= 1	
63		0 0 1 0	= 2	
.		.	.	
.		.	.	
.		.	.	

checkerBoard (topChecker)

Targeted Faults: BF

Complexity: 4n

Algorithm Steps:

- 1 up - write checker board
- 2 up - read checker board
- 3 up - write inverse checker board
- 4 up - read inverse checker board

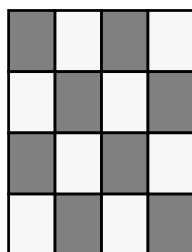
Simple march type algorithm, using checker board data pattern. Checker board data value takes into account the topology of the memory, as described in the memory model (top_column and top_word data values).

The checkerboard algorithm divides the cells into two groups (cells_1 and cells_2), such that every neighboring cell is in a different group. Figure 4-13 shows how this process forms the memory cells into a checkerboard pattern. The algorithm then writes (and reads) 0s into all cells in the cells_1 group and 1s into all cells in the cells_2 group. The algorithm repeats this process by writing (reading) 1s into cells_1 cells and 0s into cells_2 cells.

A goal of the checkerboard operations is to have 010101 patterns imposed on memory cells so that each cell's neighbors are in the opposite state. When the Nth bits are close to each other, we need to invert the 010101 pattern to 101010 at every address so that the Nth bits are toggling.

Figure 4-13. Checkerboard Algorithm (TopChecker)

Group Cells



 = cells_1

 = cells_2

Write Read

0	1	0	1
1	0	1	0
0	1	0	1
1	0	1	0

cells_1 = 0

cells_2 = 1

Write Read

1	0	1	0
0	1	0	1
1	0	1	0
0	1	0	1

cells_1 = 1

cells_2 = 0

It is important to take into account these address and data scrambling effects when testing memories using the checkerboard algorithm which is dependent upon:

- Topological address/data information.
- Consistency between logical data values and electrical data values.

The checkerboard algorithm also takes into account whether the memory under test uses muxes in its column address decoder. That is, the algorithm acts differently depending on the topology of the memory under test.

To specify the topology of the memory under test, place the following lines within the `bist_definition` of your memory model file:

```
top_column=<value>;
top_word=<0 | 1>;
```

The following describes the values for each of these statements:

- `top_column=<value>`
The `top_column` statements indicates the memory's number of words per row. The `<value>` can be any integer greater than 0, however, the integer must be a power of 2. The algorithm uses this value to ensure that the first word of a row is different than the first word of the previous row.
- `top_word=<value>`
The `top_word` statements indicates the memory's topology. The `<value>` can be either 0 or 1. The following describes how the algorithm interprets each value:
 - A 0 indicates that the memory under test does not use muxes in its column address decoder. This causes the algorithm to use the test vectors "0101....01" and "1010.....10".
 - A 1 indicates that the memory under test does use muxes in its column address decoder. This causes the algorithm to use the test vectors "000....00" and "1111....11".

Multiple memories of different topologies can share the same controller. It is only necessary that each memory model contain its own `top_column` and `top_word` statements.

See also the discussion of `top_column` and `top_word` in "[Parameters](#)" on page 57.

retentionCB

Targeted Faults: BF and DRF

Complexity: 4n

Algorithm Steps:

```

1  up  - write checker board, synchronize
2  up  - read checker board
3  up  - write inverse checker board, synchronize
4  up  - read inverse checker board

```

The RetentionCB algorithm is similar to the checkerboard algorithm in that it detects stuck-at faults for memory cells and adjacent cell shorts (assuming that the address decoder is correct). However, the RetentionCB algorithm has wait periods at the end of the two write steps to allow for retention testing. [Table 4-2](#) is for the RetentionCB algorithm and illustrates how each write step is followed by a wait period (reading the table from left to right).

In [Table 4-2](#) the first set of write and read are for data “checkerboard”; the second set of write and read are with “inverseCheckerboard”.

Table 4-2. RetentionCB Algorithm

Write	Wait	Read	Write	Wait	Read
address 0		address 0	address 0		address 0
.		.	.		.
.		.	.		.
.		.	.		.
address n-1		address n-1	address n-1		address n-1

For more information on how multiple BIST controllers are synchronized for retention testing, see [“Retention Testing”](#) on page 133.

rom1 (rom)

Targeted Faults: SAF

Complexity: 1n

Algorithm Steps:

```

1  up  - read

```

The ROM1 test algorithm reads the values from each address of the memory in increasing order, one word at a time. To determine the pass/fail state of the memory, the circuit inputs the values read from memory into a MISR and compares the signature against the known good value for the ROM.

For more information, see [“ROM Content File”](#) on page 38.

rom2

Targeted Faults: SAF

Complexity: 3n

Algorithm Steps:

```
1 up - read
2 down - read
3 up - read
```

The ROM2 test algorithm reads the values from each address of the memory in increasing order, one word at a time, and then reads the values from each address of the memory in decreasing order, one word at a time, and then the second (and final) up read catches any problems caused if the previous reads have damaged data stored in the memory or the memory itself. The read up and read down address pass will identify errors that may be masked in only one direction. This algorithm provides address and control circuitry fault detection.

addressdecoder_bg0 and addressdecoder_bg1

Targeted Faults: ADOF

Complexity: $n + 2n(1 + \log_2 n)$

Algorithm Steps:

```
addressdecoder_bg0:
1 up - write 0
2 up - write 1, shift_write 0, read 1, write 0
```

```
addressdecoder_bg1:
1 up - write 1
2 up - write 0, shift_write 1, read 0, write 1
```

Standard Address Decoder Algorithm. The algorithm is based on writing a value to a test address and checking if the base address value changes.

The address decoder fault is associated with a pair of addresses: the base address, and the testing address. Therefore address decoder faults are always detected for a pair of memory addresses.

To detect open faults in an address decoder, the algorithm writes to a neighboring address with a hamming distance of one, and checks if this write also writes into the base cell.

The addressdecoder_bg0 and the addressdecoder_bg1 algorithms are pre-defined algorithms supplied with the tool and can optionally be used as a UDA. The addressdecoder_bg0 and the addressdecoder_bg1 algorithms are used to test for stuck open PMOS address decoder faults.

The addressdecoder_bg0 and the addressdecoder_bg1 algorithms cannot be used with ROMs.

To describe this algorithm in detail, the basic steps are as follows:

Write “1” to the base address.

Write “0” to the shifted address. The value at this address will be “0”, but if it has a related open PMOS, the base address will also have a value of “0”. Next, if the base address is read, with an expect value of “1”, there will be an error as the value is “0”.

For more information on steps see: “[Step Definition](#)” on page 148. For more information on operations like “w(s_wr)w”, see “[Looped Operations](#)” on page 156.

The open PMOS address decoder has no support for the row address decoders. Currently the calculations of neighboring address is based on the assumption that the memory addressing has a row factor of one.

Port Interaction Testing Algorithm

When you specify the port interaction testing algorithm, the test is applied to all read ports of memories under test with two or more read ports. If you have both multiple-port and single-port memories, the algorithm is only applied to the multiple-port memories. Do not use the port interaction test algorithm if all memories have fewer than two read ports.

Note



This algorithm is added using the [Add Mbist Algorithms](#) command. You cannot specify any other algorithms in the same command instance. Additional algorithms must be specified in a separate command instance. This algorithm is not available for the Setup Mbist Algorithms command.

Faults Targeted by the Port Interaction Algorithm

The port interaction test algorithm detects address decoder faults (AF), checks for shorted address lines on different ports, and checks that reading from one port does not affect any other read ports.

Note



For the purpose of interpreting test results, the port interaction test algorithm is always performed last, after all other algorithms. Otherwise you might misinterpret simple faults like SAF as port interaction faults.

The port interaction test algorithm initializes applicable memories by writing a unique data value into each memory address. The write occurs on the first writable port. The algorithm then exercises all the readable ports simultaneously by reading from address zero to max, with the first port reading address *i*, the second port reading address *i*+1, and so on up to readable port *N*. By reading these “neighbor” addresses in the same cycle we are testing the interaction of the ports. If there the BIST controller sees unexpected data on any of the readable ports, then a failure has occurred.

The unique data used by the BIST controller is generated based on the address value. For example, a memory with a 5-bit address and 32-bit data will generate the following data for the first address (00001): 01-00001-00001-00001-00001-00001-00001. The inverted data for the same address would be 10-11110-11110-11110-11110-11110-11110.

Because the write step is separate from the read, it does not matter which writable port you use. The port interaction test algorithm is really a test of the read ports, not the write ports.

Note  When using Add Diagnostics Monitor to report the failmap or dout field, only the failmap or dout field of the first read port is available for a given memory during port_interaction test.

Port Interaction Example

Consider a BIST controller which tests two memories, memA and memB, whose ports are designed as follows:

memA, port 1: read/write port.

memA, port 2: read/write port.

memA, port 3: read/write port.

memB, port 1: write port.

memB, port 2: read port.

The port_interaction algorithm will perform the following actions:

1. For memory memA:
 - a. Write unique data values in each address using only port 1.
 - b. For each address in range 0 to max:
 - i. Read data at address from port 1.
 - ii. Read data at address + 1 from port 2.
 - iii. Read data at address + 2 from port 3.

These three reads occur in the same cycle. When (address + i) would exceed max, the value wraps around to zero. Read the full range of addresses.
 - c. Repeat steps a. and b. using inverted unique data values.
2. For memory memB:
 - a. Write unique data values in each address using only port 1.
 - b. For each address in range 0 to max:
 - i. Read data at address from port 2.

Since there is only one readable port the address will not wrap around in the final read. Also, the actions shown above (I and II) may be performed sequentially or concurrently, depending upon your usage of Setup Memory Test.

Port Isolation Testing Algorithm

Port isolation testing supports writing with one port while simultaneously reading with a second port. Since much of the circuitry for two ports might have runs that are “close” to each other, this allows for detecting some coupling faults that might not be otherwise detected.

The port isolation testing algorithm is not a pre-defined algorithm. You must make a UDA (user-defined algorithm), then add it with either the Add Mbist Algorithms or the Setup Mbist Algorithms command. For more information see “[User-Defined Algorithms](#)” on page 143.

A port isolation algorithm uses the following additional syntax:

- W_R operation — This operation performs the simultaneous read and write. For more information, see “[Operations](#)” on page 154.
- Port offset — This additional element to the address sequence specifies the address offset between the read and write activities in the W_R operation. For more information, see “[Address Sequences](#)” on page 150.

Note



The port isolation algorithm is not available for concurrent memory test. This algorithm is also not available for designs with any single port memories.

Types of Multiport Memories

There are two types of multiport memories that can be tested. Testing behavior is the same for both types of definitions:

- Standard dual port memories with two read_write_port definitions. This includes having either one address signal for each port, or separate read and write addresses for each port.
- Defined as a register file by using either one read_write_port or one read_port and one write_port. Such memories have a write port with separate address, data in, and control signals, and a read port with separate address, data out, and control signals.

Note



The port isolation algorithm cannot be used for memories where the address signal is shared between ports.

Types of Multiport Problems

There are two types of problems that might occur due to the two ports which are not tested by pre-defined algorithms.

The first problem involves interference while reading and writing different words in the same row. Since some of the row-select circuitry is either shared or is in proximity with other port's circuitry, interference might occur.

The second problem involves interference while reading and writing words in the same column. Although each port's column circuitry must be separate, much of the circuitry must be in close proximity to each other.

Port Isolation Testing Process

The processes described in this section are meant to test that each port's circuitry is functionally isolated from the other port (hence the term port isolation testing).

Port isolation testing works by writing to the memory using the write signals of one port while using the read signals of the other port to read data from the memory. For dual-port memories, each port takes a turn as the writer with the other port used for reads. The general approach is to write all of the memory with one pattern.

Then a *port_isolation* step is done where the opposite value is written to each location while the original value is read from a location “in front” of the writing process. Both the address increment for the writing process, and the address offset for the reading, can be set to the integer 1, or to an integer such as the number of words per row.

Depending on the memory architecture, using the row size value allows for more “intense” testing of the parts of the two port's logic that are close to each other.

There are two processes that can be specified. These processes are designed to address the two kinds of problems described previously.

- The first process involves writing at one location while simultaneously reading one location “in front” of the write location. This tests the row interference issues.
- The second process involves writing at one location while simultaneously reading data from the same column, but one row “in front” of the write location. This tests the column interference issues.

The read location will have been set during some previous process to have a value opposite what is being written.

i **Tip:** When working in ascending order, the read location will be one address or one row-size higher than the write location. When working in descending order, the read location will be one address or one row-size lower than the write location.

When the end of the memory is reached for the read location, the process will stop. The write process will continue so that the memory gets into a known state for the next algorithm step. There are several things to be aware of.

The tool cannot keep reading by wrapping around to the other end of the memory because those addresses were written to at the beginning of this algorithm step. These addresses now have the same value as the value being written. This leads to “incomplete” tests if the approach is used only with ascending or only with descending approaches. For example, for column March ascending, there is no test for interference between writing the last word in a column and reading something from the column. However, if you do both ascending and descending, the missed test from ascending will be the first test in descending. This same phenomenon occurs with single address stepping as the last word in a row is not written at the same time as another is read. Again, this is covered if both ascending and descending testing is done.

There is yet another aspect of this test that affects single-address stepping. While stepping in one direction, the last write to each row occurs while the tool is requesting a read of a word from the next row. That is, there is no simultaneous read in the same row as these writes. Again this is covered if testing is done while stepping in both ascending and descending directions.

Interactions and Limitations with BISA

The MBISTArchitect BISA support is designed to detect errors in the memory cells which can be fixed by using substitution of spare resources (for example, a spare row). The port isolation test process is testing hardware which appears to the tool as not to be part of the redundant/repair logic. Because of this, an error detected during the port isolation test that is not detected during normal memory testing should cause the memory to be considered failed.

There are two paths of possible support here. First, if we detect an error during port isolation testing, then the tool will fail the memory. Second, if/when the tool can, the tool will use the BISA information about the each memory failure as a mask so that corresponding errors detected during port isolation will be ignored.

You might need to run the BISA-related algorithms, effect repairs if required, and only then run the port isolation tests.

It is important that BISA information is not updated during port isolation testing steps. Because the port isolation testing steps are testing your non-redundant hardware, any new error detected cannot have any implications for how the redundant hardware is committed. The implication of this is that the tool must tell BISA to be inactive during port isolation tests. For more information, see “[Port Isolation User-Defined Algorithm](#)” on page 170.

Port Isolation Testing with Diagnostic Monitor

If you have enabled diagnostics, you can use the Add Diagnostic Monitor command to report the address values for a failure. The W_R operation used for port isolation tests uses separate read and write addresses. To report both the read address and write address for a failure during the W_R operation, include the following in your diagnostic monitor list:

- *addr_reg* — Captures the read failure address for all operations, including the W_R operation.
- *port_isol_addr_reg_write* — Captures the write address during the W_R operation. For all other operations, this monitor item is not used and reports a zero value.

Port Isolation Terminology (Keywords)

column march

Operating in a manner so that successive words in the same column are processed. This may be done ascending or descending.

row fast

The same as column march. Term is often used by designers.

addr_inc

Size of a row in the memory. Terminology used by MBISTArchitect. Introduced when Column March support was added.

row_size

A key word in a user-defined algorithm (UDA) definition. It indicates that the port isolation test should be done by offsetting the read address from the write address by row size. Offset value used will be the **addr_inc** value defined in the memory model library.

jump

A key word in a user-defined algorithm definition. It indicates a column march process. Mentioned here to distinguish it from the similar, but distinct, process involved in port isolation testing. The column march algorithm causes the address register to jump by **addr_inc** with each step. Each address register (read and write) will step by one during port isolation testing.

register file

A term often used to describe a memory that has a separate read and write port. Often these are used to implement a circular buffer using hardware that implements auto-incremented read and write pointers and circuitry to prevent simultaneous access of the same location by the two pointers.

Example - Port isolation Testing

This section provides examples of pseudocode for what should be done to test dual-ported RAMs and pseudocode for what should be done to test register file RAMs. For an example of the port isolation user-defined algorithm (UDA) see “[Port Isolation User-Defined Algorithm](#)” on page 170.

Pseudocode for Dual-Ported Rams (Dual Read and Write Ports)

The following are pseudocode examples only.

Dual Port Example - up, [XX = 1 | row_size]

```
for ( addr_reg = 0 to max-XX ){
    Write seed at addr_reg using port 0;
    Read inv_seed at addr_reg + XX using port 1;
}
for ( addr_reg = max-XX+1 to max-1 ) {
    Write seed at addr_reg using port 0; \
    NO READ!
}
// Repeat, writing with port 1, reading with port 0,
// BUT now writing inv_seed and reading seed.
```

Dual Port Example - down, [XX = 1 | row_size]

```
for ( addr_reg = max downto XX ){
    Write seed at addr_reg using port 0;
    Read inv_seed at addr_reg - XX using port 1;
}
for ( addr_reg = XX-1 downto 0 ) {
    Write seed at addr_reg using port 0; \
    NO READ!
}
// Repeat, writing with port 1, reading with port 0,
// BUT now writing inv_seed and reading seed.
```

Pseudocode for Register File (Separate Write and Read Ports)

The following are pseudocode examples only.

Register File Example - up, [XX = 1 | row_size]

```
for ( addr_reg = 0 to max-XX ){
    Write seed at addr_reg using write port;
    Read inv_seed at addr_reg + XX using read port;
```

```
}  
for ( addr_reg = max-XX+1 to max-1 ) {  
    Write seed at addr_reg using write port; \  
    NO READ!  
}  
// No Repeat
```

Register File Example - down, [XX = 1 | row_size]

```
for ( addr_reg = max downto XX ){  
    Write seed at addr_reg using write port;  
    Read inv_seed at addr_reg - XX using read port;  
}  
for ( addr_reg = XX-1 downto 0 ) {  
    Write seed at addr_reg using write port; \  
    NO READ!  
}  
// No Repeat
```

Retention Testing

Retention testing verifies if memory cells can retain their initial contents for a certain period of time. The time period can vary from 10ms - 80ms depending primarily on the manufacturing process and the ambient temperature during the test application.

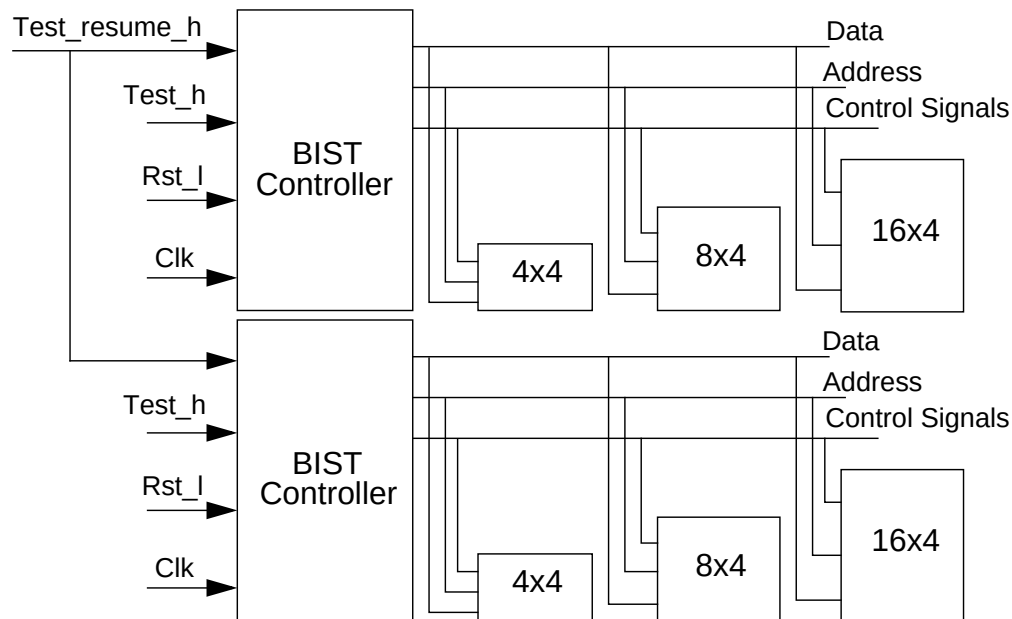
The tool's predefined retention test algorithm, RetentionCB, was introduced earlier in “[retentionCB](#)” on page 123. It is based on the Checkerboard algorithm but incurs a waiting period after each write step.

Retention Test Scheme for Multiple BIST Controllers

If a design contains multiple controllers, parallel application of the retention tests should be extended across all controllers in order to ensure a cost effective test application. Since different controllers may be testing memories of different sizes, the scheme illustrated in [Figure 4-14](#) requires complex functionality for synchronizing the different hold signals of the controllers.

[Figure 4-14](#) illustrates how the MBISTArchitect tool synchronizes multiple BIST controllers during retention tests.

Figure 4-14. Synchronized Retention Testing Across Multiple Controllers



Waiting Period

The wait period illustrated in [Table 4-2](#) on page 124 is an idle state implemented by the finite state machine of the BIST controller. Therefore, all controllers stop accessing their corresponding memories whenever they reach this idle state. When all controllers align on the idle state, the waiting period starts. When the wait period time elapses, the `test_resume_h` signal is activated (asserted) and all controllers resume test operations. This same method is used for designs that contain single BIST controllers as well. Use the computation or report generation method to derive the number of clock cycles needed before observing the retention time period. You can add deglitching registers to `test_resume_h` with the Add Signal Synchronization command.

Controlling the Retention Test Delay Time

Use the Setup Retention Cycles command to control the delay value used in the WGL and simulation testbench when waiting to activate the resume signal. You can issue this command to continue the BIST session following a retention test synchronization delay.

Retention Testing at SoC Level

When the MBISTArchitect tool inserts BIST controllers into an SoC design, each BIST controller is usually accompanied by a set of test patterns which properly initialize and run the BIST controller through its steps. After these BIST controllers have been added to the SoC design, you need to integrate these test patterns into a single SoC test set. This pattern

integration process supports both the parallel BIST and sequential BIST schemes, with the ability to combine BIST patterns in either a sequential manner or a concurrent manner. Also, the integration process is able to combine various pattern sets per instance of BIST controller, along with creating multiple pattern sets for each BIST controller.

Sequential BIST Scheme

For a sequential BIST scheme, the “non-retention/IDDQ” pattern sets are combined sequentially, while the “retention/IDDQ” pattern sets are combined concurrently. This way all retention tests happen only once for all memories in the test.

Parallel BIST Scheme

For a parallel BIST scheme, all pattern sets are combined concurrently. However all pattern sets leading up to the “retention/IDDQ” pattern sets are padded with extra patterns so that all concurrent pattern sets have the same length. This causes all “retention/IDDQ” testing to happen at the same time.

Online Algorithm Selection Capability

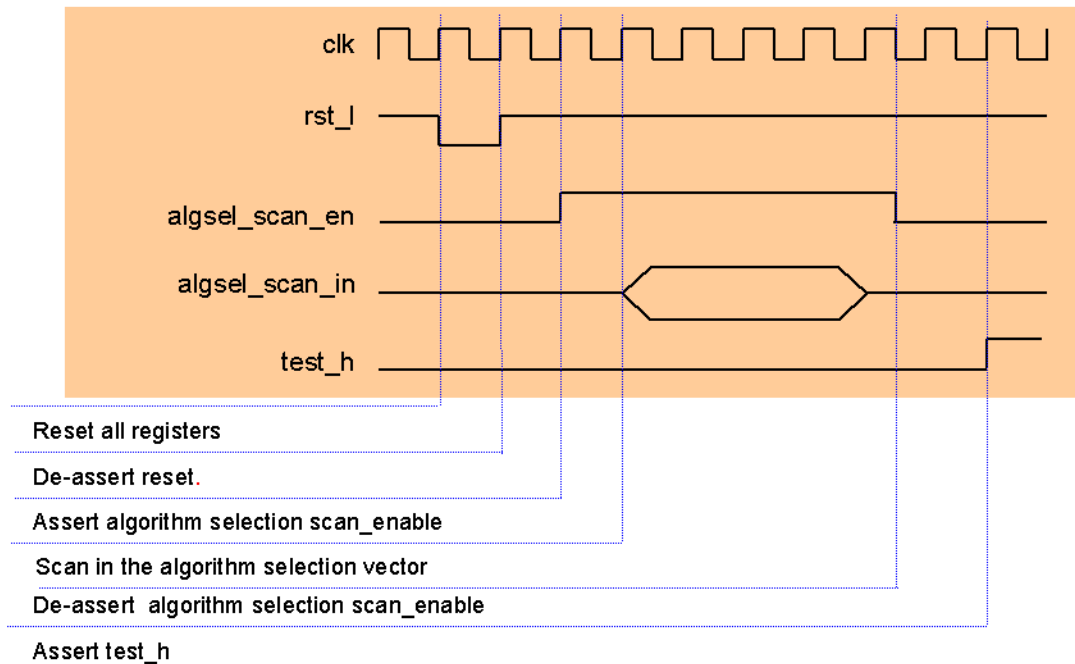
Online algorithm selection is used for runtime programmable selection of memory test algorithms from a set of hard-wired algorithms that were previously added to a memory BIST controller. The important part is that the algorithms must have been previously physically created in hardware using the Setup Algorithm Selection command.

The Setup Algorithm Selection command enables/disables support for runtime programmable online algorithm selection. When enabled, you can specify the algorithms you want to run. If no algorithm is specified, the default algorithm is March2.

Online Algorithm Selection Protocol

Figure 4-15 shows an example of the online algorithm selection protocol.

Figure 4-15. Online Algorithm Selection Protocol



Hardware Impact

Caution

Online algorithm selection increases the hardware on the chip based on the number of algorithms that are added to the BIST controller. Verify that your design will not be adversely affected by the additional on-chip hardware.

The hardware added to implement online algorithm selection depends on the type of the algorithm step sequencer implemented for the controller; for example, concurrent versus sequential, versus sequential interleaved.

Required Skip States

In general, a number of skip states will be required. Depending on the type of sequencer a minimum number of “m” skip states will be required, where “m” is the number of implemented algorithms. For a sequential un-collapsed sequential, the maximum number of skip states will be “m x n” where “n” is the number of memories tested. The skip states will add to the binary encoding size of the “tstate”, also known as the *step register variable*.

Required Shift Register

A shift register will be required to hold the skip/no-skip status of the implemented algorithms. A single bit is needed to for every algorithm, irrespective of the type of algorithm step sequencer.

Required Mux Modules

Three mux modules are required for each skip state. The mux select between skipping/not-skipping the next algorithm. Based on the skip bit value of the algorithm in the shift register, the algorithm is either executed or skipped. The three muxed variables are: the step variable, the mode variable, and the address operation variable.

Additional Hardware

You can specify the parameters of the Setup Algorithm Selection -On switch to affect the testbench simulation behavior of the MBISTArchitect tool, and the reset value of the algorithm selection register. In this case, the hardware will still be generated, even if none of the algorithms are selected, the only difference being the reset value of the selection register.

Listing the Names of the Algorithms

Use the Report Mbist Algorithm command to list out the names of the algorithms that have been selected. If the online selection is turned on, the report will have two columns: one column that displays the names of the algorithms, and another column that displays which algorithms are selected. If online selection of algorithms is turned off (using the -Off switch of the Setup Algorithm Selection command), the report will only have one column for the names of the algorithms.

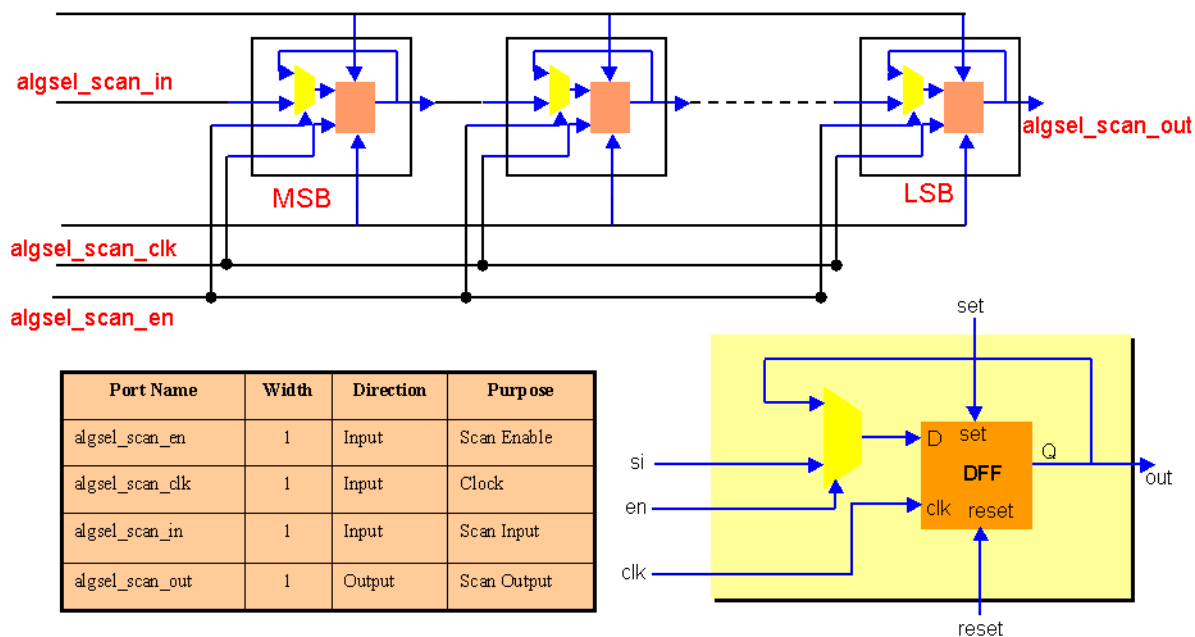
Algorithm Selection Register

You can optionally use a separate clock for the online algorithm selection register. The name of this clock is specified using the -Algsel_clk switch of the Setup Memory Clock command.

If this command and switch are not used to define a clock for online algorithm selection, no separate clock will be created, and the BIST clock will be used for loading the algorithm selection register.

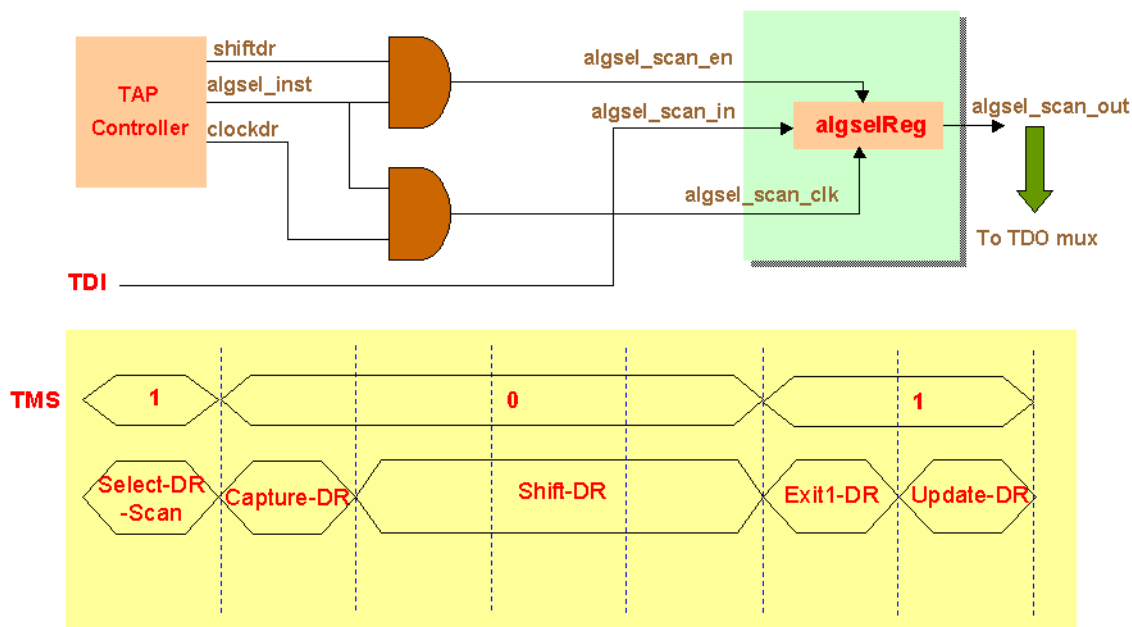
[Figure 4-16](#) shows an example of the algorithm selection registers.

Figure 4-16. Online Algorithm Selection Registers



You can optionally use a separate clock for loading the algorithm register. This is useful when integrating with JTAG as shown in Figure 4-17.

Figure 4-17. Algorithm Selection JTAG Interface



Multiport Memories

Multiport memories have a slightly different behavior with online algorithm selection. The hardware is generated on the same concept with the required skip states. The difference is in the default vectors for the testbench (and reset vector). In multiport memories, an algorithm is selected or unselected for all of the ports.

Example - Multiport Memory

The following is an example using a multiport memory. In this example the selected algorithms are: March2, March3, March1, and checkerboard.

The algorithm March2 is present on both ports, and selected to run, so it is selected to run on both ports.

The algorithm March3 is present only on port2, and it is selected to run only on port2. There will not be any hardware created that will allow March3 to run on port1.

```
add mbist algorithms 1 checkerboard march2 unique
add mbist algorithms 2 march2 march3 col_march1 march1
setup algorithm selection -on march2 march3 march1 checkerboard
report mbist algorithms
```

Report Algorithms:

Tests for Port Number 1:

Algorithm	Selected
checkerboard	Yes
march2	Yes
unique	No

Tests for Port Number 2:

Algorithm	Selected
march2	Yes
march3	Yes
col_march1	No
march1	Yes

Example - Online Algorithm Selection Step-By-Step

The following is a step-by-step example for setting up an online algorithm selection.

1. Add all of the memory models that you want to generate a controller for.
2. Optionally, specify all algorithms to be applied. If no algorithms are specified, the tool implements the March2 algorithm as the default algorithm.
3. Use the Setup Algorithm Selection command to set up support for runtime programmable online algorithm selection.

Algorithm selection on:

Turning the algorithm selection on (using the -On switch) the tool will add additional hardware to the chip to allow runtime algorithm selection. By selecting the all/subset of the implemented algorithms to this command, you can specify which of the implemented algorithms to run, and which to skip. The tool will generate a testbench which will only run the specified algorithms.

Algorithm selection off:

Turning the algorithm selection off (using the -Off switch) you will maintain the default behavior of the tool, that is, no additional hardware is added to the chip to implement runtime algorithm selection.

4. You can also see if the algorithm selection is turned on or off, and which algorithms are selected to run by using the Report Mbist Algorithm command.
5. Specify other controller parameters.
6. Use the Run command for BIST generation, and save the generated RTL.

The generated BIST controller will have the necessary hardware to run any combination of algorithms, or none of the algorithms. The generated testbench will run the specified algorithms only. The newly added interface signals to the controller are brought up to the connection module interface. Test time is updated to reflect the new skip states added to the controller algorithm step sequencer. The report algorithm step command will now reflect the newly added skip states of the sequencer.

Controller Interface Signals

The following signals and pin names are part of the controller to facilitate the online algorithm selection. [Table 4-3](#) lists the interface signal names, whether they are inputs or outputs, the signal width, and a brief description. A detailed description of each signal follows the table.

Table 4-3. Online Algorithm Interface Signals

Pin Name	Direction	Width	Description
algsel_scan_in	Input	1	Scan input
algsel_scan_en	Input	1	Scan enable
algsel_scan_out	Output	1	Scan output
algsel_scan_clk	Input	1	Clock

- **algsel_scan_in**
The scan input of the algorithm selection shift register added to the BIST controller. The first bit shifter is for the first algorithm run by the sequencer, and the last bit shifted in is for the last algorithm run by the sequencer.
- **algsel_scan_en**
The scan enable for the algorithm selection shift register. Before the BIST run is started,

this pin is activated and the desired algorithms to run are shifted. The pin is then deactivated.

The number of bits shifted in reflects the number of algorithms implemented. A “1” is shifted for an algorithm that will be run, and a “0” is shifted in for an algorithm to be skipped. The shift must be done, that is, `alg_scan_en` must be activated only while `test_h` is not activated, that is, before the test starts and while reset is inactive.

- `algsel_scan_out`
The scan output of the algorithm shift register. This signal shifts out a single bit for each shift in operation. The shifted value is bit zero of the shift register.
- `algsel_scan_clk`
The clock of the algorithm selection shift register. This clock pin will not be created by default. You must specify whether a separate clock is needed to load the algorithm selection register. See: “[Algorithm Selection Register](#)” on page 137.

You can change the pin names using the [Set Pin Name](#) command.

Refer to the Command Dictionary section of the *MBISTArchitect Reference Manual* for specific details on the [Add Mbist Algorithms](#), [Report Mbist Algorithms](#), [Setup Algorithm Selection](#), [Setup Memory Clock](#), and the [Set Pin Name](#) commands.

Comparator Test

For writable memories (RAMs), the tool employs a comparator during the read cycles of your test algorithms to determine whether the actual data read from the memory matches the expected data. The tool provides the ability to test the comparator before running the BIST. You can specify this test with the following command.

Set Comparator Test -on

When you specify comparator test, the tool prepends some new states to the BIST controller’s finite state machine. In these states the BIST controller writes a simple data background to address zero of all memories, then reads it back twice. On the first read it intentionally supplies wrong “expected data” to the comparator, on the second read it supplies the normal expected data. The comparator should return false the first time and true the second time.

Comparator test is a quick way to pre-validate the results of BIST. When you enable the comparator test, it always precedes all other tests.

Reporting Algorithms

Algorithm Clock Cycles - Determining Test Time

The Report Algorithm Steps command causes the tool to generate a report describing the number of clock cycles used by each state of the BIST controller.

When using only a single memory, or concurrent testing, the output does not include the memory field. Likewise, when using only a single port, the output does not include the port field.

Reporting Algorithm Steps

For an example on reporting the algorithm steps, see the [Report Algorithm Steps](#) command in the *MBISTArchitect Reference Manual*.

Chapter 5

User-Defined Algorithms

A pre-defined set of algorithms is included with your MBISTArchitect software installation. While the pre-defined algorithm set detects a variety of faults, it may not cover your specific fault detection needs. However, you can create additional algorithms for your designs. These user-defined algorithms (UDAs) are created using the syntax outlined in this chapter.

Note



The UDA functionality does not support march algorithms having distinct multiple port activities nor non-march-type algorithms.

The pre-defined algorithm file is available for your reference and is located at `<Tessent_Tree_Path>/lib/predefined_algorithms.mba`. Details on each of the pre-defined algorithms can be found in “[Algorithms](#)” on page 111.

Note



The pre-defined algorithm file cannot be modified. Do not attempt to include additional UDAs in this file.

Adding UDAs to the Design

The pre-defined algorithms are automatically available for use with the Add Mbist Algorithms and Setup Mbist Algorithms commands. For user-defined algorithms, you must first load the algorithm file before the algorithms are available for the Add Mbist Algorithms command. Use the following procedure to add user-defined algorithms to your design:

1. Load the file(s) containing the user-defined algorithms using the [Load Algorithms](#) command. MBISTArchitect checks each file for syntax errors when loading.

load algorithms *file_name1 file_name2*

2. Generate a list of available algorithms using the [Report Algorithms](#) command. This report will contain both pre-defined and user-defined algorithms.

report algorithms

3. Add the desired algorithms to specified ports using the [Add Mbist Algorithms](#) command. You can add a mix of pre-defined and user-defined algorithms.

add mbist algorithms 1 march2 my_algorithm

4. Verify the added algorithms using the [Report Mbist Algorithms](#) command.

report mbist algorithms

User-Defined Algorithm Language

User-defined algorithms can be created using any text editor. The following are the 3 basic elements:

- **Test Definition** — Defines a set of repetitions that create an algorithm. Each algorithm has a single test definition.
- **Repetition Definition** — Defines a set of steps to perform with a common data value. Each unique repetition is defined once and can be referenced as many times as needed.
- **Step Definition** — Defines an operation to perform on each address in a specified sequence. Each unique memory operation is defined once and can be referenced as many times as needed.

Figure 5-1. UDA Language Syntax

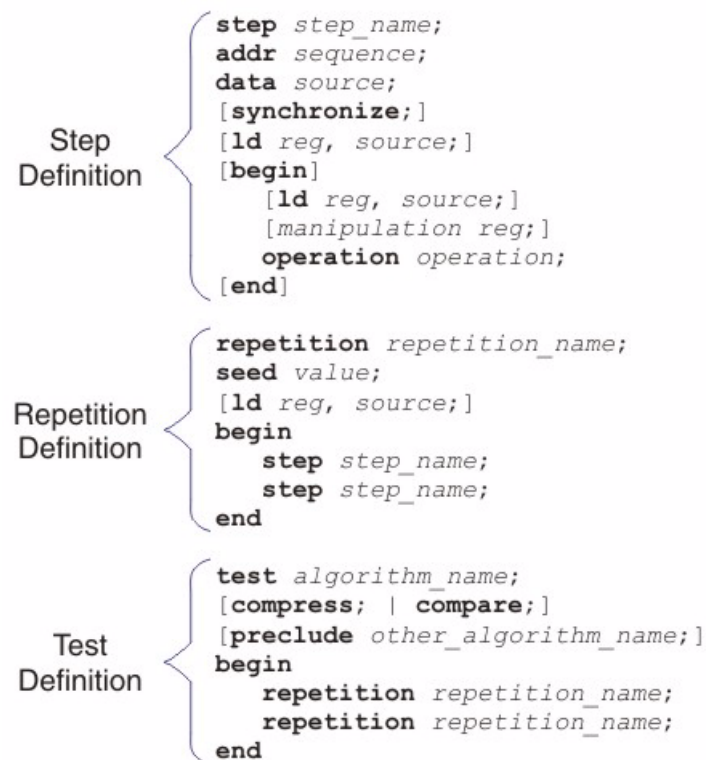


Figure 5-1 shows the syntax for the UDA language and uses the following conventions:

- **Bold** — Indicates a keyword. Enter the keywords exactly as shown.
- *Italic* — Indicates a user-supplied argument. Replace the italicized string with the appropriate value.

- Square Brackets [] — Indicates an optional clause. Do not enter the brackets.
- Vertical Bar | — Indicates mutually exclusive arguments. Do not enter the vertical bar.
- Double Slashes // — Indicates a comment. The tool ignores any text between the double slashes and the end of line character.

The UDA Language has the following rules:

1. Each line of code must be terminated by a semicolon, except for the “begin” and “end” lines.
2. The UDA language is case insensitive; an algorithm name “March” is the same as “march.”
3. To prevent unintentional overwrites, each algorithm name must be unique within a file and between all files loaded with the [Load Algorithms](#) command.
4. UDAs cannot use pre-defined algorithm names.
5. Each step must have a unique name within a file.
6. Each repetition must have a unique name within a file.
7. Each algorithm must have exactly one test definition.
8. Repetitions can only be referenced by a test definition.
9. Steps can only be referenced by a repetition definition.
10. An element must be defined in the same file prior to being referenced by another element. For example, a step must be defined prior to being referenced by a repetition. You cannot reference steps or repetitions located in another file.

Test Definition

The test definition forms the root of the algorithm. There can be multiple test definitions in a single file, but only one test definition per algorithm. A test definition has the following format:

Figure 5-2. Test Definition Usage

```

Declaration { test algorithm_name;
Preamble   { [compress; | compare;]
             [preclude other_algorithm_name;]
Statement  { begin
Sequence   { repetition repetition_name;
             repetition repetition_name;
             end

```

The test declaration defines the name of the algorithm and marks the beginning of the test definition. The test preamble contains optional settings that control the use of the defined

algorithm. The test statement sequence only contains references to repetitions. All repetitions referenced by the test definition must be defined prior to the test definition.

Arguments

- **test** *algorithm_name*;

A required keyword and string pair that specifies the name of the algorithm and identifies the block as a test definition. The *algorithm_name* should be unique within the algorithm file and between all loaded files, including the pre-defined algorithm file.

Note



If you load two files that have different definitions with the same algorithm name, the last added algorithm definition overwrites any previously loaded definition. However, you cannot overwrite a pre-defined algorithm definition. If you load a file with an algorithm name that is the same as one of the pre-defined algorithms, the new definition is ignored.

- **compress**;

An optional keyword that restricts the use of the algorithm so that it can only be used by controllers that have compressors. You cannot use both **compress** and **compare** in the same test definition.

- **compare**;

An optional keyword that restricts the use of the algorithm so that it can only be used by controllers that have comparators. You cannot use both **compress** and **compare** in the same test definition.

- **preclude** *other_algorithm_name*;

An optional keyword and string pair that restricts the use of the current algorithm with respect to another algorithm. Use this line to define algorithm incompatibilities. The algorithm specified with this keyword cannot be used in conjunction with the current algorithm in the same controller. Adding both algorithms to the same controller will produce an error.

- **begin end**

A required keyword pair that specifies the boundaries of the test statement sequence. All references to repetitions must be between these keywords. You cannot reference steps within this section. The **begin** and **end** keywords are not required if there is only one repetition reference in the test statement sequence.

Do not place a semicolon after the **begin** or **end** keyword lines.

- **repetition** *repetition_name*;

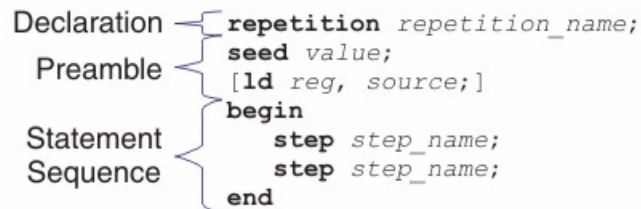
A required, repeatable keyword and string pair that references a defined repetition. Each reference should start on a new line and terminate with a semicolon. All referenced

repetitions must be defined in the file prior to the test definition. For more information, see “[Repetition Definition](#).”

Repetition Definition

The repetition definition defines a set of steps to perform with a common data value. A repetition must be defined before it is referenced by a test. There can be many repetition definitions in a single file, and a single repetition can be referenced multiple times by multiple test definitions. The repetition definition has the following format:

Figure 5-3. Repetition Definition Usage



```

Declaration { repetition repetition_name;
Preamble   { seed value;
              [ld reg, source;]
Statement Sequence { begin
                    step step_name;
                    step step_name;
                    end
  
```

The repetition declaration defines the name of the repetition and marks the beginning of the repetition definition. The repetition preamble specifies the data values to use for the referenced steps. The repetition statement sequence only contains references to steps. All referenced steps must be defined prior to the repetition.

Arguments

- **repetition** *repetition_name*;

A required keyword and string pair that specifies the name of the repetition. The repetition name must be unique within the file.

- **seed** *value*;

A required keyword and string pair that specifies the default data value. This value is used by all steps called by this repetition, unless a different value is specified by the step. Numeric values used within the user-defined algorithm language can be in a variety of bases. The default, with no prefix, is decimal. The following are valid number prefixes:

- **'b** — Binary. For example, 35 = 'b100011.
- **'d** — Decimal. For example, 35 = 'd35. For decimal, the prefix is not required.
- **'h** — Hexadecimal. For example, 35 = 'h23.
- **'o** — Octal. For example, 35 = 'o43.

The optional size preamble is a decimal number, preceding the base specifier, that specifies how many bits should be used to represent the number; therefore, the binary

representation is truncated to the specified number of bits. For example, ‘h23 represents 35, but 4’h23 represents the number 3.

Note

The seed clause is not required if all steps referenced by the repetition do not use a data source requiring a seed. For example, if all steps use a checkerboard/invcheckerboard or addr/invaddr data source, then the seed clause is not required.

- **ld** *reg, source*;

An optional, repeatable keyword and literal set that loads the data register sources. For more information on using data registers, see “UDAs with Data Registers” on page 158.

- **begin end**

A required keyword pair that specifies the boundaries of the repetition statement sequence. All references to steps must be between these keywords. You cannot reference other repetitions. The **begin** and **end** keywords are not required if there is only one step reference in the repetition statement sequence.

Do not place a semicolon after the **begin** or **end** keyword lines.

- **step** *step_name*;

A required, repeatable keyword and string pair that references a defined step. Each step reference should start on a new line and terminate with a semicolon. All referenced steps must be defined in the file prior to the repetition definition. For more information, see “Step Definition.”

Step Definition

The step definitions make up the body of the algorithm. The operation defined in a step is applied to each address in the specified sequence. There can be many steps defined in the file, and a single step can be referenced multiple times by multiple repetitions. A step must be defined before it is referenced by a repetition. The step definition has the following format:

Figure 5-4. Step Definition Usage

Declaration	{	step <i>step_name</i> ;
		addr <i>sequence</i> ;
Preamble	{	data <i>source</i> ;
		[synchronize];
		[ld <i>reg, source</i>];
Statement Sequence	{	[begin]
		[ld <i>reg, source</i>];
		[<i>manipulation reg</i>];
		operation <i>operation</i> ;
		[end]

The step declaration defines the name of the step and marks the beginning of the step definition. The step preamble contains the setup information including the address sequence and data sources to use for the step. The step statement sequence contains the operation to perform at each address in the defined address sequence.

Data register loading and manipulations can be placed in the preamble, statement sequence, or both. In general, whatever is placed in the preamble happens once per step and whatever is placed in the statement sequence happens once per address. For more information see “[UDAs with Data Registers](#)” on page 158.

Arguments

- **step** *step_name*;

A required keyword and string pair that specifies the name of the step. The step name must be unique within the file.

- **addr** *sequence*;

A required keyword and string pair that defines the sequence of addresses to which the step operation will apply. There are multiple formats available for defining an address sequence. For more information, see “[Address Sequences](#)” on page 150. By default, the step will traverse all addresses starting with address 0 and incrementing by 1.

- **data** *source*;

A required keyword and literal pair that specifies the data values used by the step operation. The available *source* values are:

- **seed** | **invseed** — Uses the direct or binary inverse data seed value as defined in the referencing repetition.
- **addr** | **invaddr** — Computes and uses the direct or binary inverse unique address data value. For memories with an address width larger than the data, the data value will be truncated. For memories with an address width smaller than the data, the data value will be concatenated with a repeating value.
- **background** | **invbackground** — Uses the direct or binary inverse data background values defined using the [Add Data Backgrounds](#) command. If multiple data backgrounds are added, the complete test algorithm is repeated once for each background. If no data backgrounds are defined, then the controller uses the default seed value from the referencing repetition.
- **checkerboard** | **invcheckerboard** — Uses a direct or binary inverse checkerboard data pattern. The exact type and value depends on how the memory is modeled. For more information on using checkerboard data, see “[checkerBoard \(topChecker\)](#)” on page 122.

- **RWr1r2** — Uses the loaded data register values. RWr1r2 is a more complex data source, where the operation data does not use the direct and inverted values. For more information, see “UDAs with Data Registers” on page 158.
- **synchronize;**
An optional keyword that causes the generated BIST controller to go into an inactive state at the end of the step. This mechanism can also be used for retention testing.
- **ld reg, source;**
An optional, repeatable keyword and literal set that loads the register data sources. For more information on using data registers, see “UDAs with Data Registers” on page 158.
- *manipulation reg;*
An optional, repeatable literal set that manipulates the loaded register data. These clauses are only valid if you have loaded data registers using the **ld** keyword. For more information on using data registers, see “UDAs with Data Registers” on page 158.
- **begin end**
An optional keyword pair that specifies the boundaries of the step statement sequence. These keywords are only required if you want to load or manipulate data registers within the step statement sequence (for each address in the sequence).
Do not place a semicolon after the **begin** or **end** keyword lines.
- **operation operation;**
A required keyword and string pair that defines the activity performed at each address in the address sequence. Only one operation is allowed per step. For more information on allowed operations, see “Operations” on page 154.

Address Sequences

The **addr** clause of the step definition describes how the BIST controller goes from one address to another address while processing the step. Any linear address sequence is possible. A linear address sequence is a sequence where the next address is deterministically derived from the current address.

The address sequence can be defined in one of the two following formats.

Format 1

addr *end, end, direction, increment* [, *portReadOffset*];

- *end, end*

A required literal and/or integer pair that specifies the address range for the address sequence. It is important to remember that when dealing with complex address range

sequences, the last value of the sequence will not necessarily be the last value of the address range. Available values are:

- **min** — Specifies the first available address as defined in the memory model.
- **max** — Specifies the last available address as defined in the memory model.
- *integer* — Specifies a particular address.
- *direction*

A required literal that specifies the direction of the address sequence. Valid *directions* are:

- **up** — Increments through the address range starting with the first *end* point.
- **down** — Reverses the address sequence. The sequence is calculated by determining the **up** sequence, then reversing the order. The sequence will start with the last value of the **up** sequence and end with the first *end* point.
- *increment*

A required literal or integer that specifies the distance between addresses in the sequence. Available *increment* values are:

- **jump** — Uses the increment value from `addr_inc` as defined in the memory model.

Jump addressing, useful for column march algorithms, can result in a BIST controller implementation that uses a number of parallel address counters. If all the memories tested by the controller have a common jump value, then a single address counter is used. If some of the memories have a different jump value, then the BIST controller uses multiple address registers.
- *integer* — Specifies the value added to the address register after each operation.
- `portReadOffset`

An optional literal or integer that specifies the address offset between the read and write activities in the W_R operation. This argument is only used for steps using the W_R operation (for example, port-isolation algorithms) where the read address is offset from the write address.

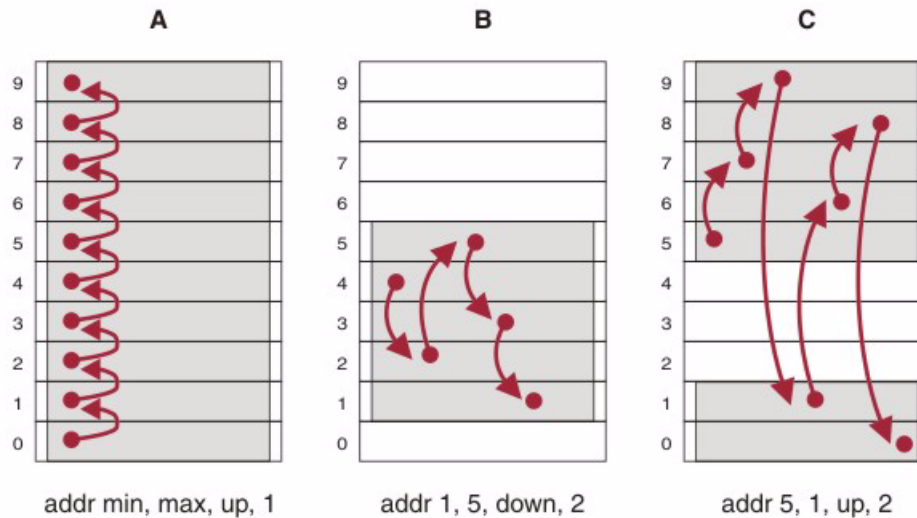
If the `portReadOffset` argument is not supplied, then the tool will assume **row_size**.

- **row_size** — Selects the offset value for each memory by referencing the *addr_inc* value in the memory model.
- *integer* — Specifies a particular offset value.

Figure 5-5 illustrates several address sequences defined using Format 1. The grey area marks the address range specified by the endpoints and the red arrows show the sequence of addresses. Example A is a simple sequence across the entire address space of the memory with an increment of 1. Example B uses a subset of the full address space of the memory with an

increment of 2. Example C also uses an increment of 2, but it has a different address range. The sequence syntax of Examples B and C may appear similar, but you can see from the illustration that the actual sequences are very different.

Figure 5-5. Address Sequence Examples



Format 2

addr function *name*, *start*, *stop* [, *count*];

- **function** *name*

A required keyword and string pair that defines the address sequence by referencing a Verilog function. The function must be defined in a Verilog file, which is loaded using the [Add Verilog Include](#) command.

- *start*, *stop*

A required literal and/or integer pair that specifies the start and stop address endpoints for the address sequence. It is important to remember that when dealing with complex address range sequences, the last value of the sequence will not necessarily be the last value of the address range. Available values are:

- **min** — Specifies the first available address as defined in the memory model.
- **max** — Specifies the last available address as defined in the memory model.
- *integer* — Specifies a particular address.

- *count*

An optional integer that defines the number of addresses that will be contained within this address sequence. If *count* is omitted, the default number of addresses will be

computed using the formula: $count = stop - start + 1$, using modulo arithmetic based on the size of the address space for the controller.

Example - Address Sequencing

The following is an example UDA named `my_uda` that describes the address sequencing.

```
step w_even_addrUp;
addr function odd_even_address, 0 , 14, 8;
data checkerboard;
operation w;

step w_odd_addrUp;
addr function odd_even_address, 1 , 15, 8;
data seed;
operation w;

step r_even_addrUp;
addr function odd_even_address, 0 , 14, 8;
data checkerboard;
operation r;

step r_odd_addrUp;
addr function odd_even_address, 1 , 15, 8;
data seed;
operation r;

repetition odd_even;
seed 0;
begin
    step w_even_addrUp;
    step w_odd_addrUp;
    step r_even_addrUp;
    step r_odd_addrUp;
end

test odd_even;
    repetition odd_even;
```

Operations

When a step is executed by the BIST controller, the specified operation is performed once at each address in the address sequence. An operation contains a string of **R** and **W** characters, which represent memory read and write activities. Do not separate the **R** and **W** characters with spaces. For example, a read-write-read activity will be represented as:

```
operation rwr;
```

Simple operations (having a single read or write activity) are taken from the description of the read or write cycle of the memory port in the memory model. More complex operations (having a combination of read and write activities) are synthesized by concatenating the individual cycle definitions from the memory model into a single definition in the specified operation order.

For port-isolation testing, there is also the **W_R** activity, which writes to one address and reads from another address at a specified offset distance. This activity should not be confused with the

shifted reads and writes described in “[Looped Operations](#)” on page 156. For more information on the W_R activity, see “[Port Isolation User-Defined Algorithm](#)” on page 170.

Operation Data

Each step of a UDA is performed independently. That is, the controller does not retain the data value used by the previous step. It is your responsibility to tell the controller what data is relevant to the current step. You tell the controller what value to expect or write using the **data** keyword for the step. The controller uses either the direct value of the specified data or the binary inverse of that data, depending on the activity as outlined below.

For simple operations:

- **Single W** — Write the specified data value.
- **Single R** — Expect the specified data value.

For complex operations:

- **W** — Write the specified data value for the first write activity and invert the data for each successive write activity. For example, if you have three write activities in an operation, the controller will write the direct data for the first write, inverse data for the second write, and direct data for the third write.
- **R** — Expect the data value used for the most recent write activity. For multiple read activities following a write activity, the controller will expect the same value for each read. For example, a WRR operation will be interpreted as: write direct data, expect direct data, and expect direct data.

Caution



If your complex operation starts with a read activity, the controller will expect the inverse data of the first write operation for that read, which is the inverse of the data specified by the step. You must make sure the last write activity from the previous step writes this value. Otherwise, your algorithm will produce errors.

- **W_R** — Write the specified data to the current address and expect the inverted data at the offset address. If your step contains this operation, you must also use [Format 1](#) for the address sequence, specifying the portReadOffset.

In most cases, the operation **w_r** cannot be mixed in the same step with any other operations. Steps preceding the **w_r** step must have left memory in its correct state. This will normally mean filled with `inv_seed`.

Example - Operation Data

The following example uses simple and complex operations. For this example, assume the data is four bits wide. Step1 increments through the memory writing the seed value to each address.

Step2 has a complex operation beginning with a read activity. Because the specified data is invseed for step2, the first read will expect the seed value, which was written by the previous step.

```
step step1;
addr min, max, up, 1;
data seed;
operation w;

step step2;
addr min, max, up, 1;
data invseed;
operation rwrrwrr;

repetition repetition1;
seed 0;
begin
    step step1;
    step step2;
end

test my_test;
begin
    repetition repetition1;
end
```

Table 5-1 illustrates the data written to the memory and expected by the controller for the above algorithm. Note the seed value was defined as 0 in the repetition.

Table 5-1. Operation Data for “my_test” Algorithm

	Step1 (seed)	Step2 (invseed)						
Activity	W	R	W	R	R	W	W	R
Data (written or expected)	0000	0000	1111	1111	1111	0000	1111	1111

Looped Operations

Looping is used primarily to test the address decoder. The [addressdecoder_bg0](#) and [addressdecoder_bg1](#) algorithms use this functionality. The address sequence, defined with the **addr** keyword, is called the *base address*. For each base address, the loop is performed on a set of neighboring addresses. The neighboring addressees are called *shifted addresses*.

For each base address, the number of shifted addresses equals the width of the address decoder; one shifted address to test each possible failing line in the decoder. If the address decoder is four bits wide, there are four shifted addresses for each base address.

The location of the shifted addresses depends on the size of the address decoder and the location of the base address. The relation between the base address and the neighboring address is as follows:

$n\text{Address} = f(\text{baseAddress}, N)$; for the N^{th} neighbor of the base address.

For PMOS open address decoder this is an XOR operator and is specified as:

$n\text{Address} = \text{xor}(\text{baseAddress}, 2^N)$;

In other words, if you have a four bit address decoder, your base address will be XORed with 0001, 0010, 0100, and 1000 to simulate each failed line in the decoder for that base address. The following table illustrates the shifted address sequence for the base address of 1010.

Table 5-2. Shifted Address Locations

	Base	Shift 0	Shift 1	Shift 2	Shift 3
2^N		0001	0010	0100	1000
Address	1010	1011	1000	1110	0010

There are two additional operation activities to allow you to read from and write to the shifted addresses. The **S_R** activity reads from the shifted address, and the **S_W** activity writes to the shifted address. The shifted activities must be within the boundary of the loop, which is defined using parentheses () in the operation string.

For example, the following operation performs a write at the base address; then, for each shifted address, it writes to the shifted address and reads from the base address:

$w(s_wr)$

Looped Operation Data

The controller uses the following conventions, in addition to those specified in “[Operation Data](#),” to determine what data is written to and expected from the memory.

- **S_W** — Writes an inverted value of the most recent **W** activity at the shifted address. The inversion for the shifted write does not affect the inversion pattern of successive **W** activities.
- **S_R** — Expects the inverted value of the most recent **W** activity at the shifted address.

For example, the operation “ $w(s_rs_wr)w$ ” is interpreted as:

1. Write the direct data to the base address.
2. For each shifted address:
 - a. Expect inverted data to the shifted address.
 - b. Write inverted data to the shifted address.
 - c. Expect the direct data from the base address.
3. Write the inverted data to the base address.

Example - Looped Operation Data

The following example is a snippet of an address decoder algorithm:

```
step wxrStepUp;  
  addr min,max,up,1;  
  data seed;  
  operation w(s_wr)w;
```

The operation “w(s_wr)w” is interpreted as:

1. Write the seed value at the base address.
2. For each shifted address:
 - a. Write the inverse seed value to the shifted address.
 - b. Expect the seed value from the base address.
3. Write the inverse seed value to the base address.

UDAs with Data Registers

Data registers can be used as data sources in a UDA instead of the usual data/inverse data sources. Using data registers allows you to specify unique values to test for more complicated faults, such as [Idempotent Coupling Faults](#).

There are three clauses where you can specify data register sources:

1. **data source;**

To use data registers for the step operations, you must specify the **RWr1r2** source for the data clause in the step definition. For example:

```
data rwr1r2;
```

When you select this data source, MBISTArchitect creates two registers, r1 and r2, and the controller will use them as a pair, similar to how seed and invseed are used. Unlike seed and invseed, the contents of r1 and r2 are independently loaded and not necessarily inverses of each other. The controller writes and expects the data using the following conventions.

For simple operations:

- **Single W** — Write the data value stored in r1.
- **Single R** — Expect the data value stored in r1.

For complex operations:

- **W** — Write the r1 data value for the first write activity and alternate between r1 and r2 values for each successive write activity. For example, if you have three write

activities in an operation, the controller will write the r1 data for the first write, r2 data for the second write, and r1 data for the third write.

- **R** — Expect the data value used for the most recent write activity. For multiple read activities following a write activity, the controller will expect the same value for each read. For example, a WRR operation will be interpreted as: write r1 data, expect r1 data, and expect r1 data.

Caution



If your complex operation starts with a read activity, the controller will expect the r2 data for that read. You must make sure the last write activity from the previous step writes this value. Otherwise, your algorithm will produce errors.

Table 5-3 shows the data registers used for the given operations.

Table 5-3. Data Registers Used for Operations

Operation	Registers
R	R(r1)
W	W(r1)
RW	R(r2) W(r1)
WR	W(r1) R(r1)
WW	W(r1) W(r2)
RWR	R(r2) W(r1) R(r1)
RWRWR	R(r2) W(r1) R(r1) W(r2) R(r2)

2. **ld** *reg, source*;

You must initialize both r1 and r2 in your algorithm. The registers can be initialized as part of the repetition definition, step definition, or both. Each register can be initialized in a separate location. For example, you can load r1 in the repetition definition and load r2 in the step definition.

Note



Loading a data register in the step definition overwrites the value, if any, specified by the referencing repetition or previous step. If a step overwrites the register value given in the repetition, the register value will remain changed for all subsequent steps, unless it is changed again by another step or repetition.

For a repetition definition, you can only load the registers in the preamble.

For a step definition, you can load the registers in the preamble, step statement sequence, or both. When in the preamble, the register will load once for the entire

address sequence. When in the step statement sequence, the register will load once for each address in the sequence.

Note

If you want to load the registers in the step statement sequence (for each address), you must include the **begin** and **end** keywords in the step definition to mark the boundary of the step statement sequence.

Available *reg* literals are:

- **r1** — Loads the specified source into r1.
- **r2** — Loads the specified source into r2.

Available register *source* literals are:

- **addr** | **invaddr** — Computes and uses the direct or binary inverse unique address data value. For memories with an address width larger than the data, the data value will be truncated. For memories with an address width smaller than the data, the data value will be concatenated with a repeating value.
- **background** | **invbackground** — Uses the direct or binary inverse data background values defined using the [Add Data Backgrounds](#) command. If multiple data backgrounds are added, the complete test algorithm is repeated once for each background. If no data backgrounds are defined, then the controller uses the default seed value from the referencing repetition.
- **seed** | **invseed** — Uses the direct or binary inverse data seed value as defined in the referencing repetition.

The following snippet shows how to initialize the data registers in a repetition definition.

```
step step1;
addr min, max, up, 1;
data rwrlr2;
operation ww; // W(r1) W(r2)

step step2;
addr min, max, up, 1;
data rwrlr2;
operation rwr; // R(r2) W(r1) R(r1)

repetition my_rep;
seed 0;
ld r1, seed;
ld r2, addr;
begin
    step step1;
    step step2;
end
```

3. *manipulation reg;*

After initializing the data registers, you can optionally manipulate the register data. Manipulation clauses are placed in the step statement sequence of the step definition. Placing the manipulation clauses in the step statement sequence means that the data is modified at each address in the sequence. You must include the **begin** and **end** keywords when using the manipulation clauses.

Note



If a register is manipulated, the register value will remain changed for all subsequent steps, unless it is modified or initialized again by another step or repetition.

Available *manipulation* literals are:

- **lsl** — Logical shift to the left. (1110 -> 1100)
- **lsr** — Logical shift to the right. (0111 -> 0011)
- **rol** — Rotation to the left. (1110 -> 1101)
- **ror** — Rotation to the right. (0111 -> 1011)

Available *reg* literals are:

- **r1** — Manipulates the data stored in r1.
- **r2** — Manipulates the data stored in r2.

The following step definition shows a register data manipulation that occurs at each address in the sequence:

```
step step1;
addr min, max, up, 1;
data rwr1r2;
ld r1, addr;
ld r2, invaddr;
begin
    rol r1;
    operation wr;
end
```

Example - UDA Using RWr1r2 to Detect Intra-Word Faults

The following example UDA uses data register syntax to support algorithms targeting intra-word idempotent coupling faults (faults occurring due to coupling between bits inside a word). One of the primary requirements of this algorithm is to have multiple data backgrounds in a single test step across the entire address space. This is possible using multiple data registers in the algorithm.

Note



This is just an example to show how multiple data backgrounds can be used in a single test step. The following algorithm may not apply to your design.

This example assumes the dofile contains “add data background x3333”. The x3333 value is referenced using the 'background' source, with the inverse as 'invBackground'. This algorithm also defines a seed value, referenced using the 'seed' source, with the inverse as 'invSeed'. The background and seed will not be specified as data sources; they will only be arguments for the 'ld' keyword. The data source here is RWr1r2, and it can take on a variety of values.

```
// intrawordCoupling.uda
//1. up - (w0101)
//2. down - (r0101, w1010)
//3. down - (r1010, w0011)
//4. down - (r0011, w1100)
//5. up - (r1100)

step write5555;
  addr min, max, up, 1;
  data seed;
  operation w;          // write seed 'h5555

step read5555writeAAAA;
  addr min, max, down, 1;
  data invSeed;
  operation rw;         // read expects seed 'h5555; write is invSeed 'hAAAA

step readAAAAwrite3333;
  addr min, max, down, 1;
  data rwrlr2;
  ld r1, background;    // 'h3333
  ld r2, invSeed;       // 'hAAAA
  operation rw;         // read expects r2; write is r1

step read3333writeCCCC;
  addr min, max, down, 1;
  data rwrlr2;
  ld r1, invBackground; // 'hCCCC
  ld r2, background;    // 'h3333
  operation rw;         // read expects r2; write is r1

step readCCCC;
  addr min, max, up, 1;
  data invBackground;
  operation r;          // read expects invBackground 'hcccc
```

```

repetition dataRegisterRep;
  seed 'h5555;
begin
  step write5555;
  step read5555writeAAAA;
  step readAAAAwrite3333;
  step read3333writeCCCC;
  step readCCCC;
end

test dataRegisterTest;
  repetition dataRegisterRep;

```

Example - Multiple Writes Using Same Data Background

The following example illustrates how data registers are useful for algorithms that need successive write activities without changing the data background in a single operation. As mentioned in “[Operation Data](#)” on page 155, the controller assumes that the data background is inverted between every successive **W** operation. When using data registers, this “inversion” means switching from r1 to r2. An algorithm with multiple write operations within the same step can be described as follows. Assume the seed is defined as 'b0101.

The following UDA uses the data registers to describe consecutive writes of the same data in a single test step.

```

// bang_test.uda
//1. up - (w0101, w1010, w0101, w1010)
//2. up - (r1010)
//3. down - (w0101, w0101, w0101, w0101)
//4. down - (r0101)

step btNormalInvertedWrites;
  addr min, max, up, 1;
  data seed;
  operation wwww;          // write seed, invSeed, seed, invSeed

step btReadInvSeed;
  addr min, max, up, 1;
  data invSeed;
  operation r;              // read invSeed

step btConsecutiveNoninvertedWrites;
  addr min, max, up, 1;
  data rwrlr2;
  ld r1, seed;
  ld r2, seed;
  operation wwww;          // write seed, seed, seed, seed

```


Setting Up The Write Enable Mask UDA Part

The keyword 'mask' is used in the 'data' source description part of the test algorithm step to denote that the step activates write enable masking.

Note



The UDA for the Write Mask algorithm can only have the operations of r,w and wr. Also, the data can only use seed or invseed. You cannot use data backgrounds with control backgrounds.

Example - Defining the Write Enable Mask Steps

The following is an example of how you might define the steps. A step defined as follows.

```
step wDataInit
  addr min,max,up,1
  data seed;
  operation w;
step wDataSetUp;
  addr min,max,up ,1;
  data seed, mask;
  operation w;
step rDataSetUp;
  addr min,max,up ,1;
  data seed, mask;
  operation r;
```

In the previous two step definitions, the value of write enable will be all 1's for step wDataInit. The data field of step wDataSetUp contains the keyword mask. This means that the controller can assign write enable values to this step as defined by the Add Control Background command. The memory will be written by activating the write enable as defined by the Add Control Background command.

The next step is rDataSetUp where the operation defined is "r". For an algorithm (like March3) the expected value will be the background written. In this case the expected value needs to be calculated based on the write enable and the background.

Consider the following definitions.

Table 5-4. Steps for a Proposed Algorithm for the Example

step	addr	data	operation
wAddrLow	min,3,up,1;	seed,mask	w
wAddrHigh	max - 3,max,up,1	seed,mask	w
rAddrLow	min,3,up,1	seed,mask	r
rAddrHigh	max - 3,max,up,1	seed,mask	r

```

step initMem;
  addr min,max,up,1;
  data seed;
  operation w;
repetition initMem;
  seed 'h0;
begin
  step initMem;
end
repetition WriteMask;
  seed 'hfffffffffffffff;
begin
  step wAddrLow;
  step wAddrHigh;
  step rAddrLow;
  step rAddrHigh;
end
test write_mask;
begin
  repetition initMem;
  repetition WriteMask;
end
add control background write_mask RAM32x4/WEN 1000 xF
add control background write_mask RAM32x8/WEN xF0 x0F

```

The sequence of algorithm will be as shown in [Table 5-5](#) (in case of concurrent controller).

Table 5-5. Sequence of Steps for Concurrent Controller

	Step Executed	RAM32x4/WEN	RAM32x8/WEN
1	initMem	1111	1111111
2	wAddrLow	1000	11110000
3	wAddrHigh	1000	11110000
4	rAddrLow	1000	11110000
5	rAddrHigh	1000	11110000
6	initMem	1111	11111111
7	wAddrLow	1111	00001111
8	wAddrHigh	1111	00001111
9	rAddrLow	1111	00001111
10	rAddrHigh	1111	00001111

For steps 1 and 6 the write enable is set to '1'. This is because “mask” was not specified while defining the step. So the write enable changes only for steps that have “mask” defined in their declaration.

In case of a sequential controller the sequence of operation will be as shown in [Table 5-6](#). The value of 'x' in the following table denotes that the controller does not care the value of the port for that step.

Table 5-6. Sequence of Steps for Sequential Non-Interleaved Controller

	Step Executed	mem var	RAM32x4/WEN	RAM32x8/WEN
1	initMem	0	1111	x
2	wAddrLow	0	1000	x
3	wAddrHigh	0	1000	x
4	rAddrLow	0	1000	x
5	rAddrHigh	0	1000	x
6	initMem	0	1111	x
7	wAddrLow	0	1111	x
8	wAddrHigh	0	1111	x
9	rAddrLow	0	1111	x
10	rAddrHigh	0	1111	x
11	initMem	1	x	11111111
12	wAddrLow	1	x	11110000
13	wAddrHigh	1	x	11110000
14	rAddrLow	1	x	11110000
15	rAddrHigh	1	x	11110000
16	initMem	1	x	11111111
17	wAddrLow	1	x	00001111
18	wAddrHigh	1	x	00001111
19	rAddrLow	1	x	00001111
20	rAddrHigh	1	x	00001111

The sequence table for sequential interleaved will be as shown in [Table 5-7](#).

Table 5-7. Sequence of Steps for Sequential Interleaved Controller

	Step Executed	mem var	RAM32x4/WEN	RAM32x8/WEN
1	initMem	0	1111	x
2	initMem	1	x	11111111
3	wAddrLow	0	1000	x

Table 5-7. Sequence of Steps for Sequential Interleaved Controller (cont.)

	Step Executed	mem var	RAM32x4/WEN	RAM32x8/WEN
4	wAddrLow	1	x	11110000
5	wAddrHigh	0	1000	x
6	wAddrHigh	1	x	11110000
7	rAddrLow	0	1000	x
8	rAddrLow	1	x	11110000
9	rAddrHigh	0	1000	x
10	rAddrHigh	1	x	11110000
11	initMem	0	1111	x
12	initMem	1	x	11111111
13	wAddrLow	0	1111	x
14	wAddrLow	1	x	00001111
15	wAddrHigh	0	1111	x
16	wAddrHigh	1	x	00001111
17	rAddrLow	0	1111	x
18	rAddrLow	1	x	00001111
19	rAddrHigh	0	1111	x
20	rAddrHigh	1	x	00001111

There are some pre-defined sets of algorithms that will be used more often. Some of them are walking1, walking0 and checkerboard.

Consider a 10bit write enable signal:

walking1 will expand itself to x001 x002 x004 x008 x010 x020 x040 x080 x100 x200.

walking0 will be expanded to x3FE x3FD x3FB x3F7 x3EF x3DF xBF x37F x2FF x1FF.

checkerboard will be expanded to x155 x2AA.

The command “Add Control Background” will support the pre-defined values like walking0, walking1 and checkerboard.

Algorithms walking0, walking1, and checkerboard will be expanded to the actual meaning for the given memory size.

If an algorithm has the “mask” keyword it will be added to the list of algorithms for the controller to synthesize (through the Add Mbist Algorithm command) there must be at least one set of control background or else the tool will report an error.

The added control background can be reported by using the report control background command.

Example - Write Enable Mask Algorithm

An example of the UDA is as follows. The data definition of steps have been highlighted in bold. The step “wSeedUp” not shown below will write the seed value to address range min to max. This is the initialization of memory. This step should not have any mask.

```

step wAddrLow;
  addr min,4,up,1;
  data invseed,mask;
  operation w;
step wAddrHigh;
  addr max - 4,max,up,1;
  data invseed,mask;
  operation w;
step rAddrLow;
  addr min,4,up,1;
  data invseed,mask;
  operation r;
step rAddrHigh;
  addr max - 4,max,up,1;
  data invseed,mask;
  operation r;
repetition WriteMaskShort_bg0;
  seed 0;
begin
  step wSeedUp;
  step wAddrLow;
  step wAddrHigh;
  step rAddrLow;
  step rAddrHigh;
end

repetition WriteMaskShort_bg1;
  seed 'hffffffffffffffffffff';
  begin
    step wSeedUp;
    step wAddrLow;
    step wAddrHigh;
    step rAddrLow;
    step rAddrHigh;
  end

test WriteMask_short
begin
  repetition WriteMaskShort_bg0;
  repetition WriteMaskShort_bg1;
end

```

After defining the above algorithm, the command Add Control Background can be used to specify the write enable signal values.

```
load algorithm writemask.mba
add mbist algorithm 1 WriteMask_short
add control background WriteMask_short RAMx/WEN x5555 xAAAA
```

There are two sets of write enable patterns specified by the command. The BIST controller will run algorithm WriteMask_short, setting the value of RAMx/WEN as x5555 (0101 0101 0101 0101) in the steps (wDataUpAddrLow, wDataUpAddrHigh, rDataUpAddrLow, rDataUpAddrHigh) and then again run the algorithm, setting the value of RAMx/WEN as xAAAA (1010 1010 1010 1010).

Note



When using a write-masked algorithm you cannot specify the controller to skip memories without a write_enable_map, so to avoid problems the tool adds a degenerate map in those memories. The affected write_enable signals will always be fully asserted.

See also: “[User-Defined Algorithm Language](#)” on page 144.

Port Isolation User-Defined Algorithm

This section defines the steps, repetitions, etc., that can support both dual-port and register file operations of the port isolation user-defined algorithm (UDA). For register files, port isolation algorithms are only valid for the first port. If you add a port isolation algorithm to the second port, the algorithm is applied, but the output from the register file(s) is ignored by the controller during testing.

See also “[Port Isolation Testing Algorithm](#)” on page 128.

The following is an example of a UDA definition and should be used for reference only, as this example has not been sufficiently tested.

```
// -----
// Steps
// -----
step w_up;                // initialize memory
    addr min,max,up,1;
    data seed;
    operation w;

step r_up;                // re-read memory after 2 w_r process
    addr min,max,up,1;
    data seed;
    operation r;

step port_isolate_up_row_test;
    addr min,max,up,1,1;
```

```

    data seed;
        operation w_r;      // w_r = port_isolate r/w pattern

step port_isolate_down_row_test;
    addr min,max,down,1,1;
    data seed;
        operation w_r;    // w_r = port_isolate r/w pattern

step port_isolate_up_column_test;
    addr min,max,up,1,row_size;
    data seed;
        operation w_r;    // w_r = port_isolate r/w pattern

step port_isolate_down_column_test;
    addr min,max,down,1,row_size;
    data seed;
        operation w_r;    // w_r = port_isolate r/w pattern

//-----
// Repetitions
// -----
repetition init_mem; // Initialize Memory to zeroes
    seed 'h00000000000000000000;
begin
    step w_up;
end

repetition p_i_up_row_test; // write 1-s, reading 0-s
    seed 'hffffffffffffffffffff;
begin
    step port_isolate_up_row_test;
end

repetition p_i_down_row_test; // write 0-s, reading 1-s
    seed 'h00000000000000000000;
begin
    step port_isolate_down_row_test;
end

repetition p_i_up_column_test; // write 1-s, reading 0-s
    seed 'hffffffffffffffffffff;
begin
    step port_isolate_up_column_test;
end

repetition p_i_down_column_test; // write 0-s, reading 1-s
    seed 'h00000000000000000000;
begin
    step port_isolate_down_column_test;
end

repetition review_mem; // Read only, expecting 0-s
    seed 'h00000000000000000000;
begin
    step r_up;
end

//-----

```

```
// Tests
//-----

test port_isolate_row_test;
begin
    repetition init_mem;
    repetition p_i_up_row_test;
    repetition p_i_down_row_test;
    repetition review_mem;
end

test port_isolate_column_test;
begin
    repetition init_mem;
    repetition p_i_up_column_test;
    repetition p_i_down_column_test;
    repetition review_mem;
end
```

Chapter 6

BIST, Memory, and the System

This chapter contains information about the details in which BIST specifically applies to your chip design and your ATE environment. You can customize the BIST generation activity according to your clocking needs and frequency requirements. You can specify concurrent or sequential test. You can generate comparators for writable memories and compressors for read only memories. You can use additional BIST insertion features to share pins or add glue logic between the BIST circuitry and your top-level ports.

The Default Scenario

Before you begin to customize your BIST, the following is the basic default scenario. At the very least, to get started with the generation phase you must specify an MBISTArchitect library file with memory model descriptions, and a dofile with the following commands:

```
load library <libname>  
add memory models <modelnames>  
run  
save bist
```

In a generation dofile of the insertion phase, the Load Library command can be in the insertion script, or omitted if you identify memories with specparams.

The BIST controller generated by this most basic generation dofile has no diagnostics and no BISA. There is one fail_h signal for all memories under test, and the signal is “sticky”, so it stays high after a failure is detected. If there is a failure, BIST continues without stopping and does not hold or restart. If the added memories are all writable, they will be tested with the March2 algorithm (also known as the MarchC+ algorithm) on all ports and evaluated by a comparator in the controller. If the memories are all ROMs, they will be tested with the “ROM1 test” algorithm on all ports and evaluated by a compressor in the memory collar. The memories will be tested concurrently (in parallel). The controller and testbench assume that the memories are driven by a clock which is in-phase with the bist_clk of the controller.

In the insertion phase you can insert the pre-generated BIST circuitry (Bottom-Up Insertion Flow) or you can generate them and insert them in one invocation of the tool (Top-Down Insertion Flow). Using pre-generated BIST circuitry, at the very least you need to invoke the tool with the chip or top netlist, the relevant HDL cell library, and an insertion dofile with the following commands:

```
load design object < controller.v>
set system mode bist
add existing controller < arguments>
insert bist logic
save design
```

This basic scenario replaces the controlled memories with collar designs, inserts the BIST controller, and adds ports and wires into your chip to connect the controller to the collars and to the top level. By default, if you have multiple controllers, there is no pin sharing and each one gets a unique path to the top-level ports or I/O Pads.

This chapter explains how to customize generation and insertion with respect to comparators, compressors, clocking schemes, sequential test, and chip-level sharing and output logic.

Previous chapters have shown how to override the defaults with respect to the algorithms generated in your controller. Later chapters will describe optional features like diagnostics, BISA, and full-speed BIST.

BIST Circuitry Interface Ports

[Table 6-1](#) and [Table 6-2](#) list the standard MBISTArchitect inputs and output ports. The ports for diagnostics, algsel, and others are optional and only occur when you use the given feature using certain tool commands. Many of the ports, but not all, should be connected to chip pins at the top level. Purely internal BIST signals are not shown in the tables.

Table 6-1. BIST Controller Input and Output Ports

Functionality	Port Name	Direction	Description/Purpose
Default MBISTArchitect controller ports	test_h	input	Enables MBISTArchitect testing.
	Test_<memport>_<i>	input	The BIST controller has one Test_<name>_i port for every non-dont_touch port in every memory added with Add Memory Models. If multiple memories, an _i suffix is used to denote the memnum.
	rst_l	input	Resets the MBISTArchitect controller.
	bist_clk	input	Controller clock.
	fail_h	output	Goes high in the presence of a failure.
	tst_done	output	Indicates MBISTArchitect test completion.

Table 6-1. BIST Controller Input and Output Ports (cont.)

Functionality	Port Name	Direction	Description/Purpose
Diagnostics	diag_scan_in	input	Scan-in port of the diagnostic register. This port is only present if you use DFTAdvisor to insert it.
	diag_scan_out	output	Scan-out port of the diagnostic register.
	diag_clk	input	Diagnostic register clock.
	restart_h	input	Present if using Restart diagnostics.
	hold_l	input	Active-low controller hold.
	debugz	input	Enables diagnostics.
	diag_monitor	output	(Optional) Parallel view of the internal diagnostics monitor register.
BISA	repairable_h	output	Indicates whether the memory is repairable or not.
	repair_data	output	Repair data scan_out pin.
	repair_data_clock	input	Repair data register clock.
	repair_data_force	output	Indicates whether the tester should collect a report.
Retention Test	start_retention_h	output	Indicates the start of retention cycles.
	test_resume_h	input	Enables test resume after retention.
Algorithm Selection	algsel_scan_in	input	Algorithm selection register scan_in.
	algsel_scan_en	input	Algorithm selection register scan enable.
	algsel_scan_out	output	Algorithm selection register scan_out
	algsel_clock	input	Algorithm selection register clock only if using Setup Memory Clock -Algsel_clk command and switch, otherwise the clock is bist_clk.

Table 6-2. BIST Collar Input and Output Ports

Functionality	Port Name	Direction	Description/Purpose
Default MBISTArchitect collar ports	<memport>	input	The BIST collar has one <memport> for every non-dont_touch port.
	Test_<memport>	input	Corresponds to the non-dont_touch memports
Memory Bypass	bp_clk	input	Bypass register clock.
	Test_mode	input	Bypass mux select.
	rst_l	input	Resets the MBISTArchitect controller.
	scan_en	input	Bypass scan enable.
	scan_in	input	Bypass scan input.
	scan_out	output	Bypass scan output.
MISR	misr_clk	input	The compressor employs this clock to shift out the completed signature.
	se	input	This signal is driven high by the tester during BIST and MISR scanout.
	compress_h	input	Used to enable the compressor function. Compressor signal active high.
	misr_scan_out	output	The signature is shifted out serially on this port.
	hold_l	input	Only if the dofile uses the Set Controller Hold command with the -On switch.
	bist_clk	input	BIST clock.
	rst_l	input	Reset active low.
	fail_h	output	Fail signal active high.
	tst_done	input	Test done.
	si	input	MISR signal input. Used to initialize the compressor with a seed value.
	misr_sout	output	MISR signal output.

Comparators Versus Compressors

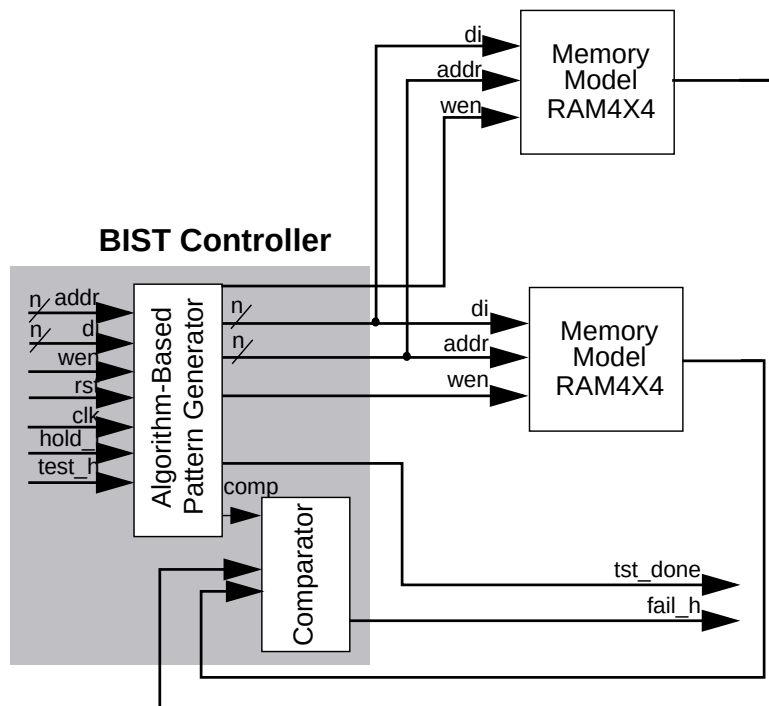
The following section explains the difference between comparators and compressors.

Comparators for RAMs

When your memory models have write ports or read_write ports, they are defined as writable memories, also referred to as RAMs. The BIST controller detects RAM failures during test with an comparator that compares actual memory data versus expected data. In a concurrent test there is one comparator per memory; in a sequential test there is just one comparator.

Figure 6-1 shows a block diagram of a BIST controller that tests two RAMs sequentially and evaluates them with a comparator.

Figure 6-1. Comparator for Two RAMs



The inputs of the comparator are normally not visible to the tester during BIST, although you can use the optional diagnostics feature to view them.

You cannot configure BIST circuitry to use a comparator with ROMs. The comparator’s “expected data” is based on algorithm steps that perform a write operation. Since you cannot write to a ROM, you cannot generate a BIST comparator for ROM test.

Compressors for ROMs

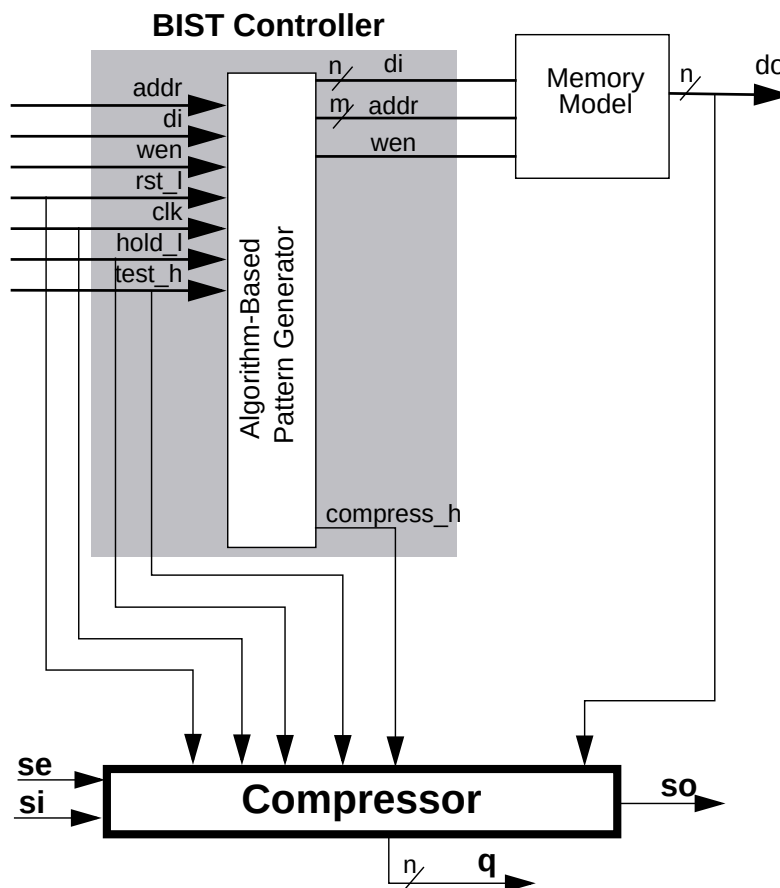
When your memory models only have read ports, they are defined as read-only memories (also known as ROMs). Since the contents of ROMs are fixed, the most efficient means of testing is to analyze the ROM contents using a compression function that computes a short signature. By default, a BIST controller for ROMs generates a compressor inside the memory collar that

instruments this function. The compressor function is a MISR (Multiple Input Shift Register or Multiple Input Signature Register).

Because the compressor is part of the memory collar, there is one compressor instance per memory instance regardless of whether the test is concurrent or sequential.

Figure 6-2 shows a block diagram of a BIST controller that tests one ROM and evaluates it with a compressor.

Figure 6-2. Compressor for a ROM



Controlling RAMs Separately From ROMs

You cannot use both a compressor and a comparator to test a given memory, nor can you use a single controller to test both RAMs and ROMs. If your chip has both RAMs and ROMs, you must run the generation phase at least twice so that one generated controller is only testing RAMs and the other generated controller is only testing ROMs.

The Compressor in the Chip

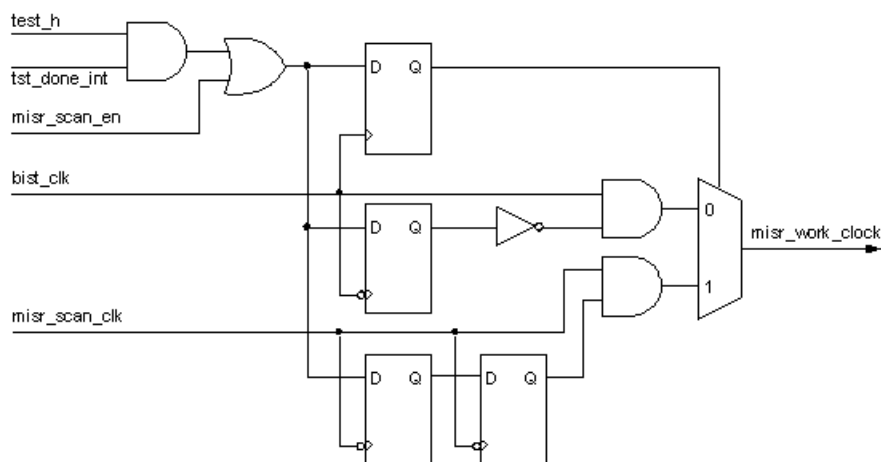
MBISTArchitect adds the following scalar ports to the BIST collar interface when using a compressor:

- **Input misr_clk** — The compressor employs this clock to shift out the completed signature. The clock gating logic chooses this clock when test_h is high, MISR scanout is enabled, and BIST determines it is ready to report. It is separate from bist_clk because the tester might require a slower frequency to receive the signature data.
- **Input se** — This signal is driven high by the tester during BIST and MISR scanout. Assert this signal high only for scanning in a seed into the MISR (on port si) before BIST.
- **Input si** — You can optionally use this port to initialize the compressor with a seed value. If you do not use a seed value, the default is zero.
- **Output misr_scan_out** — The signature is shifted out serially on this port, clocked by the misr_clk.

You can configure the compressor hardware using the [Setup Mbist Compressor](#) and [Setup Misr Polynomial](#) commands. Additionally, you can use the [Setup Observation Scheme](#) command to specify whether or not the compressor-related collar ports are connected to the BIST controller.

The compressor hardware includes a clock-gating device (shown in [Figure 6-3](#)) to switch between the regular BIST clock and the misr_clk. The reason for having two clocks is that you might want to run BIST quickly (using Full-Speed BIST) using the normal operational speed of the memory, whereas you might want to use a slower ATE clock to offload the MISR signature into your tester.

Figure 6-3. Compressor (MISR) Clock Gating



Clocking Schemes

The following section covers clocking schemes.

A single BIST controller can only operate at a single clocking frequency. If you have two memories associated with a single controller, they will be tested at the same frequency, regardless of their intended system mode frequencies. Furthermore, each port of a single memory will operate at the same frequency.

You can operate separate BIST controllers at different frequencies if they are scheduled sequentially during insertion. All controllers scheduled concurrently must use the same frequency.

Primary Versus Secondary Clocks

In the BIST/Memory system, the `bist_clk` is referred to as the primary clock and other clocks are referred to as secondary clocks. The one type of secondary clock that always exists for synchronous memory is the memory clock (referred to as `mem_clk`); however, you can configure the system to include the following optional secondary clocks:

- BISA clock (`repair_data_clock`)
- Bypass clock (`bypass_clk`)
- Compressor clock (`misr_clk`)
- Diagnostic clock (`diag_clk`)
- Online Algorithm Selection clock (`algsel_scan_clk`)

The `misr_clk` is discussed in the preceding section on Compressors, and the other clocks are discussed in later chapters. These clocks are for optional features, generally for the purpose of serially scanning data into or out of the BIST circuitry before or after test. When you enable these optional features, the clocks are automatically added to the interface of the BIST controller or collar.

You can configure primary clocks, and you can setup aspects of the secondary clocks using one of the following commands:

Setup Controller Clock
Setup Diagnostic Clock
Setup Memory Clock

The `bist_clk` and `mem_clk` must be driven at the same frequency; however, they can be inverses of each other. The following sections describe how you can configure BIST to invert one or both of these two clocks.

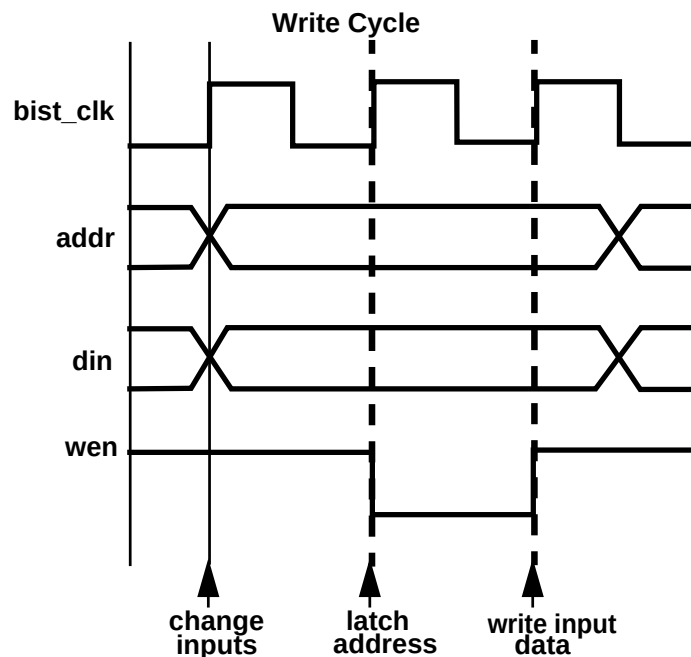
You can specify the BIST clock period using the Setup Clock Period command. This will be used in the testbench and the optional synthesis script. It does not otherwise affect the generation of pipelining inside the BIST controller.

Asynchronous Memories

If your memory model does not have a pin declaration for type “clock”, then it is asynchronous. However, you still must define the read/write cycle definition into the model, because the BIST controller is still synchronous and needs to know what to do on each cycle of the `bist_clk`.

Figure 6-4 shows the timing diagram for the BIST controller’s write cycle to an asynchronous memory.

Figure 6-4. Asynchronous Memory Write Cycle



A change on the address bus `addr` starts the write cycle. After a minimum settling time, the write enable signal `wen` goes active low which causes the memory to latch the new address. Sometime before the `wen` signal goes inactive high, new data is placed on the `din` bus and allowed to settle. When `wen` goes inactive high, the data is written to memory and the write cycle ends.

Modifying the Memory Clock

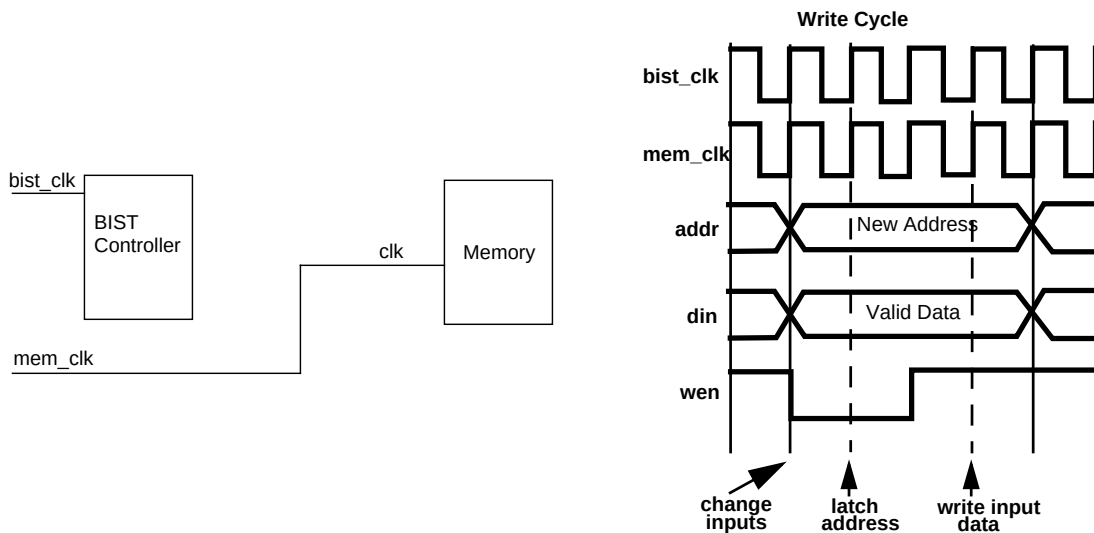
The following sections cover muxing and inverting the memory clock.

The Original System Memory Clock

Normally the BIST memory collar contains a mux for every port in the memory model, except those designated as dont_touch or already BIST-ready. The muxes each have three inputs: system data, test data, and a selector input. During BIST, the selector is activated to choose the test data. Clocks are always special; however, and you might not wish to allow muxing of the clock.

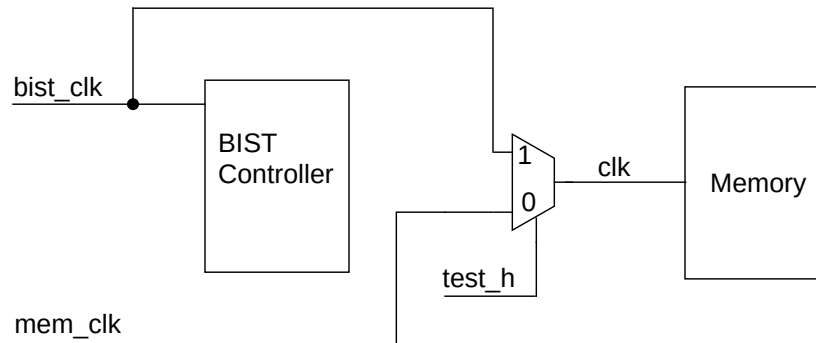
In the default scenario, or if you use the Setup Memory Clock –System command and switch, the BIST controller and the memory are clocked directly by the tester, with no intervening mux. This setup is also called the “gate clock off scheme.” In this scheme the clocks are expected to be the same frequency and in-phase. [Figure 6-5](#) shows the block diagram with the system memory clock setup, and the timing diagram for the BIST controller’s write cycle to the memory.

Figure 6-5. Setup Memory Clock -System



Muxing the Memory Clock

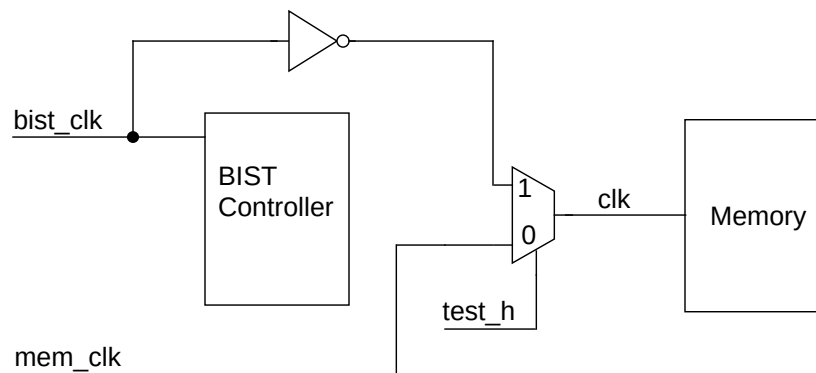
If you want to mux the memory clock between your chip clock and a test clock, use the “Setup Memory Clock –Test” command and switch. In this configuration when `test_h` is activated the memory will receive the same clock that is driving the controller and `bist_clk` so the memory and the controller will be running at the same frequency and in-phase. However, your chip clock can be any frequency and any phase. [Figure 6-6](#) shows the block diagram with this basic test clock setup. The timing diagram for this setup is nearly identical to that shown in [Figure 6-5](#), except that here the memory `clk` port waveform is delayed by the propagation time of `mem_clk` through the mux.

Figure 6-6. Setup Memory Clock -Test Noinvert

Muxing and Inverting the Memory Clock

When the BIST controller clock and memory clock are running at the same frequency but out-of-phase, it is possible to speed up the read/write cycles and make test complete sooner. When the clocks are in phase, the controller can make an access request and must wait a full cycle for the memory to respond; however, when the clocks are 180 degrees out of phase, the wait time is a half-cycle instead of a full-cycle. The half-cycle is saved at every test address and can add up to considerable total savings in test time.

The tool determines whether such optimization is possible and if so it generates the appropriate changes in the finite state machine of the controller. If you want to setup out of phase clocks using the test clock scheme with an inversion on the memory clock, then use the “Setup Memory Clock –Test Invert” command and switch. [Figure 6-7](#) shows the block diagram with the inverted test clock setup.

Figure 6-7. Setup Memory Clock -Test Invert

Inverting the BIST Controller Clock

The previous section explained how you can save a half-cycle at every address by making the memory clock 180 degrees out of phase with the controller clock. Instead of inverting the memory clock, you could invert the controller clock using the “Setup Controller Clock -Negative” command and switch. This achieves the same timing optimization, but also gives you the flexibility to choose whether or not to mux the memory clock.

Special Controls

The following sections cover special controls.

Hold

Normally when the BIST controller detects a memory failure it does not stop; it just keeps running the algorithms and comparing for failures until test is done. However you can use the command and switch “Set Controller Hold –On” to request a hold_l input port on the controller interface. The tester can activate the hold_l signal (active low) at any time and the controller will pause testing until the signal is deactivated. The hold_l input is synchronous with bist_clk.

Reset

The controller always has a rst_l input port. The tester can activate this signal (active low) at any time and the controller will clear all internal state. The rst_l input is asynchronous and instantly interrupts BIST.

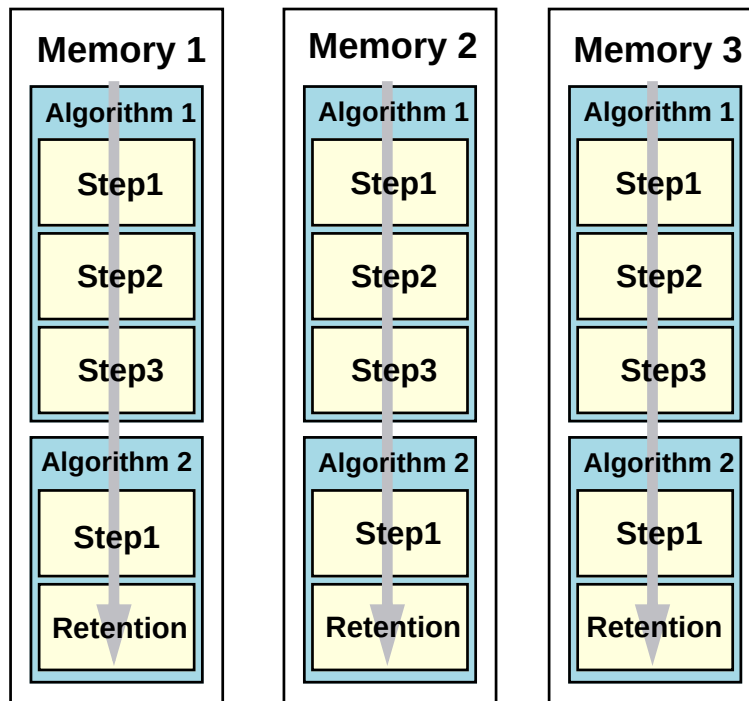
Concurrent Versus Sequential Testing of Memories

The following sections explain the difference between concurrent and sequential testing of memories.

Concurrent Memory Test

When a BIST controller is testing more than one memory, by default the controller tests them in parallel. This requires controller hardware and routing proportional to the number of memories being tested, but the total test time is only as fast as the single slowest memory. If three memories would normally take 1000 cycles each to be tested individually, the total concurrent test time is just 1000 cycles. [Figure 6-8](#) illustrates the ordering of concurrent test.

Figure 6-8. Concurrent Memory Test

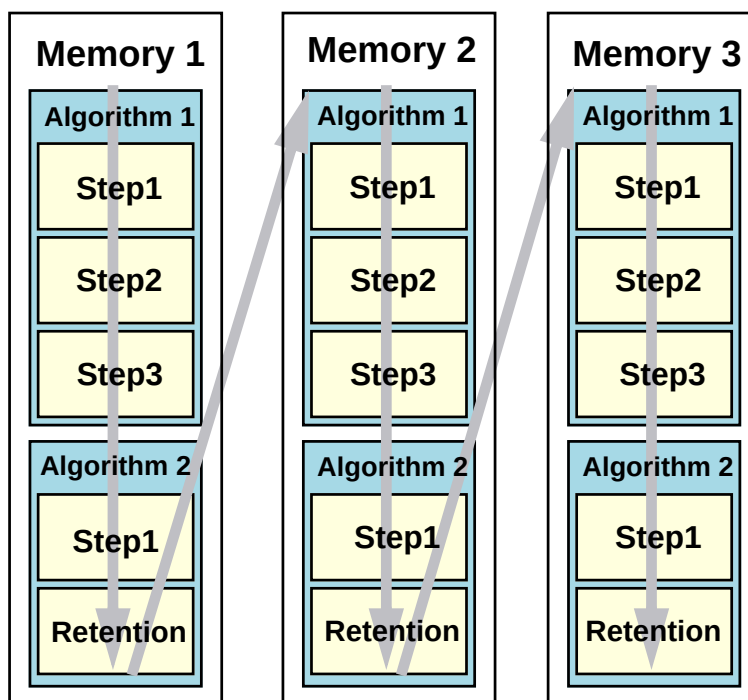


Sequential-Contiguous Memory Test

Concurrent test is not always the preferred method. You might not prefer the extra hardware, or it may cause bus contention in a scenario where the memories under test share a tri-state data bus. An alternative approach is sequential testing, where only one memory is tested at a time. This reduces the total amount of controller hardware, and the total test time is the sum of the test time for all memories. If three memories would normally take 1000 cycles each to be tested individually, the total sequential test time is 3000 cycles. Routing is the same for sequential and concurrent test.

The tool offers two kinds of sequential test: contiguous and interleaved. [Figure 6-9](#) illustrates the ordering of sequential-contiguous test.

Figure 6-9. Sequential-Contiguous Algorithm Ordering



In a sequential-contiguous test, the controller performs all algorithms for the first memory, then performs all algorithms for the second memory, and so on. To enable this mode of test, use the following command and switches:

Setup Memory Test –sequential contiguous

The RetentionCB algorithm presents a problem for sequential-contiguous test. In retention test the controller writes data to a memory, then waits for a “long” time (on the order of milliseconds) before reading it. Performing this algorithm on multiple memories in sequential-contiguous test can make the total test time very long. If you are not using retention test this is not an issue, but if you are using it, the solution to this problem is described in the next section. For more information, see “[retentionCB](#)” on page 123.

Sequential-Interleaved Memory Test

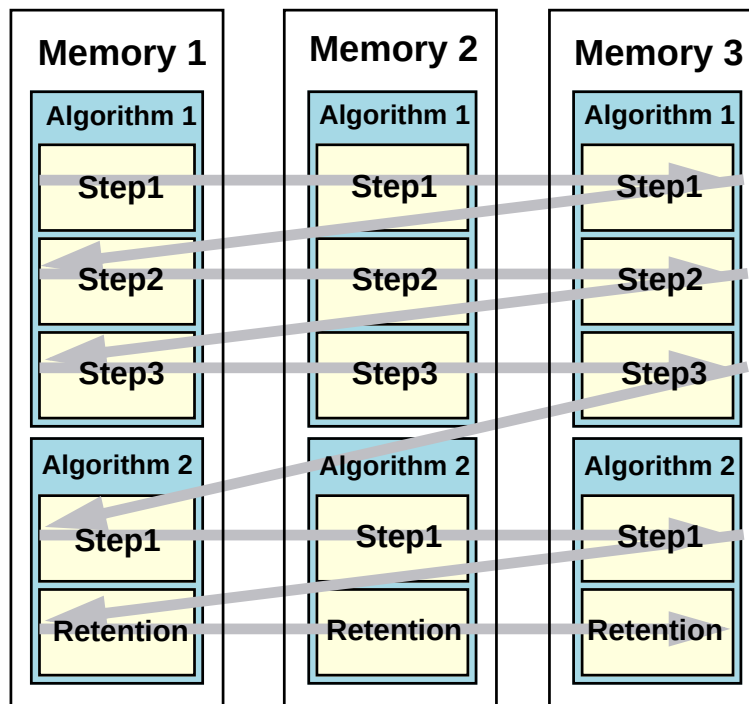
If you are performing sequential test on multiple memories with the RetentionCB algorithm, there is an optimization to avoid spending too much time on the retention step: sequential-interleaved test. In a sequential-interleaved test, the controller performs the first step of the first algorithm on the first memory, then performs the first step of the first algorithm on the second memory, and so on. Then it moves to the second step of the first algorithm for all memories. The retention step of the RetentionCB algorithm takes a “long” time (on the order of milliseconds), but with sequential-interleaved test the time cost of retention is effectively incurred only once for all memories. If three memories would normally take 1000 cycles each to be tested individually, but in each case 800 cycles are devoted to the RetentionCB algorithm,

then in sequential-interleaved test the total test time is 1400 cycles. To enable this mode of test, use the following command and switches:

Setup Memory Test –sequential interleaved

Figure 6-10 illustrates the ordering of sequential-interleaved test.

Figure 6-10. Interleaved Sequential Algorithm Ordering



Top-Level Insertion Circuitry

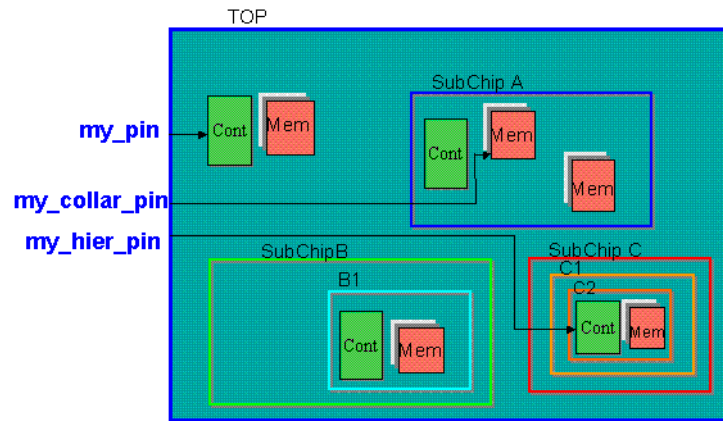
Top-Level Pin Mapping

Top-level pin mapping maps a specified controller or collar pin to an existing or newly created pin at the top level by using the Add Pin Mapping command. The connection to the internal wire or net can be done by using the -Notop switch. If the top-level pin does not exist, the tool creates a new pin with the specified name. [Figure 6-11](#) shows an example of top-level pin mapping.

By default all controller pins are mapped to newly created pins at the top level. You can share with existing pins on the top level by specifying the preference mapping with the Add Pin Mapping and Add Pin Sharing commands.

For more information and examples, see the [Add Pin Mapping](#) and [Add Pin Sharing](#) commands in the *MBISTArchitect Reference Manual*.

Figure 6-11. Top-Level Pin Mapping Example

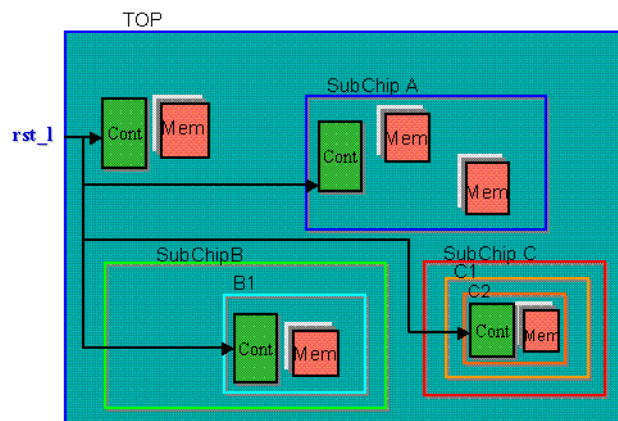


Controller Pin Sharing

Controller pin sharing maps specified pin types of a controller to a single pin at the top level using the [Add Pin Sharing](#) command. With this command, you also identify pins on different controllers to be shared. Pin type examples are as follows: `bist_clk`, `rst_l`, and `debugz`, to name a few. [Figure 6-12](#) shows an example of controller pin sharing.

For multiple controllers, you might want to map specific pin type for all controllers to a single pin at SoC.

Figure 6-12. Controller Pin Sharing Example

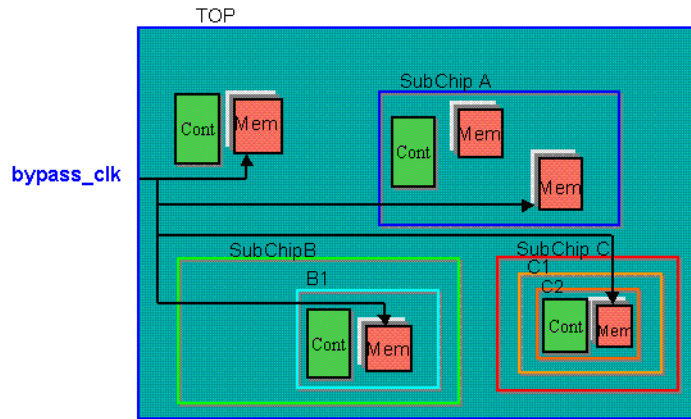


Collar Pin Sharing

Collar pin sharing maps specified pin types of a collar to a single pin at the top level using the [Add Pin Sharing](#) command. This command connects all memory block signals with specified

pin types to the top level. Pin type examples are as follows: `bypass_clk`, `compress_h`, and `test_mode`, to name a few. Figure 6-13 shows an example of collar pin sharing.

Figure 6-13. Collar Pin Sharing Example

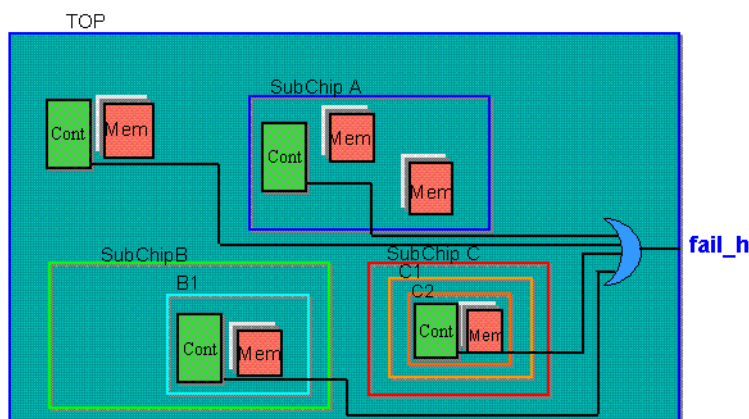


Adding Output Logic

You can add output logic using the Add Pin Sharing and Add Pin Mapping commands. These commands allow you to take the specified pin type of all controllers and put them through a logic gate to a single pin. For multiple controllers, you can gate a particular pin type of all the controllers to a single pin at the SoC level.

Pin type examples are as follows: `fail_h`, `tst_done`, and `start_retention`, to name a few. Figure 6-14 shows an example of adding output logic.

Figure 6-14. Adding Output Logic Example



Sharing Top-Level Bidirectional Ports

The BIST controllers and collars have the ability to share top-level bidirectional ports. This section explains this functionality.

During the MBIST testing mode, the top-level bidirectional port (bidi port) is configured as an input or output port depending on the controller's or the collar's pin type. If the pin type of the controller or collar is an input, the top-level bidi port is configured as an input port. Conversely, if the pin type of the controller or collar is an output, the top-level bidi port is configured as an output port. This is achieved by inserting the necessary logic to control the bidi port enable(s). In order to control the bidi enables, the bidi port must be connected to a bidi I/O pad.

Figure 6-15 shows an example of the logic inserted to configure a bidi port as an input port during test; whereas Figure 6-16 shows an example of the logic inserted to configure a bidi port as an output port during testing.

Figure 6-15. Bidi Port Configured as an Input Port

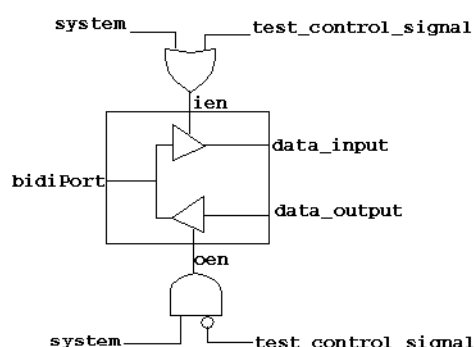
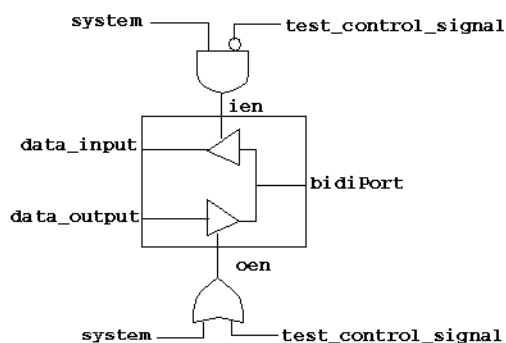


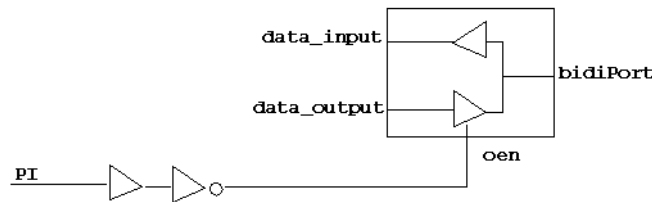
Figure 6-16. Bidi Port Configured as an Output Port



The majority of the bidi I/O pads come with one enable only (oen), and in such cases the logic inserted to control the input enable (ien) is eliminated. Also, the logic inserted to control the I/O pad enables is adjusted in case the polarity of the output enable (or input enable) is active low.

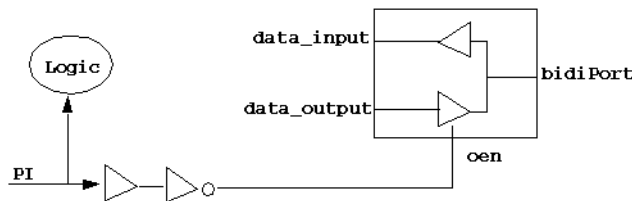
Prior to inserting the control logic to control the bidi enables, the tool traces back to find out if a bidi enable is directly driven by a primary input. If the tool finds that a bidi enable is directly driven by a primary input, the tool will not insert any control logic, and will force the primary input to a proper value (0 or 1) during test. The tool will trace back through buffers and inverters only. [Figure 6-17](#) shows an example of a primary input directly controlling a bidi output enable.

Figure 6-17. Bidi Port Controlled by a Primary Input (PI)



In the case where there is any fan out in the path between the primary input and the bidi I/O pad enable as shown in [Figure 6-18](#), the tool will insert the control logic shown in [Figure 6-15](#) or [Figure 6-16](#) depending on the type of controller/collar pin to be mapped.

Figure 6-18. Bidi Port Controlled by a Primary Input (PI) with Fan Out



The previous paragraphs and figures explained the bidi port default behavior of the tool. To overwrite this default behavior, you must instruct the tool to not insert any control logic. To instruct the tool to not insert any control logic, you use the -NOControl switch of either the Add Pin Mapping, or the Add Pin Sharing commands. In this case, you must force the bidi enable using a test_setup procedure.

For more information on the test_setup procedure, refer to the [Add Pin Mapping](#) or the [Add Pin Sharing](#) commands, and the -NOControl switch.

Bidirectional Enable Signal Behavior

During the loading of the algorithm selection register, the test_h signal is low, therefore it cannot be used to control the bidi enable of the algorithm selection related signals. Because of

this, `algsel_scan_en` is used instead of `test_h` as a control signal to control the bidi I/O pad enables in case any of the following signals: `algsel_scan_in`, `algsel_scan_out`, or `algsel_scan_clk` are mapped to a bidi port. Also, `algsel_scan_en` is used to control the mux (if any) in the case of mapping `algsel_scan_out`.

In the case of parallel loading of algorithm selection through primary inputs (only `algsel_scan_in` is created as a vector), the signal `test_h` will be used as a test control signal to control bidi enables.

Similar to the previous algorithm selection case, during scanning in a misr seed, `test_h` is low, therefore it cannot be used to control the bidi I/O pad enable of misr related signals. Therefore, `misr_scan_en` is used instead of `test_h` to control `misr_scan_in` and `misr_clk` if any of these signals are mapped to bidi ports. At the time of scanning out the misr signature, the signal `misr_scan_en` is low, therefore `test_h` is used to control any muxes and/or bidi enables in case `misr_scan_out` is mapped to a bidi port.

Table 6-3 lists the `test_control_signal` for different pins.

Table 6-3. `test_control_signal` for Different Pins

Signal Type	<code>test_control_signal</code>
algorithm selection signals	<code>algsel_scan_en</code>
misr signals, except <code>misr_scan_out</code>	<code>misr_scan_en</code>
<code>test_h</code>	no control signal
<code>algsel_scan_en</code>	no control signal
<code>misr_scan_en</code>	no control signal
<code>rst_l</code>	no control signal
all remaining signals	<code>test_h</code>

For the following signals: `test_h`, `algsel_scan_en`, `misr_scan_en`, and `rst_l`, the tool cannot insert any control logic. Therefore, the tool will trace to see if there is a direct PI to control the bidi enable. If not, the tool will issue an error stating that it cannot map this signal to a bidi port. You can bypass this error by using the **-NOControl** switch to force the tool to do the mapping, and also to control the bidi enable you need to force it through a `test_setup` procedure to the proper control value (0 or 1).

BIST Diagnostic Scheme

The tool provides diagnostic capability by instantiating a diagnostic block and connecting it to the signals within the controller. The tool generates the definition of this diagnostic block with the default name <controller>_diag and writes this definition to the output RTL file.

Note



If you specify the use of an alternative or user supplied diagnostic block, the tool will not provide the standard definition of the block. It is your responsibility to ensure that this alternative block is available at RTL compilation and elaboration time.

Diagnostic Clock Control

In the default implementation, a BIST controller clock is used to observe diagnostic data. However, it is desirable to control the rate at which the diagnostic information should be observed (for example, BIST clock or the diagnostic clock).

You can use the Setup Diagnostic Clock command with the -Slow_tester_clk switch to drive the observation scheme. When you issue this command, a diagnostic clock pin named diag_clk will be added to the controller pin list. This pin will be toggled at half BIST clock during the testbench, and is used to drive the observation scheme of the diagnostic information.

Interface to Diagnostic Block

The diagnostic block uses the interface defined in the following table.

Table 7-1. Diagnostic Block Interface

Signal Type	Direction	Function
start	Input signal	Taken high to initiate diagnostic. Connected to controller generated fail_mon signal(s)
monitor	Input signal	Vector signal connected in controller to data to be scanned out by diagnostic block.
size	Parameter	Instance parameter set to control the size of the monitored data above.

Table 7-1. Diagnostic Block Interface (cont.)

Signal Type	Direction	Function
enable	Input signal	Taken high to enable the diagnostic block. Diagnostic block will be inactive if this signal is low. Connected to the debugz input of the BIST controller, if present, or tied active otherwise.
diag_clk	Input signal	Diagnostic clock, used to scan the data out of the diagnostic block. This clock must be no faster than the BIST controller clock. See also “ Understanding the Diagnostic Scanout Output Data ” on page 194
ctrl_clk	Input signal	BIST controller clock, used to control the diagnostic block sampling of the start signal and monitored data.
dout	Output signal	Single bit of diagnostic data. Connected to the BIST controller diagnostic data output.
hold	Output signal	Taken high when the diagnostic block requires the BIST controller to enter the held state.

Understanding the Diagnostic Scanout Output Data

The scanned out diagnostic data can be used for debugging.

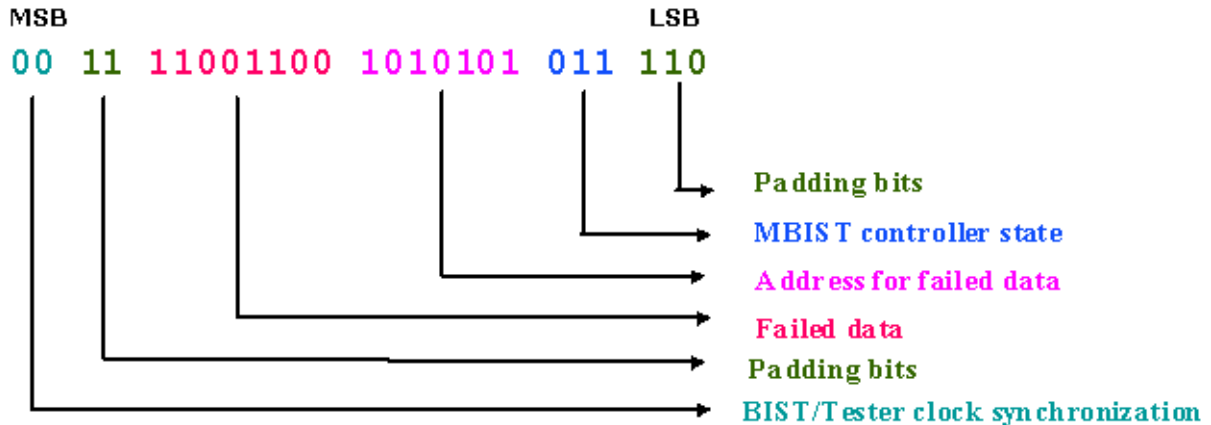
This example examines the diagnostic output data from stuck-at fault “11001100” at address “10101010”. The diagnostic data scanned out by diag_clk can be broken down as “0011110011001010101011110” (from MSB to LSB). [Table 7-2](#) breaks down this example diagnostic output data.

Table 7-2. Diagnostic Data Explanation

Diagnostic Output Data	Bit Meaning
00	BIST/Tester clock synchronization (MSB)
11	Padding bits
11001100	Failed data
1010101	Address for failed data
011	MBIST controller state
110	Padding bits (LSB)

[Figure 7-1](#) shows the same information as [Table 7-2](#) in a pictorial format.

Figure 7-1. Diagnostic Output Data



Bit Sequence of Diagnostic Scan Out Data

The diagnostic scanout stays 0 (after the 110 postamble) for an unspecified number of BIST cycles until the failflag is deasserted.

Selection of Scan Out Data

You can select the exact data to be scanned out and the ordering of the data. This choice is made from a set of data items available within the controller. The set includes:

- **tstate**
The current major state of the BIST controller. From this it is possible to determine exactly what algorithm step is being performed and, in the case of sequential memories, what memory.
- **addr_reg**
The value of the address register within the BIST controller.
- **addr**
The value of the address outputs, supplied to the memory.
- **rw_state**
The current position within the sequence of activities that occur at each address. From this it is possible to determine what part of a complex cycle has been executed.
- **dout**
The data read from the memory, that failed the comparison.
- **failmap**
The failing bitmap, the exclusive or of the expected data and the data read from the memory.

- memnum
The memory number for the memory under test, in the case of sequential memory testing.

Scanning Out Diagnostic Data

By default the BIST controller uses the BIST clock to scan out the diagnostic data.

You have the ability to monitor each failure for a predefined amount of data (addr, tstate, dout, rw_state, etc.).

In order to extract the failing data, the BIST controller requires the controller's hold capability, as well as additional functionality to download the failing data on every occurrence of a miscompare.

The MBISTArchitect tool provides the ability to add this functionality to the BIST controller so that the failing data is scanned out of the device on every miscompare, with minimal impact on silicon real estate and routing overhead. To specify diagnostics, use the command `Set Controller Debug -on`, and to specify the hold capability, use the command `Set Controller Hold -on`.

When the set controller hold -On switch is used, the tool generates a hold_l pin on the controller. If hold_l is activated low (asserted low), the controller and (if present) the compressor are held and no internal registers are updated. The hold_l is controlled by you. The name, hold_l, is the default and can be changed using the [Set Pin Name](#) command.

The BIST controller operates as follows: When a miscompare occurs, fail_h is activated and the BIST controller starts scanning the failing data out of the controller through scan_out. Once the failing data is scanned out, fail_h is deactivated and the BIST controller resumes the test. The scan out operation is repeated on every occurrence of miscompare.

There are several options available in the MBISTArchitect tool when diagnostics is enabled.

- The option to choose synchronization between the BIST clock and diagnostic clock, or not.
- The option to hold the BIST controller after each failure, or not.
- The -restart and -norestart option.

By default, the MBISTArchitect tool does the synchronization between diagnostic and BIST clock and hold the BIST controller after each failure.

For more information and examples, see the [Set Controller Debug](#), and [Set Controller Hold](#) commands in the *MBISTArchitect Reference Manual*.

Field Size (Diagnostic Register Width)

One major issue is that the software does not have the ability to calculate how big the fields are until after the Run command is issued when you are in the BIST Generation mode.

For example, if the signal dout is included, the width of the dout part is the total width of data from all memories. The same is true of address(es).

Another problem is that the fail_map is chosen as the widest memory, or the sum of all of the memories for concurrent testing.

Another complication is that field size can (and in some cases has to) change if more memories are added. Some of the values in the code are only valid after other code has executed within the Run command in BIST Generation mode.

See also the [Report Diagnostic Monitor](#) command in the *MBISTArchitect Reference Manual*.

Generating a BIST Controller with Diagnostic Capabilities

By default, the tool will indicate failures to ensure that a bad part is rejected. However, it is often necessary to diagnose failures to identify their cause. In this case, data is needed that indicates exactly which patterns resulted in mismatches. This data can then be processed to identify specific memory faults.

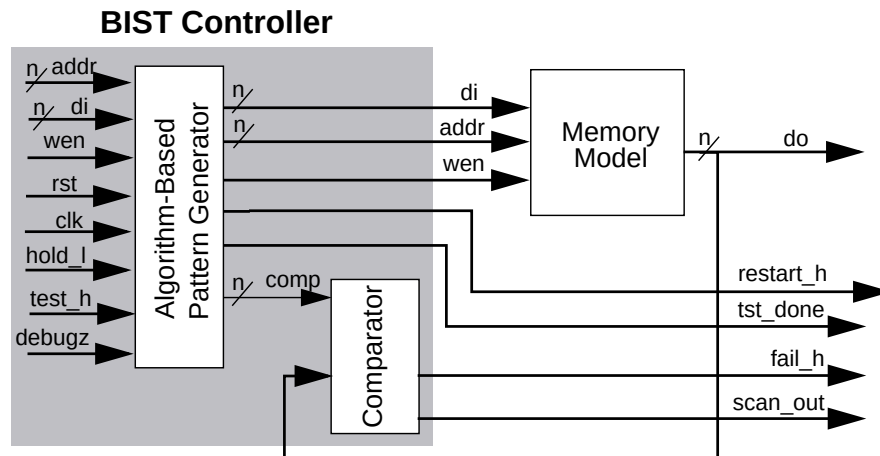
In order to extract the failing data, the BIST controller requires the controller's hold capability as well as additional functionality to download the failing data on every occurrence of a mismatch. The tool provides the ability to add this functionality to the BIST controller, so that the failing data is scanned out of the device on every mismatch, with a minimal impact on silicon real estate and routing overhead.

[Figure 7-2](#) shows example architecture resulting from the tool's diagnostic capability. In addition to the hold_l input signal, the tool generates an additional input port (debugz) and output ports scan_out, and restart_h.

- Restart (restart_h)
An active high signal that is activated when the BIST controller is in a restart mode, and is deactivated when the controller successfully restarts the BIST. The signal restart_h is only present if you have specified diagnostics with restart. This signal is used only for testbench purposes.
- Test Done (tst_done)
When high, the tst_done signal indicates the completion of the self-test operation.
- Fail (fail_h)
The pass/fail flag for the BIST controller.

- Scan Output (scan_out)
(Debug only) This is the scan output port for diagnosing serially scanned out failing data. This signal works with the hold_l and the debugz signals.

Figure 7-2. BIST Architecture Using Diagnostic Functionality



- Addresses (addr)
The address inputs to the memory array.
- Data inputs (di)
The data inputs to the memory array.
- Write enables (wen)
The write enables that control memory read/write operations.
- Reset (rst)
An active-low signal that resets the finite state machine.
- Clock (clk)
The clock for the finite BIST controller.
- Hold (hold_l)
An optional active-low signal that forces the BIST controller to stop processing and maintain its current state.
- Test (test_h)
An active-high signal that enables the BIST controller. When test_h is high, self-test is in progress. When test_h is low, the hold_l signal is activated to discontinue the clocking of the BIST controller and conserve power.
- Diagnostic Mode (debugz)
(Debug only) The diagnostic mode enable signal. When debugz is low, the BIST controller performs the default memory tests. When debugz is high, the diagnostic mode is enabled. Works with hold_l and scan_out.

When you specify diagnostics, the BIST controller operates in one of two modes controlled by debugz. In either mode, the tst_done signal is activated when the testing process is finished. The modes and operation of the fail_h and scan_out ports are as follows.

Note

The behavior of fail_h described next assumes the tool generated comparator logic with “setup comparator failflag -singlefail” in effect, which is the default.

- Normal Mode (debugz = ‘0’)
When debugz is set to ‘0’, the BIST controller performs the default test. In this mode, the scan_out port is set to ‘0’, as no fail data is downloaded. The fail_h port is activated on the first failure and remains high for the remainder of the test.

Diagnostics with BIST Full-Speed

Whenever the BIST controller is stopped due to a defect during diagnostics running at-speed, the BIST will restart the algorithm from the very beginning. It disables error detection until after the point at which the hold occurred.

For restart, the BIST controller is reset and the entire BIST process starts over from the beginning. The restart takes place immediately after the data from all pending failures is processed. The sequence of events in a restart is as follows:

1. A failure is first detected in the controller.
2. In case of diagnostics with hold, the BIST controller is stopped immediately. In case of diagnostics with nohold, the BIST controller continues.
3. For diagnostics with hold, the controller is stopped until the diagnostics data is completely scanned out. A restart is then initiated.
4. For diagnostics with nohold, if the information pertaining to the first failure is completely shifted out before a second failure occurs, the BIST controller simply continues its operations.

However, if a second failure is detected before the information related to the first failure is scanned out, the controller is stopped and the diagnostic information associated with the first and second failure is scanned out. The controller then continues from the point where it was stopped due to second failure.

Note

The scan out process includes a user-specifiable number of recovery cycles after the data is scanned out. This period allows the tester to detect end of data, to finish processing the data and to set up for any future data. The recovery cycles are part of the complete process.

5. When a restart is initiated, error detection is disabled until the controller reaches the point that resulted in the halt.
6. When the point where the controller was stopped is reached, the normal BIST operation resumes to detect further failures. Both testing and the memory operations are already back to at-speed mode at this point.

Setting the Diagnostic Mode in MBISTArchitect

In order to synthesize the diagnostic functionality into the BIST controller, the following conditions must be met:

1. The BIST controller must use a comparator for verification. You cannot enable diagnostics for ROMs.
2. Only algorithms supporting the comparator can be used. You cannot use the rom1 (rom) or rom2 algorithms.

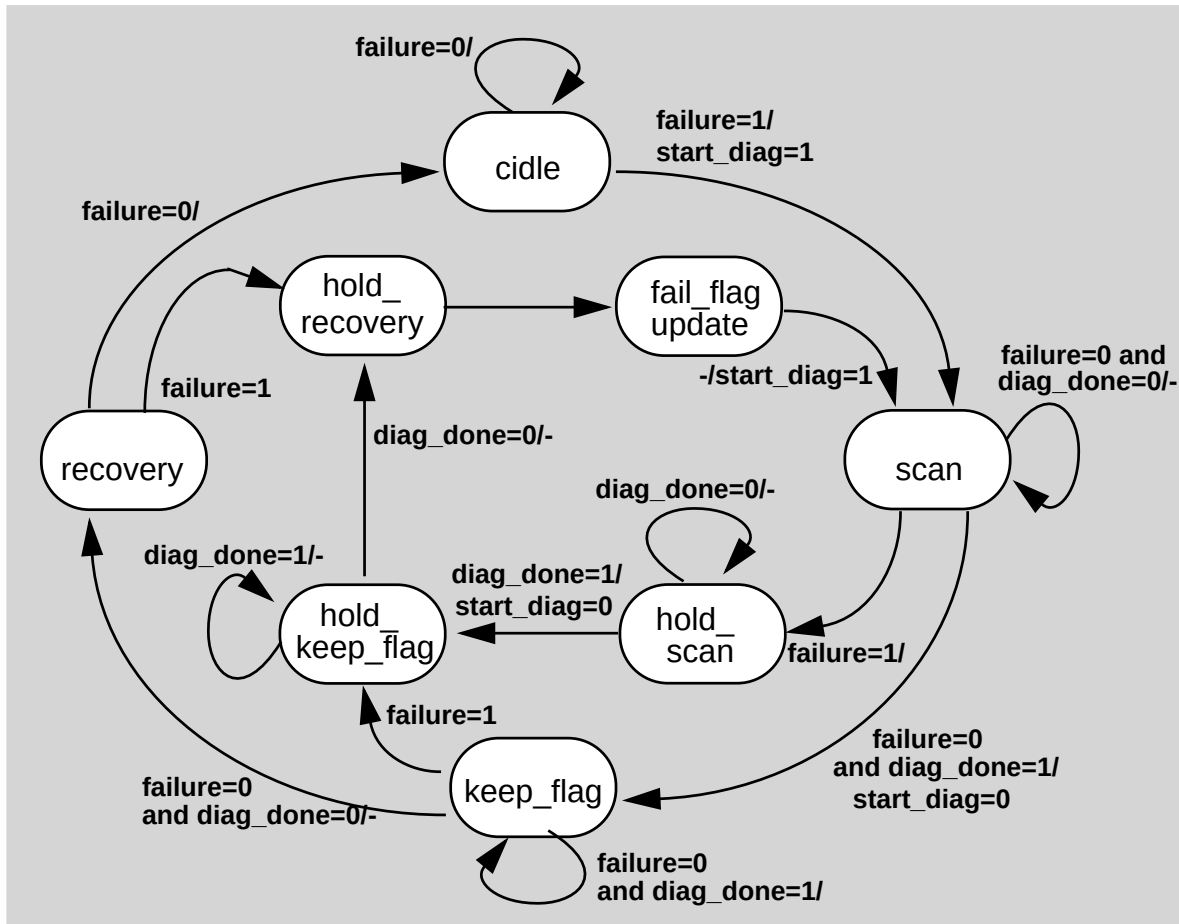
Diagnostic Clock Domains

There are two clock domains for the diagnostic process in the MBIST controller. One clock controls the diagnostic clock domain that scans out diagnostic data to the Automatic Test Equipment (ATE). A second clock domain is run by the MBIST clock that operates everything except the diagnostic data scan_out and operates at a faster clock speed. In default operation, the tool operates with these clocks in a synchronized relationship. When you turn synchronization on, these clocks become synchronized by passing information between the domains.

Synchronization Between MBIST Clock and Diagnostic Clock

The following two figures are state diagrams explaining the relationship between the clock domains. [Figure 7-3](#) diagrams the state of the tool's diagnostic control process. This process operates in the MBIST clock domain and communicates with the diagnostic scan process that is outlined in [Figure 7-4](#). Note that [Figure 7-4](#) shows the state of the diagnostic clock domain in non-synchronized functionality.

Figure 7-3. Diagnostic Control Process in MBIST Clock Domain



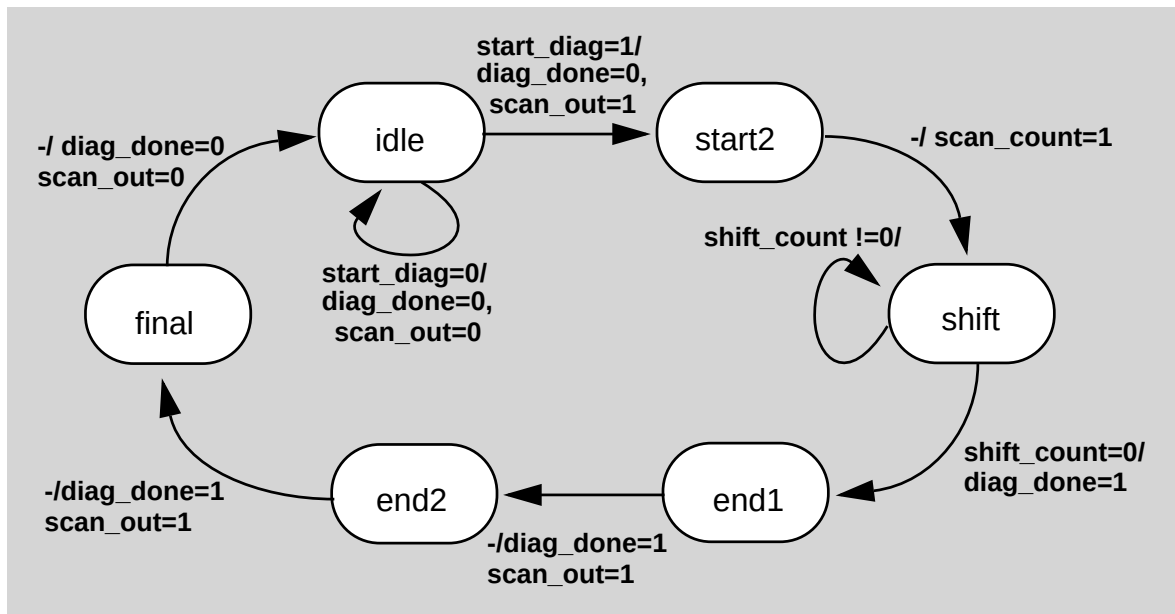
Initially, after you apply reset, the state of the diagnostic control process is in *cidle* as shown in [Figure 7-3](#), and the diagnostic scan process state is in *idle* as shown in [Figure 7-4](#).

When the MBIST controller detects a failure (*failure=1*) it activates the *fail_h* signal to inform the ATE externally and uses the internal signal “*start_diag*,” in [Figure 7-3](#), to inform the diagnostic scan process to scan out diagnostic data.

In the mean time, diagnostic scan process remains at “*shift*” state until diagnostic data are scanned out completely. If you have chosen to run the diagnostic process in the default mode, with synchronization on, the “*shift*” state will also freeze the MBIST controller operation until the scan operation completes.

Alternately, if you have chosen the asynchronous process as shown in [Figure 7-3](#), the MBIST controller operation is not held unless a second failure is detected before the scan operation is completed. In that event, the control-state advances to “*hold_scan*” and stays there until the scan operation completes.

Figure 7-4. Diagnostic Scan Process in Diagnostic Clock Domain



After diagnostic scan process scans out the diagnostic data completely, it sends “diag_done,” seen here in [Figure 7-4](#), to the diagnostic control process. Also, the “final” state resets the “diag_done” signal.

Setting Recovery and Hold Recovery States

In [Figure 7-3](#), fail_h goes high when a failure is detected. This happens between states “idle” and “scan”. The ATE should run the diagnostic clock only when fail_h is high. Thus, fail_h has to stay high until the diagnostic scan state ([Figure 7-4](#)) reaches idle state. In [Figure 7-3](#), fail_h stays high until state keep_flag detects diag_done is 0. Similarly, if second failure happens before scan out is complete, second fail_h data will be stored internally. This fail_h data will be updated at fail_flag_update state.

[Figure 7-3](#), shows that fail_h is active when the controller moves from “idle” state to “scan” state or from “fail_flag_update” state to “scan” state, and turns off when it either moves from “keep_flag” state to “recovery” state or from “hold_keep_flag” state to “hold_recovery” state.

ATE normally runs at slower clock rates than the on-chip clocks for the DUT; therefore, at-speed testing of embedded memories requires the BIST clock to run at a higher frequency than the ATE.

Depending on your choices for diagnostics modes (see command [Set Controller Debug](#) in the *MBISTArchitect Reference Manual*), it is possible for the BIST section to detect and record a second failure before the data from a first failure has all been scanned out. The tool supports a

recovery process that causes fail_h to be deactivated between the end of the scan out of the first failure, and reactivated to indicate another failure to be scanned out. Recovery consists of a number of BIST clock cycles. The number required depends on the BIST clock period, the time the ATE needs to handle recording of the first failure's data, and to set up and notice that the fail_h is deactivated.

Avoiding Timing Problems

As described above, “start_diag” and “diag_done” signals are communicated between two non-synchronous clock domains, one as sender, the other as receiver. To avoid timing problems, the sender has to hold the signal stable long enough to make sure the receiver has enough time to receive it. Also, the receiver should try to capture the signal several times. It is possible that the signal arriving time and clock edge can change at the same time. Typically, at least two clock-cycle captures are used. Therefore, the sender has to make sure the signal is stable enough for the receiver to capture at least twice.

In [Figure 7-3](#), “start_diag” starts when it moves from “cidle” state to “scan” state and stays active until it receives “diag_done,” which indicates “start_diag” is received. In [Figure 7-4](#), “start_diag” is captured by “idle” state and “start2” state to have at least two captures.

In [Figure 7-4](#), “diag_done” starts when it moves from “shift” state to “end1” state and stays active until it reaches “final” state. In other words, it will stay active for about 3 diagnostic clock cycles. In [Figure 7-3](#), “diag_done” signal is captured by scan state or hold_scan state to indicate scan data shifts out completely. As shown in [Figure 7-4](#), with 5 extra diagnostic clock cycles used for synchronization, scan_out data is padded with two cycles of “11” at the beginning and three cycles of “110” at the end of diagnostic data scan. To control the MBIST freeze operation, an internal signal “int_hold” is used to freeze MBIST operation. Signal “int_hold”, which is not shown in [Figure 7-3](#), is active when the diagnostic control process is in at “hold_scan” state, “hold_keep flag” state, “hold_recovery” state. It is off when the state is at “cidle” state and “fail_flag update” state. At other states, it will be active only if another failure is detected, otherwise, it is off.

As described above, “start2” state is added to make sure that the “start_diag” signal is captured at least twice. However, the diagnostic clock is generally activated by the ATE only after active fail_h is received, which is about the same time as “start_diag” is being activated. Since an ATE is relatively slow compared to MBIST operation, there is no risk of “start_diag” and “diagnostic clock” changing at the same time. Therefore, it is not necessary to have “start2” state.

Similarly, “end1” and “end2” states are added to make sure “diag_done” signals stays active for three diagnostic clock cycles. Since diagnostic clock cycles are generally slower than MBIST clock cycles, it is possible that one diagnostic clock cycle is long enough for the receiver to capture the sender's signal at least twice so that it can get out of the “scan” state in [Figure 7-3](#). Therefore, for a very slow diagnostic clock, we can remove “end1” and “end2” states. So, for a relatively slow ATE and ATE controlled diagnostic clock, we can reduce extra diagnostic clock cycles from 5 to 2.

Fail_h Behavior

The fail_h signal has two possible uses:

- To indicate that an error has been detected.
- To indicate that there is diagnostics data that the tester should off load when the diagnostics is running.

Because fail_h is actually a signal to the tester, which is considered to be running asynchronously with the BIST controller, some rules are needed for proper functionality.

First, in non-diagnostics mode, there are two general modes of fail_h. The modes are related to whether or not the tester can use fail_h as an edge signal, or as a level signal. It is often the case that a tester is running more slowly than the BIST controller. If fail_h is activated and then deactivated after one BIST clock cycle, the tester might miss it. So, a primary choice that is allowed is for the fail_h signal to be activated only on cycles where the BIST is seeing an error or fail_h can be activated and stay activated as soon as the first error is seen. The latter would allow a slower automatic tester to see the signal as a level signal and optionally to abort the test as soon as possible.

Fail_h can be interpreted, at the end of the BIST process, as a GO/NO-GO flag for the device under test. This behavior is considered correct when a device, which includes a diagnostics section, is run without diagnostics enabled.

When a device is run with diagnostics enabled, the fail_h signal can be viewed as a diagnostics_data_ready signal. It is a signal telling the tester that there is data to be off-loaded. This interpretation, as well as several constraints, has some implications for the signal behavior. Some of these impact behavior at the end of each set of diagnostic data and at the end of the BIST process.

With diagnostics enabled, fail_h is a level signal used to implement handshaking between the BIST controller and the tester. The signal is activated and stays activated until the tester issues enough diagnostics clocks to off load the set of data. A somewhat subtle issue is that after the data is off-loaded, the fail_h signal has to be deactivated long enough for the tester to detect its deactivated reliably. This period is specified as a number of BIST clock periods using the -recover option. At the end of the tester receiving the data from one error event, there will be “recover” BIST clocks before a second error can be reported, that is, before fail_h can be activated.

This delay is required for many testers. It can be eliminated by setting the recover value to zero. This would be feasible if the tester can handle retrieving and storage of the second error’s data immediately.

If at the end of the BIST process, there is no error data pending, then it is reasonable that this signal not be activated. If an error occurs near the end of the BIST process, then this signal will be activated during the time when BIST tst_done signal normally becomes activated.

There are several different ways for this to lead to confusion. For example, if the BIST controller is running faster than the tester (often the case), it is possible for the BIST to get done and wish to activate `tst_done` before the tester actually detects the `fail_h` signal. If the `fail_h` signal was allowed to be activated to indicate the original meaning of failed device, then the tester would not be able to decide which meaning it has at this point.

Because of this and other possible ambiguities, if there is no diagnostics data to be off-loaded at the end of the BIST test, `fail_h` will not be activated when diagnostics mode is enabled.

Chapter 8

BISA for Repair

Memories often take up a large fraction of the area on an SoC and often have very small features. Both factors mean that manufacturing faults in the memories might have a substantial impact on yield. One method to eliminate this impact is to build redundancy into the memory such that sections impacted by defects can be operationally replaced with spare sections.

When a memory has errors, the chip can be repaired by enabling the redundant resources, which override the bad part of the memory and allow it to operate normally. This small investment in die area represented by a few extra memory columns or rows can greatly improve yield. As part of this process, test engineers need a way to identify bad portions of the chip so they can choose whether or not to activate the redundant memory resources.

The MBISTArchitect tool addresses this need with a capability called BISA (Built-In Self-repair Analysis). When you enable BISA and specify a repair strategy for your memories, the tool generates on-chip logic to track memory errors found during BIST, and produces a summary report at the conclusion of testing. BISA simplifies the task of tracking errors and identifying your memories as error-free, repairable, or failed. The BISA Combined Report tells you which memories are repairable based upon their BIST results and their redundancy characteristics.

BISA is compatible with diagnostics. You may activate either or both of these features in the same BIST controller to obtain information about memory errors.

BISA Rules and Limitations

The following list outlines the BISA functionality capabilities and limitations:

- BISA does not generate redundancy in your memory, and it does not generate the logic to activate the redundancy; it is assumed that your memory architecture already contains redundancy and repair logic.
- You must use the [Add BISA Hardware](#) command to activate BISA. Even if you declare your column repair strategy for a memory in the memory model, for column repair [Method 2 \(Old Syntax\)](#), no BISA hardware will be generated for a memory until it is explicitly requested by the Add BISA Hardware command.
- You can specify a different repair strategy for each memory model. However, you cannot have both column and row repair strategies for a single controller.
- If BISA is enabled, you should not use algorithms that detect defects in the address decoders, port hardware, or write mask signals and not the memory array. Such faults

can manifest as array defects and may lead the BISA hardware to falsely conclude that a memory is repairable.

- When executing March algorithms for any port, the BISA hardware accumulates the failing information regardless of what port is used to read the data; therefore, you can not use the BISA report to determine which port caused a failure.
- BISA is intended to detect cell-array defects, but is also active for algorithms like port_interaction and port_isolation, which are intended to test memory ports and not the memory cell array. You could determine the type of error detected (cell-array or port error) using diagnostics by observing the algorithm tstate when fail_h goes high; but the BISA report would not indicate what algorithm is active when an error is detected. However, the bisa_active signal is 0 for all port_interaction steps and for steps with the port_isolation w_r operation. If an error occurs while the bisa_active signal is 0 then the repairable_h signal is also 0, which indicates a port error because port errors cannot be repaired.

Mixing Memories With and Without Redundancy

It is possible for a single BIST controller to test a combination of memories with and without redundancy and repair logic. MBISTArchitect supports this situation by allowing you to activate BISA for the specific memories you choose. For BISA, there are two types of memories:

- **RR memory** — A memory that includes redundant resources and repair logic. An RR memory is deemed repairable if the errors do not exceed the redundant resources. If an RR memory has more errors than it can repair then it has failed.
- **NR memory** — A memory which does not have redundant resources or repair logic. If an NR memory has any errors then it has failed.

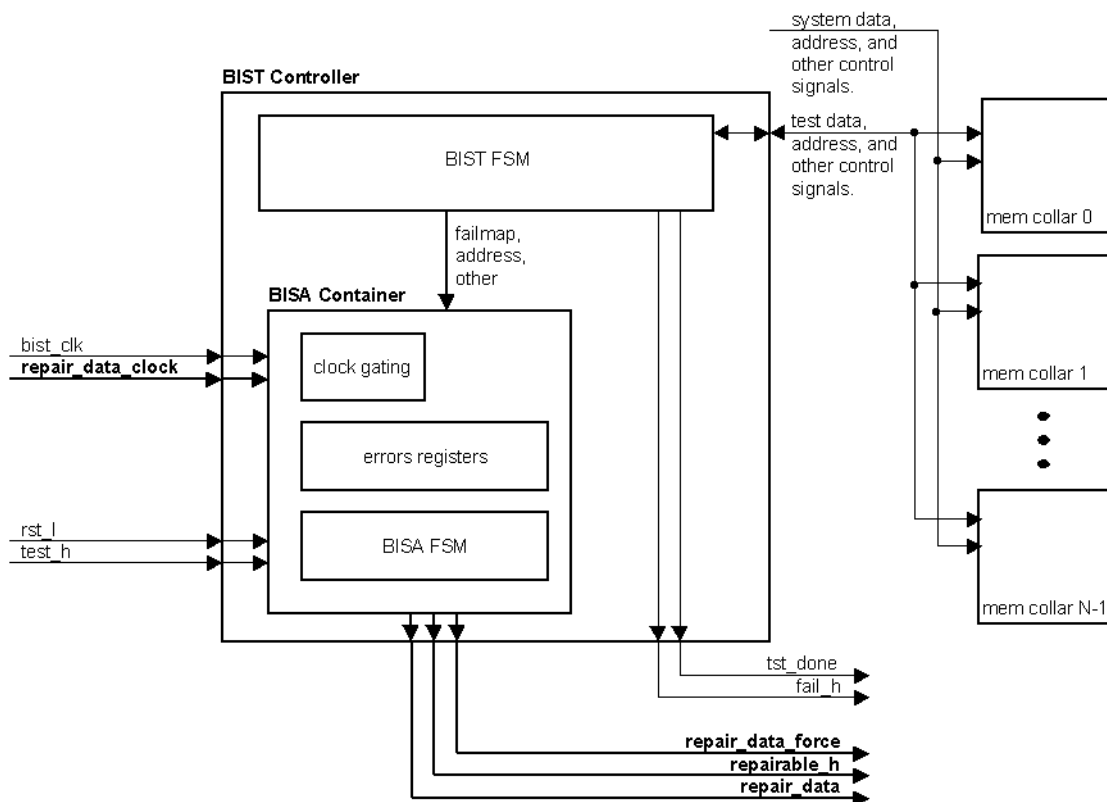
If you declare a repair strategy and activate BISA generation for a memory, then the tool interprets it as an RR memory. The tool interprets other added memory models as NR. Do not add BISA hardware to your design if all your memories are NR.

The BISA combined report will not explicitly analyze the errors seen on an NR memory. However, you will know implicitly whether or not any NR memories have failed by looking at the repairable_h port: if it is high, then all NR memories were completely error-free during test; if it is low but the Unit Reports for your RR memories show no fails, then you can deduce that the overall chip failure was due to errors in one or more NR memories. By similar reasoning, if the repair_data_force port is low, then all memories were completely error-free, both RR and NR memories. If you need to see the exact failure data of NR memories, use the Add Diagnostic Monitor command.

BISA Block Diagram

The BISA container tracks tested memory errors at-speed and provides a summary report at shift frequency. Figure 8-1 shows how BISA relates to BIST and the memory collars. BISA contains the finite state machine (FSM) logic, errors registers, and internal clock gating. The BISA block is instantiated within the BIST controller and obtains many of the signals used during BIST, such as the BIST clock, the current test address, the miscompare failmap, control signals like test_h and rst_l, and BIST-generated internal signals, which tell whether the failmap is valid. The miscompare failmap is a vector of errors on the currently tested memory or memories, and it is the XOR of the actual data word versus the expected data word.

Figure 8-1. BISA Block Diagram



The use of BISA adds four new scalar ports to the BIST controller interface:

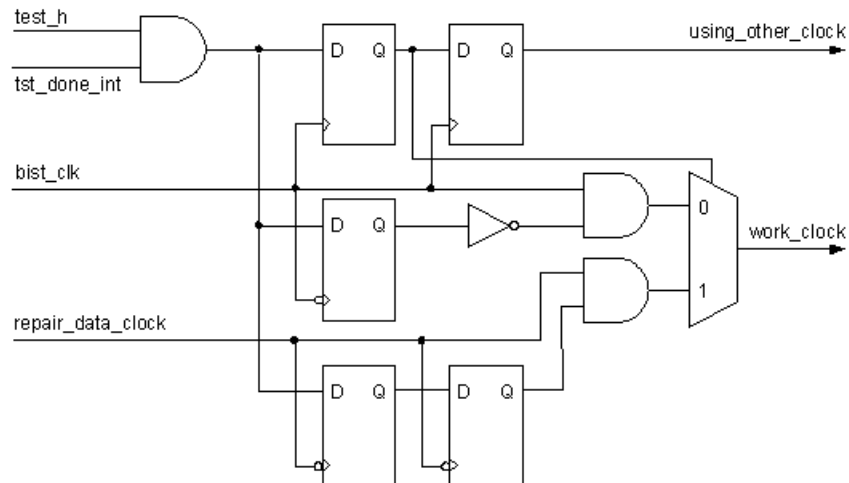
- **Input repair_data_clock** — BISA employs this clock to shift out the repair data. The clock gating logic chooses this clock while test_h is high and tst_done has gone high. It is separate from bist_clk because the tester may require a slower frequency to receive the repair data.
- **Output repair_data_force** — If this port goes high with tst_done, then the repair data is valid and the tester will record it. If it stays low, then no errors were detected in any of the memories and there is no need for a repair report.

- **Output repairable_h** — This port goes high with `tst_done` if memory errors were detected but do not exceed the available redundant resources, in which case the memories are repairable. It stays low if there are no errors, or if errors exceed resources.
- **Output repair_data** — The summary report is shifted out serially on this port, clocked by the `repair_data_clock`. For a description of the report, see “[BISA Report Format](#)” on page 214.

For information on the synchronization of the BISA ports, see the “[BISA Timing Diagram](#)” on page 220.

BISA employs a clock gating device, shown in [Figure 8-2](#), to switch between the regular BIST clock and the `repair_data_clock`. The reason for the distinction is that you might want to run BIST using the normal operational speed of the memory, whereas you might want to use a slower ATE clock to offload the BISA combined report into your tester.

Figure 8-2. BISA Clock Gating



Activating BISA With a Column Repair Strategy

Use one of the following two methods for activating BISA with a column repair strategy:

- **Method 1** — You declare the repair strategy when activating BISA with the Add BISA Hardware command. Method 1 uses the most current functionality and is the preferred method.
- **Method 2** — You must declare the repair strategy in the memory model. Method 2 uses the syntax from previous versions of MBISTArchitect.

 **Note** Method 2 will be phased out in future releases. It is recommended that you transition your dofiles to use the syntax in Method 1.

You can use Method 1 even if you already have a strategy declared in the memory model. MBISTArchitect will ignore the strategy in the memory model and use the strategy provided with the `-Column` switch of the `Add BISA Hardware` command.

Method 1 (Preferred)

1. Instantiate memories for a controller using the [Add Memory Models](#) command.
2. Activate BISA generation and declare the column repair strategy for one or more memories using the [Add BISA Hardware](#) command with the following syntax:

```
ADD BISA Hardware -COLumn nredundant [ -Format Bits | Index ]
[ -All | -MEMids list... ]
```

Method 2 (Old Syntax)

1. Declare each column repair strategy in the memory model(s) using the **repair** keyword in the parameter statements. Use the following library file syntax inside the `bist_definition` of the memory model:

```
repair logical_column [ <nspares> [ bits | index ] ] ;
```

Arguments:

- **nspares** — An optional positive integer that cannot be less than 1. This specifies the number of spare columns in the memory. The default value is 1.
 - **bits | index** — An optional keyword that specifies **bits mode** or **index mode** reporting. The default is bits mode.
2. Instantiate memories for a controller using the [Add Memory Models](#) command.
 3. Activate BISA generation for one or more memories using the [Add BISA Hardware](#) command with the following syntax:

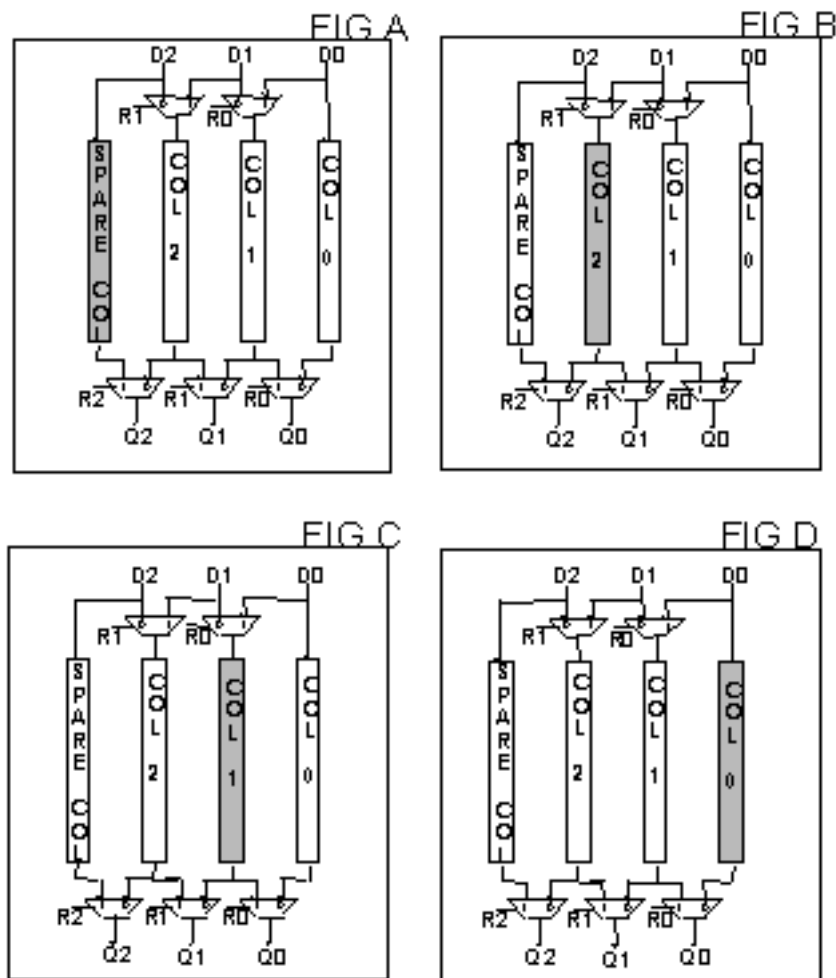
```
Add BISA Hardware {-All | memory_number...}
```

BISA assumes that the architecture of your column repair permits a spare column to be used to replace any bad column in the memory. If `addr_inc` is greater than 1, then each redundant resource may be used to repair a bit of a single word on the row.

[Figure 8-3](#) shows how you might add mux hardware to the column decoder and data bus in order to implement a column repair strategy for your memory. In [Figure 8-3](#), the memory is defined with `data_size 3`, `addr_inc 1`, and one spare column resource. In each of the four sub figures, the shaded column is the one made inactive by the repair logic.

For more information on the definition of `addr_inc` and other related memory model parameters, see “[Parameters](#)” on page 57.

Figure 8-3. Redundant Column Repair Example



[Table 8-1](#) gives the values of R0, R1, and R2, which you would apply to repair an error in one memory column in this example.

Table 8-1. Example Column Repair Controls

Bad Column	R2	R1	R0	Figure
No errors	0	0	0	Fig A
Column 2	1	0	0	Fig B
Column 1	1	1	0	Fig C
Column 0	1	1	1	Fig D

The previous table is just an example, and the exact column repair logic is dependent upon your memory architecture and does not affect BISA.

BISA reports column errors in one of two formats: bits mode and index mode. With bits mode reporting, BISA maintains a column vector with one bit for each column. The column vector bits are 0 for good columns and 1 for bad columns. The BISA report for a column-repairable memory in bits mode shifts out the entire column vector. With index mode reporting, BISA encodes the index of each 1 in the column vector, starting at the vector MSB, and reports up to N column indices, where N is the number of spare columns. Either way, the columns are reported in their logical order, regardless of the value of the optional memory model parameter `top_word`.

The column vector is always internally present because when a test is running at-speed, there might not be enough time to capture the column error and encode its index in one `bist_clk` cycle. Therefore, reporting in bits mode is the default; but you can configure BISA to take extra cycles and report in index mode instead. The optional process of encoding vector bits to indices is completed before the repair data begins shifting out.

Note that in a sequential-contiguous test with multiple memories, the bits-mode column vector only reports 1s for the first N errors, starting at the vector MSB. The reason is that the internal column vector register is cleared and re-used for each memory, and the first N error indices of each memory are saved separately. This is more area-efficient for the BISA container than allocating one column vector for each memory.

For more information about sequential-contiguous testing, see [Figure 6-8](#) on page 185.

Activating BISA With a Row Repair Strategy

Use the following method to active BISA with a row repair strategy:

1. Instantiate memories for a controller using the [Add Memory Models](#) command.
2. Activate BISA generation and declare the row repair strategy for one or more memories using the [Add BISA Hardware](#) command with the following syntax:

```
ADD BISA Hardware -ROW nredundant [ -All | -MEMids list... ]
```

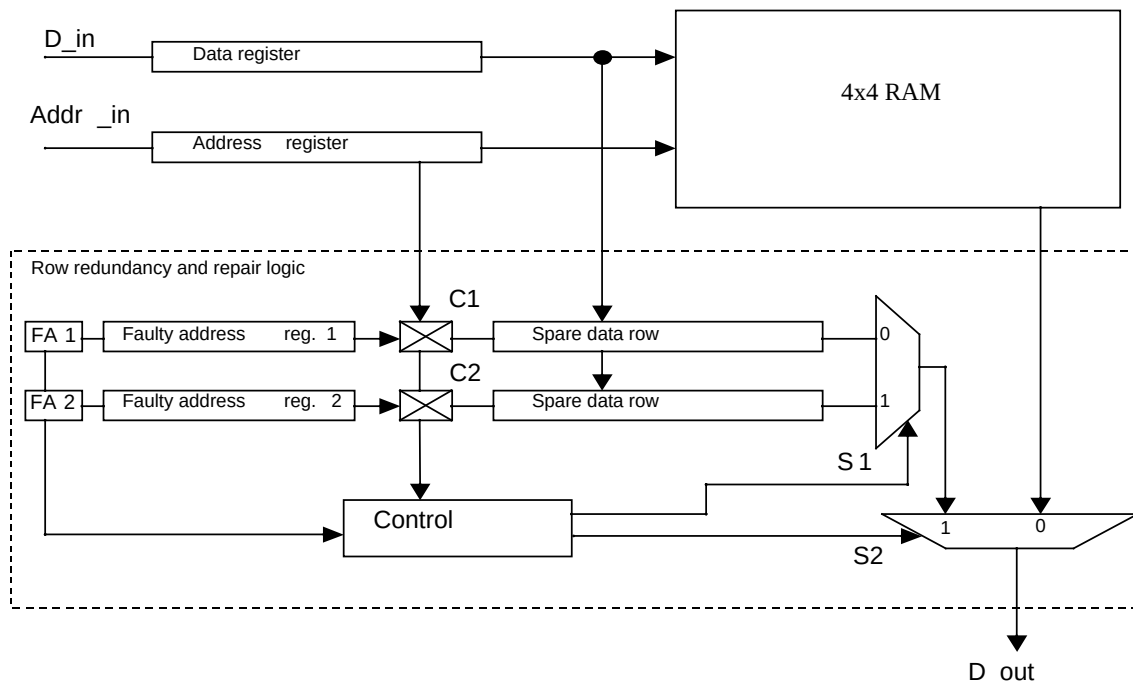
Use the Add BISA Hardware command multiple times to declare different strategies for each memory.

BISA assumes that the architecture of your row repair permits a spare row to replace any bad row in the memory. If `addr_inc` is greater than 1, then each redundant row replaces all words on the original row.

[Figure 8-4](#) shows how you might add mux hardware on the row decoder and output bus to implement a row repair strategy. Your row repair logic may differ, but this does not affect BISA. The example memory is defined with `data_size 4`, `addr_inc 1`, and two spare rows. When

the address matches a faulty row address (XNOR comparator), the spare data row is shifted out instead of the original row.

Figure 8-4. Redundant Row Repair Example

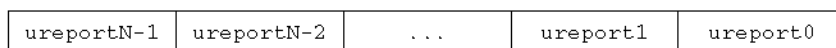


For more information on the definition of `addr_inc` and other related memory model parameters, see “[Parameters](#)” on page 57.

BISA Report Format

When testing concludes and BISA analysis is complete, the tester begins toggling the `repair_data_clock` to start shifting out the BISA combined report, which is a concatenation of unit reports (shown in [Figure 8-5](#) as *ureport*) from each BISA-active memory. The combined report appears serially on the `repair_data` port, starting with bit 0 of the first unit report:

Figure 8-5. BISA Combined Report



Each unit report for a memory consists of one or more report blocks followed by a repairable-bit, shown as “R” in the following figure.

Figure 8-6. BISA Unit Report

R	blockN-1	blockN-2	...	block1	block0
---	----------	----------	-----	--------	--------

Each report block is formatted as shown in the following figure, where “V” is the valid bit.

Figure 8-7. BISA Report Block

V	memid	error_scene
---	-------	-------------

In the three previous diagrams the MSB (the last bit shifted out) is on the left and the LSB (the first bit shifted out) is on the right. The following are the definitions of the data fields.

- **Repairable bit** — Set to 1 if the memory was error-free or repairable; set to 0 if the memory failed.
- **Valid bit** — Set to 1 if the error scene contains real data; otherwise set to 0. For column repair index mode, the error scene is not valid unless there was an error to report. For column repair bits mode, the error scene is always valid; in bits mode, there is only one report block and it represents the column vector.
- **Memid** — A 0-based memory id, using the order of arguments given in the Add Memory Model command. This field is omitted if there is only one added memory.
- **Error Scene** — For row repair, this contains the row address (in the case of descrambling, the address to be scanned out is the address that is applied to the descrambling logic). For column repair, this contains the column vector or column index (as declared with the column repair strategy). Scene size is standardized to the largest scene size used in any unit report, so that every report block is the same number of bits. Shorter error scenes are left-padded with 0s to the MSB.

All zeros in a column vector means “no errors,” whereas a valid all zeros column index means “error in column 0.” All zeros in a row address means “error in row 0” if the valid bit is 1.

 **Note** NR memories are not part of the combined report, and there is no combined report at all if all memories were error-free during test.

Example - BISA Report for Column Bits and Column Index

The following example shows a memory model description of two column-repairable memories; one using bits mode reporting and the other using index mode reporting.

```
//column-repair.lib
model memA (...)(...
    bist_definition( ...
        data_size = 8;
        addr_inc = 1;
        repair logical_column 1 bits;
        ...

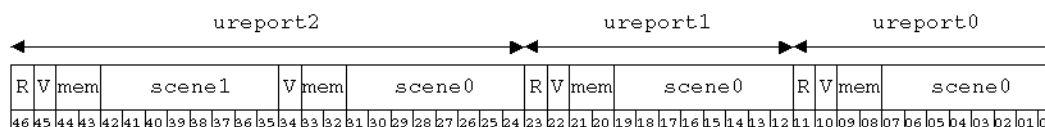
model memB (...)(...
    bist_definition( ...
        data_size = 16;
        addr_inc = 4;
        repair logical_column 2 index;
        ...
```

The memory models might be instantiated in a BISTGEN dofile as follows:

```
//bistgen1.do
add memory model memA memA memB
add bisa hardware
```

The previous example adds 2 memA with 1 spare column each and 1 memB with 2 spare columns. The column vectors for memA have $8 \times 1 = 8$ bits. The column index for memB has $\log_2(16 \times 4) = 6$ bits. The error scene in all cases will be 8 bits, so the column index fields are left-padded with two 0s. Since there are three added memories, the memid field is 2 bits wide. [Figure 8-8](#) shows the field format of the combined report.

Figure 8-8. Combined Report for Column Repair Example



The following is the testbench format of the combined report with example values:

```
#
# ** memory number(msb downto lsb) : 00
# ** repairable bit : 0
# ** valid bit : 1
# ** column vector(msb downto lsb) : 01000011
#
```



```
#  ** memory number(msb downto lsb) : 01
#  ** repairable bit : 1
#  ** valid bit : 1
#  ** column vector(msb downto lsb) : 01000000
#
#  ** memory number(msb downto lsb) : 10
#  ** repairable bit : 0
#  ** valid bit : 1
#  ** column index(msb downto lsb) : 00000000
#  ** valid bit : 1
#  ** column index(msb downto lsb) : 00000001
#
```

In this example there were three bad columns in mem#0 (columns 0, 1, and 6), one bad column in mem#1 (column 6), and more than two bad columns in mem#2 (at least columns 0 and 1).

Example - BISA Report with Row Repair

Consider the following memory model description with two memories:

```
model memA (...) (...)
  bist_definition(
    data_size = 8;
    addr_inc = 1;
    address_size = 9;
  )
...
model memB (...) (...)
  bist_definition(
    data_size = 16;
    addr_inc = 4;
    address_size = 12;
  )
...
```

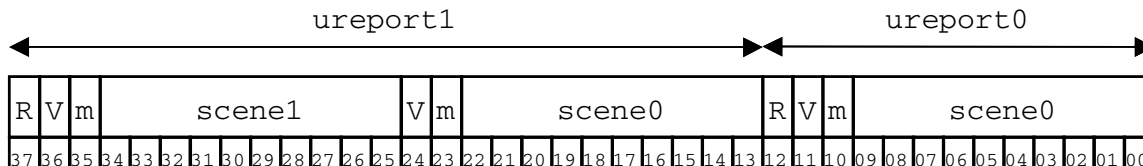
The memory models are instantiated and BISA is activated in the dofile as follows:

```
add memory models memA memB
add bisa hardware -row 1 -memids 0
add bisa hardware -row 2 -memids 1
report bisa hardware
```

```
Built-In Self-Analysis Hardware Setup
Memid Model Strategy Nredundant Noriginal Format
0 memA row 1 512
1 memB row 2 1024
```

Taking into account the values of `addr_inc` and `address_size`, we know that memA has $2^9/1 = 512$ rows and memB has $2^{12}/4 = 1024$ rows. The row address size for memA is 9 bits and for memB is 10 bits. Because the error scene uses the largest scene size for all scenes, the row address reported for memA will be left-padded with a 0. Because there are two memories, the memid field is 1 bit wide. [Figure 8-9](#) shows the combined report for this example.

Figure 8-9. Combined Report for Row Repair Example



The following is the testbench format of the combined report:

```
#
# ** memory number(msb downto lsb) : 0
# ** repairable bit : 0
# ** valid bit : 1
# ** row address(msb downto lsb) : 0001000011
#
# ** memory number(msb downto lsb) : 1
# ** repairable bit : 1
# ** valid bit : 1
# ** row address(msb downto lsb) : 0000000101
# ** valid bit : 0
# ** row address(msb downto lsb) : 0000000000
#
```

In this example, there was more than one bad row in memory 0 (row address 67, i.e. the 68th row) and exactly one bad row in memory 1 (row address 5, i.e. the 6th row). Therefore, memory 0 (memA) failed and memory 1 (memB) is repairable. Only the first bad row is shown in memory 0 because there is only one bisa redundancy register to store the location of bad rows.

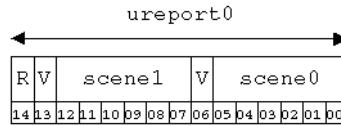
Example - BISA Report with Memid Field Omitted

This example modifies the BISTGEN dofile from the previous example to add just one memory.

```
//bistgen2.do
add memory model memB
add bisa hardware
```

With just one added memory, it is not necessary to include the memid field in the combined report. Also, unlike the previous example, it is not necessary to pad the error scene, so the column index is simply 6 bits wide. [Figure 8-10](#) shows the field format of the combined report.

Figure 8-10. Combined Report for Memid Omitted Example



The following is the testbench format of the combined report, with example values:

```
#
# ** repairable bit : 1
# ** valid bit : 1
# ** column index(msb downto lsb) : 100001
# ** valid bit : 0
# ** column index(msb downto lsb) : 000000
#
```

In this example the tested memory had one bad column, whose 0-based index is 33 (the 34th column in the row). Since the memory has two spare columns, it is repairable.

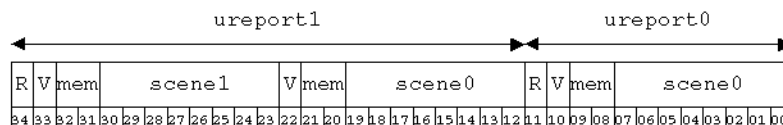
Example - BISA Report with RR and NR Memories

Using the memory model description file from [Example - BISA Report for Column Bits and Column Index](#), this example modifies the BISTGEN dofile to add three memories but designates one of them as NR, that is, it has no redundancy or repair logic.

```
//bistgen3.do
add memory model memA memA memB
add bisa hardware 1 3
```

Two instances of memA are specified; however, the first memA includes redundancy and repair logic (RR) whereas the second memA does not (NR). The added memB is also RR. Even though two RR memories are specified, there are still three memories total, so the memid field uses two bits to uniquely identify its associated memory. [Figure 8-11](#) shows the field format of the combined report.

Figure 8-11. Combined Report for RR and NR Example



The following is the testbench format of the combined report, with example values:

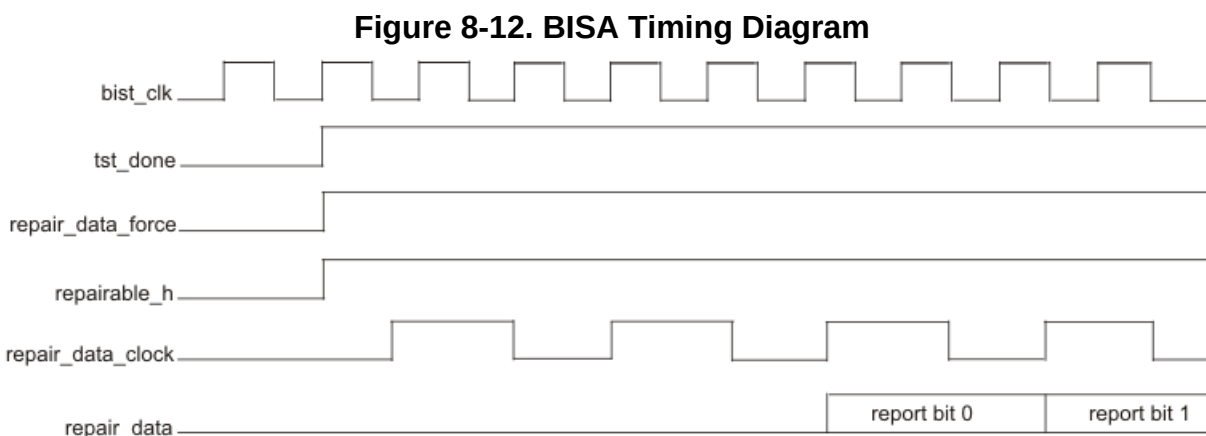
```
#
```

```
# ** memory number(msb downto lsb) : 00
# ** repairable bit : 1
# ** valid bit : 1
# ** column vector(msb downto lsb) : 00000000
#
# ** memory number(msb downto lsb) : 10
# ** repairable bit : 1
# ** valid bit : 1
# ** column index(msb downto lsb) : 00001100
# ** valid bit : 1
# ** column index(msb downto lsb) : 00000001
#
```

In this example, mem#0 was error-free and there were two defective columns in mem#2 (columns 12 and 1). The value of repairable_h is not shown; if it is high, then mem#1 was error-free, otherwise mem#1 failed.

BISA Timing Diagram

Figure 8-12 illustrates the timing of the BISA repair signals as seen on the BIST Controller interface.



In Figure 8-12, the example timing diagram shown defines the **bist_clk** and **repair_data_clock** with asynchronous waveforms, representing a scenario where the tester has a slow clock for offloading repair data, and the tester initiates the slow clock at an arbitrary time after **tst_done** goes high.

When **tst_done** goes high, memory test and BISA processing has completed. If a combined report is applicable, the **repair_data_force** signal goes high at the same time as **tst_done**. Similarly, if **repairable_h** is going to be activated (asserted), it will happen in this same cycle. (These three signals remain high even after the combined report has completely shifted out.) After **tst_done** goes high, the fast **bist_clk** is no longer used inside BISA and the tester should begin toggling the **repair_data_clock**. With the third positive edge or rising edge (posedge) of **repair_data_clock**, BISA shifts out the first bit of the combined report. After the last report bit

has been shifted out, it remains on the repair_data port even if the repair_data_clock continues to toggle.

Note that any available diagnostics output is shifted out before tst_done goes high. For more information, see “[Scanning Out Diagnostic Data](#)” on page 196.

Chapter 9

Controller Test Description Language

The Controller Test Description Language (CTDL) is used to provide information to test the controllers in an SoC design. Information required for testing controllers includes information required for initialization of SoC controllers, and instance-specific controller_access protocols.

CTDL is a mechanism to exchange information about a controller's testing resources. Testing resources are divided into information needed for operational modes, and information used for test vector re-use.

CTDL uses a common language with required information contained in multiple files. Information required in CTDL includes:

- A mandatory controller description that identifies the I/Os and their type, and identifies special signals.
- CTDL describes the “controller_access” and “controller_test” procedures required for each controller.
- CTDL describes SoC level timeplates which can be used to replace controller timeplates when mapping patterns.

Information found in the CTDL syntax can be grouped into two major areas:

- Controller test description.
- Controller test access.

This information is supplied in separate files called the Controller Test Description file and the Controller Test Access file. You must provide a unique controller test description file for each controller being tested.

The Controller Test Description file and Controller Test Access file are separate files, but they actually share a common syntax and a common parser. The controller test description file is generated through the BIST generation process, while the controller test access file is generated through the BIST insertion process.

CTDL Syntax Conventions

The following rules apply to CTDL syntax. Words that are **bold** denote keywords. Words that are *italicized* denote lexical elements such as identifiers, strings or numbers. Plain words are syntax rules which are further described. Rules consist of the rule name, followed by a colon,

and then the syntax elements of the rule. The rule ends with a semicolon. A bar “|” character stands for “or”.

Optional elements are described by using the square brackets []. Lists are described by using square brackets with an ellipsis “...”, such as [...]. Lists of one or more have the first element outside the square brackets, such as pin_name [,pin_name]. Lists of zero or more have the first element within the square brackets.

CTDL syntax is based on the test procedure file syntax used in the MBISTArchitect™ tools. The syntax in the examples show the CTDL syntax and the test procedure file syntax.

Controller Test Description File Contents

The Controller Test Description file should contain the following parts:

- [Controller Declaration Block](#)
- [Timeplate Definitions](#)
- [Controller Test Procedure](#)

Example - Controller Test Description File

The following is an example controller test description file.

```
// CTDL for ram256x16_multi_bist
// MBISTArchitect v8.2004_3.10-prerelease Wed Jun  2 08:57:23 PDT 2004
// Filename      : ram256x16_multi_bist.v.ctdf
// Date          : Thu Jun  3 14:17:57 2004
```

```
core ram256x16_multi_bist =
    output Test_a_0[7:0];
    output Test_ws_0;
    output Test_ds_0;
    output Test_as_0;
    output Test_ceb_0;
    output Test_web_0;
    output Test_oeb_0;
    output Test_in_0[15:0];
    output Test_a_1[7:0];
    output Test_ws_1;
    output Test_ds_1;
    output Test_as_1;
    output Test_ceb_1;
    output Test_web_1;
    output Test_oeb_1;
    output Test_in_1[15:0];
    output Test_a_2[7:0];
    output Test_ws_2;
    output Test_ds_2;
    output Test_as_2;
```



```
        output Test_ceb_2;
        output Test_web_2;
        output Test_oeb_2;
        output Test_in_2[15:0];
        output tst_done;
        output fail_h;
        input Test_out_0[15:0];
        input Test_out_1[15:0];
        input Test_out_2[15:0];
        input test_h;
        input clk;
        input rst_l;

        clock clk;
        clock_lo rst_l;
    end;
```

Controller Declaration Block

The controller declaration consists of a block that names the controller and describes the controller I/Os. The information provided by this block verifies that I/O types (inputs and outputs) in the controller description file match the controller I/Os in the netlist.

In addition to pin directions, certain pin types can also be specified (clock, test_en, clock_lo, and so on) to help guide the construction of controller access methods. The declaration block must always precede references. The controller declaration section consists of a controller definition statement that has the following format.

```
core controller_name =
    controller_statement
    [controller_statement ...]
end;
```

The controller_statement defines the ports on the controller's boundary using the pin_type statement. Special ports that reference the ports and I/Os are assigned followed by the access method. The clock, clock_lo, and test_en statements do not declare new signals, but add information to signals which are already declared with this pin_type statement.

```
controller_statement:
    pin_type pin_name [, pin_name ...];
    clock pin_name [, pin_name ...];
    clock_lo pin_name [, pin_name ...];
    test_en pin_name [, pin_name ...];
    default access_method method;
```

- **core controller_name**
A string that specifies the name of the controller which is being defined.
- **pin_type pin_name**
A pair of strings that identifies ports (on the controller) as an input or output and

specifies the name of the port. The literal values for the `pin_type` argument are input or output.

- **clock pin_name**
A string that specifies signal names used as clocks which are active high. Clock pins are those which can cause a state element to change state.
- **clock_lo pin_name**
A string that specifies signal names used as clocks which are active low. Clock pins are those which can cause a state element to change state.
- **test_en pin_name**
A string that specifies the names of the test mode pin for the controller

Timeplate Definitions

The `core_test` procedure defined in the CTDF (Controller Test Description File) must use timeplates which are core timeplates. This is done by including a controller statement in the timeplate definition and by only referencing controller pins in the timeplate statements.

Before any force events in the `core_test` procedure can be mapped to I/O pins, a SoC timeplate needs to be defined for the controller timeplate to map to. The SoC timeplate is identified in the CTAF (Controller Test Access File). An SoC timeplate is one that does not use the controller statement, but instead uses SoC I/O pins in all of its timeplate statements.

Note



The only legal names for controller timeplates are TP0 and TP_BIST.

Timeplate Definition Statement

The timeplate definition statement has the following format:

```
timeplate timeplate_name =  
    [core controller_name]  
    timeplate_statement  
    [timeplate_statement ...]  
    period time;  
end;
```

The Timeplate_statement

Timeplates specify timing edges to be used on specific controller or SoC pins. The `timeplate_statement` declares specific input values or pins, specific output values or pins to monitor, how the clock should be pulsed, and the default timeplate.

The `timeplate_statement` example below lists statements that can be included in a `timeplate_statement`; not all of these are required.

```
timeplate_statement:
    force_pi time;
    measure_po time;
    force pin_name value;
    measure pin_name time;
    pulse pin_name time width;
```

Note

All times and widths are based upon the default of 1 ns. To change the default, use the `set time scale` statement. For more information, refer to “[Changing the Default Time Scale](#)” on page 230.

- **timeplate timeplate_name**
A string that specifies the name of the timeplate being declared for the controller.
- **core controller_name**
A string that specifies the name of the controller associated with the timeplate.
- **force_pi time**
An integer that specifies the time within a tester’s cycle (period) where an SoC input starts applying (forcing) the data. The data remains valid for the specific time period throughout the mapping process. At the end of the pattern, testers will generally maintain this value.
- **measure_po time**
An integer that specifies the time within a tester’s cycle (period) where the expected data is compared against the SoC pin data. It is assumed that this is an edge compare.
- **bidi_measure_po time**
An integer that specifies output values that the tester needs to monitor through some access mechanism.
- **force pin_name value**
A string that specifies the pin name and an integer that specifies the time for which a pin is to be forced to a specific logic throughout the mapping process.
- **measure pin_name time**
A string that specifies the pin name and an integer that specifies the time period for which a pin needs to be monitored.
- **pulse pin_name time width**
A string that specifies the pin name, an integer that specifies at what time to start the pulse, and an integer that specifies the width of the pulse. This statement can only reference clock pins that have been declared. The sum of the time and width must be less than the period.

- **period time**
An integer that defines the period of a tester cycle.

Timeplate Example

```
timeplate TP0=  
    core ram256x16_multi_bist;  
    force_pi 0;  
    measure_po 90;  
    period 100;  
end;
```

Controller Test Procedure

This procedure specifies the mechanism by which a controller may be configured for test or put in the test mode. Since there can be multiple test modes for a controller, there can be multiple controller_test procedures.

The Controller_test Procedure

The controller_test procedure is similar to the controller isolation procedure except for two new statements, the optional probe statement and the pattern_file statement. The controller-specific test procedure has the following format.

```
procedure controller_test mode L=  
    core controller_name;  
    timeplate timeplate_name;  
    pattern_file filename =  
        [pattern_statement ...]  
    end;  
    probe pin_name [, pin_name ...];  
    cycle =  
        cycle_statement  
        [cycle_statement ...]  
    end;  
end;
```

The Pattern_file Declaration Statement

The pattern_file declaration statement identifies a pattern file to use to provide the test patterns for testing this controller in this particular test mode. You can end the pattern_file statement with a semicolon after the filename, or optionally add additional pattern statements that optionally specify fault coverage and fault type measured statements. For more information on its format, refer to the argument description in [“The Cycle Statement”](#) on page 229.

The Probe Statement

The probe statement specifies which I/Os of the controller need to be “contacted” for test.

```
probe pin_name [, pin_name...];
```

The Cycle Statement

The final portion of the test procedure is the cycle statement. Cycles contain event statements that specify what happens during that cycle. Single or multiple cycles are used to specify what sequence of events are used to place the controller into this particular test mode. A hold statement in the controller_test procedure is in effect from the time it is specified, until the controller is removed from that test mode.

```
cycle_statement:  
    hold pin_name value;  
    expect pin_name value;  
    force pin_name value;  
    pulse pin_name time width;
```

Note



The cycle procedure in the controller_test procedure is similar to the controller_isolate procedure. The controller_test procedure can have hold, force, pulse, and expect statements, and these statements can occur in multiple cycles.

- **core controller_name**
A string that specifies the name of the controller that this test procedure is associated with.
- **timeplate timeplate_name**
A string that specifies the name of the timeplate being referenced.
- **pattern_file filename**
A string that specifies which pattern file will be used to provide the test patterns for testing this controller in this particular test mode. If more than one pattern_file statement is specified in a controller_test procedure, the pattern files are parsed in order and concatenated. The pattern file statement can take two forms, the first form just specifies the file name and is followed by a semicolon.
- **format format_type**
A literal that specifies the format in which to save the pattern file. Currently, valid format type options are:
 - **WGL**
This option writes the patterns in the WGL format.
- **probe pin_name**
A string that specifies which I/Os of the controller need to be “contacted” for test.

- **hold pin_name value**
A string that specifies the name of the pin and an integer (0 or 1) that defines the hold value for the pin. It implies that a particular primary input (PI) needs to be held at a certain value as long as the controller is in that particular mode (from time it is issued, until the end of this procedure). Any subsequent hold statement referencing the same primary input in the same test procedure or applied sub-procedures is illegal. The set of conditions specified by a hold statement can be viewed as static constraints (static during that particular mode) which ensure that the controller is “held in a particular test mode” in that particular state. In this example, the controller is held in a controller_test state.
- **expect pin_name value**
A string that specifies the name of an output pin which has a known value after the controller is put into a test state. In this case, the expect statement specifies that output y is at a logic value 1 throughout the test mode.
- **force pin_name value**
A string that specifies a name of a controller pin that is to be forced to a specific logic value throughout the mapping process. This pin must be probed. This value is valid until another force statement changes it, or some other pattern activity changes it.
- **pulse pin_name time width**
A string that specifies the pin name, an integer that specifies at what time to start the pulse, and an integer that specifies the width of the pulse. This statement can only reference clock pins that have been declared. The sum of the time and width must be less than the period.

Controller Test Procedure Example

```
procedure core_test run_bist =  
    core ram256x16_multi_bist;  
    timeplate TP0;  
    probe tst_done, fail_h, clk, rst_l;  
    pattern_file ram256x16_multi_bist.wgl;  
    cycle =  
        hold test_h 1;  
    end;  
end;
```

Changing the Default Time Scale

You can change the default time scale of 1 ns, used in the CTDL files, by adding the set time scale statement. This statement is used outside any other declaration. The CTDL files do not have to have the same time scale.

Different Time Scales

It is sometimes required to have CTDL files that have different time scales. An example of this is when multiple memory BIST controllers are used that have different timing requirements.

When different time scales are specified in different procedure or CTDL files, all timing information being loaded will be converted to the smallest of the time scales specified, and this time scale will become the current time scale. See the following [Example - Using with Different Time Scales](#).

Time Scale Statement

The basic syntax of the statement is:

```
set time scale <time_value> <units>;
```

Where the set time scale <time_value> and <units> is an integer and string pair that specifies the time scale and units for the CTDL file.

The valid units are: “fs”, “ps”, “ns”, “us”, and “ms”.

Example - Using the Same Time Scale

```
set time scale 10.0 ns;

core arm1 =
  input a, b, c, e, si, sen, clk;
  output x, y, so;
  inout v, w;
  enable e;
  clock clk;
  scan_in si;
  scan_en sen;
  scan_out so;
  default access_method parallel;
  access_method parallel = clk, si, so;
  access_method serial_hold = e
end;
```

Example - Using with Different Time Scales

my_file_1

```
set time scale 1.0 ns;
```

my_file_2

```
set time scale 10.0 ps;
```

Independent of the order in which these two files are parsed, when the parsing is finished, the time scale will be set to 10.0 ps, and all of the timing information in the my_file_1 file will be converted from a time scale of 1.0 ns to a time scale of 10.0 ps.

Controller Test Access File Contents

The Controller Test Access File (CTAF) describes a mechanism by which stimulus can be delivered to the controller input pins, and the controller outputs can be observed via the SoC I/Os. Not all I/Os of a controller need to be accessed from the I/Os of a chip. Only the I/Os essential for testing the controller need to be accessed.

A controller test access file is required by the integration phase of the tool. While a Controller Description File (CTDF) is only needed for each type of controller (and not for each controller instance), a controller test access file contains information associated with each instance or usage of a controller. The information for each instance can be in separate files, or all of the information can be present in a single file (it can even be present in the controller test description file).

The controller test access file must contain the following:

- [Controller Instance Declaration](#)
- [Controller Access Procedure](#)
- [Timeplates for Test Patterns](#)

Example - Controller Test Access File

An example of a Controller Test Access File (CTAF) is shown below. An explanation of each of the sections in the file follows.

```
// -----  
// File Type:      Controller Test Access File  
// Date Created:   Thu Jun  3 14:17:57 2004  
// Tool Version:   v8.2004_3.10-prerelease   Wed Jun  2 08:57:23 PDT 2004  
// -----  
  
timeplate soc_timeplate =  
    force_pi 0 ;  
    measure_po 90 ;  
    period 100 ;  
end;  
  
core_instance /cntrl_ram256x16 =  
    core ram256x16_multi_bist ;  
    map test_h = test_h_1, tst_done = tst_done_1, fail_h = fail_h_1 ;  
    map clk = bist_clk_3, rst_l = rst_l_1 ; end;  
  
procedure core_access =
```



```
timeplate soc_timeplate ;
core_instance /cntrl_ram256x16 ;
cycle =
    force bist_clk_1 0 ;
    force rst_l_1 1 ;
    force test_h_1 0 ;
end;
end;
```

Controller Instance Declaration

The controller instance declaration provides a description of how the controller I/Os map to the SoC I/Os for a particular instance of a controller. It also maps the controller timeplates and names the scan group. The controller_instance declaration specifies the instance name of the controller, the name of the controller, and any mapping information known about how the SoC I/Os connect to the controller instance I/Os. The controller_instance declaration refers to both the controller and SoC I/Os.

The Controller Instance Statement

The controller_instance statement contains the following information:

```
core_instance instance_name =
    core controller_name;
    map controller_pin_name = soc_pin_name [, controller_pin_name =
        soc_pin_name];
    map_timeplate controller_tp_name = soc_tp_name [, controller_tp_name =
        soc_tp_name];
end;
```

The Map Statement

In a map statement, the controller I/Os are on the left side of the equation, while the chip I/Os are on the right side of the '=' sign.

The map statement(s) specify the controller SoC I/O mappings which map controller input to specific chip input.

- **controller_instance instance_name**
A string that specifies the name of a controller. This must specify a complete path.
- **core controller_name**
A string that specifies the name of the controller.
- **map controller_pin_name = soc_pin_name**
A pair of strings that maps the controller input to specific chip input. In the code example, two map statements specify the controller-SoC I/O mapping (the two statements could also be combined into one) which specify the following mapping:

controller input *a* maps to chip input *in1*, controller input *b* maps to chip input *in2* and so on. In a map statement, the controller I/Os are on the left-hand side, while the chip I/Os are on the right-hand side of the equal sign.

- **map_timeplate controller_tp_name = soc_tp_name**
A pair of strings that maps the controller's timeplate definition to the SoC's timeplate definition. The two timeplates must have equivalent pin statements. If a controller pin is pulsed in the controller timeplate, the SoC pin that the controller pin is mapped to, must also be pulsed. The same is true for pins that are forced. For more information on mapping timeplates, refer to "[Automatic Timeplate Mapping](#)" on page 238.

Controller Access Timeplate Definition

All controller_test and controller_isolate procedures defined in the Controller Test Description file must use timeplates which are controller timeplates. This is done by including the controller statement in a timeplate definition and by only referencing controller pins in the timeplate statements.

In the controller test access file, before the controller_test or controller_isolate procedures can be mapped to SoC I/O pins, a SoC timeplate needs to be defined. An SoC timeplate is one that does not use the controller statement, but instead uses SoC I/O pins in all of its timeplate statements.

The Controller Test Access file defines an SoC timeplate, called "soc_timeplate", which is a system timeplate that lists the SoC I/O pins test parameters. In order for these two timeplates to be used in the mapped patterns, the timeplate needs to be mapped using the map_timeplate statement within the controller instance declaration.

Timeplate Definition

The timeplate definition contains the following information:

```
timeplate timeplate_name =  
    [core controller_name]  
    timeplate_statement  
    [timeplate_statement ...]  
    period time;  
end;
```

A timeplate definition specifies the timing edges used on signals and the timing for a single tester cycle. The timeplate definition lists the appropriate syntax for the timeplate_statement, followed by a definition of each statement, and an example timeplate_statement. This statement lists all options that can be included in a timeplate_statement, not all of these are required.

```
timeplate_statement:  
    force_pi time;  
    measure_po time;  
    force pin_name value;
```

```
measure pin_name time;  
pulse pin_name time width;
```

Controller Access Procedure

The `controller_access` procedure specifies the sequence of events that are needed to activate the path to the controller specified in the mapping statements in the `controller_instance` declaration. The `controller_access` procedure is required to have an instance statement to specify which `controller_instance` it is associated with.

The `controller_access` procedure is applied before the `controller_test` procedure is applied, and the held values are valid until the end of the `controller_isolate` procedure, after all of the patterns from the controller test pattern file have been applied (from WGL file).

Controller_access Procedure Syntax

The `controller_access` procedure has the following format:

```
procedure controller_access =  
    core_instance instance_name;  
    timeplate timeplate_name;  
    cycle definition  
    [cycle definition]  
end;
```

Timeplate Statement

The `timeplate` statement in the `controller_access` procedure specifies which timeplate to use for the events in the `controller_access` procedure. The `controller_access` procedure contains cycle definition and cycle statements that identify how testing must occur during the cycle.

```
cycle_definition:  
    cycle =  
        cycle_statement  
        [cycle_statement...]  
    end;  
  
cycle_statement:  
    force pin_name value;  
    hold pin_name value;  
    pulse pin_name time width;
```

- **controller_instance instance_name**
A string that specifies the name of the controller instance being accessed.
- **timeplate timeplate_name**
A string that specifies the name of the timeplate being referenced.

- **hold pin_name value**
A string that specifies the name of the SoC pin to force to a specific logic value throughout the mapping process. This statement results in the tool inserting the appropriate logic to maintain this value during test mode. It implies that a particular primary input (PI) needs to be held at a certain value as long as the controller is in that particular mode (from time it is issued, until the end of this procedure). Any subsequent hold statement referencing the same primary input in the same test procedure or applied sub-procedures is illegal.
- **force pin_name value**
A string that specifies a name of a SoC pin that is to be forced to a specific logic value throughout the mapping process. This value is valid until another force statement changes it, or some other pattern activity changes it.
- **pulse pin_name time width**
A string that specifies the pin name, an integer that specifies at what time to start the pulse, and an integer that specifies the width of the pulse. This statement can only reference clock pins that have been declared. The sum of the time and width must be less than the period.

Example - Controller Access Procedure

```
procedure core_access =  
  timeplate soc_timeplate ;  
  core_instance /cntrl_ram256x16 ;  
  cycle =  
    force bist_clk_1 0 ;  
    force rst_l_1 1 ;  
    force test_h_1 0 ;  
  end;  
end;
```

Timeplates for Test Patterns

There are two types of timeplates used in the various CTDL files and in controller pattern files:

- **Controller timeplates**
Controller timeplates are any timeplates that describe timing at the boundary of a controller.
- **SoC timeplates**
SoC timeplates describe timing at the periphery of the SoC device under test.

Controller Timeplates

Controller timeplates include a controller statement which identifies the controller that the timeplate belongs to. Controller timeplates only reference controller pins defined in the controller declaration. All controller_test and controller_isolate procedures defined in the controller test description file must use timeplates that are controller timeplates.

Note

The only legal names for controller timeplates are TP0 and TP_BIST.

In addition to the timeplates defined in the CTDL files, there are also timeplates defined in the pattern file(s) referenced by the controller_test procedures. Timeplates defined in the pattern files are treated as controller timeplates.

SoC Timeplates

SoC timeplates are identical to timeplates used in the test procedure files in Mentor Graphics tools.

All controller timeplates (from the CTDL files and from the pattern files) contain timing information that is valid for applying test data directly to the controller, but the timing data might not be valid for applying the test data to the SoC I/Os (timing skews due to mux logic or other UDL logic). Therefore it might be necessary for you to provide new SoC timeplates for the final integrated patterns. This can be done by using the map_timeplates statement in the controller_instance declaration in the test access file. It is also possible that the test data is not very timing sensitive and therefore the timing in the controller timeplates can be used at the SoC periphery. If this is the case, you can let the tool map the controller timeplates to SoC timeplates automatically.

Map_timeplate Statement

The map_timeplate statement occurs in the controller_instance declaration and is used to explicitly map a controller timeplate to an SoC timeplate. This statement maps the controller timeplate (which is defined in the pattern file) to the SoC timeplate. The two timeplates must have equivalent pin statements. That is, if a controller pin is pulsed in the controller timeplate, the SoC pin that the controller pin is mapped to must also be pulsed. The same is true for pins which are forced. Another way to think of this is that the SoC timeplate must have the same statements as the controller timeplate, except that the controller pins are replaced with the mapped-to SoC pins and the timing can be different.

Note



The controller timeplate does not have to be one that is defined in the Controller Definition file; it could be one defined in the pattern file. For additional syntax information and an example, refer to “[Controller Instance Declaration](#)” on page 233.

Automatic Timeplate Mapping

If you do not specify a `map_timeplate` statement for a controller timeplate, then the tool automatically maps the controller timeplate to an SoC timeplate. This is done by creating an SoC timeplate that has the same name of the controller timeplate with the controller instance name added to the front.

If there are two controller instances, “ins1” and “ins2”, and the controller being instanced has one timeplate called “tp1” in its pattern file, the final pattern set may contain two timeplates called “ins1_tp1” and “ins2_tp1”. The pin names in the controller timeplate will be replaced by the SoC pin names that the controller pins are mapped to.

Timeplate Optimization

Because the automatic timeplate mapping could possibly create a large number of timeplates in the final integrated test data, there is a timeplate optimization algorithm that is used for automatically created SoC timeplates. When a controller timeplate is automatically mapped to an SoC timeplate, before that timeplate is added to the list of used timeplates, the tool will first search through the list of existing SoC timeplates looking for a timeplate that exactly matches the new SoC timeplate. An exact match is a timeplate that has the same timeplate statements and the same timing. If a match is found, the existing matching timeplate is used and the newly created SoC timeplate is discarded.

Example - Timeplate Optimization

If there are two instances of a controller, called “ins1” and “ins2”, and the WGL pattern file for this controller uses a controller timeplate called `TP_BROAD`. If the controller instance “ins1” is the first one translated, then an SoC timeplate called `ins1_TP_BROAD` will be created.

If the second controller instance, “ins2”, has its controller pins mapped to the SoC pins in such a way that the timeplate for “ins2” matches the timeplate for “ins1”, then the tool will find that `ins1_TP_BROAD` is an exact match to the timeplate that it would create for “ins2”, so it will not create a second SoC timeplate but instead use `ins1_TP_BROAD` again.

Chapter 10

Full-Speed BIST

This chapter describes available options for high-speed testing. There are two fundamental types:

- **At-Speed** — The BIST clock runs at the same speed as the system clock. Pipelining may be used to adjust the timing the BIST controller.
- **Full-Speed** — The memory model is converted to support overlapping read and write cycles. The model compaction allows the controller to initiate a read or write operation at every clock cycle, reducing the total number of cycles required for BIST.

You can also use a combination of these types.

Pipelining allows you to run test at higher frequencies. Full-speed BIST allows you to run test with a smaller number of cycles. You can use these features together or in isolation, although the optimizations performed in Full-Speed BIST automatically incur a certain amount of internal controller pipelining.

Current Memory BIST Status

In a typical design with memory BIST, the BIST controller performs two primary functions to the memories under test: One, it provides the test stimulus, and two, it checks the response. A BIST controller can test multiple memories.

Figure 10-1. Memory BIST Controller

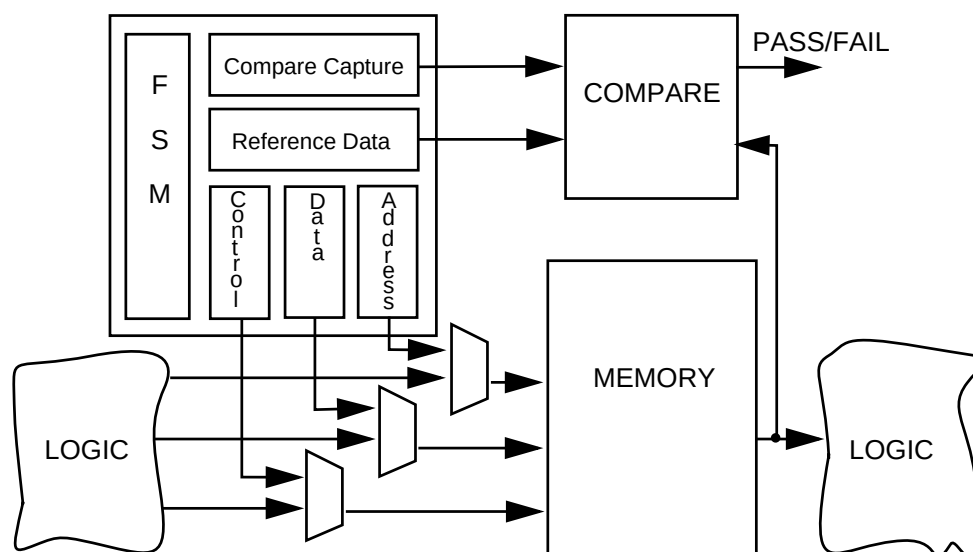


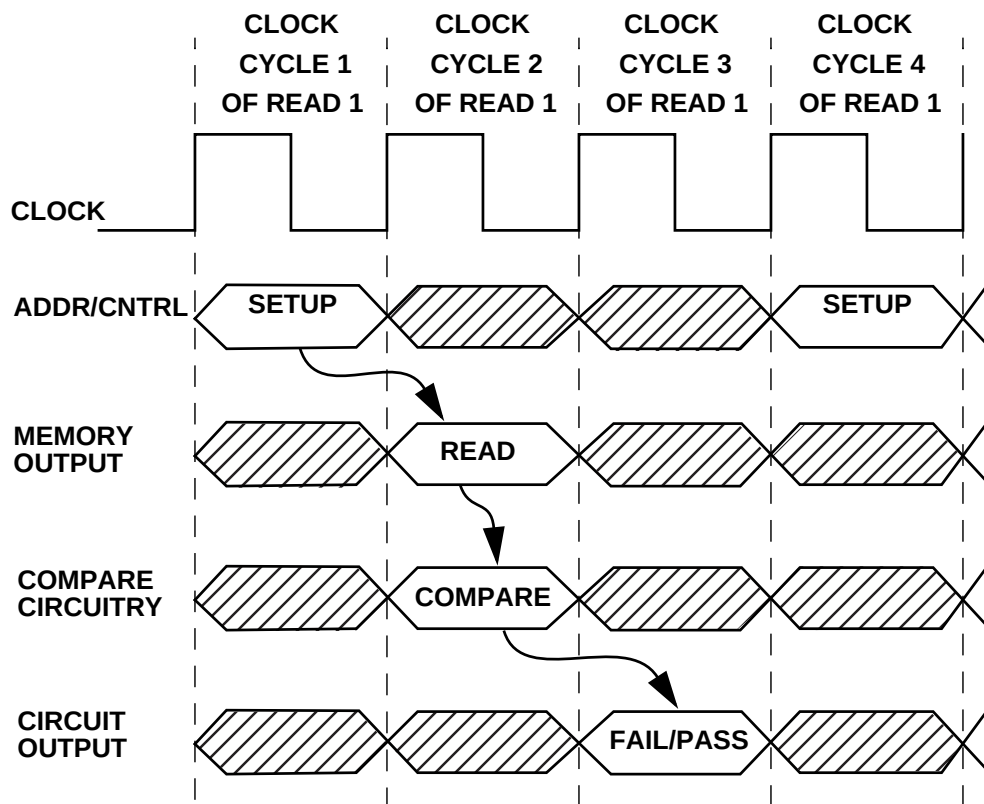
Figure 10-1 demonstrates one memory tested by one BIST controller. The BIST controller has sequential behavior. The clock controlling its state transitions can be from either an internal clock generator or an external source. To avoid clock synchronization problems during the BIST operation, the same clock source must control both the BIST controller and the memories it tests. Here we assume all memories are synchronous memories.

To properly perform read or write operations for synchronous memories, the BIST controller must first generate read/write setup signals before the memory clock is active. The examples presented in this section assume that all read/write setup signals are synchronous signals and all memories and the BIST controller are activated at rising edge.

Since the BIST controller and its memories use the same clock, a typical read/write operation requires two clock cycles. During the first clock cycle, the BIST controller generates all the necessary read/write setup signals for the memories under test. A read/write operation occurs at the edge of memory clock during the second clock cycle. In single clock memory BIST operation, this is called data latency.

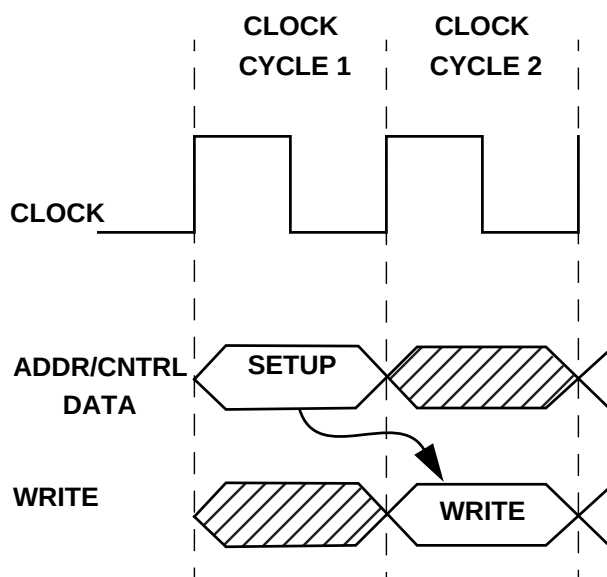
In addition, memory BIST controllers typically use comparators to verify the data read out from the memories. Since memory outputs are not ready until the edge of the second clock, the result of the comparator will be captured at the third clock cycle. Therefore, a BIST controller requires three clock cycles to perform a complete, isolated, read operation, as shown in Figure 10-2.

Figure 10-2. Read Operation



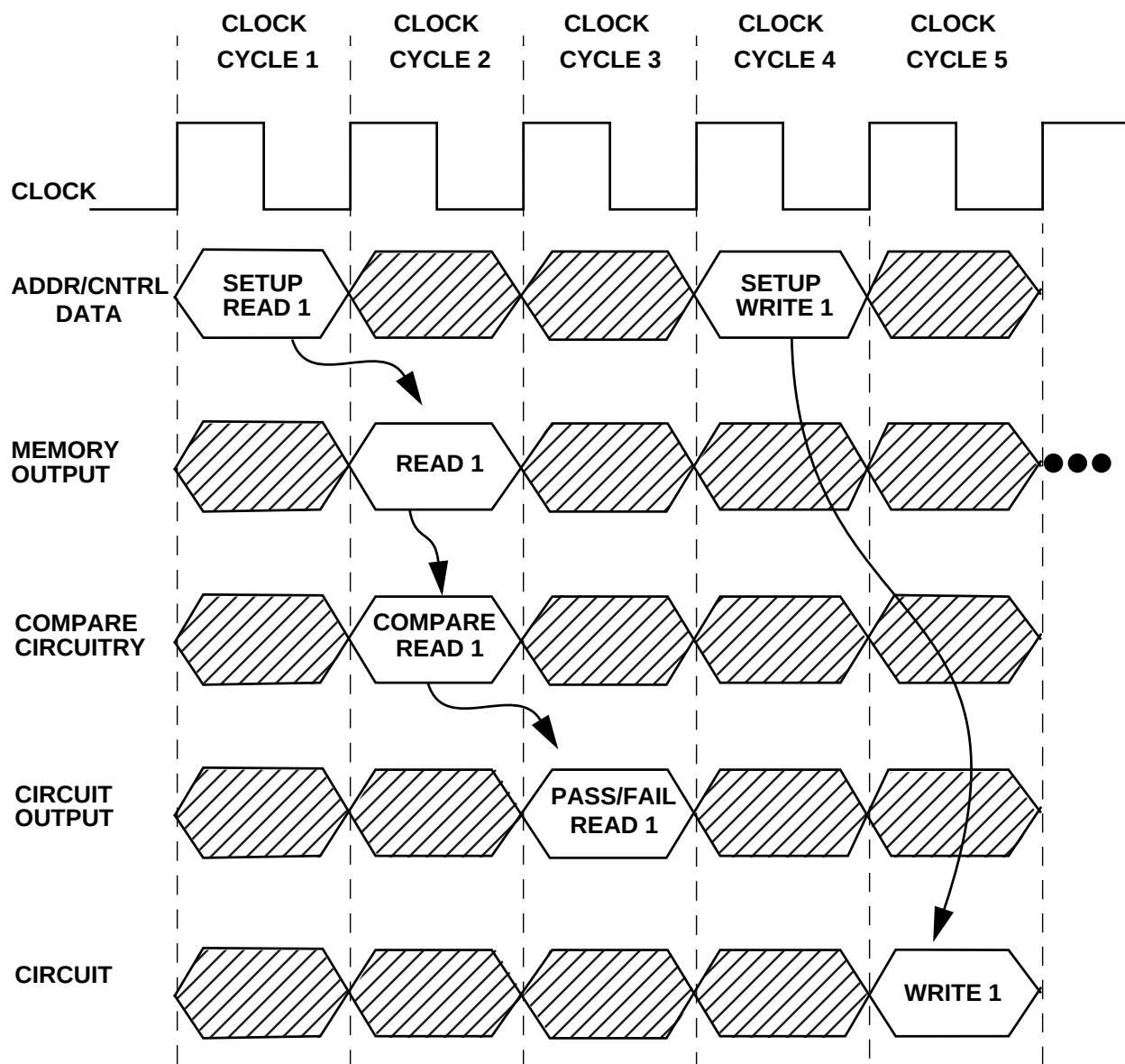
A BIST controller requires two clock cycles to complete an isolated write operation, as shown in [Figure 10-3](#).

Figure 10-3. Write Operation



Typically, a memory BIST controller requires six cycles to do two consecutive read operations, and four cycles to do two consecutive write operations. It requires five cycles to do one read operation followed by one write operation, as shown in [Figure 10-4](#).

Figure 10-4. Consecutive Read/Write Operations



Understanding At-Speed Testing

Because memories are getting larger and denser, design and test engineers need to ensure higher memory test quality to ensure overall chip quality. In addition to static functional tests, timing and stress tests are necessary to detect chip operation problems.

At-speed BIST operation generally means the BIST operation is capable of exercising the memories at chip clock frequency. However, at-speed operation is not sufficient to detect all timing faults. Even if the BIST controller design is operated at the chip clock frequency, its data latency prevents testing if the memory can change address and read out different data from different addresses at every cycle. Without this test, the BIST operation may not ensure adequate memory quality.

A single clock memory BIST controller can launch a read or write operation on each active clock edge, thus enabling timing and stress testing as part of the BIST operation. The MBIST Full-Speed™ feature of the tool implements this type of BIST operation. Besides improved test quality, this feature significantly reduces test time. For example, consecutive read/write operation in [Figure 10-4](#) requires five clock cycles which can be done in two clock cycles with MBIST Full-Speed BIST™ operation.

Understanding Full-Speed Testing

The MBISTArchitect tool can create BIST logic that supports a variety of clocking schemes. With reference to the at-speed pipeline testing process, it is worth reviewing a number of possibilities. In some cases, the “best” or “natural” approach for non-at-speed testing is not the same as for at-speed.

Keep in mind that at-speed pipeline testing has constraints. The MBISTArchitect tool cannot create logic to test memory such as Double Data Rate (DDR) memory that uses both the positive and negative edges to produce results. Also, the at-speed testing process will not work for asynchronous memories because testing it depends on observing results from a previous cycle while changing the control inputs for the next cycle. If the memory is asynchronous, its outputs could change too soon.

The first issue for consideration is what clock edge a synchronous memory uses. This is normally decided by the designer or the technology long before BIST issues are considered. Most memories are designed to work from positive edges. This is usually indicated to the MBISTArchitect tool by way of two versions of the setup memory clocking command.

This again sets up a mux that allows the memory clock to be driven by a chip clock or by an inverted version of the clock that drives the BIST.

For more information see the [Setup Memory Clock](#) command in the *MBISTArchitect Reference Manual*.

Outside of the at-speed process, there can be reasons for specifying that the BIST controller should use the same or opposite edge as the memory. In general, some of these considerations are dependent upon your design and test preference.

Choosing to have the BIST use the clock edge opposite the one used by the memory causes BIST changes to appear on the memory inputs shortly after the opposite edge from which the

timing checks are used for setup and hold rules. This opposite clock edge mode works best when the main errors of concern are stuck-at or other coupling faults.

Normally, an opposite clock edge situation is set up in one of two ways.

When the memory clock is positive, you might use the following command:

setup controller clock -negative

This assumes the default setting: Setup Memory Clock -System

When the memory clock is negative, you might use the following command:

setup memory clock -test invert

By implication, since this is default: Setup Controller Clock -positive

Given that specific clock edges have been chosen, then there are two default choices for setting up for MBIST Full-Speed pipelining. Based on the polarities of the controller and memory clocks, either a two-stage or three-stage pipeline must be defined by you. With different polarities, a two-stage pipeline is required, with the comparison happening one cycle earlier. With the same polarity, a three-stage pipeline is required, with the comparison happening one cycle earlier.

These choices are very general. Correct pipeline setup depends on how your memory actually works. In particular, if your memory has its own pipelining that will need to be taken into account.

Also the command Setup Full_speed -On will check the clock edges requested, do some analysis of your memories, and attempt to set up the pipelining for you.

Pipelined Read/Write Operations

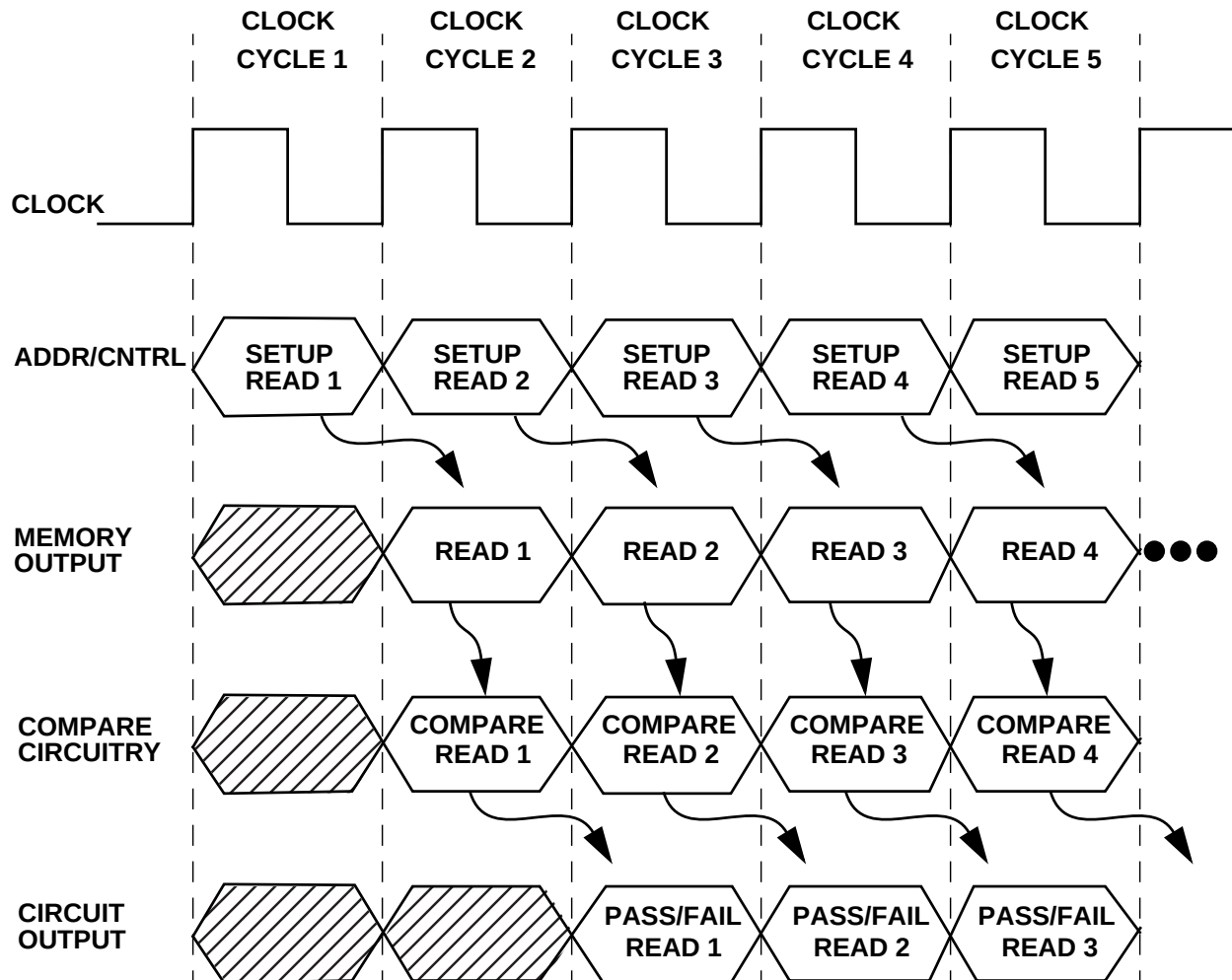
Because of the data latency in single clock memory BIST operation, pipeline circuitry is required to enable the BIST controller to perform MBIST Full-Speed read/write operations. The pipeline is used to temporarily separate the needed events at each clock cycle of read/write operations.

A three-stage pipeline can be used to compress the three-cycle read operation as shown in [Figure 10-2](#), into single cycle read. In this case, the first stage does the read setup, which may include read address change, read enable activation, or output enable activation. The second stage activates the read clock and provides the reference data for read data output comparison. The third stage captures the comparison result. In other words, inside the BIST controllers, all signals needed for the read operation are generated at the rising edge of the same clock.

A pipeline register is needed to create a one-cycle delay at the memory clock signal. A pipeline register is needed at the reference data to make sure the comparator is delayed by one cycle. Similarly, an MBIST Full-Speed BIST controller needs a pipeline register to create a two-cycle

delay at the capture signal which activates the capturing of the results from the comparator. Since memory clock is repeated every cycle, it does not need to be delayed and does not need pipeline register. [Figure 10-4](#) shows the pipelined consecutive read operations. [Figure 10-5](#) shows the pipelined design for MBIST Full-Speed memory BIST operation.

Figure 10-5. MBIST Full-Speed Pipelined Read Operations



In addition, a two-state pipeline can compress the two-cycle write operation, as shown in [Figure 10-3](#), into single cycle write. The first stage does the write setup which may include write address change, write data change, or write enable activation. The second stage activates the write clock. Similarly, all signals needed for write operation are generated at the rising edge of the same clock inside the BIST controllers. Here, only the memory clock needs to be delayed one cycle to achieve MBIST Full-Speed operation. [Figure 10-7](#) shows the pipelined consecutive write operations.

Figure 10-6. MBIST Full-Speed Pipelined BIST Controller

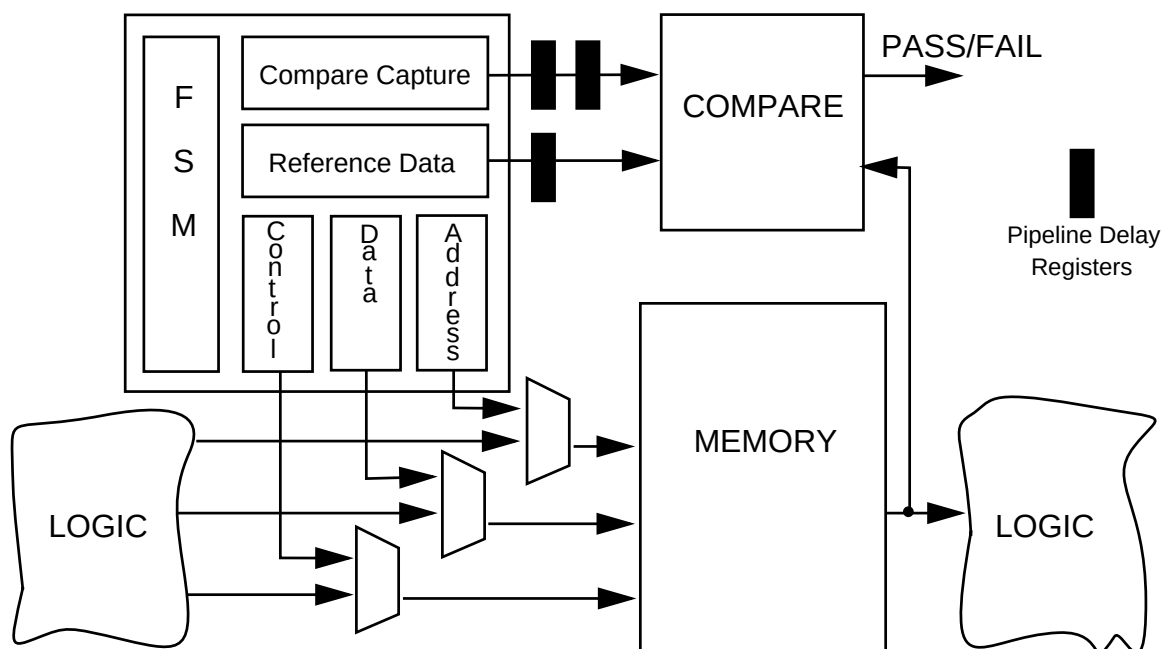
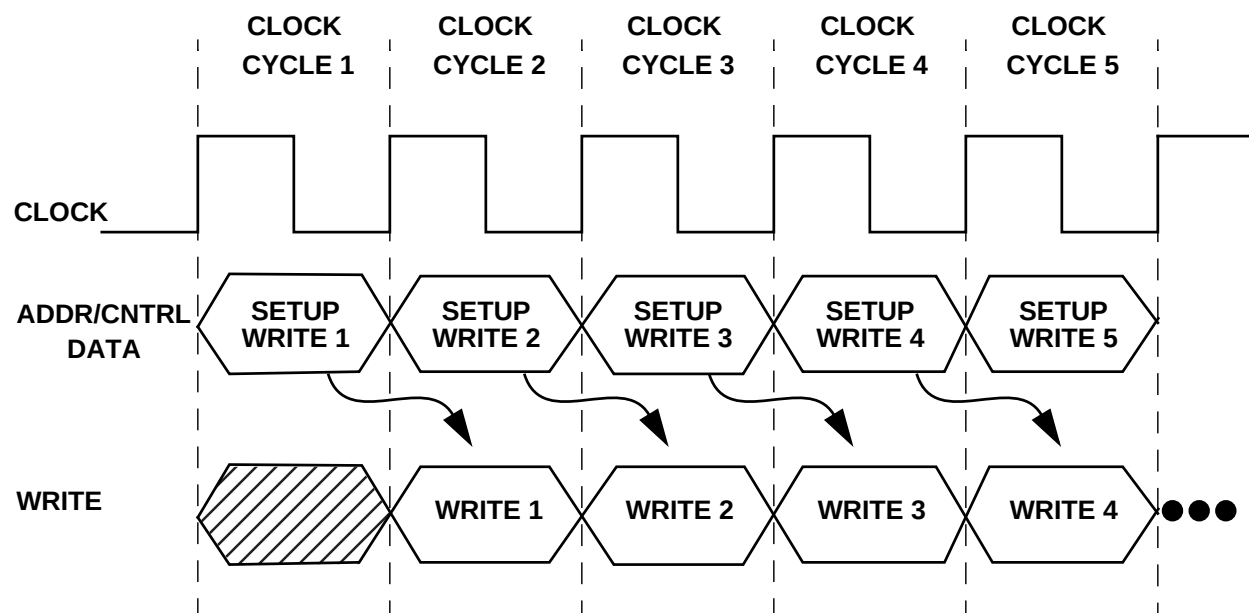
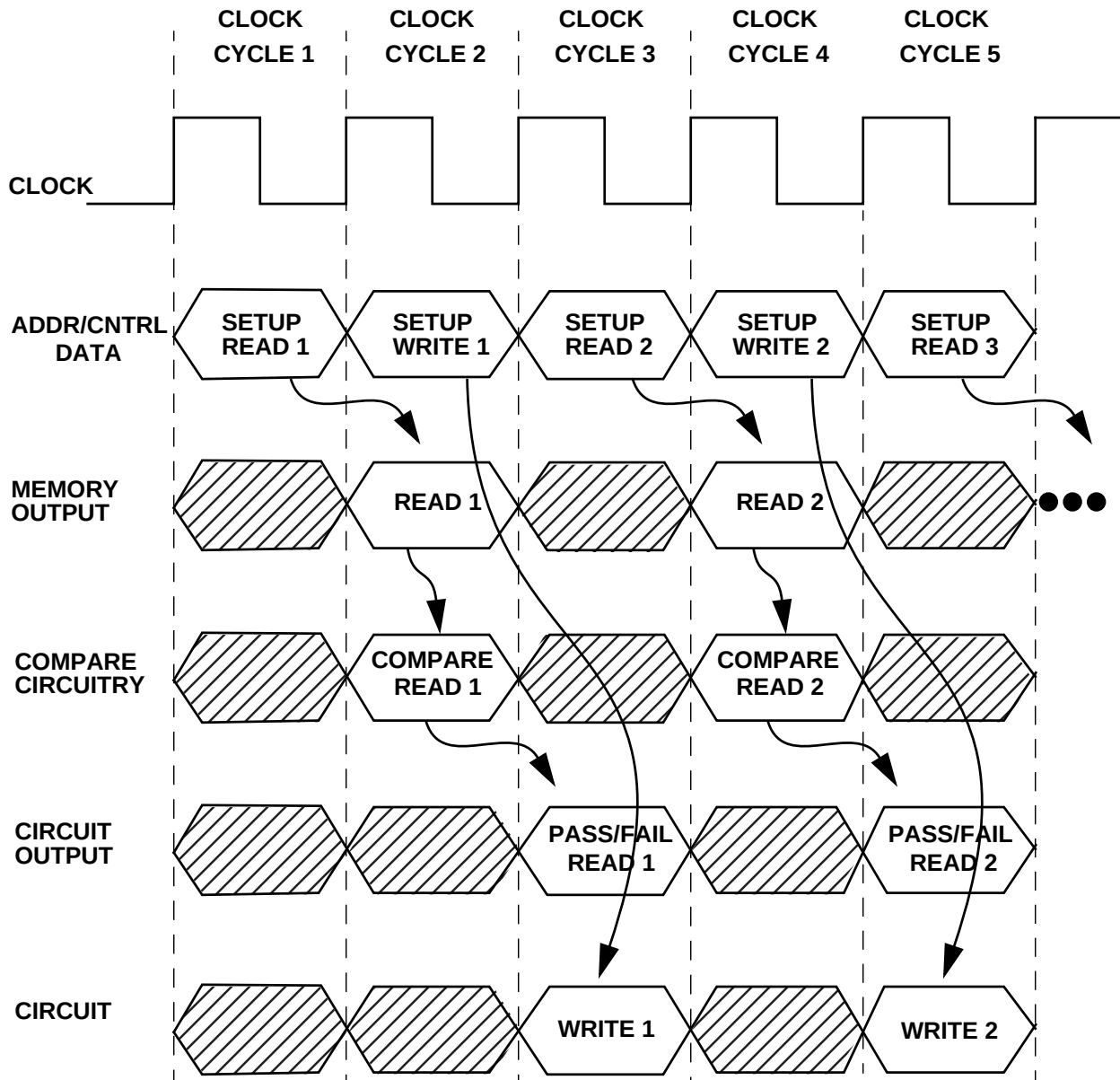


Figure 10-7. MBIST Full-Speed Pipelined Write Operation



Similarly, [Figure 10-8](#) shows the pipelined consecutive read/write operations.

Figure 10-8. Pipelined Consecutive Read/Write Operations



Problems To Consider with Full-Speed

See the [Set Controller Debug](#) command in the *MBISTArchitect Reference Manual*.

Practical Considerations for Full-Speed

In some memory designs, two different clocks can control the read and write operations separately. The read clock is not needed during the write operation, and the write clock is not

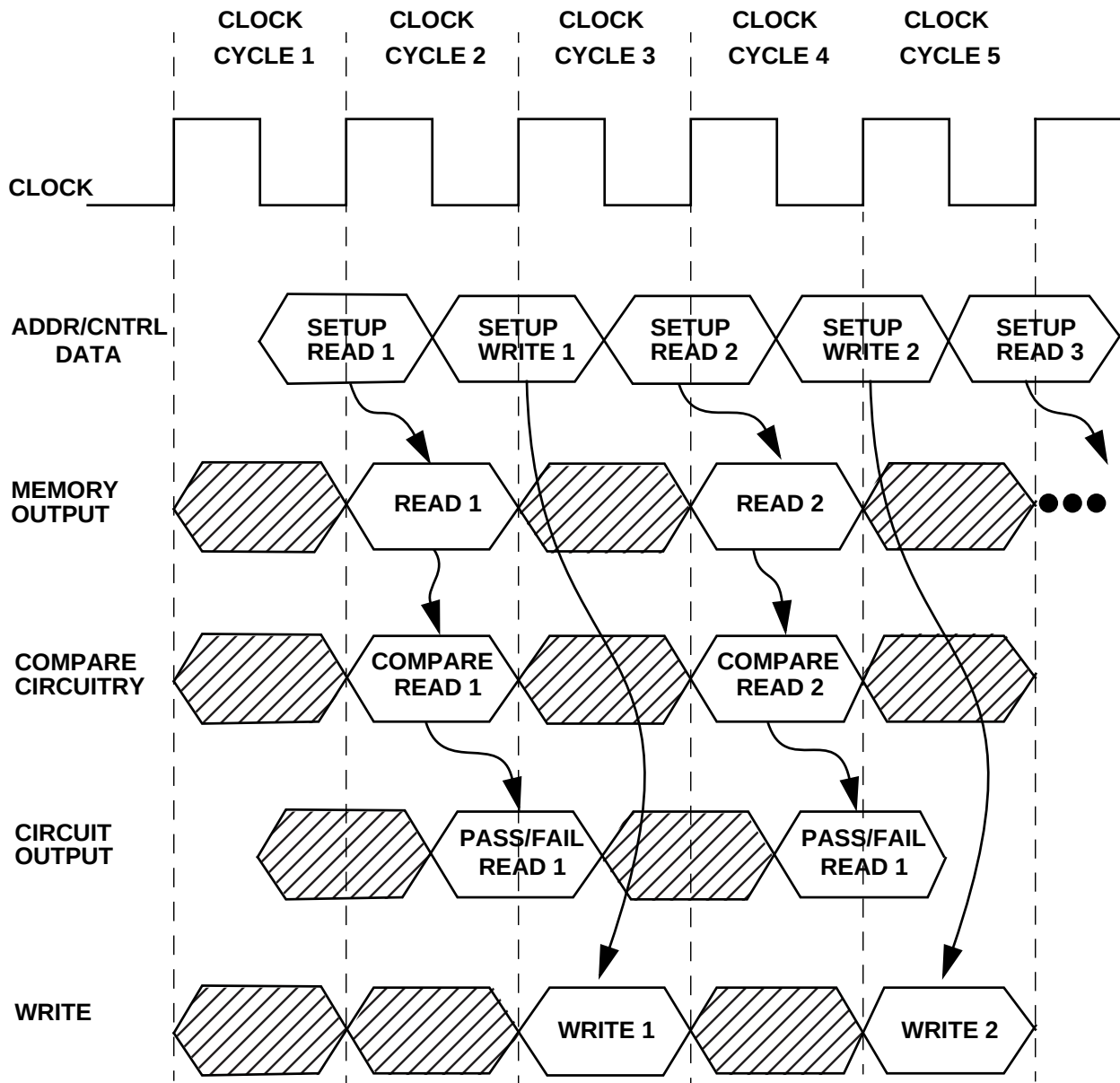
needed during the read operation. However, as long as the active read clock has no impact to the write operation, and the active write clock has no impact to the read operation, a simple solution is to provide a free-running clock to both sources. This way, the pipeline register is not needed to create a one-cycle delay of the memory clock.

The discussion in previous sections assumes all setup signals are synchronous. However, in some memory designs, setup signals may need to be active before and after the clock edge. For example, certain memory designs require the output enable signal to be active before the read clock edge, and after the read clock edge during read operation. To support MBIST Full-Speed consecutive read/write operation shown in [Figure 10-8](#), the output enable signal must be active at every cycle. To simplify the design of MBIST Full-Speed pipelined BIST controllers, all asynchronous read/write setup signals should be always active.

As shown in [Figure 10-1](#), the BIST controller and the memories under test use the same clock source, and all are activated at the rising edge. After each memory's clock is activated, that memory's data, address, and control signals have to hold stable longer than its hold time requirement. Since memories, in general, are much larger than the BIST controller, the wiring delays between memory clock and other signals can be large.

Without proper knowledge of layout information, it is hard to prevent hold time violation during design synthesis. This is a kind of timing closure problem. To avoid this problem, some designers prefer to use the negative edge for the BIST controller to have approximately a half-cycle hold time tolerance. In that case, all setup signals and comparator capture would be done at the negative edge, and the read operation in [Figure 10-2](#) would be changed to the read operation seen in [Figure 10-9](#). In [Figure 10-9](#) the events that are advanced by a half cycle are activated at the negative edge. The data latency is reduced from three cycles to two cycles.

With MBIST Full-Speed BIST operation, capture has to happen in cycle one since compare results can change in cycle two. In other words, inside an MBIST Full-Speed BIST controller, only the pipeline register to create the one-cycle delay at capture signal is needed. Since there is no delay needed in the reference data, no pipeline register is needed. For example, the read/write operation in [Figure 10-8](#) would be changed to that in [Figure 10-10](#). Then only a two-stage pipeline is needed.

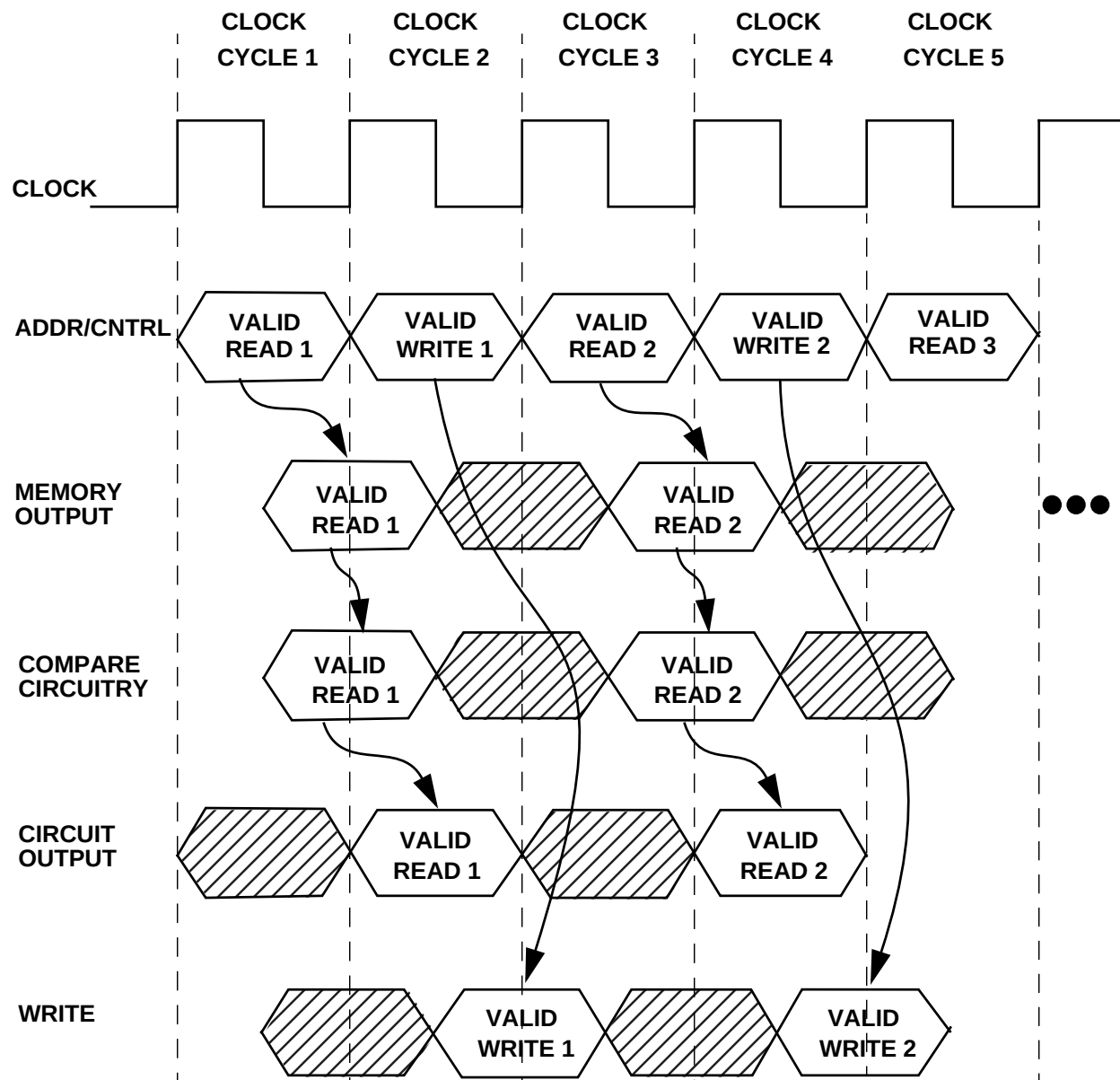
Figure 10-9. Pipelined Read/Write with Negative-edge BIST Controller

Negative-edge BIST Controllers

Negative-edge BIST controllers have one problem. They are finite state machines, which are tested by scan methodology such as ATPG. During this scan testing, scan chains in memory BIST controllers and the scan chains in other logic will be activated at different clock edges, which complicates scan operations.

Some designers prefer to use negative edge at memories and the positive edge at the BIST controllers to avoid hold time violation. However, it is usually not practical to do scan testing on memories.

Figure 10-10. Pipelining Read/Write Operations with Negative-edge



Because memories have two clock sources, using the positive edge clock during system operation and the negative edge clock during BIST operation, the read operation in [Figure 10-2](#) will be as in [Figure 10-10](#). The data latency is reduced as well. For example, the read/write operation in [Figure 10-8](#) would be changed as in [Figure 10-10](#). Once again, only a two-stage pipeline is needed.

Using the negative-edge can increase hold time tolerance; however, it reduces the setup time tolerance. As shown in [Figure 10-9](#) and [Figure 10-10](#) about half a cycle is allowed for setup time. Keep in mind that some timing analyzer tools do not handle multiple clock edges very well.

Pipelining

At high frequency test you might incur timing violations in BIST with the default settings. The logic inside the default BIST controller's finite state machine might not meet your frequency requirements, or perhaps the interconnect flight time between the controller and the memory collar may be too large. Register pipelining allows you to successfully perform BIST at high frequency. This section describes the options offered by the command "Setup Pipeline Registers". You can use as many of these options as needed; use a separate Setup Pipeline Registers command for each one.

Pipelining the Expect Process

By default the BIST controller evaluates RAMs using a comparator, whose two inputs are: one, the actual data from the RAM, and two, the expected data based on what the current algorithm previously wrote to the test address. You can request pipelining of the expect process with this command:

Setup Pipeline Registers `expect_process` on

When you use this command, the BIST controller will use a special indexing method to produce the expected data, and the indexing logic will be pipelined to line up with the actual data arriving from the RAM.

Figure 10-11. No Expect Indexing

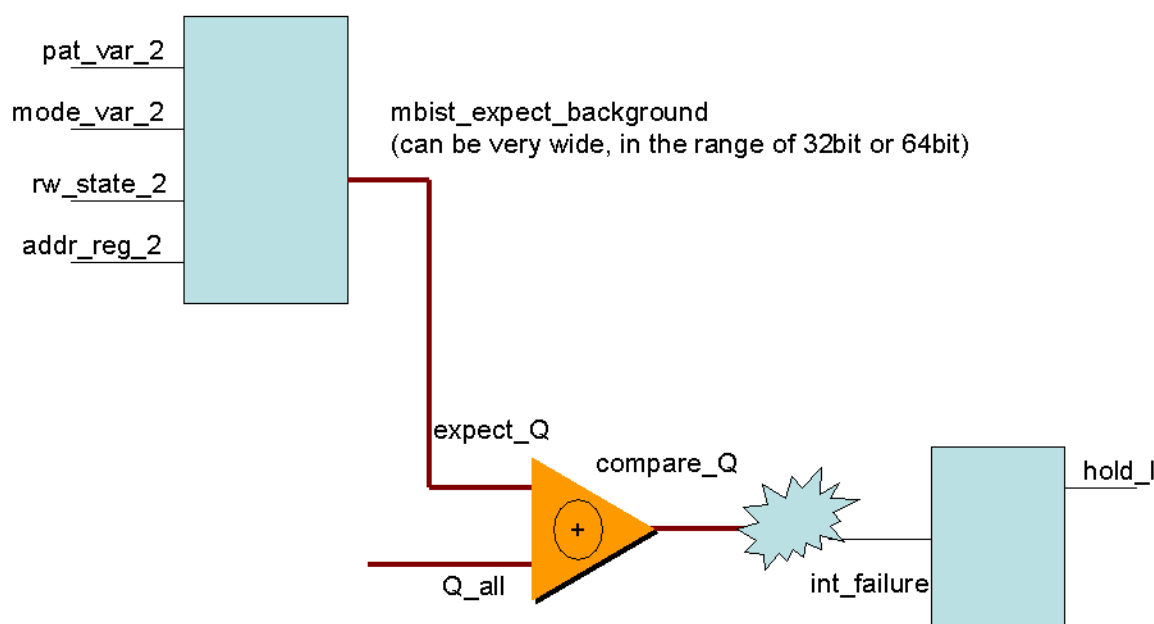
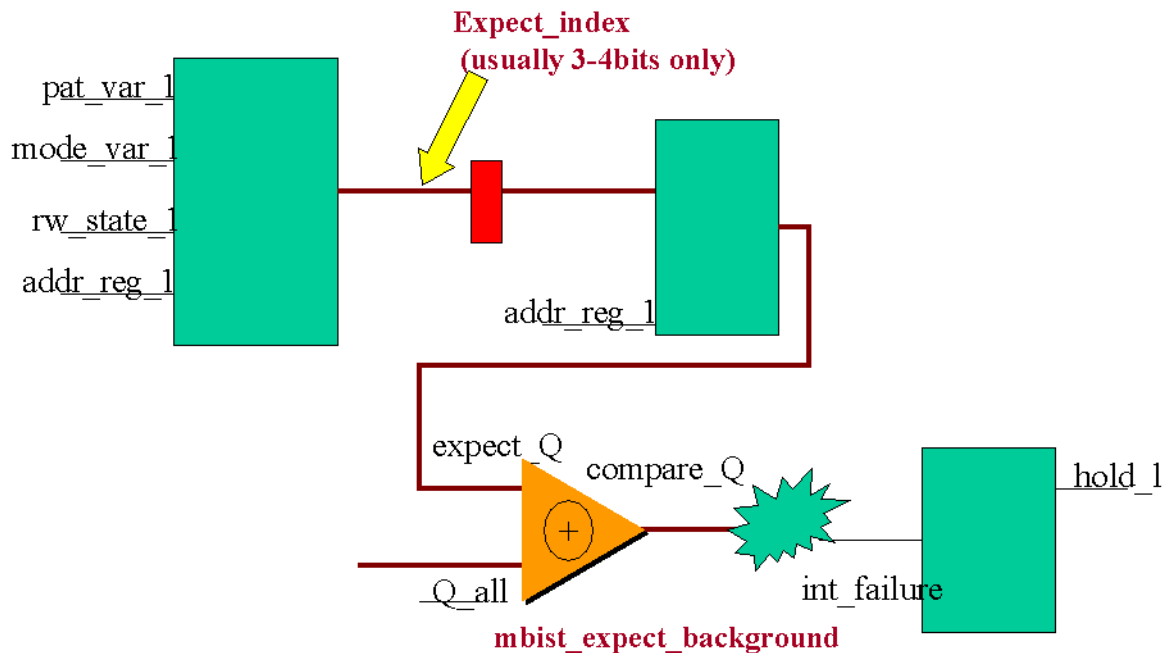


Figure 10-12. Expect Indexing and Pipelining



Comparator Result Pipelining

By default the BIST controller's comparator forwards its result (ok or miscompare) in the same cycle to a cloud of logic which determines whether the failure value is relevant. For example, when concurrently testing two memories of different address sizes, in the upper address space the controller needs to ignore miscompare values attributed to the smaller memory. The comparator is typically synthesized to a large cone of XOR gates and there might not be enough time in one cycle to calculate the comparison and evaluate the result. You can request that a pipeline register be placed between the comparison calculation and its evaluation with this command:

setup pipeline registers compare_result on



Note

MBISTArchitect implicitly enables `compare_result` pipelining when you use diagnostics with restart.

Figure 10-13. Restart Diagnostics

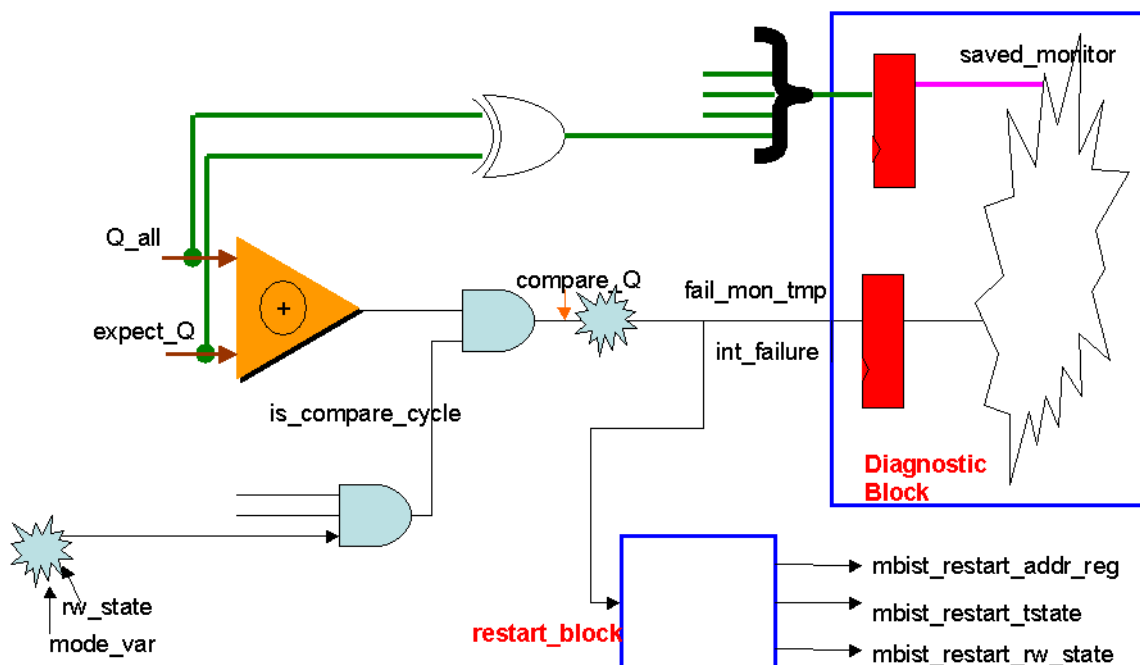


Figure 10-14. Comparator Result Pipelining (Restart)

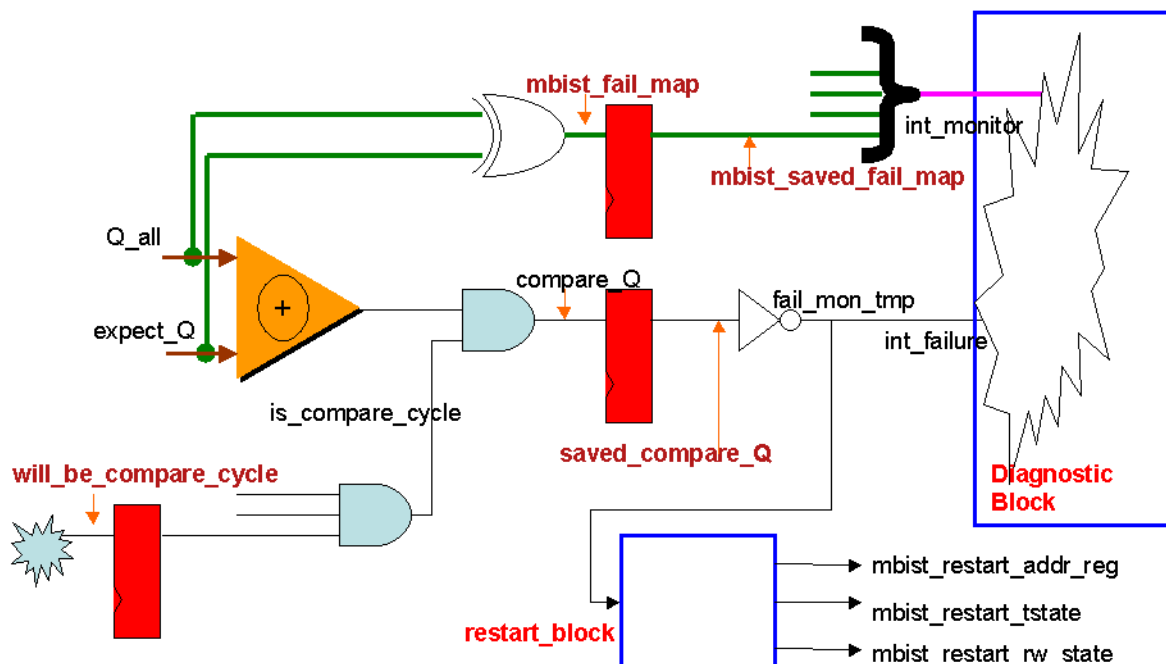


Figure 10-15. Hold / Nohold Diagnostics

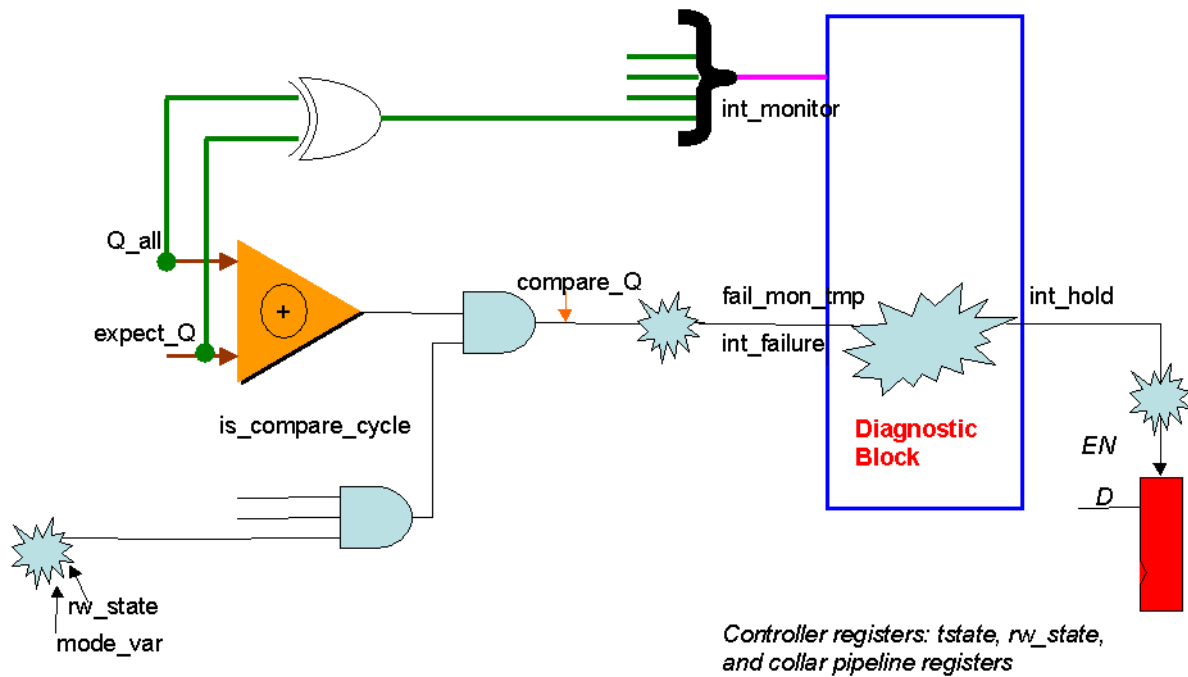
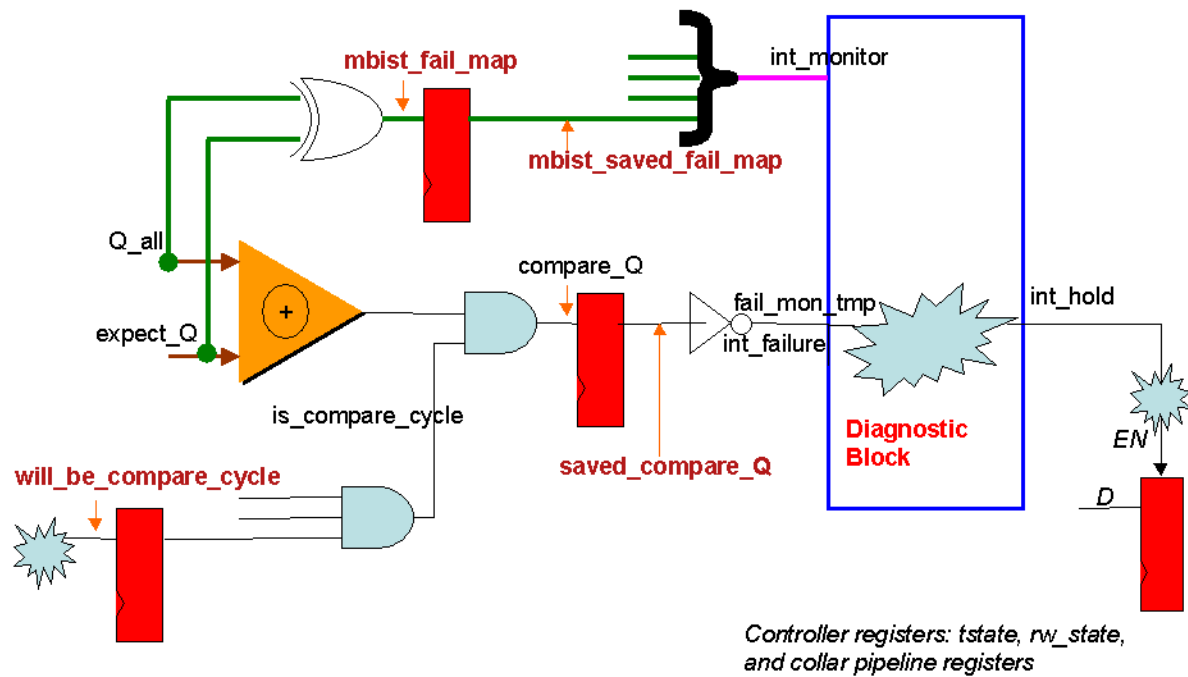


Figure 10-16. Comparator Result Pipelining (Hold / Nohold)



Memory I/O Pipelining

The read/write_cycle declarations in the memory model description tell the BIST controller what to do cycle-by-cycle when reading from and writing to a memory. By default the BIST controller performs memory transactions immediately when it reaches a given algorithm step. However, due to the physical floor plan separation of the controller and the collar, or perhaps due to the particular timing of your memory, there may not be enough time for the controller and the memory to properly communicate. Your design might already take this into account: you might be planning to add pipeline registers of your own after you are finished using the MBISTArchitect tool. Or you might want to request that the tool instantiate these registers for you. Regardless, if you know the total number of pipeline stages you want, use one or both of the following commands:

```
Setup Pipeline Registers output_block <stages>  
Setup Pipeline Registers input_block <stages>
```

The output_block and input_block are defined from the point-of-view of the memory, not the controller. Memory inputs include data_in, address, and control signals. Memory outputs include only data_out.

In the previous commands, the value of stages is the total number of pipeline registers you will have when you are done. If you do not use any additional arguments, the tool assumes you will be manually adding them later. On the other hand, if you want the tool to instantiate these registers for you, use the previous commands with an “-instance” argument. Choose whether you want register instances in the BIST controller or in the collar:

```
Setup Pipeline Registers output_block <stages>  
-instance collar <nstages_in_collar> controller <nstages_in_controller>  
  
Setup Pipeline Registers input_block <stages> -instance  
all collar <nstages_in_collar> controller <nstages_in_controller>
```

Note that if nstages_in_collar plus nstages_in_controller is less than nstages, the tool assumes that you are planning to manually add the remaining pipeline registers later.

Memory Output Pipelining

Figure 10-17. No Output Pipeline Instance

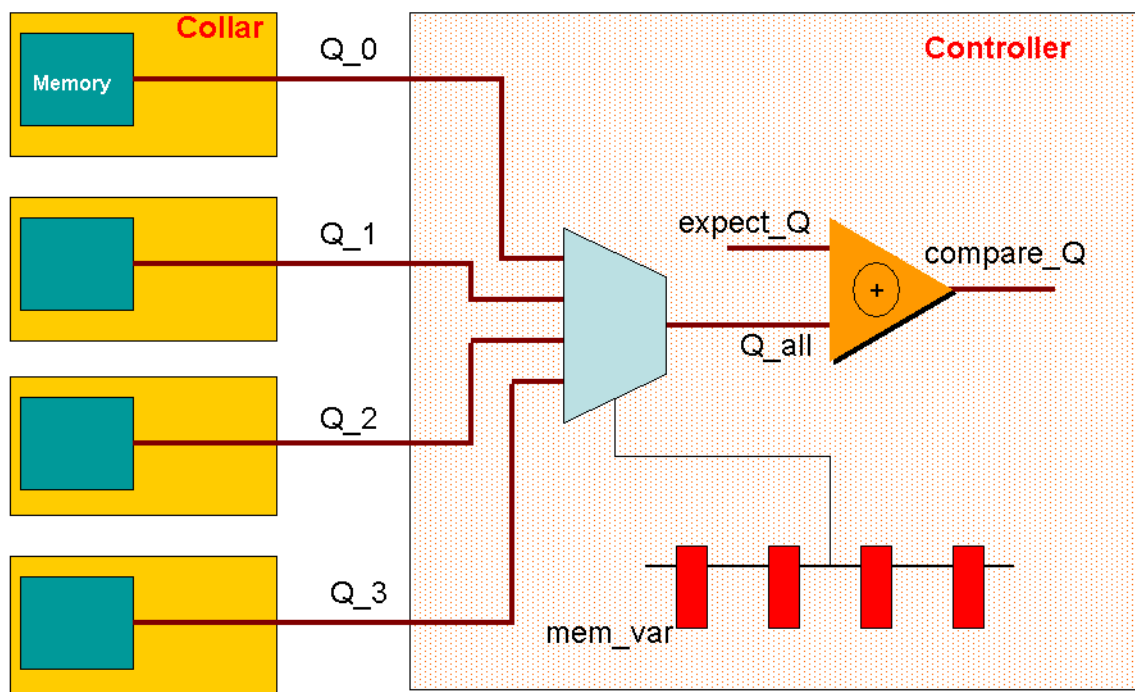


Figure 10-18. Output Pipeline Instance In Collar

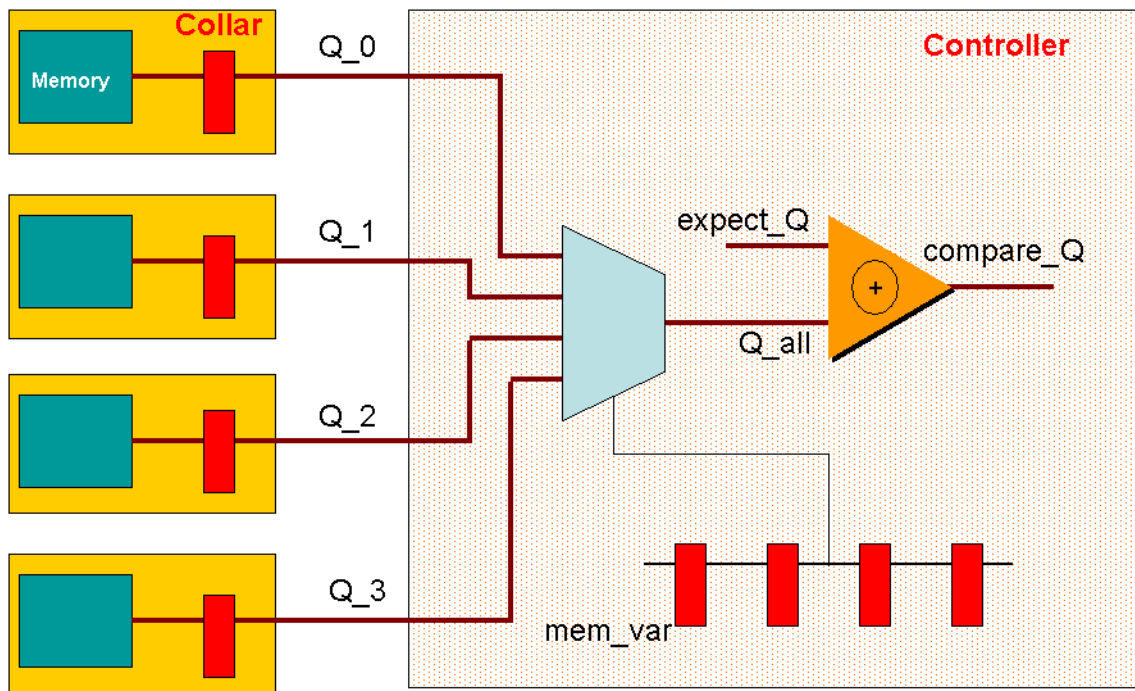
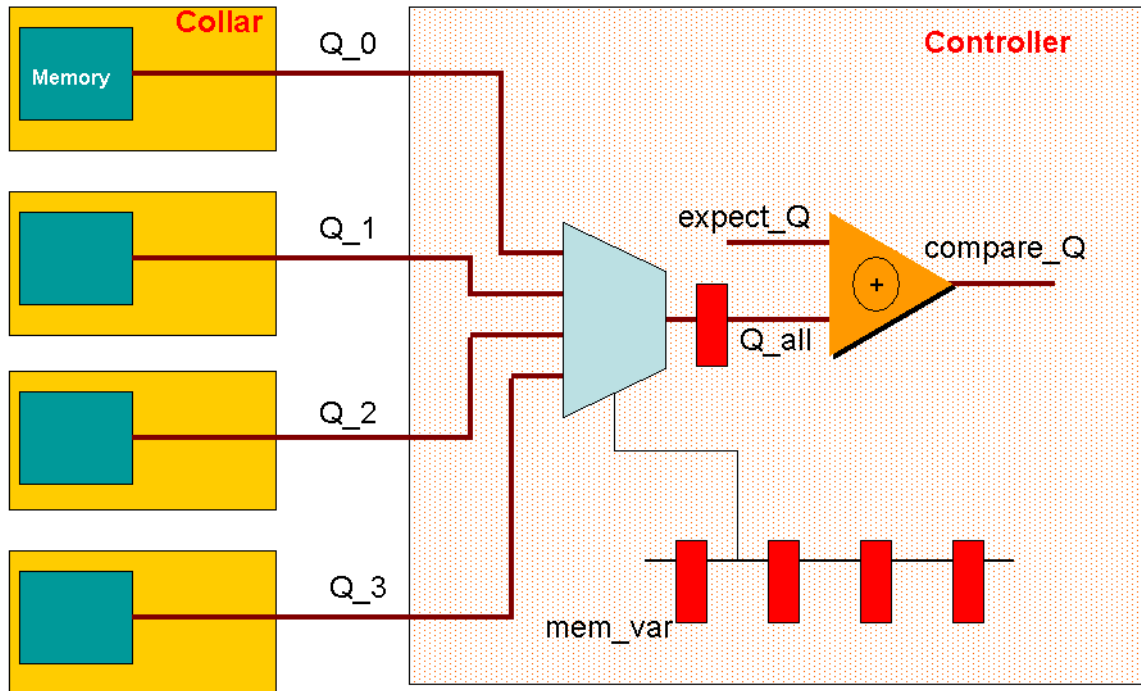


Figure 10-19. Output Pipeline Instance In Controller



Memory Input Pipelining

Figure 10-20. No Input Pipeline Instance

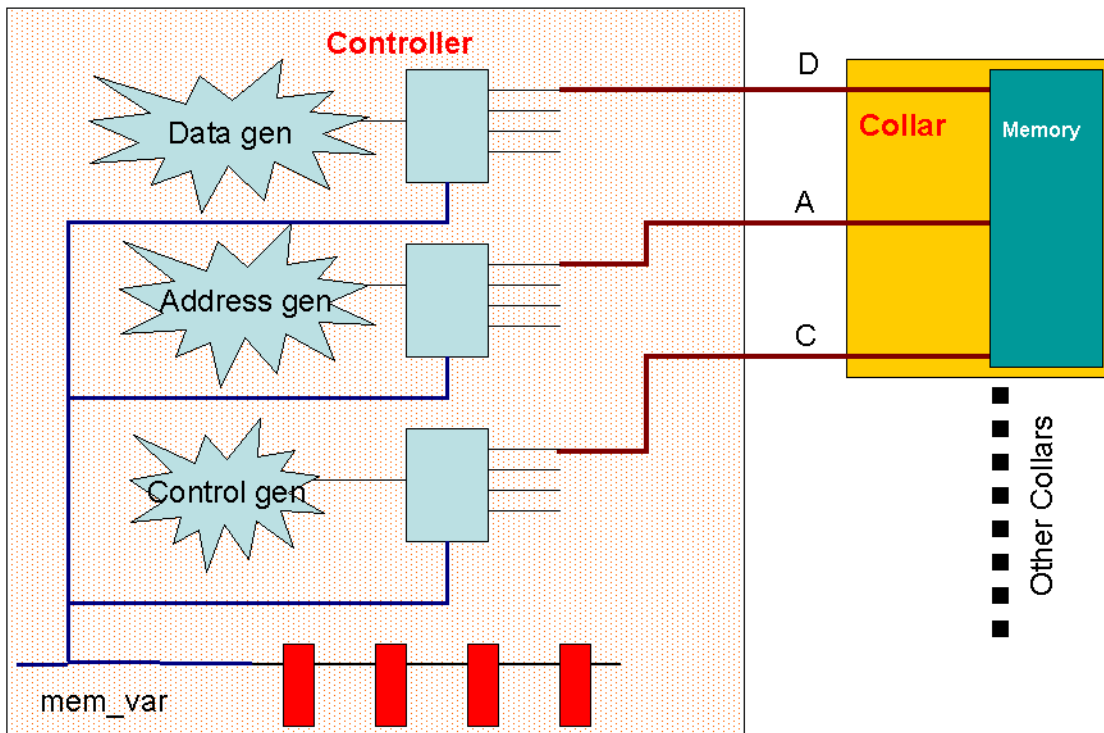


Figure 10-21. Input Pipeline Instance In Collar

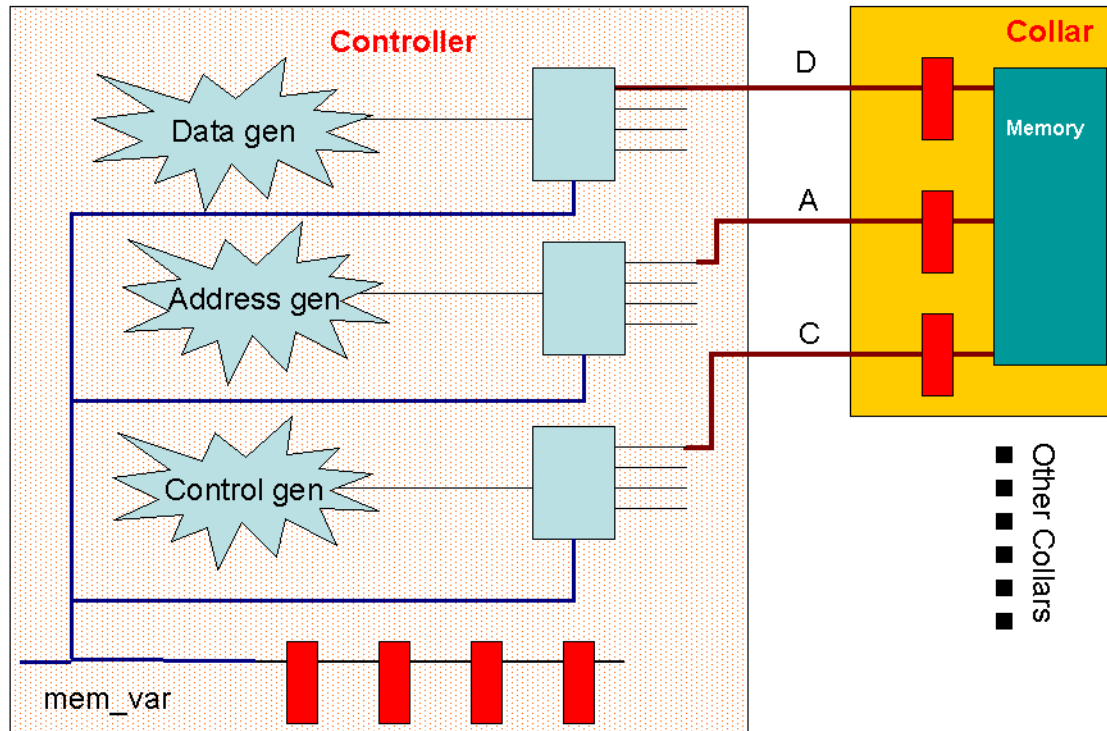
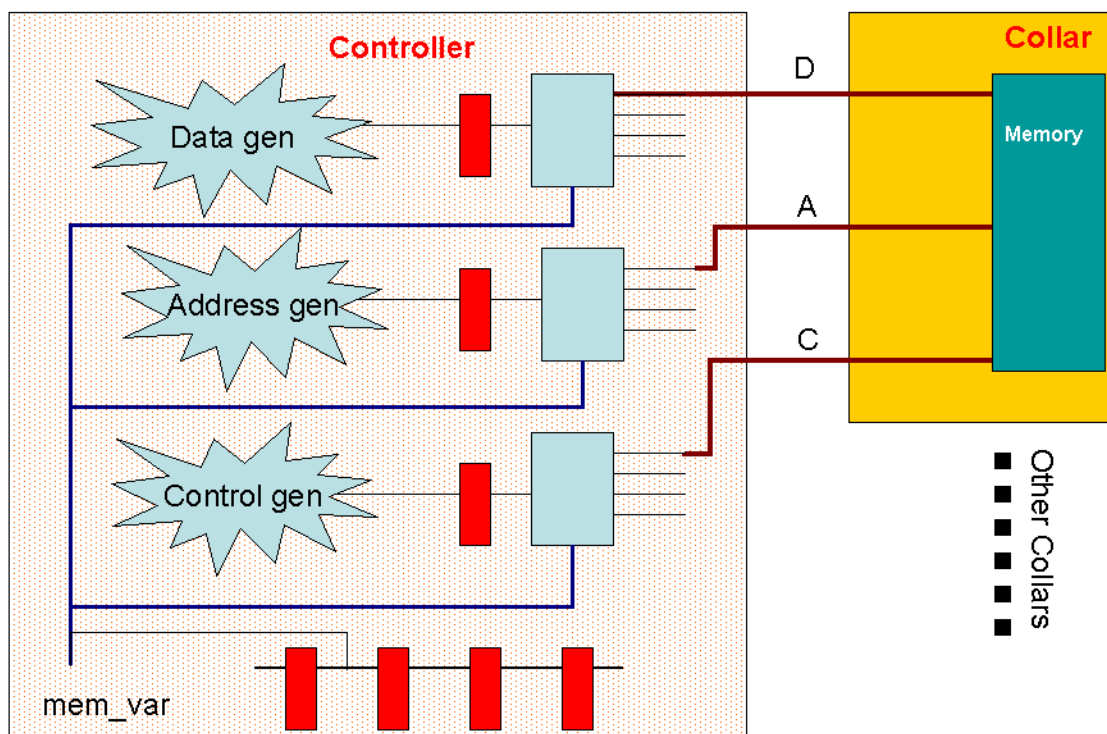


Figure 10-22. Input Pipeline Instance In Controller



Chapter 11

Diagnosing Memory Failures

You can use the Diagnosis tool to diagnose memory modules that contain a BIST controller generated and inserted with MBISTArchitect.

The Memory Diagnosis Process	261
Memory Diagnosis Requirements and Input Files	263
Preparing the ATE Failure Log for Memory Diagnosis	264
Running Memory Diagnosis	265
Interpreting Memory Diagnosis Results	267
Command Reference	269

The Memory Diagnosis Process

Memory diagnosis is a process for debugging failures found during post-silicon testing of memory modules. Memory diagnosis requires input files generated during the MBIST generation and insertion phases. These provide a location map of the failures found in the memory modules and associated BIST circuitry. Consequently, you use the failure location map information for either debugging failures or generating a repair scheme.

The following diagram shows the stages in the MBIST creation and diagnosis process. Refer to [Table 11-1](#) for details.

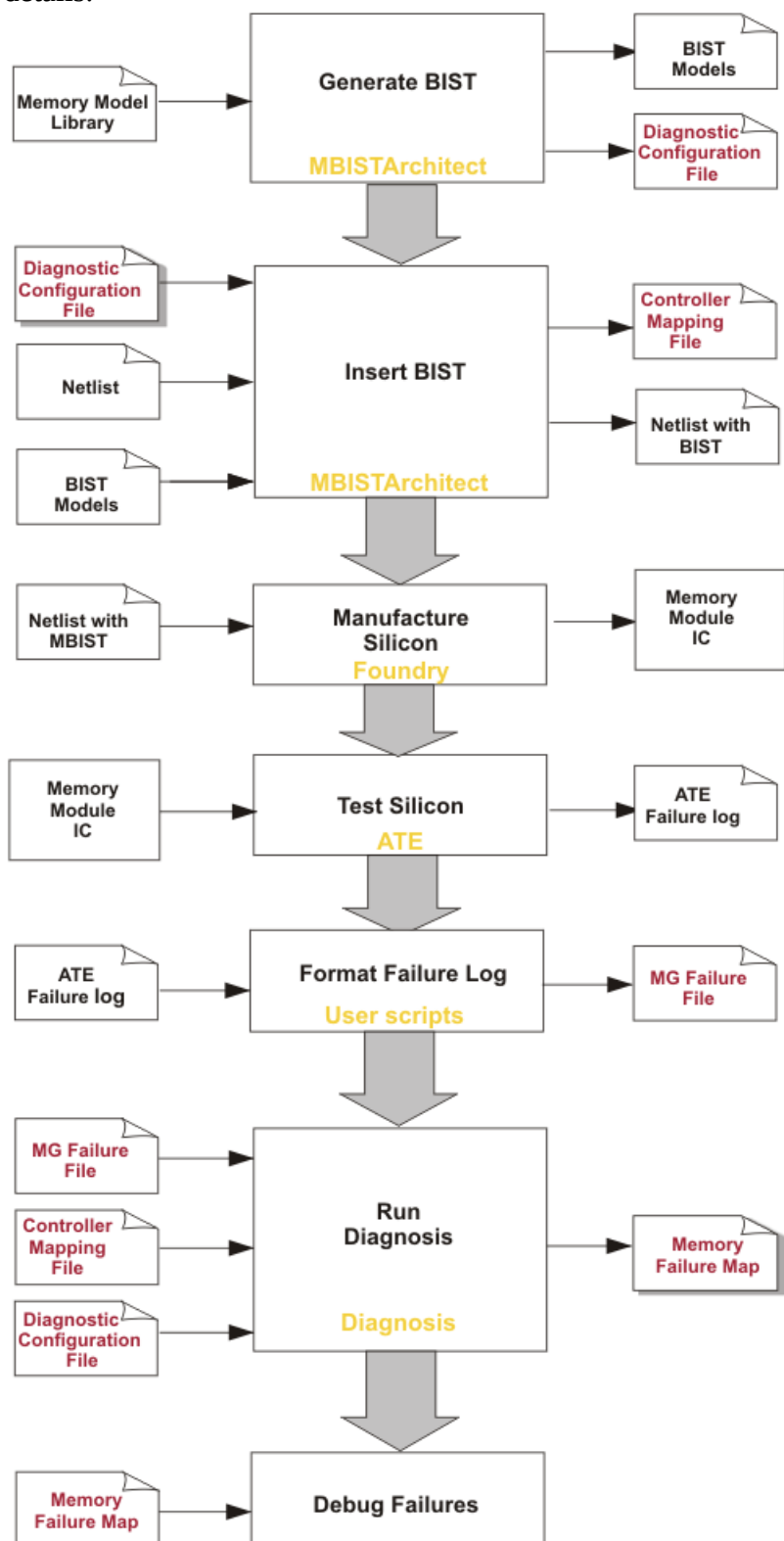


Table 11-1. Memory Diagnosis Process

Stages	Description
Generate BIST	The test engineer uses the memory model library and MBISTArchitect to generate BIST circuitry and the diagnostic configuration file for diagnosis. See “Creating the Diagnostic Configuration File.”
Insert BIST	The test engineer uses MBISTArchitect to insert the BIST circuitry into the design netlist and generate a controller mapping file for diagnosis. See “Creating the Controller Mapping File.”
Manufacture Silicon	The foundry manufactures the design into a silicon IC.
Test Silicon	The test engineer tests the failing IC on an ATE. The ATE outputs a failure log that lists the failures within the memory module IC.
Format Failure Log	The test engineer creates scripts to convert the ATE failure log into an ASCII format compatible with the Diagnosis tool. See “Preparing the ATE Failure Log for Memory Diagnosis.”
Run Diagnosis	The test engineer uses the Diagnosis tool to run diagnosis on the ASCII failure file and produce a location map for the failures returned by the ATE. See “Running Memory Diagnosis.”
Debug Failures	The test engineer uses the failure location map to locate and debug the memory module failures.

Memory Diagnosis Requirements and Input Files

- **BIST Controller** — MBISTArchitect must generate the BIST controller(s) associated with the memories you are testing.
- **Diagnostic Configuration File** — Contains the BIST circuit controller state and memory configuration information you use for diagnosis. You create the diagnostic configuration file using MBISTArchitect; MBISTArchitect writes the configuration file during the BIST circuitry generation (*file_name.diagcgb*). See [“Creating the Diagnostic Configuration File.”](#)
- **Controller Mapping File** — Contains the mapping between the memory model information, the associated chip-level BIST controller instance, and the memory collar instances. MBISTArchitect creates this controller mapping file when the application inserts the BIST circuitry into the design netlist. See [“Creating the Controller Mapping File.”](#)
- **ATE Failure Log** — Contains the failure information from post-silicon testing of the memories. You must convert the ATE failure log into a format compatible with the

Diagnosis tool. For more information, see “[Preparing the ATE Failure Log for Memory Diagnosis](#).”

Preparing the ATE Failure Log for Memory Diagnosis

The typical industry practice is to create scripts that convert ATE failure logs into the necessary format. Depending on the ATE, the specific format of the failure log varies, but each BIST controller has a body of failing data where each line of binary data corresponds to a diagnostic monitor.

Rules

Use the following rules when creating an ASCII failure file for diagnosis:

- Place all the failing information for a single die in one failure file. Each failure file should only contain information for one die.
- Precede all comment text with a pair of slashes (/).
- Define the keywords you use to create the failure file in the same order they are presented in [Table 11-2](#).

Keywords

[Table 11-2](#) lists the keywords you use to create an ASCII failure file for diagnosis. For an example, see “[ASCII Failure File Example](#).”

Table 11-2. ASCII Failure File Keywords

Keyword (s)	Usage Rules
tracking_info_begin <i>user_defined_text</i> tracking_info_end	<i>Optional.</i> Use these keywords to place user-defined tracking information in a failure file. Use these keywords only once to create a single tracking information section for the entire file. This information is not used for diagnosis; it is placed in the diagnosis report verbatim.
preamble {on off}	Required. Use this keyword to indicate whether the failure data includes the preamble bits.
postamble {on off}	Required. Use this keyword to indicate whether the failure data includes the postamble bits.

Table 11-2. ASCII Failure File Keywords

Keyword (s)	Usage Rules
failures_begin controller <i>controller1_pathname</i> begin <i>binary_failure_data1</i> end controller <i>controller2_pathname</i> begin <i>binary_failure_data2</i> end failures_end	Required. Use one set of these keywords to enter the failing data associated with each controller. You can represent the failing data in binary (1 and 0) or signal strength (H and L). Place the failing data between these keywords on separate lines using the following nested keywords: • controller — identifies the controller pathname • begin — precedes the binary failure data • end — terminates the binary failure data These nested keywords can be repeated for each controller you put in the failure file.

ASCII Failure File Example

```
tracking_info_begin
wafer id: 2
die id: 100
tracking_info_end
preamble on
postamble on
failures_begin
controller /U1/cnt11
begin
HLLLLHHHHHLLLHHH
LLLLHHHHHLLLLLL
HHLLLLLLLLHLHLHL
...
end
controller /U1/cnt12
begin
LLLHHHHLLLLLLHHHH
LLLHHHHHHHHLHLHL
...
end
controller /U1/U12/cnt13
begin
HLHLHLHLHL
...
end
failures_end
```

Running Memory Diagnosis

Use the following procedure for running the memory diagnosis.

Prerequisites

- ASCII failure file must be available and correctly formatted. See “[Preparing the ATE Failure Log for Memory Diagnosis](#).”
- Controller mapping file and diagnostic configuration file must be available and located in the same directory. See “[Creating the Controller Mapping File](#)” and “[Creating the Diagnostic Configuration File](#).”

Procedure

1. From the Linux/UNIX shell, invoke Diagnosis for memory diagnosis. For example:

```
Tessent_Tree_Path/yieldassist -mbistdiag -logfile logfile_name
```

where:

- ***Tessent_Tree_Path*** — The location of the software tree.
- **-mbistdiag** — A switch that invokes Diagnosis in the memory diagnosis mode.
- **-logfile *logfile_name*** — The name of the logfile to save the session transcript to.

The Diagnosis tool invokes in a command line session.

2. Load the controller mapping file. For example:

```
dofile cntl.map
```

where:

- **dofile** — The command used to load the controller mapping file.
- ***cntl.map*** — The name of the MBISTArchitect-created controller mapping file to load.

3. Run the diagnosis. For example:

```
diagnose failures u1_failure_file -output mem_fail_map
```

where:

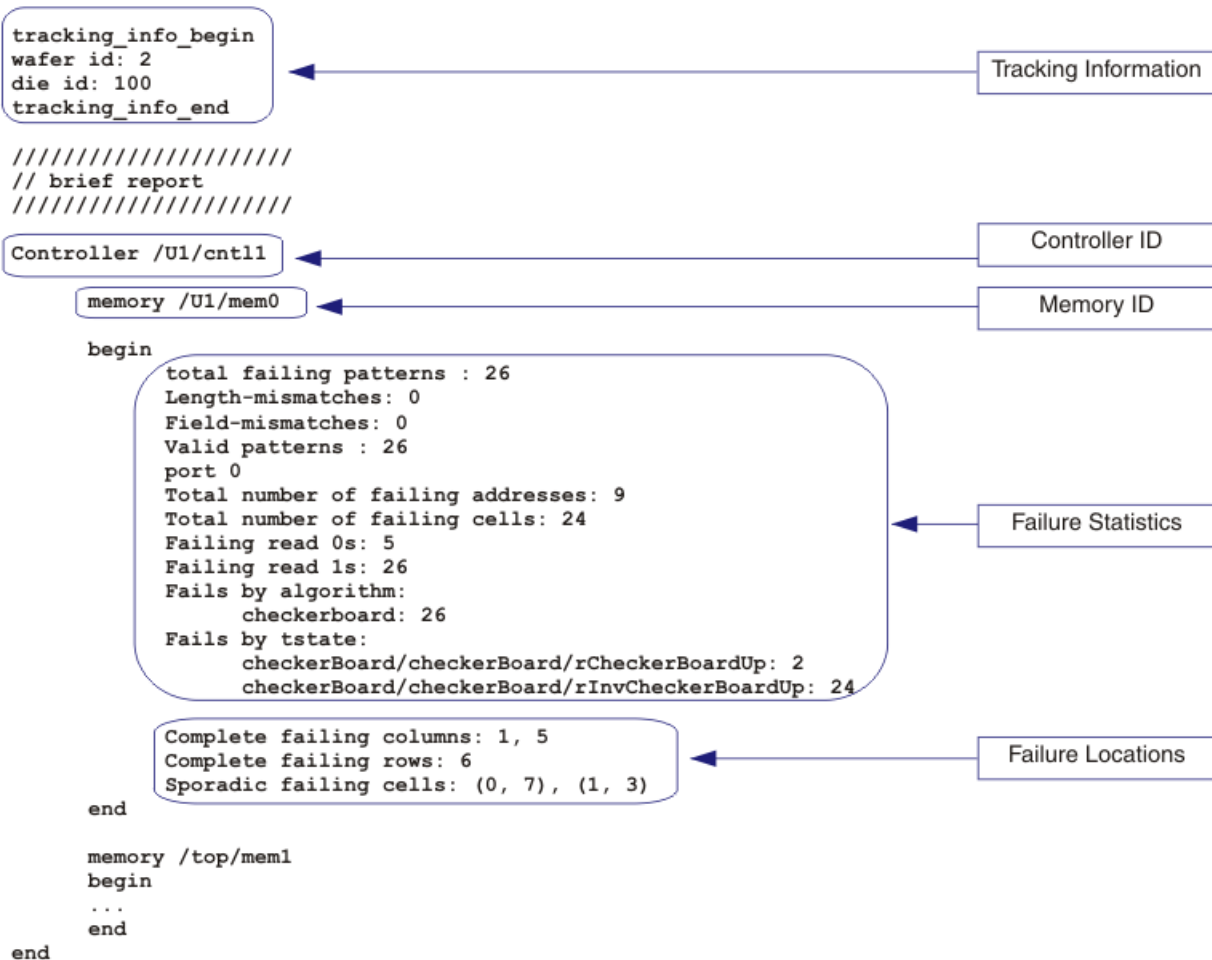
- ***u1_failure_file*** — The ATE failure log in ASCII format compatible with the Diagnosis tool.
- ***mem_fail_map*** — The name of the file to write the diagnosis results to.

The failure file is diagnosed and the results are saved to the specified file. See the [Diagnose Failures](#) command.

Interpreting Memory Diagnosis Results

The diagnosis results are categorized by controller. Failing columns and rows indicated in the diagnosis report are logical columns and rows.

Diagnosis Report Example



Verbose Diagnosis Report Example

If “[Set Diagnosis -Report verbose](#)” is used, the following information is reported for each diagnostic monitor at the top of the diagnosis report:

```

//////////
// verbose report
//////////
  
```

```
controller /U1/cnt11
begin
monitor :1000011111000111
begin
failing memory: /U1/mem0
failing port: 0
failing address: 0x0
expected value: 1111
actual value : 1010
failing bits : 0101
failing algorithm step: march2/march2/rwrBackgroundUp
failing read : 2
end
monitor: 0011110000011100
begin
.....
.....
end
.....
end
controller /U1/cnt11
begin
diagnostic monitor
.....
end
```

If you test memories concurrently, one diagnostic monitor may contain failing information from multiple memories. This detailed monitor information can help to diagnose the root cause of a failure.

Command Reference

Table 11-3 summarizes the Diagnosis commands you use for running diagnosis on memory modules with the BIST circuitry.

Table 11-3. Memory Diagnosis Command Summary

Command	Description
Add Controller Mapping	Loads specified controller mapping information for the diagnosis session.
Delete Controller Mapping	Deletes the specified controller mapping information from the current diagnosis session.
Diagnose Failures	Diagnoses the failures in the specified failure file.
Echo	Issues a user-defined string to the transcript or specified pathname.
Exit	Terminates the software tool session.
Help	Displays the usage syntax and system mode for the specified command.
History	Displays a list of previously executed commands.
Report Controller Mapping	Displays mapping information for the specified controllers.
Report Diagnostic Monitor	Displays failure information associated with the specified diagnostic monitor.
Set Diagnosis	Sets the diagnosis reporting parameters.
Set Dofile Abort	Specifies whether the tool aborts or continues dofile execution when an error is detected.
System	Passes a specified command to the operating system for execution.

Add Controller Mapping

Tools supported: Diagnosis

Scope: Memory Diagnosis mode

Usage

ADD COntroller Mapping *diagnostic_config_file_name controller_instance_pathname memory_collar_instance_name...*

Description

Adds controller mapping information for the diagnosis session.

The Add Controller Mapping command associates each controller instance with a corresponding diagnosis configuration file and memory collar instances for diagnosis. The memory under test is wrapped within each memory collar.

You must enter the memory collars in the same order that they were entered in the memory model during BIST generation. When multiple controllers are instantiated, you must enter each controller individually with the Add Controller Mapping command.

Arguments

- ***diagnostic_config_file_name***
Required string that specifies the name of the diagnostic configuration file associated with the BIST controller.
- ***controller_instance_pathname***
Required string that specifies the pathname for the controller instance associated with the memory being diagnosed.
- ***memory_collar_instance_name...***
Required, repeatable string that specifies the pathname of a memory collar instance associated with the memory being diagnosed. The memory under test is wrapped within each of the specified memory collars. You must enter the memory collars in the same order that they were entered in the memory model during BIST generation.

Examples

The following example establishes mapping between the diagnostic configuration file *u1_diagcfg*, a controller *U1/cnt11*, and the associated memory collars/memory modules *collar1*, *collar2*, *collar3*.

```
add controller mapping u1_diagcfg U1/cnt11 collar1 collar2 collar3
```

Related Commands

[Delete Controller Mapping](#)

Delete Controller Mapping

Tools supported: Diagnosis

Scope: Memory Diagnosis mode

Usage

DELEte COntroller Mapping **-All** | *controller_instance_pathname...*

Description

Deletes the specified controller mapping information from the current diagnosis session.

Arguments

- **-All**
Required switch that deletes all controller mapping.
- *controller_instance_pathname*
Required, repeatable string that specifies the pathname for the controller instance to delete.

Examples

The following example deletes the controller *U1/cnt11* from the current diagnosis session.

delete controller mapping U1/cnt11

Related Commands

[Add Controller Mapping](#)

Diagnose Failures

Scope: Scan diagnosis mode

Usage

DIagnose FAilures ***failure_filename*** [-Output *report_filename*] [-Replace]

Description

The Diagnose Failures command performs a diagnosis of the failing scan test and chain test patterns in the specified failure file.

- ***failure_filename***
A required string that specifies the name of the file that contains the failing test pattern information for diagnosis.
- -Output *report_filename*
An optional switch and string pair that directs the tool to write the diagnostic report in ASCII format to the specified file. By default, the diagnostic report displays on screen.
- -Replace
An optional switch that directs the tool to overwrite existing diagnostic report files in either ASCII or CSV format. By default, the tool will not overwrite existing files.

Echo

Tools Supported: Diagnosis

Diagnosis Scope: Memory Diagnosis

Usage

ECHo **string** [{ > | >>} *file_pathname*]

Description

Issues a user-defined string to the transcript or a specified pathname.

Arguments

- **string**
A required string that specifies the value to echo to the transcript.
- **>*file_pathname***
An optional redirection operator and pathname pair used at the end of the argument list for creating or replacing the contents of *file_pathname*.
- **>>*file_pathname***
An optional redirection operator and pathname pair string used at the end of the argument list to append value to the contents of the *file_pathname*.

Examples

The following example appends *foo* to the contents of the /U1/log.file.

```
echo foo >>log.file
```

Related Commands

None

Exit

Tools Supported: Diagnosis

Diagnosis Scope: Memory Diagnosis

Usage

EXIt [-Force]

Description

Terminates the application tool program.

Arguments

- -Force

An optional switch that explicitly specifies to not save the session and to immediately terminate the tool session.

Help

Tools Supported: Diagnosis

Diagnosis Scope: Memory Diagnosis

Usage

HELp [*command_name*] [-MANual]

Description

Displays the usage syntax and system mode for the specified command.

The help command displays useful information for a selected command. You can display the usage and syntax of a command by typing “help” and the command name. You can display a list of certain groups of commands by typing “help” and a *keyword* such as Add, Delete, Set, and so on.

Arguments

- *command_name*
An optional string that either specifies the name of the command for which you want help or specifies one of the following keywords whose group of commands you want to list: add, delete, set, setup, or write. If you do not supply a *command_name*, the default display is a list of all the valid command names.
- -MANual
An optional string that specifies to also display the reference manual description for the specified command. If you use this switch without specifying a command name, the tool opens the product bookcase, giving access to all the manuals for that product group.

History

Tools Supported: Diagnosis

Diagnosis Scope: Memory Diagnosis

Usage

HIStory [*list_count*] [-Nonumbers] [-Reverse] [-Save *filename*]

Description

Displays a list of previously executed commands.

The History command is similar to the Korn shell (ksh) **history** command in Unix. By default, this command displays a list of all previously executed commands, including all arguments associated with each command, starting with the oldest.

Note



The HISTFILE and HISTSIZE ksh environment variables do not control the command history of the tool.

You can perform command-line editing if you set the VISUAL or EDITOR ksh environment variable to either emacs, gmacs, or vi editing. Please see the ksh(1) man page for specifics on the various editing modes.

A leading number precedes each command line in the history list that indicates the order in which the commands were entered.

Arguments

- *list_count*
An optional integer that specifies for the tool to display only the specified number (*list_count*) of most the recently executed commands. If no *list_count* is specified, the tool displays all previously executed commands.
- -Nonumbers
An optional string that specifies for the tool to display the history list without the leading numbers. This is useful for creating dofiles. The default displays the leading numbers.
- -Reverse
An optional switch that specifies for the tool to display the history list starting with the most recent command rather than the oldest.
- -Save *filename*
An optional switch and string pair that specifies the name of a file (*filename*) to write the command history to.

Report Controller Mapping

Tools supported: Diagnosis

Scope: Memory diagnosis mode

Usage

REPort COntroller Mapping **-All** | *controller_instance_pathname...*

Description

Displays mapping information for the specified controllers.

Arguments

- **-All**
Required switch that reports all controller mapping.
- *controller_instance_pathname*
Required, repeatable string that specifies the pathname of the controller instance to display mapping for.

Examples

The following example displays all the mapping information for the current diagnostic session.

```
report controller mapping -all

// command: report controller mapping -a
Controller : /E1/cntl1
  Config File : ascii_dout_only.cfg
  Memory 0   : /E1/mem1 (RRA8X5topword0)
  Memory 1   : /E1/mem2 (RRA8X5topword1)

Controller : /E1/cntl2
  Config File : ascii_dout_plus_failmap.cfg
  Memory 0   : /E1/mem3 (RRA8X5topword0)
  Memory 1   : /E1/mem4 (RRA8X5topword1)

Controller : /E1/cntl3
  Config File : ascii_failmap_only.cfg
  Memory 0   : /E1/mem5 (RRA8X5topword0)
```

Related Commands

[Add Controller Mapping](#)

[Delete Controller Mapping](#)

Report Diagnostic Monitor

Tools supported: Diagnosis

Scope: Memory Diagnosis mode

Usage

REPort Diagnostic Monitor **-All** | *controller_instance_pathname...*

Description

Displays mapping information associated with the specified diagnostic monitor.

Arguments

- **-All**
Required switch that displays information for all the diagnostic monitors associated with the current diagnosis session.
- *controller_instance_pathname*
Required, repeatable string that specifies the pathname for the controller instance to display diagnostic monitor information for.

Examples

The following example displays the diagnostic monitor information for all the controllers in the current diagnosis session.

```
report diagnsotic monitor -all
// command: report diagnostic monitor -a
Controller: /E1/cnt11
```

Monitor	Width
preamble	3 (optional 011)
addr	12
dout	5
tstate	3
addr_reg	3
memnum	1
postamble	2 (optional 11)

```
Controller: /E1/cnt12
```

Monitor	Width
preamble	3 (optional 011)
addr	12
dout	5
tstate	3
addr_reg	3
failmap	5
memnum	1
postamble	2 (optional 11)

```
Controller: /E1/cnt13
```

Monitor	Width
preamble	3 (optional 011)
addr	12
tstate	3
addr_reg	3
failmap	5
memnum	1
postamble	2 (optional 11)

Related Commands

[Add Controller Mapping](#)

Set Diagnosis

Tools Supported: Diagnosis

Diagnosis Scope: Memory Diagnosis

Usage

SET DIAGNOSIS [-Report {Verbose | Brief }]

Description

Sets the diagnosis reporting parameters.

Arguments

- -Report {Verbose | Brief}
An optional switch and literal pair that specifies the format of the diagnosis reports.

Set Dofile Abort

Tools Supported: Diagnosis

Diagnosis Scope: Memory Diagnosis

Usage

SET DOfile Abort **ON** | **OFF** | **exit**

Description

Specifies whether the tool aborts or continues dofile execution if it detects an error condition.

By default, if an error occurs during the execution of a dofile, processing stops, and the line number causing the error in the dofile is reported. The Set Dofile Abort command lets you to turn this functionality off so that the tool continues to process all commands in the dofile.

Arguments

- **ON**

A literal that halts the execution of a dofile upon the detection of an error. When in batch mode the tool exits on error, but when in interactive mode the tool returns to the session prompt. This is the default upon invocation of the tool.

- **OFF**

A literal that forces dofile processing to complete all commands in a dofile regardless of error detection.

- **exit**

A literal that directs the tool to exit the session if it detects an error while executing a dofile regardless of whether invoked in batch or interactive mode.

System

Tools Supported: Diagnosis

Diagnosis Scope: Memory Diagnosis

Usage

SYStem *os_command*

Description

Passes the specified command to the operating system for execution.

The system command executes one operating system command without exiting the currently running application.

Arguments

- *os_command*
A required string that specifies any legal operating system command.

There are several ways to get help when setting up and using Tessent software tools. Depending on your need, help is available from documentation, online command help, and Mentor Graphics Support.

Documentation

The Tessent software tree includes a complete set of documentation and help files in PDF format. Although you can view this documentation with any PDF reader, if you are viewing documentation on a Linux file server, you must use only Adobe® Reader® versions 8 or 9, and you must set one of these versions as the default using the MGC_PDF_READER variable in your *mgc_doc_options.ini* file.

For more information, refer to “[Specifying Documentation System Defaults](#)” in the *Managing Mentor Graphics Tessent Software* manual.

You can download a free copy of the latest Adobe Reader from this location:

<http://get.adobe.com/reader>

You can access the documentation in the following ways:

- **Shell Command** — On Linux platforms, enter **mgcdocs** at the shell prompt or invoke a Tessent tool with the -Manual invocation switch. This option is available only with Tessent Shell and the following classic point tools: Tessent FastScan, Tessent TestKompress, Tessent Diagnosis, and DFTAdvisor.
- **File System** — Access the Tessent bookcase directly from your file system, without invoking a Tessent tool. From your product installation, invoke Adobe Reader on the following file:

```
$MGC_DFT/doc/pdfdocs/_bk_tessent.pdf
```

- **Application Online Help** — You can get contextual online help within most Tessent tools by using the “help -manual” tool command:

```
> help dofile -manual
```

This command opens the appropriate reference manual at the “dofile” command description.

Mentor Graphics Support

Mentor Graphics software support includes software enhancements, access to comprehensive online services with SupportNet, and the optional On-Site Mentoring service.

For details, refer to this page:

<http://supportnet.mentor.com/about>

If you have questions about a software release, you can log in to SupportNet and search thousands of technical solutions, view documentation, or open a Service Request online:

<http://supportnet.mentor.com>

If your site is under current support and you do not have a SupportNet login, you can register for SupportNet by filling out a short form here:

<http://supportnet.mentor.com/user/register.cfm>

All customer support contact information is available here:

<http://supportnet.mentor.com/contacts/supportcenters/index.cfm>

Appendix B

Design Rules Checking

The Design Rules

Most error messages are accompanied by a line number and file name to help you resolve the error condition. Some error messages relate to problems with internally generated data, and therefore will not have a line number and file name to report. In cases where it is possible to identify this information, the following informational message will accompany the error message to report file name and line number:

```
The following occurred at line L in file F.
```

Where L is the line number and F is the file_name. For example, a syntax error (P1) in a test procedure file would produce the following two messages:

```
The following occurred at line 10 in file testproc.  
Syntax error in line number 10. (P1-1).
```

CTDF Rule Checking

CTDF rules check the controller definitions in the CTDF against the netlist data. The default handling for all CTDF rules is an error.

CTDF1 Rule

No controllers are found (ctdf is not loaded), resulting in the following message:

```
No controllers found.
```

CTDF2 Rule

No instance in the netlist is associated with the controller, resulting in the following message:

```
Controller controller_name does not exist in the netlist.
```

CTDF3 Rule

A pin is in the netlist, but the pin is not declared in the CTDF, resulting in the following message:

```
Pin pin_name of controller controller_name is not included in CTDF.
```

CTDF4 Rule

A pin is declared in the CTDF but the pin is not declared in the netlist, resulting in the following message:

```
Pin pin_name of controller controller_name does not exist in netlist.
```

CTDF Rule Summary Message

After all of the CTDF rule messages are output, a summary message is output as follows:

```
Totally there are n CTDF rule violation.
```

Where n is the number of rule violations.

Integration Rule Checking (I Rules)

Integration Rule 1 (I1)

The controller provider must create a controller test description file that contains information necessary to construct access mechanisms to the controller (via RTL DFT synthesis) and also to construct the chip-level test vectors once such an access mechanism is in place. This file should include information about the controller input/output and controller access and isolation methods.

The controller should be specified in the controller declaration that names the controller and describes the controller inputs, outputs, and bidirectionals. There must be at least one controller and controller instance specified.

If the controller or corresponding controller instance is not specified, the following error message is generated:

```
No P defined. (I1)
```

Where P is the name of the missing controller or controller instance.

You can correct this error condition by specifying the controller in the controller declaration.

Integration Rule 2 (I2)

Each controller instance or controller module described in the Controller Test Description file must exist in the design netlist. If a controller instance or module is not found in the design netlist, the following error message is generated:

```
Controller Test Access File does not have information of controller
instance P. (I2)
Instance P in controller access files is not found in the design. (I2)
Module M in controller definition files is not found in the design. (I2)
```

Where P is the controller instance name and M is the module name.

You can correct this error condition by specifying a controller declaration in your controller test description file.

Integration Rule 3 (I3)

The controller declaration consists of a block that names the controller, and describes the controller's inputs, outputs, and bidirectionals. As part of the controller declaration, you declare the ports on the controller's boundary, controller inputs, outputs, and signals in the pin_type statement. Special ports that reference the ports and input/outputs are then defined, followed by the access method. a check is made to see if the pins in the controller declaration match the pins on the controller boundary. If the pins do not match, one of the following error messages is generated:

```
Controller declaration file of controller C does not have pin P which
exists in the design. (I3)
Controller declaration file of controller C has pin P which does not exist
in the design. (I3)
Pin P in controller declaration of controller C has wrong pin direction.
(I3)
```

Where C is the controller name, and P is the pin name.

You can correct this error condition by changing the pins specified in the controller declaration so that they match the pins on the controller boundary.

Integration Rule 4 (I4)

Within the controller declaration, the clock statement specifies signals that are used as clocks. Clock pins are those which can cause a state element to change state. Clocks, sets, and resets are typically identified. It is required that every clock pin defined in the controller declaration must be mapped to an SoC clock pin.

A verification is made to check that each clock pin in the controller declaration is mapped to an SoC clock pin. If the pins are not mapped or are mapped incorrectly, one of the following error messages is generated:

```
Clock pin P in controller declaration of controller C is not mapped to an
SoC clock pin. (I4)
Clock pin P in controller declaration of controller C is mapped to SoC pin
S which is not a clock pin. (I4)
Clock pin P in controller declaration of controller C is mapped to SoC pin
S which is a clock pin with inconsistent clock off state. (I4)
```

Where P is the clock pin name, C is the controller name, and S is the SoC pin name.

You can correct this error condition by verifying that each clock pin defined in the controller declaration is mapped to an SoC clock pin.

Integration Rule 8 (I8)

This error message indicates an error in mapping. If all mapped pins of a controller do not have a direct path to the corresponding mapped SoC pin after the controller_access procedure is applied, then it generates one of the following messages:

```
N path between pin "P" of controller instance "C" (M) and SoC pin "S"
cannot be established. (I8)
N path between pin "P" of controller instance "C" (M) and SoC pin "S" has
different party as specified in controller access file. (I8)
```

Where N is the pin direction (either input or output); P is the pin name; C is the controller instance name; M is the controller ID; and S is the SoC pin name.

You can correct this error condition by reviewing your controller_access procedure for inconsistency in pin definitions within mapping statements.

Integration Rule 9 (I9)

The probe pin_name statement specifies which input/outputs of the controller needs to be “contacted” for test or require access to the SoC inputs.

If any pins are identified with probe statements that are not mapped to an SoC pin, then it generates the following message:

```
Probed pin P of controller instance C is not mapped to an SoC pin. (I9)
```

Where P is the pin name and C is the instance name.

You can correct this error condition by verifying that all pins identified with probe statements are mapped to an SoC pin.

Integration Rule 11 (I11)

All signals occurring in the controller pattern file must be identified in either the applicable probe lists or in the controller definition. Unidentified signals in the controller pattern file will invoke the I11 rule and generate the following error message:

```
Signal name from filename is not in description_string name. (I11)
```

If the default for this rule is changed to anything less than error, the extra signals and pattern information will be ignored.

Integration Rule 12 (I12)

All signals occurring in the controller definition and the applicable probe lists must also be present in the pattern file being translated. Signals missing from the pattern file will invoke the following error message:

```
Signal name from controller controller_name is not in pattern file  
file_name. (I12)
```

If the default for this rule is changed to anything less than error, then an 'X' value will be used for that signal during pattern translation.

Pad Rule Checking

The pad rules validate design associations between SoC pins and pads.

PAD1 Rule

The SoC pin should have a pad unless it is specified as excluded or dont_touch. The default handling is:

```
Error message: SoC pin pin_name has no pad  
Warning message: SoC pin pin_name has no pad
```

PAD2 Rule

SoC pin has more than one pad. The default handling is an error.

```
Error message: SoC pin pin_name has more than one pad.
```

PAD3 Rule

An SoC pin and its pad have different direction. The default handling is an error.

Error message: SoC pin `pin_name` has wrong direction pad `inspathname`.

PAD4 Rule

Input pad and IO pad should have exactly one `data_in` signal. The default handling is an error.

Error message: A `pin_type` (`data_in`) is associated with more than one pin in PAD design `design_name`

PAD5 Rule

Input pad and IO pad should have one `data_in`. The default handling is an error.

Error message: PAD design `design_name` has no `pin_type` `data_in`

PAD6 Rule

`Data_in` should be output pin. The default handling is an error.

Error message: `Data_in` `pin_name` should be output pin in the PAD design `design_name`

PAD7 Rule

Output pad and IO pad should have exactly one `data_out`. The default handling is an error.

Error message: A `pin_type` (`data_out`) is associated with more than one pin in PAD design `design_name`

PAD8 Rule

Output pad and IO pad should have one `data_out`. The default handling is an error.

Error message: PAD design `design_name` has no `pin_type` `data_out`

PAD9 Rule

Data_out should be an input pin. The default handling is an error.

Error message: Data_out pinname should be input pin in the PAD design
design_name

PAD10 Rule

A Bidi pad should have one output_enable or both one output_enable and one input_enable.
The default handling is an error.

Error message: PAD design design_name has no pin_type output_enable

PAD11 Rule

A Bidi pad should not have more than one output_enable. The default handling is an error.

Error message: PAD design design_name has more than one pin_type
output_enable

PAD12 Rule

A Bidi pad should not have more than one input_enable. The default handling is an error.

Error message: PAD design design_name has more than one pin_type
input_enable

PAD13 Rule

Output_enable should be input pin. The default handling is an error.

Error message: Output_enable pin_name should be input pin in the PAD
design

PAD14 Rule

Input_enable should be input pin. The default handling is an error.

Error message: Input_enable pin_name should be input pin in the PAD design design_name

PAD15 Rule

All pads should have exactly one io_pin. The default handling is an error.

Error message: A pin_type (io_pin) is associated with more than one pin in PAD design design_name

PAD16 Rule

All pads should have one io_pin. The default handling is an error.

Error message: PAD design design_name has no pin_type io_pin

PAD17 Rule

Io_pin direction checking. The default handling is an error.

Error message: Io_pin pin_name has incorrect pin direction in the PAD design design_name

PAD18 Rule

Un-recognized pin attribute. The default handling is an error.

Error message: Pin pin_name has un-recognized pin attribute in the PAD design design_name

PAD19 Rule

Un-recognized cell attribute. The default handling is an error, and it can be changed to warning, note, or ignore.

Error message: Design design_name has un-recognized pin attribute attribute_name

PAD20 Rule

Multiple cell attributes. The default handling is an error.

Error message: Design design_name has multiple cell attributes

PAD21 Rule

SoC attaches to wrong pad pin. The default handling is an error.

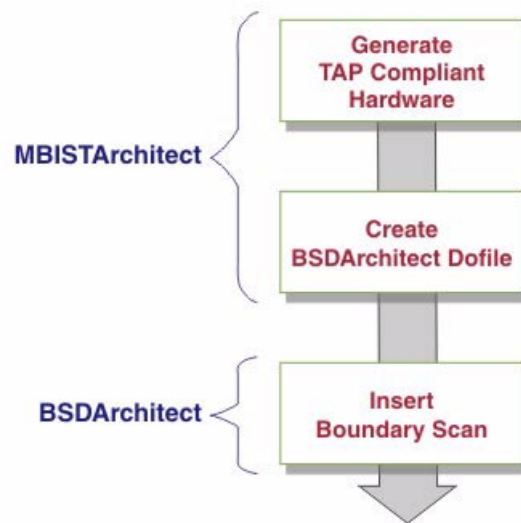
Error message: SoC pin_name attaches to a pad inspathname at pin pin_name without io_pin type.

Appendix C

MBISTArchitect Flow with BSDArchitect

The Memory BIST logic inserted by MBISTArchitect can be controlled through a TAP controller generated by BSDArchitect. The process for creating this logic is shown in [Figure C-1](#).

Figure C-1. Memory BIST to Boundary Scan Process



Generating TAP Compliant Hardware

The BIST hardware is created during the generation phase. Most BIST configurations are ready to use with a TAP controller. However, if your design has diagnostics or repair enabled, a few modifications are required for the BIST controller to properly connect to a BSDArchitect TAP controller. Use the “[Set Bsdarchitect ON](#)” command in your MBISTArchitect generation dofile(s) to make these modifications and create the TAP compliant BIST hardware. If your design does not use diagnostics or BISA, you do not need to include this command in your generation dofile(s).

Creating the BSDArchitect Dofile

After you have generated the TAP compliant BIST circuitry, MBISTArchitect can create a dofile for BSDArchitect, which generates a TAP controller and connects it to the BIST controller(s).

To create the BSDArchitect dofile, use the [Save Driver Files](#) command with the -Bsda switch during the BIST mode of the insertion phase. For example:

```
save driver files -bsda bsda_dofile.do
```

Inserting Boundary Scan

Once you have created the BSDArchitect dofile and exited the MBISTArchitect insertion session, you can insert the boundary scan logic by invoking BSDArchitect. You can include the dofile in the [bsdarchitect](#) invocation command or you can use the [Dofile](#) command after invocation. For example:

```
bsdarchitect design.v -dofile bsda_dofile.do
```

The dofile generated by MBISTArchitect can be used in either the Internal or External BSDArchitect flow.

Examples

The following examples illustrate the process for creating both memory BIST and boundary scan logic for designs with particular MBISTArchitect features enabled:

- “[Simple Configuration](#)” on page 296
- “[Diagnostics](#)” on page 301
- “[Retention](#)” on page 306
- “[MISRs](#)” on page 309
- “[Repair](#)” on page 314
- “[Online Algorithm Selection](#)” on page 317

Simple Configuration

The following example describes the process for creating both memory BIST and boundary scan logic for a design with a simple configuration. This BIST configuration has two controllers that are tested sequentially. Retention, diagnostics, MISRs, BISA, and online algorithm selection are not enabled for either controller.

Generating TAP Compliant Hardware

For the simple configuration, there are no connections that conflict with Boundary Scan standards. Therefore, you do not need to use the [Set Bsdarchitect](#) command in your generation dofiles. The following generation dofile was used for this simple controller configuration:


```
//-----
// Generation Dofile
//-----
add memory models ram4x4
add mbist algorithms 1 march2
set bist insertion -on
setup file naming -ctdl mbist1.ctdl \
    -bist mbist1.v\
    -conn mbist1_conn.v\
    -test mbist1_tb.v\
    -script mbist1_dcscript \
    -wgl mbist1.wgl
run
report mbist algorithms
report algorithm steps
save bist -verilog -replace -Script
exit
```

Creating the BSDArchitect Dofile

To create the BSDArchitect dofile, include the [Save Driver Files](#) command in the insertion dofile. The following insertion dofile was used for this simple configuration:

```
//-----
// Insertion Dofile
//-----
load library mbist.lib
report memory instances
set system mode bist
add new controller myctrl1 -dofile mbist1.do /core_b/mem_b
add new controller myctrl2 -dofile mbist2.do /mem_a
report memory instances
report controllers
insert bist logic
save design -include none -replace
save driver files -bsda bsda.do -replace
set system mode int
add pattern translation -all
integrate patterns
save patterns patterns.v -verilog -rep
exit -force
```

The following dofile was created by MBISTArchitect for the simple configuration. The components of this dofile are discussed below.

```
1 // -----
2 // BSDArchitect Dofile
3 // -----
4
5 /*** Create Reset register to reset before the start of mbist operation
6 add bscan instruction mbist_reset -reg BSCAN_BYPASS_REG
7
8 /*** Create mbist register and target an instr for mbist operations
9 add external register mbist_reg 4
```

```
10 add bscan instruction mbist_instr -reg mbist_reg
11
12 /*** Create the register interface
13 set external_register interface mbist_reg -capture tst_done_mgc_1
fail_h_mgc_1 tst_done_mgc_2 fail_h_mgc_2 -update test_h_mgc_1
test_h_mgc_2
14
15 /*** Connecting controller resets to BSDA reset
16 add port connection rst_l_mgc_1 "mbist_reset NAND updateir"
17 add port connection rst_l_mgc_2 "mbist_reset NAND updateir"
18
19 /*** Connecting Bist clock
20 add port connection bist_clk_mgc_1 "buf tck"
21 add port connection bist_clk_mgc_2 "buf tck"
22
23 /*** Specify non-top ports
24 add nontop port test_h_mgc_1
25 add nontop port tst_done_mgc_1
26 add nontop port fail_h_mgc_1
27 add nontop port test_h_mgc_2
28 add nontop port tst_done_mgc_2
29 add nontop port fail_h_mgc_2
30
31 /*** WARNING
32 /*** The Cycles in the following "set testbench parameters" commands
33 /*** are based on MBISTA tool best estimates. User may need to modify
34 /*** the number of cycles if necessary
35
36 set testbench parameters -instr mbist_instr -test_event mbist_1
-shift_in 10 -shift_out 10xx -wait_in_run_test_idle_state 370
37 set testbench parameters -instr mbist_instr -test_event mbist_2
-shift_in 01 -shift_out xx10 -wait_in_run_test_idle_state 134
38
39 /*** Run, save and quit
40 run
41 save bscan -replace
42 save patterns jtag_patterns.wgl -format wgl -replace
43 exit -force
```

This dofile has the following components. The hardware and connections created by the dofile are illustrated in [Figure C-2](#).

- **Setup Reset Control** — To support resetting the BIST controllers, the following are required:
 - a. Create a boundary scan instruction that will enable the controller reset. When you create the instruction, a new signal is created at the TAP interface with the same name as the instruction. The BIST controller reset will be connected to the TAP controller reset. See [line 6](#).
 - b. Connect the instruction signal to the reset signal in each controller. The signal is not connected directly; it is connected with the updateir signal through a NAND gate. See [lines 16 and 17](#).

- **Setup BIST Interface** — To control BIST, you must create an interface between the signals in the TAP controller and the signals in the BIST controller. This is accomplished by the following:
 - a. Create a new register for interfacing with the controllers. The size of the register depends on the number of BIST controllers in the design. See line 9.
 - b. Create a boundary scan instruction to target the new register. See line 10.
 - c. Connect the register to the controller signals. See line 13.

Note

If you mapped any controller pins to internal pins during the MBISTArchitect session, you must replace the internal pin path names in the BSDArchitect dofile with the names of the external pins driving those signals. For example, the path names in the following command must be updated:

```
set external_register interface ret_reg_1 -capture /U1/inp[1] /U1/inp[0]
```

The names are replaced by the corresponding external pin names:

```
set external_register interface ret_reg_1 -capture topOut[1] topOut[0]
```

- **Connect Clocks** — The BIST clock is connected to the boundary scan clock TCK through a buffer. This is the default clock behavior. See lines 20 and 21.

This example uses TCK to drive the BIST clock. However, many designs use a PLL or system clock instead of TCK because TCK is a slow clock. To connect these clocks, the Add Port Connection command is replaced with the following commands:

```
add testbench clock <system / PLL reference clock> -period <t1>
set testbench parameters -tck_period <t2>
```

These commands are preceded by a warning message that instructs you to replace the generated clock paths. For example, using a PLL internal clock produces the following message:

```
/** WARNING: Internal pin ( /PLL/p11_clk ) should be replaced with
// the external pin that drives it.
//
add testbench clock /PLL/p11_clk -period 100
set testbench parameters -tck_period 200
```

You must replace the internal PLL clock path with the name of the external clock pin (in this case topPLLclk).

If the BIST clock is driven by a top-level system clock, the following commands are used:

```
add testbench clock system_clk1 -period 50
set testbench parameters -tck_period 100
```

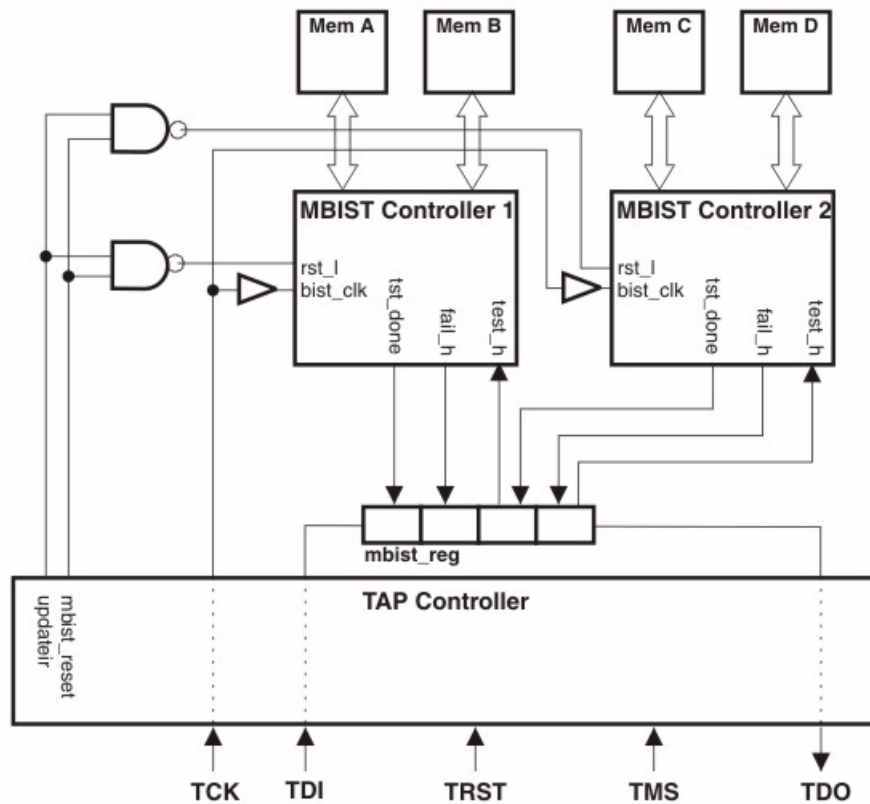
In the above clock examples, it is assumed that the BIST clock speed is twice that of TCK.

- **Place Ports** — By default, BSDArchitect connects the top-level BIST ports to the top level of the design and adds boundary scan cells on each port. A series of [Add Nontop Port](#) commands keeps the top-level BIST ports as internal pins so that no boundary scan cells are added. See lines [24-29](#).
- **Define Test Behavior** — Once the BIST interface is created using the mbist_reg register, the behavior must be defined. The [Set Testbench Parameters](#) commands define the control signals and data timing for the BIST controllers. The cycles represent the number of cycles required for the particular controller to complete BIST. The cycle values determine how long the TAP controller waits in the test/idle before checking for test failure. See lines [36](#) and [37](#).

Note

This example uses sequential testing. Using concurrent groups will change this section of the BSDArchitect dofile. Instead of having one Set Testbench Parameters command per controller, there is one command per concurrent group. For example, if the design has four controllers and controllers 1 and 2 are specified as a concurrent group, there will be three Set Testbench Parameters commands: one command for the concurrent group (controllers 1 and 2), one command for controller 3, and one command for controller 4.

Figure C-2. Default BSDArchitect Configuration



Diagnostics

The following example is a simple design with two controllers, both having diagnostics enabled. These controllers are tested sequentially. The following sections describe the differences between the previous [Simple Configuration](#) example and configurations with diagnostics.

Generating TAP Compliant Hardware

To include diagnostics for a controller, you must include the [Set Controller Debug](#) command in your generation dofile. A register is created in the controller to scan out the diagnostic data. By default, this register does not have an input at the controller boundary, which is not supported by BSDArchitect. To make the design TAP compliant, you must also include the [Set Bsdarchitect](#) command in the generation dofile(s). This will create a dummy input pin for the diagnostics register.

The [Setup Diagnostic Clock](#) command is optional for the diagnostics feature in MBISTArchitect. However, it is required that diagnostics hardware use a separate clock when controlling diagnostics through the TAP controller. To specify a separate clock for diagnostics,

you must include the “Setup Diagnostic Clock -Slow_tester_clk” command in your generation dofile.

Because there are no `diag_scan_enable` signals, the diagnostic clock is used to shift out the diagnostic data. Consequently, *diag_clk* cannot be a free running clock. A free running *diag_clk* would cause constant shifting of the diagnostics register. The *diag_clk* also cannot be shared among controllers. This would cause all controllers to simultaneously shift out diagnostic data.

The following generation dofile was used for this example:

```
//-----  
// Generation Dofile  
//-----  
add memory models ram4x4  
add mbist algorithms 1 march2  
set bist insertion -on  
set controller debug -on  
set diag clock -slow_tester_clk  
set bsdarchitect -on  
setup file naming -ctdl mbist1.ctdl \  
    -bist mbist1.v\  
    -conn mbist1_conn.v\  
    -test mbist1_tb.v\  
    -script mbist1_dcscript \  
    -wgl mbist1.wgl  
run  
report mbist algorithms  
report algorithm steps  
save bist -verilog -replace -Script  
exit
```

Creating the BSDArchitect Dofile

The insertion dofile used for this example is the same as the simple configuration example. The insertion dofile must include the [Save Driver Files](#) command to create the BSDArchitect dofile.

The following dofile was created by MBISTArchitect for the simple configuration with diagnostics. The components of this dofile are discussed below.

```
1 // -----  
2 // BSDArchitect Dofile  
3 // -----  
4  
5 /*** Create Reset register to reset before the start of mbist operation  
6 add bscan instruction mbist_reset -reg BSCAN_BYPASS_REG  
7  
8 /*** Create mbist register and target an instr for mbist operations  
9 add external register mbist_reg 6  
10 add bscan instruction mbist_instr -reg mbist_reg  
11  
12 /*** Create the register interface
```

```
13  set external_register interface mbist_reg -capture tst_done_mgc_1
    fail_h_mgc_1 tst_done_mgc_2 fail_h_mgc_2 -update test_h_mgc_1
    hold_1_mgc_1 debugz_mgc_1 test_h_mgc_2 hold_1_mgc_2 debugz_mgc_2
14
15  /*** Connecting controller resets to BSDA reset
16  add port connection rst_1_mgc_1 "mbist_reset NAND updateir"
17  add port connection rst_1_mgc_2 "mbist_reset NAND updateir"
18
19
20  /*** Connecting Bist clock
21  add port connection bist_clk_mgc_1 "buf tck"
22  add port connection bist_clk_mgc_2 "buf tck"
23
24  /*** Specify non-top ports
25  add nontop port test_h_mgc_1
26  add nontop port restart_h_mgc_1
27  add nontop port tst_done_mgc_1
28  add nontop port diag_clk_mgc_1
29  add nontop port fail_h_mgc_1
30  add nontop port hold_1_mgc_1
31  add nontop port debugz_mgc_1
32  add nontop port diag_scan_in_mgc_1
33  add nontop port diag_scan_out_mgc_1
34  add nontop port test_h_mgc_2
35  add nontop port restart_h_mgc_2
36  add nontop port tst_done_mgc_2
37  add nontop port diag_clk_mgc_2
38  add nontop port fail_h_mgc_2
39  add nontop port hold_1_mgc_2
40  add nontop port debugz_mgc_2
41  add nontop port diag_scan_in_mgc_2
42  add nontop port diag_scan_out_mgc_2
43
44  add core register diag_reg_1 diag_scan_in_mgc_1 diag_scan_out_mgc_1 -len
    25
45  add bscan instruction diag_instr_1 -reg diag_reg_1
46  add port connection diag_clk_mgc_1 "tck AND diag_instr_1 AND shiftdr"
47
48  add core register diag_reg_2 diag_scan_in_mgc_2 diag_scan_out_mgc_2 -len
    29
49  add bscan instruction diag_instr_2 -reg diag_reg_2
50  add port connection diag_clk_mgc_2 "tck AND diag_instr_2 AND shiftdr"
51
52  /*** WARNING
53  /*** The Cycles in the following "set testbench parameters" commands
54  /*** are based on MBISTA tool best estimates. User may need to modify
55  /*** the number of cycles if necessary
56
57  set testbench parameters -instr mbist_instr -test_event mbist_1
    -shift_in 111000 -wait_in_run_test_idle_state 136
58
59  set testbench parameters -instr diag_instr_1 -strobe_cycle 25
60
61  set testbench parameters -instr mbist_instr -test_event mbist_2
    -shift_in 000111 -wait_in_run_test_idle_state 532
62
63  set testbench parameters -instr diag_instr_2 -strobe_cycle 29
64
```

```
65  /*** Run, save and quit
66  run
67  save bscan -replace
68  save patterns jtag_patterns.wgl -format wgl -replace
69  exit -force
```

This dofile has the following components. The hardware and connections created by the dofile are illustrated in [Figure C-3](#) with the components specific to diagnostics highlighted in blue.

- **Setup Reset Control** — This is done exactly the same as the [Simple Configuration](#), which creates an instruction and connects that signal to the reset of each controller. See lines [6](#), [16](#), and [17](#).
- **Setup BIST Interface** — The method for creating and connecting the *mbist_reg* register is the same as the simple configuration. However, the size of the register increases to accommodate the additional *hold_1* and *debugz* signals used for diagnostics. See lines [9](#), [10](#), and [13](#).
- **Connect Clocks** — The BIST clock is connected to the boundary scan clock TCK through a buffer, just as the simple configuration. See lines [21](#) and [22](#).

For diagnostic clock connections, see “Setup Diagnostic Interface.”

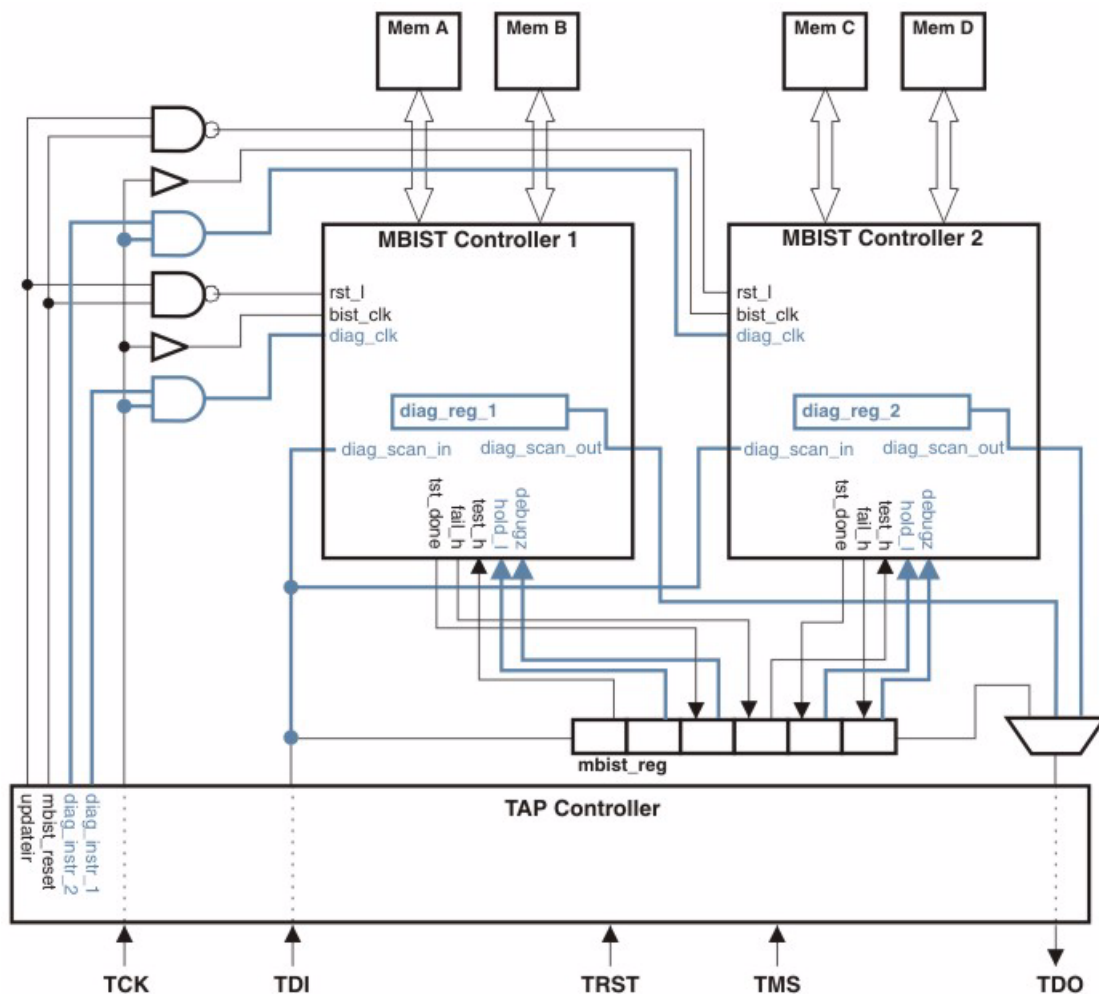
- **Place Ports** — Just like the simple configuration, a series of Add Nontop Port commands prevents the internal ports from being connected at the top level with boundary scan cells. See lines [25-42](#).
- **Setup Diagnostic Interface** — To scan out the diagnostic data for each controller, the following components are required:
 - a. Define a register to hold the diagnostic data for each BIST controller. Because the register already exists in the BIST controller, the Add Core Register command is used to identify the input, output, and length of the register. The length of the core register depends on the hold and restart behavior you selected in your generation dofile. See lines [44](#) and [48](#).
 - b. Define an instruction to shift out the diagnostic data for each controller. The signals created by these instructions are used to control the diagnostic clock, which shifts out the contents of the diagnostic register. Because each diagnostic register must be accessed independently, there must be one instruction per controller. See lines [45](#) and [49](#).
 - c. Connect the diagnostic clock signals to the instructions. The *diag_clk* signal for each controller is driven by the corresponding diagnostics instruction ANDed with TCK and *shiftdr*. See lines [46](#) and [50](#).
- **Define Test Behavior** — The Set Testbench Parameters commands define the behavior for each of the controllers.

- The first command (*mbist_instr*) starts BIST, with diagnostics enabled, for the first controller by loading high values for *test_h*, *hold_l*, and *debugz*. The cycles specified equals the time required to complete BIST for that controller. See line 57.
- The second command (*diag_instr*) activates the diagnostic clock and register to scan out the diagnostic data. The diagnostic clock will be pulsed for the specified number of *strobe_cycles* while in the shift-DR state. The *strobe_cycles* specified equals the length of the core register. See line 59.

Once the diagnostic data is scanned out, the first controller will restart/resume for the same number of cycles specified in the first instruction to complete BIST. The process of loading the *diag_instr* and restarting is repeated until the diagnostic data scanned out is all zeros.

- The above is repeated for the second BIST controller. See lines 61 and 63.

Figure C-3. Boundary Scan Configuration with Diagnostics



Retention

The following example is a simple design with two controllers, tested sequentially, both using retention testing. The following sections describe how the process differs from the [Simple Configuration](#) example.

Generating TAP Compliant Hardware

To enable retention testing for a controller, your generation dofile must add either the pre-defined [retentionCB](#) algorithm or a user-defined algorithm that uses the **synchronize** keyword in at least one write step. For more information see the UDA “[Step Definition](#)” on page 148. This example uses the RetentionCB algorithm for both controllers.

A typical retention test contains a write step followed by a wait period (retention time) and a read step. Once the controller completes the write step, it asserts the *start_retention_h* signal and enters retention. The controller will come out of retention, de-asserting *start_retention_h*, when the *test_resume_h* signal is asserted.

```
//-----  
// Generation Dofile  
//-----  
add memory models ram8x4  
add mbist algorithms 1 march2 retentionCB  
set bist insertion -on  
setup file naming -ctdl mbist2.ctdl \  
    -bist mbist2.v\  
    -conn mbist2_conn.v\  
    -test mbist2_tb.v\  
    -script mbist2_dcscript \  
    -wgl mbist2.wgl  
run  
report algorithm steps  
save bist -verilog -replace -Script  
exit
```

Creating the BSDArchitect Dofile

The insertion dofile used for this example is the same as the simple configuration example. The insertion dofile must include the [Save Driver Files](#) command to create the BSDArchitect dofile.

The following dofile was created by MBISTArchitect for the simple configuration with retention. The components of this dofile are discussed below.

```
1  // -----  
2  // BSDArchitect Dofile  
3  // -----  
4  
5  /*** Create Reset register to reset before the start of mbist operation  
6  add bscan instruction mbist_reset -reg BSCAN_BYPASS_REG  
7
```

```
8  //*** Create mbist register and target an instr for mbist operations
9  add external register mbist_reg 4
10 add bscan instruction mbist_instr -reg mbist_reg
11
12 //*** Create the register interface
13 set external_register interface mbist_reg -capture tst_done_mgc_1
   fail_h_mgc_1 tst_done_mgc_2 fail_h_mgc_2 -update test_h_mgc_1
   test_h_mgc_2
14
15 //*** Connecting controller resets to BSDA reset
16 add port connection rst_l_mgc_1 "mbist_reset NAND updateir"
17 add port connection rst_l_mgc_2 "mbist_reset NAND updateir"
18
19 //*** Connecting Bist clock
20 add port connection bist_clk_mgc_1 "buf tck"
21 add port connection bist_clk_mgc_2 "buf tck"
22
23 //*** Specify non-top ports
24 add nontop port test_h_mgc_1
25 add nontop port tst_done_mgc_1
26 add nontop port fail_h_mgc_1
27 add nontop port start_retention_h_mgc_1
28 add nontop port test_resume_h_mgc_1
29 add nontop port test_h_mgc_2
30 add nontop port tst_done_mgc_2
31 add nontop port fail_h_mgc_2
32 add nontop port start_retention_h_mgc_2
33 add nontop port test_resume_h_mgc_2
34
35 add external register retention_reg_1 2
36 add bscan instruction retention_instr_1 -reg retention_reg_1
37 set external_register interface retention_reg_1 -capture fail_h_mgc_1
   tst_done_mgc_1
38 add port connection test_resume_h_mgc_1 "retention_instr_1 AND updateir"
39
40 add external register retention_reg_2 2
41 add bscan instruction retention_instr_2 -reg retention_reg_2
42 set external_register interface retention_reg_2 -capture fail_h_mgc_2
   tst_done_mgc_2
43 add port connection test_resume_h_mgc_2 "retention_instr_2 AND updateir"
44
45 //*** WARNING
46 //*** The Cycles in the following "set testbench parameters" commands
47 //*** are based on MBISTA tool best estimates. User may need to modify
48 //*** the number of cycles if necessary
49
50 set testbench parameters -instr mbist_instr -test_event mbist_1
   -shift_in 10 -wait_in_run_test_idle_state 245
51
52 set testbench parameters -instr retention_instr_1 -test_event
   retention_1 -shift_in x -shift_out 00 -wait_in_run_test_idle_state 120
53 set testbench parameters -instr retention_instr_1 -test_event
   retention_2 -shift_in x -shift_out 01 -wait_in_run_test_idle_state 9
54
55 set testbench parameters -instr mbist_instr -test_event mbist_2
   -shift_in 01 -wait_in_run_test_idle_state 389
56
```

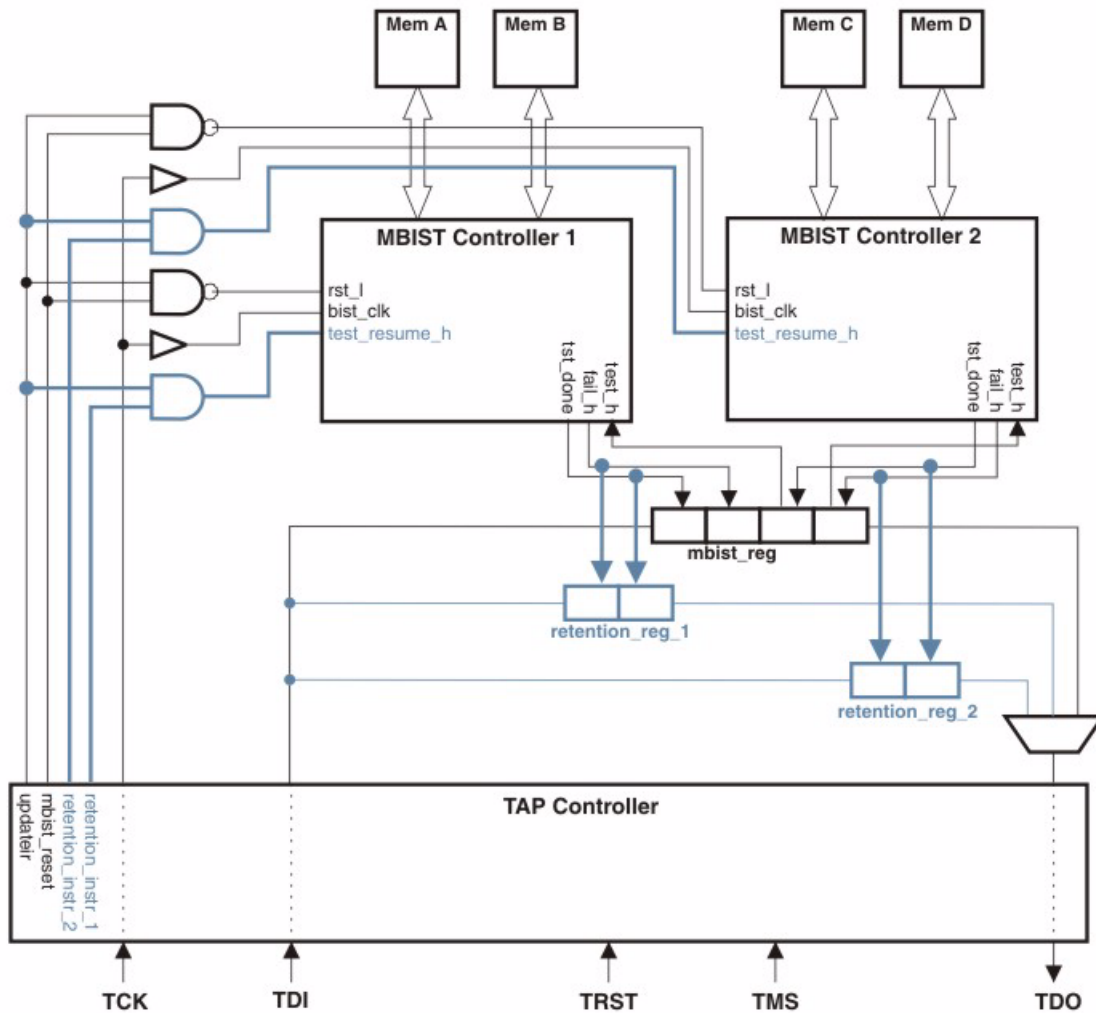
```
57 set testbench parameters -instr retention_instr_2 -test_event
   retention_3 -shift_in x -shift_out 00 -wait_in_run_test_idle_state 140
58 set testbench parameters -instr retention_instr_2 -test_event
   retention_4 -shift_in x -shift_out 01 -wait_in_run_test_idle_state 281
59
60 /*** Run, save and quit
61 run
62 save bscan -replace
63 save patterns jtag_patterns.wgl -format wgl -replace
64 exit -force
```

This dofile has the following components. The hardware and connections created by the dofile are illustrated in [Figure C-4](#) with the components specific to retention highlighted in blue.

- **Setup Reset Control** — This is done exactly the same as the [Simple Configuration](#), which creates an instruction and connects that signal to the reset of each controller. See lines [6](#), [16](#), and [17](#).
- **Setup BIST Interface** — The creation and connection of the *mbist_reg* register is the same as the simple configuration example. See lines [9](#), [10](#), and [13](#).
- **Place Ports** — Just like the simple configuration, a series of Add Nontop Port commands prevents the internal ports from being connected at the top level with boundary scan cells. See lines [24-33](#).
- **Setup Retention Interface** — To capture the retention signals, the following components are required for each controller using retention testing:
 - a. Define a register for each BIST controller. See lines [35](#) and [40](#).
 - b. Define an instruction to control each register. See lines [36](#) and [41](#).
 - c. Connect the registers to the controller output signals. See lines [37](#) and [42](#).
 - d. Connect the instruction signals to the BIST controllers. See lines [38](#) and [43](#).
- **Connect Clocks** — The BIST clock is connected to the boundary scan clock TCK through a buffer, just as the simple configuration.
- **Define Test Behavior** — The [Set Testbench Parameters](#) commands define the behavior for each of the controllers. The retention testing changes these significantly. For each controller in this design, there will be three parameters defined:
 - a. *mbist_instr* — Start BIST for a controller. The number of cycles is equal to the time from the start of BIST until the end of the first retention period. See lines [50](#) and [55](#).
 - b. *retention_instr*, *test_event1* — Assert test_resume_h to continue BIST after the first retention period. The cycles define the time between the end of the first retention period and the end of the second retention period. See lines [52](#) and [57](#).
 - c. *retention_instr*, *test_event2* — Assert test_resume_h to continue BIST after the second retention period. The cycles define the time between the end of the second retention period and the end of BIST for the controller. See lines [53](#) and [58](#).

Note

The parameters defined depends on the algorithms you have selected. This design is using the RetentionCB algorithm, which has two retention steps. If you are using a UDA, the number of retention steps and the parameters may be different.

Figure C-4. Boundary Scan Configuration with Retention

MISRs

This simple example has two controllers tested sequentially. The first controller has a single ROM and the second controller has two ROMs. Each ROM has a 32-bit MISR.

Generating TAP Compliant Hardware

To use MISRs, you must associate one or more ROM instances with the controller. Mixing RAM and ROM instances for a controller is not allowed. For each ROM instance, MBISTArchitect automatically adds a MISR. The size and tap locations are determined automatically, but you can specify custom MISR information using the [Setup Misr Polynomial](#) command.

The following generation dofile was used for one of the controllers in this example. The bold lines are the commands used to set up the MISR hardware.

```
//-----  
// Generation Dofile  
//-----  
add memory models RR128X8_ROM -File rom_init_128x8  
setup misr polynomial -size 32  
set bist insertion -on  
set design name controller -module first_rom  
set design name collar -module first_collar_rom  
set design name misr -module first_misr  
setup file naming -ctdl mbist_rom1.ctdl \  
    -bist mbist_rom1.v \  
    -conn mbist_rom1_conn.v \  
    -test mbist_rom1_tb.v \  
    -script mbist_rom1_dcscript \  
    -wgl mbist_rom1.wgl  
run  
report algorithm steps  
save bist -verilog -script -replace  
exit
```

Creating the BSDArchitect Dofile

The insertion dofile used for this example is the same as the simple configuration example. The insertion dofile must include the [Save Driver Files](#) command to create the BSDArchitect dofile.

The following dofile was created by MBISTArchitect for the simple configuration with MISRs. The components of this dofile are discussed below.

```
1  // -----  
2  // MGC BSDA Dofile  
3  // -----  
4  
5  /*** Create Reset register to reset before the start of mbist operation  
6  add bscan instruction mbist_reset -reg BSCAN_BYPASS_REG  
7  
8  /*** Create mbist register and target an instr for the mbist operations  
9  add external register mbist_reg 2  
10 add bscan instruction mbist_instr -reg mbist_reg  
11  
12 /*** Create the register interface
```

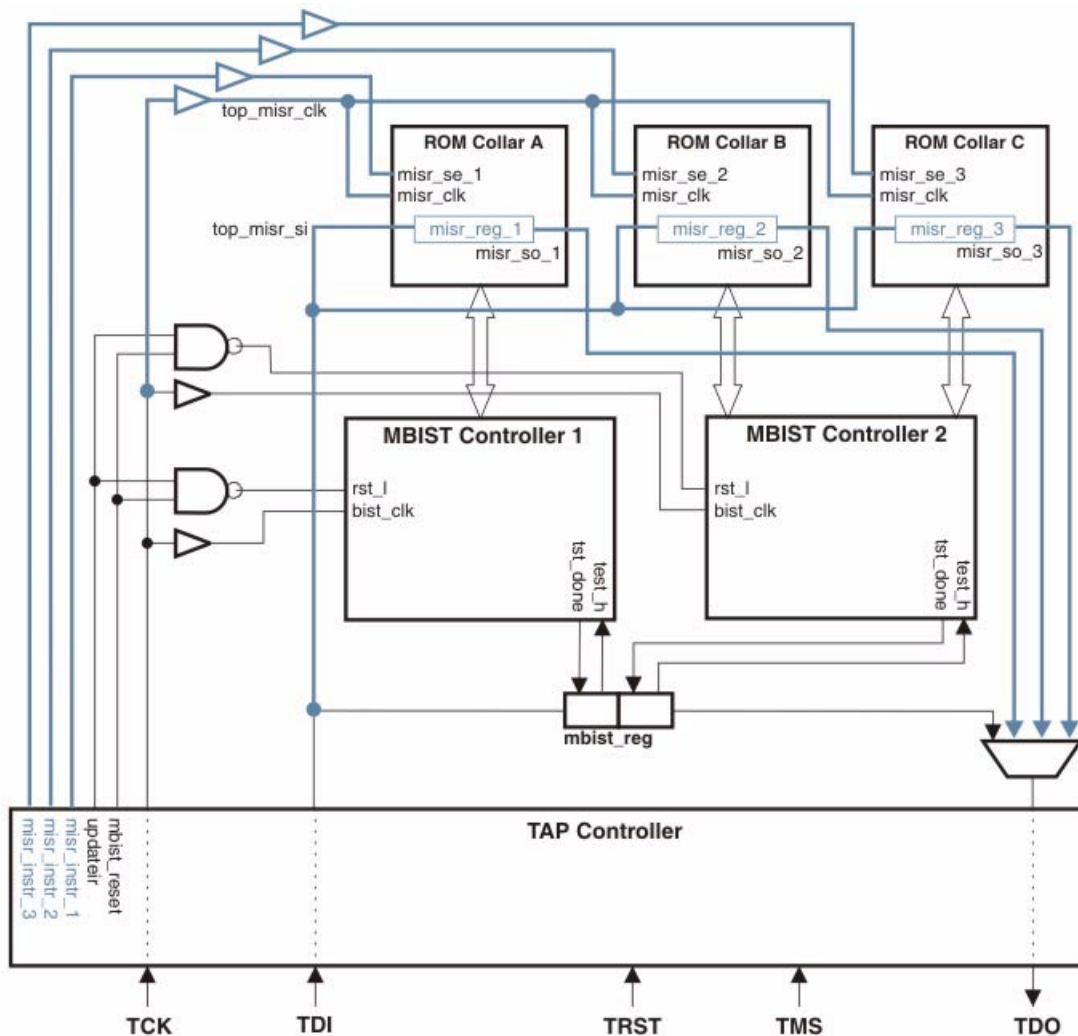
```
13 set external_register interface mbist_reg -capture tst_done_mgc_1
   tst_done_mgc_2 -update test_h_mgc_1 test_h_mgc_2
14
15 /*** Connecting controller resets to BSDA reset
16 add port connection rst_l_mgc_1 "mbist_reset NAND updateir"
17 add port connection rst_l_mgc_2 "mbist_reset NAND updateir"
18
19 /*** Connecting Bist clock
20 add port connection bist_clk_mgc_1 "buf tck"
21 add port connection bist_clk_mgc_2 "buf tck"
22
23 /*** Specify non-top ports
24 add nontop port test_h_mgc_1
25 add nontop port tst_done_mgc_1
26 add nontop port test_h_mgc_2
27 add nontop port tst_done_mgc_2
28 add nontop port top_misr_si
29 add nontop port misr_so_1
30 add nontop port misr_se_1
31 add nontop port misr_so_2
32 add nontop port misr_se_2
33 add nontop port misr_so_3
34 add nontop port misr_se_3
35
36 add core register misr_reg_1 top_misr_si misr_so_1
37 add bscan instruction misr_instr_1 -reg misr_reg_1
38 add port connection top_misr_clk "buf tck"
39 add port connection misr_se_1 "misr_instr_1 AND shiftdr"
40
41 add core register misr_reg_2 top_misr_si misr_so_2
42 add bscan instruction misr_instr_2 -reg misr_reg_2
43 add port connection misr_se_2 "misr_instr_2 AND shiftdr"
44
45 add core register misr_reg_3 top_misr_si misr_so_3
46 add bscan instruction misr_instr_3 -reg misr_reg_3
47 add port connection misr_se_3 "misr_instr_3 AND shiftdr"
48
49 /*** WARNING
50 /*** The Cycles in the following "set testbench parameters" commands
51 /*** are based on MBISTA tool best estimates. User may need to modify
52 /*** the number of cycles if necessary
53
54 set testbench parameters -instr mbist_instr -test_event mbist_1
   -shift_in 10 -shift_out 1x -wait_in_run_test_idle_state 386
55
56 set testbench parameters -instr misr_instr_1 -test_event misr_1
   -shift_out 0001011101011101010101011101000001 -strobe_cycle 32
57
58 set testbench parameters -instr mbist_instr -test_event mbist_2
   -shift_in 01 -shift_out x1 -wait_in_run_test_idle_state 386
59
60 set testbench parameters -instr misr_instr_2 -test_event misr_1
   -shift_out 0001011101011101010101011101000001 -strobe_cycle 32
61
62 set testbench parameters -instr misr_instr_3 -test_event misr_3
   -shift_out 0001011101011101010101011101000001 -strobe_cycle 32
63
64 /*** Run, save and quit
```

```
65  run
66  save bscan -replace
67  save patterns jtag_patterns.wgl -format wgl -replace
68  save patterns jtag_patterns.v -format verilog -replace
69  exit -force
```

This dofile has the following components. The hardware and connections created by the dofile are illustrated in [Figure C-5](#) with the components specific to MISR hardware highlighted in blue.

- **Setup Reset Control** — This is done the same as the [Simple Configuration](#), which creates a reset instruction and connects that signal to the reset of each controller. See lines [6](#), [16](#), and [17](#).
- **Setup BIST Interface** — The creation and connection of the *mbist_reg* register is similar to the simple configuration example. Notice that the register has a length of 2 instead of 4. This is because the *fail_h* signal is not used for ROM testing. Failure is determined by comparing the MISR signature. See lines [9](#), [10](#), and [13](#).
- **Place Ports** — Just like the simple configuration, a series of Add Nontop Port commands prevents the internal ports from being connected at the top level with boundary scan cells. See lines [24](#) - [34](#).
- **Setup MISR Interface** — The following are done for each MISR in the design. See lines [36](#) - [47](#).
 - a. Identify the MISR as an internal register. Notice that the specified input is the same for each register. For this example, the MISR inputs were shared during BIST insertion.
 - b. Define an instruction for shifting out the MISR signature and connect the instruction signal to the shift enable of the register.
 - c. Connect the misr clocks to TCK. In this example, the individual *misr_clk* signals were shared under the *top_misr_clk* signal during BIST insertion and are connected to TCK with a single BSDArchitect command.
- **Connect Clocks** — The BIST clock connects to the boundary scan clock TCK through a buffer, just as the simple configuration.
- **Define Test Behavior** — The [Set Testbench Parameters](#) commands define the behavior for each controller and MISR. The test behavior for this design includes the following for each controller:
 - a. Reset the controller and assert *test_h* to start BIST. Check if *tst_done* is asserted at the end of the test time. See lines [54](#) and [58](#).
 - b. Load the *misr_instr* instruction to connect the *misr_reg* register between TDI and TDO and shift out the MISR signature. See lines [56](#), [60](#), and [62](#).

Figure C-5. Boundary Scan Configuration with MISRs

**Note**

To capture the collar's fail_h signal into the BSDArchitect external register, use the [Add Pin Mapping](#) command to map the fail_h signal to the top level as in the following example:

```
add pin mapping misr_fail_h /rom_1/fail_h
```

The following is a snippet of the resulting BSDArchitect dofile:

```
14 /*** Create the register interface
15 set external_register interface mbist_reg -capture tst_done_mgc_1
   misr_fail_h -update test_h_mgc_1
```

Repair

This simple example has two controllers tested sequentially with BISA hardware. Each controller has one associated memory instance.

Generating TAP Compliant Hardware

To set up BISA hardware for your design, specify the repair strategy using the [Add BISA Hardware](#) command.

BSDArchitect will identify a core register for scanning out the BISA report. However, MBISTArchitect does not create an input for this register by default. You must include the “Set Bsdarchitect -ON” command in your generation dofile to create TAP-compliant hardware. This adds a dummy input pin, named *repair_data_in*, for the BISA report register.

```
//-----  
// Generation Dofile  
//-----  
add memory model ram4x4  
add mbist algorithms 1 march2  
set bist insertion -on  
set bsdarchitect -on  
add bisa hardware -row 2  
report bisa hardware  
setup file naming -ctdl mbist1.ctdl \  
    -bist mbist1.v \  
    -conn mbist1_conn.v \  
    -test mbist1_tb.v \  
    -script mbist1_dcscript \  
    -wgl mbist1.wgl  
run  
report mbist algorithms  
report algorithm steps  
save bist -verilog -replace -script  
exit
```

Creating the BSDArchitect Dofile

The insertion dofile used for this example is similar to the simple configuration example. The insertion dofile must include the [Save Driver Files](#) command to create the BSDArchitect dofile.

The following dofile was created by MBISTArchitect for the simple configuration with repair. The components of this dofile are discussed below.

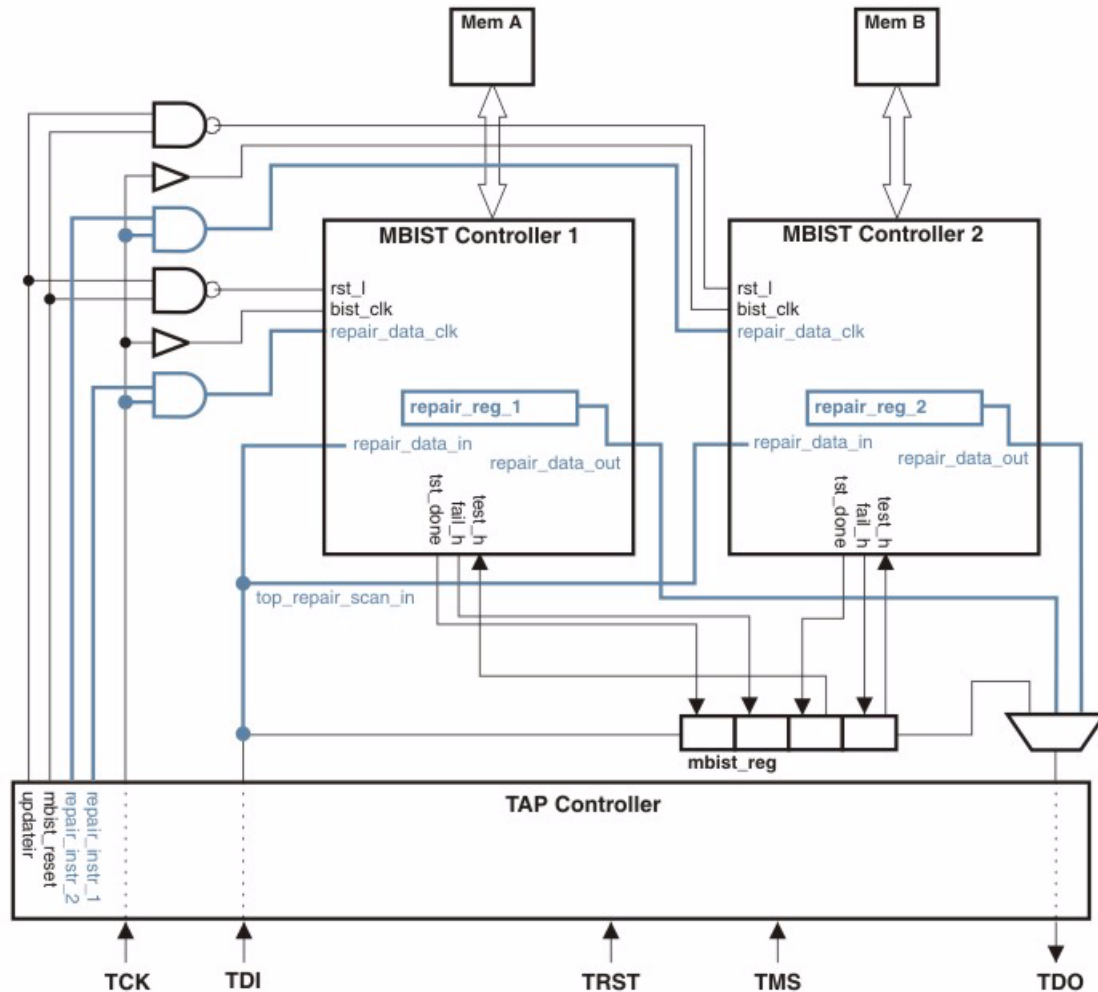
```
1  // -----  
2  // MGC BSDA Dofile  
3  // -----  
4  
5  /*** Create Reset register to reset before the start of mbist operation  
6  add bscan instruction mbist_reset -reg BSCAN_BYPASS_REG  
7
```

```
8  /*** Create mbist register and target an instruction for the mbist
   operations
9  add external register mbist_reg 4
10 add bscan instruction mbist_instr -reg mbist_reg
11
12 /*** Create the register interface
13 set external_register interface mbist_reg -capture tst_done_mgc_1
   fail_h_mgc_1 tst_done_mgc_2 fail_h_mgc_2 -update test_h_mgc_1
   test_h_mgc_2
14
15 /*** Connecting controller resets to BSDA reset
16 add port connection rst_l_mgc_1 "mbist_reset NAND updateir"
17 add port connection rst_l_mgc_2 "mbist_reset NAND updateir"
18
19 /*** Connecting Bist clock
20 add port connection bist_clk_mgc_1 "buf tck"
21 add port connection bist_clk_mgc_2 "buf tck"
22
23 /*** Specify non-top ports
24 add nontop port test_h_mgc_1
25 add nontop port repair_data_out_mgc_1
26 add nontop port repair_data_clk_mgc_1
27 add nontop port tst_done_mgc_1
28 add nontop port fail_h_mgc_1
29 add nontop port top_repair_scan_in
30 add nontop port test_h_mgc_2
31 add nontop port repair_data_out_mgc_2
32 add nontop port repair_data_clk_mgc_2
33 add nontop port tst_done_mgc_2
34 add nontop port fail_h_mgc_2
35
36 add core register repair_reg_1 top_repair_scan_in repair_data_out_mgc_1
   -len 7
37 add bscan instruction repair_instr_1 -reg repair_reg_1
38 add port connection repair_data_clk_mgc_1 "tck AND repair_instr_1"
39
40 add core register repair_reg_2 top_repair_scan_in repair_data_out_mgc_2
   -len 4
41 add bscan instruction repair_instr_2 -reg repair_reg_2
42 add port connection repair_data_clk_mgc_2 "tck AND repair_instr_2"
43
44 /*** WARNING
45 /*** The Cycles in the following "set testbench parameters" commands
46 /*** are based on MBISTA tool best estimates. User may need to modify
47 /*** the number of cycles if necessary
48
49 set testbench parameters -instr mbist_instr -test_event mbist_1
   -shift_in 10 -shift_out 10xx -wait_in_run_test_idle_state 135
50
51 set testbench parameters -instr repair_instr_1 -shift_in x -shift_out x
   -wait_in_run_test_idle_state 7
52
53 set testbench parameters -instr mbist_instr -test_event mbist_2
   -shift_in 01 -shift_out xx10 -wait_in_run_test_idle_state 534
54
55 set testbench parameters -instr repair_instr_2 -shift_in x -shift_out x
   -wait_in_run_test_idle_state 4
56
```

```
57  /*** Run, save and quit
58  run
59  save bscan -replace
60  save patterns jtag_patterns.wgl -format wgl -replace
61  exit -force
```

This dofile has the following components. The hardware and connections created by the dofile are illustrated in [Figure C-6](#) with the components specific to repair highlighted in blue.

- **Setup Reset Control** — This is done the same as the [Simple Configuration](#), which creates a reset instruction and connects that signal to the reset of each controller. See lines [6](#), [16](#), and [17](#).
- **Setup BIST Interface** — The creation and connection of the *mbist_reg* register is the same as the simple configuration example. See lines [9](#) and [13](#).
- **Place Ports** — Just like the simple configuration, a series of Add Nontop Port commands prevents the internal ports from being connected at the top level with boundary scan cells. See lines [24](#) - [37](#).
- **Setup Repair Interface** — The following is performed for each controller:
 - a. Identify the BISA report register as a core register. Notice that same input is specified for both registers. This is because the dummy input pins were shared during BIST insertion with the pin name *top_repair_scan_in*. See lines [36](#) and [40](#).
 - b. Define an instruction for shifting out the BISA report. See lines [37](#) and [41](#).
 - c. Connect the repair clock signal by ANDing TCK with the corresponding repair instruction signal. See lines [38](#) and [42](#).
- **Connect Clocks** — The BIST clock is connected to the boundary scan clock TCK through a buffer, just as the simple configuration.
- **Define Test Behavior** — The [Set Testbench Parameters](#) commands define the behavior for each of the controllers. The test behavior for this design includes the following for each controller:
 - a. Reset the BIST controller and assert *test_h* to begin BIST. Check the values of *tst_done* and *fail_h* when BIST completes. See lines [49](#) and [53](#).
 - b. Load the *repair_instr* instruction, to connect the BISA register between TDI and TDO, and shift out the contents of the BISA report. The size of the repair register equals the sum of all repair data registers defined in the BISA container module. See lines [51](#) and [55](#).

Figure C-6. Boundary Scan Configuration with Repair

Online Algorithm Selection

Online algorithm selection is a feature that allows you to select individual algorithms from the programmed set during runtime. The selection occurs by loading a vector with each bit corresponding to an algorithm. This vector can be loaded either serially or in parallel into the selection register. This example shows both configurations as the BSDArchitect implementation for each is quite different. The default configuration is serial loading.

Serial Load Algorithm Selection

The serial load configuration example has two controllers tested sequentially. The first controller has 3 programmed algorithms and the second controller has 4 programmed algorithms.

Generating TAP Compliant Hardware

The following dofile is for a design with serial load online algorithm selection. The commands specific to algorithm selection are highlighted. However, the set memory clock command is optional. This command sets up a separate clock signal for shifting in the algorithm selection vector.

```
//-----  
// Generation Dofile  
//-----  
add memory model ram4x4  
add mbibst algorithms 1 march2 march1 march3  
set bist insertion -on  
set alg selection -on  
set memory clock -algsel algsel_clk  
setup file naming -ctdl mbist1.ctdl \  
-bist mbist1.v\  
-conn mbist1_conn.v\  
-test mbist1_tb.v\  
-script mbist1_dcscript \  
-wgl mbist1.wgl  
run  
report mbist algorithms  
report algorithm steps  
save bist -verilog -replace -script  
exit
```

Creating the BSDArchitect Dofile

The insertion dofile used for this example is the same as the simple configuration example. The insertion dofile must include the [Save Driver Files](#) command to create the BSDArchitect dofile.

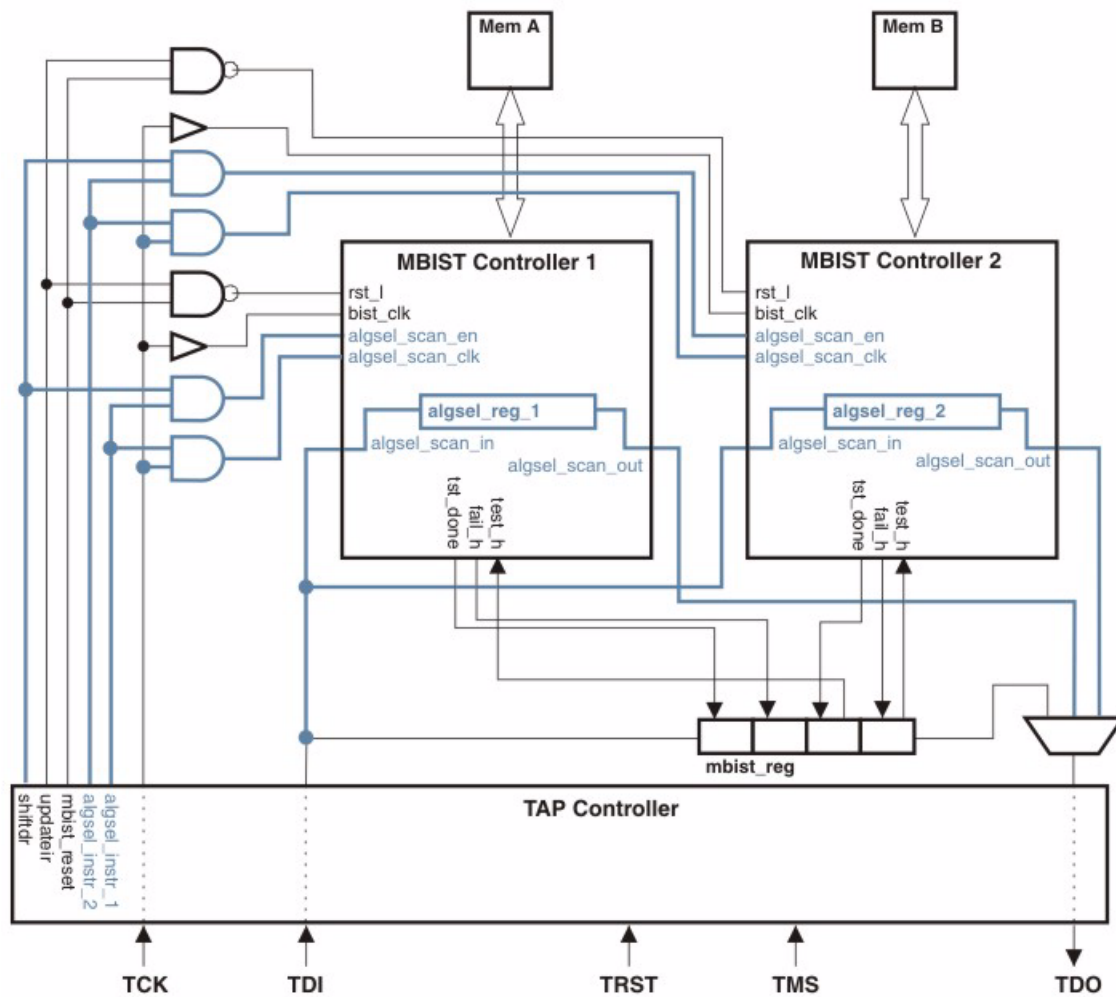
The following dofile was created by MBISTArchitect for the simple configuration with serial load online algorithm selection. The components of this dofile are discussed below.

```
1 // -----  
2 // MGC BSDA Dofile  
3 // -----  
4  
5 /*** Create Reset register to reset before the start of mbist operation  
6 add bscan instruction mbist_reset -reg BSCAN_BYPASS_REG  
7  
8 /*** Create mbist register and target an instruction for the mbist  
operations  
9 add external register mbist_reg 4  
10 add bscan instruction mbist_instr -reg mbist_reg  
11  
12 /*** Create the register interface  
13 set external_register interface mbist_reg -capture tst_done_mgc_1  
fail_h_mgc_1 tst_done_mgc_2 fail_h_mgc_2 -update test_h_mgc_1  
test_h_mgc_2  
14  
15 /*** Connecting controller resets to BSDA reset
```

```
16 add port connection rst_l_mgc_1 "mbist_reset NAND updateir"
17 add port connection rst_l_mgc_2 "mbist_reset NAND updateir"
18
19 /*** Connecting Bist clock
20 add port connection bist_clk_mgc_1 "buf tck"
21 add port connection bist_clk_mgc_2 "buf tck"
22
23 /*** Specify non-top ports
24 add nontop port algsel_scan_en_mgc_1
25 add nontop port test_h_mgc_1
26 add nontop port algsel_scan_out_mgc_1
27 add nontop port algsel_scan_in_mgc_1
28 add nontop port algsel_scan_clk_mgc_1
29 add nontop port tst_done_mgc_1
30 add nontop port fail_h_mgc_1
31 add nontop port algsel_scan_en_mgc_2
32 add nontop port test_h_mgc_2
33 add nontop port algsel_scan_out_mgc_2
34 add nontop port algsel_scan_in_mgc_2
35 add nontop port algsel_scan_clk_mgc_2
36 add nontop port tst_done_mgc_2
37 add nontop port fail_h_mgc_2
38
39 add core register algsel_reg_1 algsel_scan_in_mgc_1
   algsel_scan_out_mgc_1 -len 3
40 add bscan instruction algsel_instr_1 -reg algsel_reg_1
41 add port connection algsel_scan_clk_mgc_1 "tck AND algsel_instr_1"
42 add port connection algsel_scan_en_mgc_1 "shiftdr AND algsel_instr_1"
43
44 add core register algsel_reg_2 algsel_scan_in_mgc_2
   algsel_scan_out_mgc_2 -len 4
45 add bscan instruction algsel_instr_2 -reg algsel_reg_2
46 add port connection algsel_scan_clk_mgc_2 "tck AND algsel_instr_2"
47 add port connection algsel_scan_en_mgc_2 "shiftdr AND algsel_instr_2"
48
49 /*** WARNING
50 /*** The Cycles in the following "set testbench parameters" commands
51 /*** are based on MBISTA tool best estimates. User may need to modify
52 /*** the number of cycles if necessary
53
54 set testbench parameters -instr algsel_instr_1 -shift_in 111 -shift_out
   111 -wait_in_run_test_idle_state 3
55
56 set testbench parameters -instr mbist_instr -test_event mbist_1
   -shift_in 10 -shift_out 10xx -wait_in_run_test_idle_state 333
57
58 set testbench parameters -instr algsel_instr_2 -shift_in 1111 -shift_out
   1111 -wait_in_run_test_idle_state 4
59
60 set testbench parameters -instr mbist_instr -test_event mbist_2
   -shift_in 01 -shift_out xx10 -wait_in_run_test_idle_state 926
61
62 /*** Run, save and quit
63 run
64 save bscan -replace
65 save patterns jtag_patterns.wgl -format wgl -replace
66 exit -force
```

This dofile has the following components. The hardware and connections created by the dofile are illustrated in [Figure C-7](#) with the components specific to serial online algorithm selection highlighted in blue.

- **Setup Reset Control** — This is done exactly the same as the [Simple Configuration](#), which creates a reset instruction and connects that signal to the reset of each controller. See lines [6](#), [16](#), and [17](#).
- **Setup BIST Interface** — The creation and connection of the *mbist_reg* register is the same as the simple configuration example. See lines [9](#), [10](#), and [13](#).
- **Place Ports** — Just like the simple configuration, a series of Add Nontop Port commands prevents the internal ports from being connected at the top level with boundary scan cells. See lines [24](#) - [36](#).
- **Setup Online Algorithm Selection Interface** — The following is created for each controller:
 - a. Identify the algorithm selection register as a core register. See lines [39](#) and [44](#).
 - b. Define an instruction for shifting in the algorithm selection vector. See lines [40](#) and [45](#).
 - c. Connect the algsel instruction to the corresponding scan enable port. See lines [42](#) and [47](#).
 - d. Connect the algorithm selection clock. See lines [41](#) and [46](#).
- **Connect Clocks** — The BIST clock is connected to the boundary scan clock TCK through a buffer, just as the simple configuration.
- **Define Test Behavior** — The [Set Testbench Parameters](#) commands define the behavior for each of the controllers. The test behavior for this design includes the following for each controller:
 - a. Reset the BIST controller and load the algorithm selection instruction and shift in the algorithm selection vector. The vector specified here selects all of the algorithms. See lines [54](#) and [58](#).
 - b. Assert *test_h* to start BIST with the selected algorithms. Check the values of *tst_done* and *fail_h* when BIST completes. See lines [56](#) and [60](#).

Figure C-7. Boundary Scan Configuration with Serial Algorithm Selection

Parallel Load Algorithm Selection

The parallel load configuration example has two controllers tested sequentially. The first controller has 3 programmed algorithms and the second controller has 4 programmed algorithms.

Generating TAP Compliant Hardware

The following dofile is for a design with parallel load online algorithm selection. Notice that the set memory clock -algsel command is not included. This is because shifting is not needed for a parallel load algorithm selection register.

```
//-----
// Generation Dofile
//-----
add memory model ram4x4
```

```
add mbibst algorithms 1 march2 march1 march3
set bist insertion -on
set alg selection -on -parallel
setup file naming -ctdl mbist1.ctdl \
  -bist mbist1.v\
  -conn mbist1_conn.v\
  -test mbist1_tb.v\
  -script mbist1_dcscript \
  -wgl mbist1.wgl
run
report mbist algorithms
report algorithm steps
save bist -verilog -replace -script
exit
```

Creating the BSDArchitect Dofile

The insertion dofile used for this example is the same as the simple configuration example. The insertion dofile must include the [Save Driver Files](#) command to create the BSDArchitect dofile.

The following dofile was created by MBISTArchitect for the simple configuration with parallel load online algorithm selection. The components of this dofile are discussed below.

```
1  // -----
2  // MGC BSDA Dofile
3  // -----
4
5  /*** Create Reset register to reset before the start of mbist operation
6  add bscan instruction mbist_reset -reg BSCAN_BYPASS_REG
7
8  /*** Create mbist register and target an instruction for the mbist
   operations
9  add external register mbist_reg 9
10 add bscan instruction mbist_instr -reg mbist_reg
11
12 /*** Create the register interface
13 set external_register interface mbist_reg -capture tst_done_mgc_1
   fail_h_mgc_1 tst_done_mgc_2 fail_h_mgc_2 -update test_h_mgc_1
   test_h_mgc_2 algsel_scan_in_mgc_1[2] algsel_scan_in_mgc_1[1]
   algsel_scan_in_mgc_1[0] algsel_scan_in_mgc_2[3] algsel_scan_in_mgc_2[2]
   algsel_scan_in_mgc_2[1] algsel_scan_in_mgc_2[0]
14
15 /*** Connecting controller resets to BSDA reset
16 add port connection rst_l_mgc_1 "mbist_reset NAND updateir"
17 add port connection rst_l_mgc_2 "mbist_reset NAND updateir"
18
19 /*** Connecting Bist clock
20 add port connection bist_clk_mgc_1 "buf tck"
21 add port connection bist_clk_mgc_2 "buf tck"
22
23 /*** Specify non-top ports
24 add nontop port test_h_mgc_1
25 add nontop port algsel_scan_in_mgc_1[2]
26 add nontop port algsel_scan_in_mgc_1[1]
27 add nontop port algsel_scan_in_mgc_1[0]
```

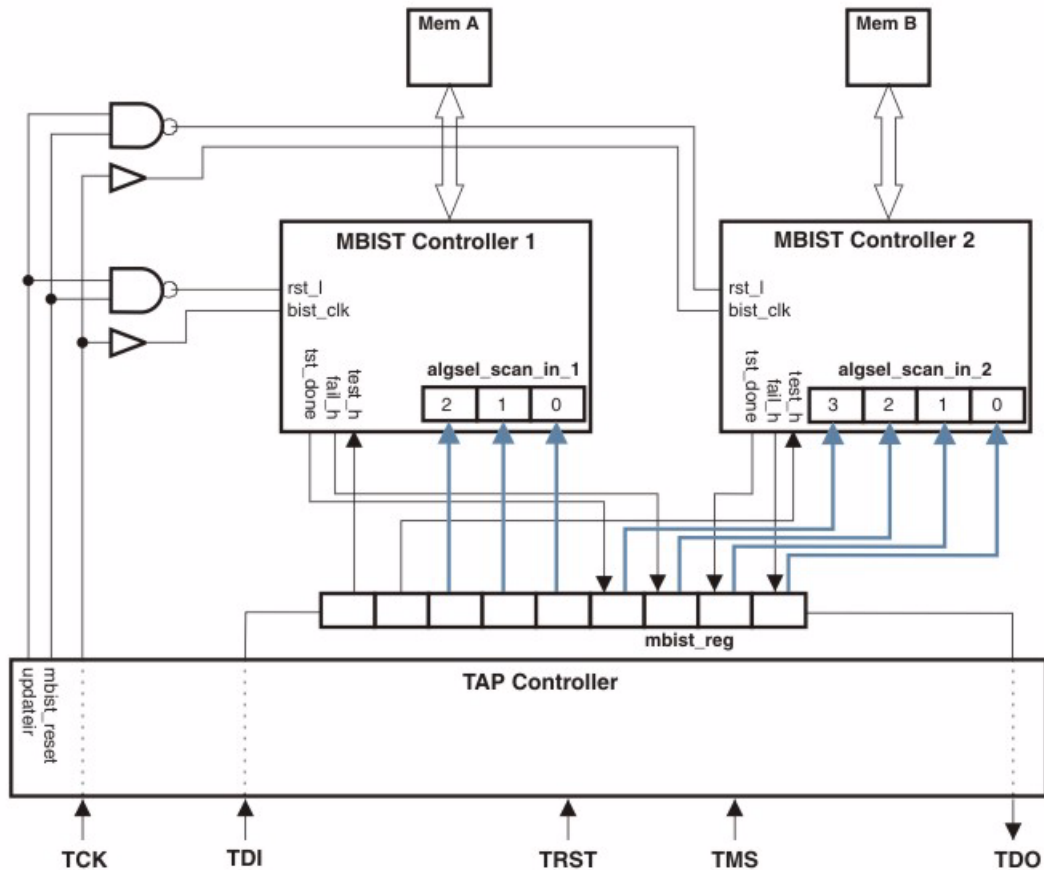
```
28 add nontop port tst_done_mgc_1
29 add nontop port fail_h_mgc_1
30 add nontop port test_h_mgc_2
31 add nontop port algsel_scan_in_mgc_2[3]
32 add nontop port algsel_scan_in_mgc_2[2]
33 add nontop port algsel_scan_in_mgc_2[1]
34 add nontop port algsel_scan_in_mgc_2[0]
35 add nontop port tst_done_mgc_2
36 add nontop port fail_h_mgc_2
37
38 /*** WARNING
39 /*** The Cycles in the following "set testbench parameters" commands
40 /*** are based on MBISTA tool best estimates. User may need to modify
41 /*** the number of cycles if necessary
42
43 set testbench parameters -instr mbist_instr -test_event mbist_1 -
  shift_in 101110000 -shift_out 10xx -wait_in_run_test_idle_state 333
44
45 set testbench parameters -instr mbist_instr -test_event mbist_2 -
  shift_in 010001111 -shift_out xx10 -wait_in_run_test_idle_state 926
46
47 /*** Run, save and quit
48 run
49 save bscan -replace
50 save patterns jtag_patterns.wgl -format wgl -replace
51 exit -force
```

This dofile has the following components. The hardware and connections created by the dofile are illustrated in [Figure C-8](#) with the components specific to parallel online algorithm selection highlighted in blue.

- **Setup Reset Control** — This is done exactly the same as the [Simple Configuration](#), which creates an instruction and connects that signal to the reset of each controller. See lines [6](#), [16](#), and [17](#).
- **Setup BIST Interface** — The creation of the *mbist_reg* register is similar to the simple configuration example, but the size and connections are different. This register is used for both the BIST interface and the algorithm selection interface. Each bit of the algorithm selection registers corresponds to a bit in the *mbist_reg* register. See lines [9](#), [10](#), and [13](#).
- **Place Ports** — Just like the simple configuration, a series of Add Nontop Port commands prevents the internal ports from being connected at the top level with boundary scan cells. See lines [24](#) - [36](#).
- **Setup Online Algorithm Selection Interface** — The online algorithm selection interface is combined with the *mbist_reg* BIST interface. No additional instructions or register declarations are required.
- **Connect Clocks** — The BIST clock is connected to the boundary scan clock TCK through a buffer, just as the simple configuration.

- **Define Test Behavior** — The [Set Testbench Parameters](#) commands define the behavior for each of the controllers. The test behavior for this design includes the following for each controller:
 - a. Reset the BIST controller and assert *test_h* to start BIST, simultaneously selecting algorithms by parallel loading the algorithm selection vector from *mbist_reg*. Check the values of *tst_done* and *fail_h* when BIST completes. See lines [43](#) and [45](#).

Figure C-8. Boundary Scan Configuration with Parallel Algorithm Selection



Appendix D

Pre-Defined Algorithm File Contents

The following code is contained in the pre-defined algorithm file. For more information on the syntax used in this file, see “[User-Defined Algorithm Language](#)” on page 144.

```
// Copyright (C) Mentor Graphics Corporation 1999 All Rights Reserved
//
// Standard Memory Bist algorithms
//
// This file contains the standard set of algorithms loaded when
// the memory bist kernel is initialised.
//

//*****
//
// march1
//
// Summary:
//   Optimised version of standard MarchC algorithm, redundant read 0
//   removed from the middle of the algorithm.
//
// Size:
//   10n
//
// Algorithm:
//   up   - write 0
//   up   - read 0, write 1
//   up   - read 1, write 0
//   down - read 0, write 1
//   down - read 1, write 0
//   down - read 0
//

step wSeedUp;
  addr min, max, up, 1;
  data seed;
  operation w;

step rwInvSeedUp;
  addr min, max, up, 1;
  data invSeed;
  operation rw;

step rwSeedUp;
  addr min, max, up, 1;
  data seed;
  operation rw;

step rwInvSeedDown;
  addr min, max, down, 1;
  data invSeed;
```

```
        operation rw;

step rwSeedDown;
    addr min, max, down, 1;
    data seed;
    operation rw;

step rSeedDown;
    addr min, max, down, 1;
    data seed;
    operation r;

repetition march1;
    seed 0;
begin
    step wSeedUp;
    step rwInvSeedUp;
    step rwSeedUp;
    step rwInvSeedDown;
    step rwSeedDown;
    step rSeedDown;
end

test march1;
    repetition march1;

//*****
//
// march2
//
// Summary:
//     Modified version of standard MarchC algorithm.
//     Data can come from seed (0) or user specified run time
//     data background value.
//
// Size:
//     14n
//
// Algorithm:
//     up    - write 0
//     up    - read 0, write 1, read 1
//     up    - read 1, write 0, read 0
//     down  - read 0, write 1, read 1
//     down  - read 1, write 0, read 0
//     down  - read 0
//
step wBackgroundUp;
    addr min, max, up, 1;
    data background;
    operation w;

step rwrInvBackgroundUp;
    addr min, max, up, 1;
    data invBackground;
    operation rwr;

step rwrBackgroundUp;
```

```

    addr min, max, up, 1;
    data background;
    operation rwr;

step rwrInvBackgroundDown;
    addr min, max, down, 1;
    data invBackground;
    operation rwr;

step rwrBackgroundDown;
    addr min, max, down, 1;
    data background;
    operation rwr;

step rBackgroundDown;
    addr min, max, down, 1;
    data background;
    operation r;

repetition march2;
    seed 0;
begin
    step wBackgroundUp;
    step rwrInvBackgroundUp;
    step rwrBackgroundUp;
    step rwrInvBackgroundDown;
    step rwrBackgroundDown;
    step rBackgroundDown;
end

test march2;
    repetition march2;

//*****
//
// march3
//
// Summary:
//   Modified version of standard March2 algorithm, final two steps
//   are removed. Data can come from seed (0) or user specified run
//   time data background value.
//
// Size:
//   10n
//
// Algorithm:
//   up   - write 0
//   up   - read 0, write 1, read 1
//   up   - read 1, write 0, read 0
//   down - read 0, write 1, read 1
//

// uses step wBackgroundUp
// uses step rwrInvBackgroundUp
// uses step rwrBackgroundUp
// uses step rwrInvBackgroundDown

repetition march3;

```

```
    seed 0;
begin
    step wBackgroundUp;
    step rwrInvBackgroundUp;
    step rwrBackgroundUp;
    step rwrInvBackgroundDown;
end

test march3;
    repetition march3;

//*****
//
// col_march1
//
// Summary:
//     Same as March1, except address increment value is taken from the
//     memory models' addr_inc value.
//
// Size:
//     10n
//
// Algorithm:
//     up   - write 0
//     up   - read 0, write 1
//     up   - read 1, write 0
//     down - read 0, write 1
//     down - read 1, write 0
//     down - read 0
//
step wSeedUpCol;
    addr min, max, up, jump;
    data seed;
    operation w;

step rwInvSeedUpCol;
    addr min, max, up, jump;
    data invSeed;
    operation rw;

step rwSeedUpCol;
    addr min, max, up, jump;
    data seed;
    operation rw;

step rwInvSeedDownCol;
    addr min, max, down, jump;
    data invSeed;
    operation rw;

step rwSeedDownCol;
    addr min, max, down, jump;
    data seed;
    operation rw;

step rSeedDownCol;
    addr min, max, down, jump;
```



```

    data seed;
    operation r;

repetition col_march1;
    seed 0;
begin
    step wSeedUpCol;
    step rwInvSeedUpCol;
    step rwSeedUpCol;
    step rwInvSeedDownCol;
    step rwSeedDownCol;
    step rSeedDownCol;
end

test col_march1;
    repetition col_march1;

//*****
//
// unique
//
// Summary:
//     Simple march type algorithm, using data for each address that is
//     based on the address.
//
//     For memories where the address bus is larger than the data bus
//     the data value is computed by using addition to reduce the
//     address bus down to the data bus size; the address bus is sliced,
//     using the data bus width, and these are added together to give
//     the data value.
//
//     Conversely, for other memories, the data value is computed by
//     catenating copies of the address value enough times to fill the
//     data bus.  If necessary the most significant copy of the address
//     value will be sliced to exactly fit the data bus width.
//
// Size:
//     5n
//
// Algorithm:
//     up    - write 0
//     up    - write address unique
//     up    - read address unique
//     up    - write inverse address unique
//     up    - read inverse address unique
//
// uses step wSeedUp

step wAddrUniqUp;
    addr min, max, up, 1;
    data addr;
    operation w;

step rAddrUniqUp;
    addr min, max, up, 1;
    data addr;

```

```
        operation r;

step wInvAddrUniqUp;
    addr min, max, up, 1;
    data invAddr;
    operation w;

step rInvAddrUniqUp;
    addr min, max, up, 1;
    data invAddr;
    operation r;

repetition unique;
    seed 0;
begin
    step wSeedUp;
    step wAddrUniqUp;
    step rAddrUniqUp;
    step wInvAddrUniqUp;
    step rInvAddrUniqUp;
end

test unique;
    repetition unique;

//*****
//
// checkerBoard
//
// Summary:
//     Simple march type algorithm, using checker board data pattern.
//     Checker board data value takes into account the topology of the
//     memory, as described in the memory's model (top_column and
//     top_word data values).
//
//     N.B. In previous versions of MBISTArchitect this algorithm wasn't
//     sensitive to the memory topology.
//
// Size:
//     4n
//
// Algorithm:
//     up    - write checker board
//     up    - read checker board
//     up    - write inverse checker board
//     up    - read inverse checker board
//
step wCheckerBoardUp;
    data checkerboard;
    addr min, max, up, 1;
    operation w;

step rCheckerBoardUp;
    data checkerboard;
    addr min, max, up, 1;
    operation r;
```

```

step wInvCheckerBoardUp;
  data invcheckerboard;
  addr min, max, up, 1;
  operation w;

step rInvCheckerBoardUp;
  data invcheckerboard;
  addr min, max, up, 1;
  operation r;

repetition checkerBoard;
begin
  step wCheckerBoardUp;
  step rCheckerBoardUp;
  step wInvCheckerBoardUp;
  step rInvCheckerBoardUp;
end

test checkerBoard;
  repetition checkerBoard;

//*****
//
// retentionCB
//
// Summary:
//   Similar to checherboard, but with synchronization at the end of
//   the write states
//
// Size:
//   4n
//
// Algorithm:
//   up - write checker board, synchronize
//   up - read checker board
//   up - write inverse checker board, synchronize
//   up - read inverse checker board
//

step wCheckerBoardUpSynchronize;
  synchronize;
  data checkerboard;
  addr min, max, up, 1;
  operation w;

step wInvCheckerBoardUpSynchronize;
  synchronize;
  data invcheckerboard;
  addr min, max, up, 1;
  operation w;

repetition retentionCB;
begin
  step wCheckerBoardUpSynchronize;
  step rCheckerBoardUp;
  step wInvCheckerBoardUpSynchronize;
  step rInvCheckerBoardUp;
end

```

```
test retentionCB;
    repetition retentionCB;

//*****
//
// topChecker
//
// Summary:
//     Since the checkerBoard algorithm is now sensitive to the memory
//     topology this algorithm is just a synonym for checkerBoard.
//

test topChecker;
    repetition checkerBoard;

//*****
//
// Rom1
//
// Summary:
//     Reads data from all memory locations.
//
//     N.B. Requires MISR to compressor data values.
//
// Size:
//     1n
//
// Algorithm:
//     up    - read
//

step rRomUp;
    data seed;
    addr min, max, up, 1;
    operation r;

repetition rom;
    seed 0;
    step rRomUp;

test rom1;
    compress;
    repetition rom;

// retain old definition of ROM for backwards compatibility
test rom;
    compress;
    repetition rom;

//*****
//
// Rom2
//
// Summary:
//     Improved ROM test algorithm (see "Design-tor-test for digital
```

```

//    IC's and embeded core systems", Alfred L. Crouch, Prentice Hall,
1999
//    page 237).
//
//    Read and compress up then down and up the address space
//
//    N.B. Requires MISR to compressor data values.
//
// Size:
//    3n
//
// Algorithm:
//    up    - read
//    down  - read
//    up    - read
//
step rRomDown;
    data seed;
    addr min, max, down, 1;
    operation r;

// redeclare this, because currently we don't support
// multiple instances of the same step in repetition
step rRomUpAgain;
    data seed;
    addr min, max, up, 1;
    operation r;

repetition rom2;
    seed 0;
begin
    step rRomUp;
    step rRomDown;
    step rRomUpAgain;
end

test rom2;
    compress;
    repetition rom2;

//*****
//
// AddressDecoder
//
// Summary:
//    Standard Address Decoder Algorithm.
//    The algorithm is based on writing a value to a test address
//    and checking if the base address value changes.
//
//    There are two standard algorithm supported.
//    They are same algorithm with two different backgrounds.
//
//    Algorithm with background 0 : addressdecoder_bg0
//    Algorithm with background 1 : addressdecoder_bg1
//
// Size:
//    n+ 2n(1+logn)  *log base is 2*

```

```
//
// Algorithm:
//   addressdecoder_bg0:
//     up - write 0
//     up - write 1, shift_write 0, read 1, write 0
//
//   addressdecoder_bg1:
//     up - write 1
//     up - write 0, shift_write 1, read 0, write 1
//

step wwrwAddressDecoder;
  data invseed;
  addr min,max,up,1;
  operation w(s_wr)w;

repetition addressdecoder_bg0;
  seed 0;
begin
  step wSeedUp;
  step wwrwAddressDecoder;
end

repetition addressdecoder_bg1;
  seed 1;
begin
  step wSeedUp;
  step wwrwAddressDecoder;
end

test addressdecoder_bg0;
begin
  repetition addressdecoder_bg0;
end

test addressdecoder_bg1;
begin
  repetition addressdecoder_bg1;
end
```

— Symbols —**'include**importance on location, [40](#)**— A —**ABF faults, [112](#)Adding output logic, [189](#)addr, [198](#)

Address

multi-bit, [52](#)

Address descrambling

address subsection, [62](#)definition, [60](#)definition example, [63](#)deriving the address information, [63](#)syntax example, [62](#)

Address scrambling

address subsection, [62](#)syntax example, [62](#)Address sequences example, [151](#)

Addresses

addr, [198](#)

Addressing

fast column addressing (address scrambling), [76](#)fast row addressing (address incrementation), [76](#)

Algorithm clock cycles

reporting, [142](#)Algorithm selection register, [137](#)

Algorithms

Checkerboard, [122](#)Col_March 1, [119](#)fault types, [111](#)March 2, [116](#)March C+, [116](#)online algorithm selection, [135](#)Port Interaction Test, [126](#)port interaction test, [126](#)port isolation testing, [128](#)port isolation testing for UDA, [170](#)RetentionCB, [124](#)ROM Test, [124](#), [125](#)TopChecker, [122](#)Unique Address, [120](#)user-defined algorithm, [143](#)write enable mask, [164](#)Array notation, [53](#)Assert event example, [67](#)Assert state, [54](#), [55](#)Assert statement, [66](#)fix modifier, [67](#)fix modifier for optimization example, [74](#)

Asynchronous memory

no memory clock, [181](#)

At-speed

BIST operation, [243](#)pipeline testing, [243](#)

Attribute

dont_touch, [54](#)dont_touch array notation example, [55](#)dont_touch ports example, [55](#)**— B —**

Background

values, [149](#)BF faults, [112](#)

Bidi ports

sharing, [190](#)

Bidirectional buses

control of, [54](#)

Bidirectional ports

sharing, [190](#)

BISA

activating, [210](#)bits mode reporting, [213](#)block diagram, [209](#)Built-In Self-repair Analysis, [207](#)column errors, [213](#)

- column index, [215](#)
- column repair, [211](#)
- column vector, [213](#), [215](#)
- combined report, [216](#)
- index mode reporting, [213](#)
- NR memories, [208](#)
- report block, [215](#)
- report example, [216](#)
- row repair, [213](#)
- RR memories, [208](#)
- timing diagram, [220](#)
- unit report, [214](#)
- yield improvement, [207](#)
- BIST
 - algorithms, [111](#)
 - connection file, [42](#)
 - controller model file, [40](#)
 - coupling faults, [112](#)
 - diagnostic scheme, [193](#)
 - generating, [20](#)
 - generation and insertion, [20](#)
 - inserting, [20](#)
 - stuck-at faults, [113](#)
 - testing of output enables, [76](#)
 - transition faults, [114](#)
- BIST controller
 - configuration format, [41](#)
 - diagnostic capabilities, [197](#)
 - functions, [239](#)
- BIST insertion, [20](#)
 - BIST mode, [21](#)
 - Integration mode, [21](#)
 - Setup mode, [21](#)
- BIST scheme
 - parallel, [135](#)
 - sequential, [135](#)
- bist_definition example, [71](#)
- BIST-ready memory support, [76](#)
- Bits mode reporting, [213](#)
- Block-based tool flow, [32](#)
- Bottom-up tool flow, [28](#)
- Bridge coupling faults, [112](#)
- Bypass-ready memories, [83](#)
- C —
- CFid faults, [112](#)
- CFin faults, [112](#)
- Change statement, [66](#)
- Checkerboard algorithm, [122](#)
- Chip enable signal, [91](#)
- chip_enable
 - toggling, [67](#)
- clk, [198](#)
- Clock
 - clk, [198](#)
 - diagnostic clock control, [193](#)
- Clock domains
 - for the diagnostic process, [200](#)
- Clock synchronization, [240](#)
 - between MBIST and diagnostic clocks, [200](#)
- Col_March 1 algorithm, [119](#)
- Collar pin sharing, [188](#)
- Collar specify block
 - with specparam, [109](#)
- Column errors
 - BISA, [213](#)
- Column index
 - BISA, [215](#)
- Column repair
 - BISA, [211](#)
- Column vector
 - BISA, [213](#), [215](#)
- Combined report
 - BISA, [216](#)
- Commands
 - mbistarchitect, [16](#)
 - reset state, [18](#)
 - user-defined algorithm, [143](#)
- Compile script
 - example netlist, [28](#)
 - example test bench, [28](#)
- Configuration file
 - creating, [45](#)
- Controller
 - available data items, [195](#)
 - controller test procedure, [228](#)
 - declaration block, [225](#)
 - declaration in CTDF, [225](#)
 - instance definition, [233](#)
 - pin sharing, [188](#)
- Controller access

- timeplate definition, [234](#)
 - Controller instance declaration
 - map statement, [233](#)
 - Controller interface signals
 - for online algorithm selection, [140](#)
 - Controller mapping file
 - creating, [45](#)
 - Controller test access, [223](#)
 - Controller test access file, [232](#)
 - example, [232](#)
 - Controller test description, [223](#)
 - Controller test description file, [42](#), [224](#)
 - Controller test description language, [223](#)
 - syntax, [223](#)
 - Controller timeplates, [237](#)
 - legal names, [237](#)
 - Controller_access procedure
 - CTAF, [235](#)
 - syntax, [235](#)
 - timeplate statement, [235](#)
 - controller_instance statement, [233](#)
 - controller_statement, [225](#)
 - controller_test procedure, [228](#)
 - Coupling faults, [112](#)
 - AND bridging faults, [112](#)
 - bridge, [112](#)
 - idempotent, [112](#)
 - inversion, [112](#)
 - OR bridging faults, [112](#)
 - state, [113](#)
 - testing for, [112](#)
 - cs signal, [91](#)
 - CTAF
 - controller access timeplate, [234](#)
 - controller instances, [233](#)
 - controller_access procedure
 - definition, [235](#)
 - example, [232](#)
 - mapping controller and SoC pins, [233](#)
 - mapping timeplates, [234](#)
 - CTDF
 - controller declaration, [225](#)
 - controller test procedure, [228](#)
 - description, [224](#)
 - example file, [224](#)
 - pattern file format, [229](#)
 - rule checking, [285](#)
 - time scale example, [231](#)
 - time scale syntax, [231](#)
 - time scale, changing, [230](#)
 - timeplates, [226](#)
 - CTDL
 - controller test access file, [232](#)
 - controller test description file, [224](#)
 - description, [223](#)
 - syntax, [223](#)
 - Cycle definitions, [65](#)
 - minimum cycle requirement, [67](#)
 - read_cycle, [65](#)
 - write_cycle, [65](#)
 - cycle_statement, [229](#)
- D —
- Data background values, [149](#)
 - Data buses
 - multi-bit, [52](#)
 - Data descrambling definition, [60](#)
 - example, [63](#)
 - Data input signal, [92](#)
 - Data inputs
 - di, [198](#)
 - Data latency, [240](#)
 - Data output signal, [91](#)
 - Data scrambling
 - reasons for, [59](#)
 - Data seed, [149](#)
 - debugz, [198](#)
 - Default CTDL time scale
 - changing, [230](#)
 - Descrambling
 - address definition, [60](#)
 - address definition example, [63](#)
 - data definition, [60](#)
 - data definition example, [63](#)
 - descrambling_definition, [61](#)
 - Design file, [37](#)
 - Design rules, [285](#)
 - checking, [285](#)
 - CTDF rule checking, [285](#)
 - I rules, [286](#)
 - Integration rule checking, [286](#)

PAD rule checking, 289
 Determining test time, 142
 di, 198
 Diagnostic behavior
 of MBISTArchitect, 200
 Diagnostic block
 interface, 193
 Diagnostic clock
 control, 193
 domains, 200
 synchronization with MBIST clock, 200
 Diagnostic data
 scanning out, 196
 Diagnostic mode
 debugz, 198
 Diagnostic output data
 example of output, 194
 Diagnostic scan out
 bit sequence, 195
 Diagnostic scheme
 BIST, 193
 Diagnostics
 algorithms, 111
 Diagnostics configuration file
 creating, 45
 Dofile, 37
 default, 22
 default contents, 22
 example of an MBIST template (dofile), 27
 example of an MBISTArchitect dofile, 27
 Dont_touch
 array notation example, 55
 attribute, 54
 ports example, 55
 DRC, 285
 Driver syntax
 for specparam, 108
 — E —
 Event statements, 65
 assert statement, 66
 change statement, 66
 Example
 collar specify block with specparam, 109
 controller test access file, 232
 CTDF, 224

dual port synchronous RAM8x2, 98
 input PAD model specparam, 107
 MBIST template (dofile), 27
 MBISTArchitect dofile, 27
 memory modeling, 85
 multiple control enables and dont_touch,
 101
 output PAD model specparam, 107
 RAM with vector write-enable signals, 103
 RAM4x8, 86
 run simulation script, 28
 single port synchronous RAM8x2, 93
 test bench and netlist compile script, 28
 Verilog memory model specparam, 105
 Expect event example, 68
 Expect statement
 move modifier, 67

— F —

Fail flag
 fail_h, 197
 fail_h, 197
 Fail_h behavior, 204
 Fast column addressing (address scrambling),
 76
 Fast row addressing (address incrementation),
 76
 Fault types, 111
 Faults
 coupling faults, 112
 stuck-at, 113
 transition faults, 114
 Faults detected, 111
 Port Interaction Test algorithm, 126
 Unique Address algorithm, 120
 Fix modifier, 67
 for assert statement example, 74
 Fix modifier uses, 75
 Full-speed design with pipeline, 243

— G —

Generating BIST, 20

— H —

Hardware impact
 due to online algorithm selection, 136

HDL BIST

- connection file, [42](#)
- controller model file, [40](#)

Hold

- hold_l, [198](#)

— I —

I rules checking, [286](#)

Idempotent coupling faults, [112](#)

Include

- importance of location, [40](#)

Index mode reporting, [213](#)

Input PAD model specparam

- example, [107](#)

Inputs

- addr, [198](#)
- clk, [198](#)
- debugz, [198](#)
- di, [198](#)
- hold_l, [198](#)
- rst, [198](#)
- test_h, [198](#)
- wen, [198](#)

Inputs to MBISTArchitect, [37](#)

- design file, [37](#)
- dofile, [37](#)
- library file, [37](#)

Inserting BIST, [20](#)

Instances controller declaration, [233](#)

Integration rule checking, [286](#)

Inversion coupling faults, [112](#)

Invoking MBISTArchitect, [16](#), [17](#)

— L —

Legal characters

- for memory modeling, [49](#)

Library file, [37](#)

list_of_pins, [50](#)

— M —

Map statement

- for controller instance declaration, [233](#)

map_timeplate statement, [237](#)

Mapping

- controller and SoC pins, [233](#)
- timeplates, [234](#), [237](#)

- timeplates, automatic, [238](#)

Mapping file

- creating, [45](#)

March 2 algorithm, [116](#)

March C+ algorithm, [116](#)

March1 algorithm, [115](#)

MBIST

- example template (dofile), [27](#)

MBIST clock

- synchronization with diagnostic clock, [200](#)

MBIST controller functions, [239](#)

MBIST Full-Speed operation, [243](#)

MBISTArchitect

- algorithms, [111](#)
- diagnostic behavior, [200](#)
- example dofile, [27](#)
- exiting, [19](#)
- invoking, [17](#)
- loading libraries, [18](#)
- output file naming, [39](#)
- overview of tool, [15](#)
- quitting, [19](#)
- resetting, [18](#)
- running, [16](#), [17](#)
- specparam driver syntax, [108](#)
- usage flow, [19](#)

MBISTArchitect inputs, [37](#)

- design file, [37](#)
- dofile, [37](#)
- library file, [37](#)

MBISTArchitect outputs, [39](#)

- controller test description file, [42](#)
- HDL BIST connection file, [42](#)
- HDL BIST controller model file, [40](#)
- pattern files, [42](#)
- synthesis driver file, [44](#)

Memories, bypass-ready, [83](#)

Memory model

- template, [51](#)

Memory model example

- array notation example, [70](#)
- port and cycle example, [69](#)

Memory model naming, [49](#)

Memory model specparam, [105](#)

Memory model statement, [50](#)

Memory model syntax

- array notation, [53](#)
- bist_definition, [50](#)
- cycle definitions, [65](#)
- descrambling definition, [60](#)
- header example, [50](#)
- list_of_pins, [50](#)
- model statement, [50](#)
- model_name, [50](#)
- parameters, [57](#)
- pin declarations, [51](#)
- port definition, [64](#)
- testing output enables, [76](#)

Memory modeling

- case sensitivity, [49](#)
- issues, [49](#)
- legal characters, [49](#)
- scalar pin ordering, [49](#)
- syntax for model naming, [49](#)

Memory modeling descriptions

- basic information, [49](#)

Memory modeling examples, [85](#)

- dual port synchronous RAM8x2, [98](#)
- multiple control enables and dont_touch, [101](#)
- RAM with vector write-enable signals, [103](#)
- RAM4x8, [86](#)
- single port synchronous RAM8x2, [93](#)

Memory models

- naming, [49](#)

mgc_dft_cell_type

- Specparam, [108](#)

mgc_dft_connect

- Specparam, [108](#)

mgc_pin_type

- Specparam, [108](#)

model_name, [50](#)

Modelfile format, [38](#)

Modes

- BIST insertion, [21](#)
- switching between, [21](#)

Move modifier, [67](#), [91](#)

Multi-bit

- address, [52](#)
- data buses, [52](#)

Multiport memories

- example with online algorithm selection, [139](#)
- with online algorithm selection, [139](#)

— N —

Naming memory models, [49](#)

Negative-edge BIST controllers, [249](#)

Netlist

- example compile script, [28](#)

Non-retention/IDDQ pattern sets, [135](#)

NR memory, [208](#)

— O —

OBF faults, [112](#)

oe signal

- declaration, [90](#)

Online algorithm selection, [135](#)

- algorithm selection register, [137](#)
- controller interface signals, [140](#)
- for multiport memories, [139](#)
- hardware impact, [136](#)
- listing names of algorithms, [137](#)
- multiport memory example, [139](#)
- protocol, [135](#)
- step-by-step example, [139](#)

Operations

- read, [154](#)
- shifted read and shifted write, [157](#)
- write, [154](#)

Optimization of timeplates, [238](#)

Output enable

- signal declaration, [90](#)

Output file naming, [39](#)

Output PAD model specparam

- example, [107](#)

Outputs

- fail_h, [197](#)
- restart_h, [197](#)
- scan_out, [198](#)
- tst_done, [197](#)

Outputs from MBISTArchitect, [39](#)

- controller test description file, [42](#)
- HDL BIST connection file, [42](#)
- HDL BIST controller model file, [40](#)

pattern files, [42](#)
synthesis driver file, [44](#)

— P —

PAD rule checking, [289](#)
Parallel bist scheme, [135](#)
Pattern file format from CTDF, [229](#)
Pattern files, [42](#)
pattern_file declaration statement, [228](#)
Pin declarations, [51](#)
pin_type statement, [225](#)
Pins
 mapping controller and SoC, [233](#)
Pipeline circuitry, [244](#)
Pipelined read/write operations, [244](#)
Port definition, [64](#)
 event statements, [65](#)
Port Interaction Test
 faults detected, [126](#)
Port interaction test algorithm, [126](#)
Port isolation testing algorithm, [128](#)
Port isolation testing algorithm for UDA, [170](#)
probe statement, [229](#)
Procedure
 controller test, [228](#)
Protocol
 online algorithm selection, [135](#)

— R —

RAM model example, [69](#)
Read operations, [154](#)
Read_cycle, [65](#)
read_cycle example, [71](#)
Read/write cycle optimization example, [70](#)
repair keyword, [210](#)
repair_data, [210](#)
repair_data_clock, [209](#)
repair_data_force, [209](#)
repairable_h, [210](#)
Report block
 BISA, [215](#)
Report memory instances, [21](#)
Reset
 rst, [198](#)
Reset state command, [18](#)
Resetting MBISTArchitect, [18](#)

Restart signal
 restart_h, [197](#)
restart_h, [197](#)
Retention test, [133](#)
Retention test delay time
 controlling, [134](#)
Retention test scheme
 for multiple BIST controllers, [133](#)
Retention/IDDQ pattern sets, [135](#)
RetentionCB algorithm, [124](#)
ROM Content file
 Modelfile format, [38](#)
ROM Test algorithm
 by issuing an Add Mbist Algorithm
 command, [38](#)
 Modelfile format, [38](#)
 ROM content file, [38](#)
 rom1, [124](#)
 rom2, [125](#)
Row repair
 BISA, [213](#)
RR memory, [208](#)
rst, [198](#)
Rule checking
 CTDF, [285](#)
 I rules, [286](#)
 Integration, [286](#)
 PAD, [289](#)
Run simulation script
 example, [28](#)
Running MBISTArchitect, [16](#)

— S —

Scalar pin ordering
 for memory models, [49](#)
Scan out data
 ordering, [195](#)
 selecting, [195](#)
Scan output
 scan_out, [198](#)
scan_out, [198](#)
SCF faults, [113](#)
Scrambling
 data, [59](#)
Seed, [149](#)
Sequential bist scheme, [135](#)

- Set controller hold -on, [196](#)
- Shell commands
 - mbistarchitect, [16](#)
- Shifted read and shifted write operations, [157](#)
- Signals
 - addr, [198](#)
 - clk, [198](#)
 - controller interface for online algorithm selection, [140](#)
 - cs, [91](#)
 - debugz, [198](#)
 - di, [92](#), [198](#)
 - do, [91](#)
 - fail_h, [197](#)
 - hold_l, [198](#)
 - oe, [90](#)
 - restart_h, [197](#)
 - rst, [198](#)
 - scan_out, [198](#)
 - test_h, [198](#)
 - tst_done, [197](#)
 - wen, [198](#)
- SoC timeplates, [237](#)
- Specparam, [104](#)
 - collar specify block example, [109](#)
 - driver syntax, [108](#)
 - example for Verilog memory model, [105](#)
 - example of bidirectional ports, [108](#)
 - for memory model, [105](#)
 - mgc_dft_cell_type, [108](#)
 - mgc_dft_connect, [108](#)
 - mgc_pin_type, [108](#)
- SRAM modeling example, [95](#)
- SRAM read and write respective timing
 - example, [71](#)
- Starting the tool, [17](#)
- State coupling faults, [113](#)
- Step register variable, [136](#)
- Stuck-at faults, [113](#)
- Synchronization
 - between MBIST and diagnostic clocks, [200](#)
- Synchronous memories, [240](#)
- Syntax
 - CTDL, [223](#)
 - UDA, [144](#)
- Syntax rules
 - for memory modeling issues, [49](#)
- Synthesis driver file, [44](#)
- T —
- Test
 - test_h, [198](#)
- Test access file, [232](#)
- Test bench
 - example compile script, [28](#)
- Test done
 - tst_done, [197](#)
- Test patterns
 - timeplates for, [236](#)
- Test time
 - computation, [133](#)
 - determining, [142](#)
 - report generation, [133](#)
- test_control_signal, [192](#)
- test_h, [198](#)
- Testing
 - output enables, [76](#)
 - rom1 algorithms, [124](#)
 - rom2 algorithms, [125](#)
- Time scale
 - changing in CTDF, [230](#)
 - example, [231](#)
 - syntax, [231](#)
- Timeplate statement
 - for controller_access procedure, [235](#)
- Timeplates
 - automatic mapping, [238](#)
 - controller, [237](#)
 - legal names, [237](#)
 - controller access definition, [234](#)
 - CTDF definition, [226](#)
 - definition statement, [226](#)
 - mapping, [234](#), [237](#)
 - optimization, [238](#)
 - SoC, [237](#)
 - test patterns, [236](#)
- Timing problems
 - avoiding, [203](#)
- Tool
 - starting, [16](#)
- Tool flows, [24](#)

- block-based flow, [32](#)
- bottom-up flow, [28](#)
- top-down flow, [24](#)
- top_column, [58](#)
- top_column statements, [123](#)
- top_word, [58](#)
- top_word statements, [123](#)
- TopChecker algorithm, [122](#)
- Top-down tool flow, [24](#)
- Top-level pin mapping, [187](#)
- Transition faults, [114](#)
- tst_done, [197](#)
- tstate, [136](#)

— U —

- UDA, [143](#)
 - syntax, [144](#)
- Unique Address algorithm, [120](#)
 - faults detected, [120](#)
- Unit report
 - BISA, [214](#)
- User-defined algorithms, [143](#)
 - addressdecoder_bg0, [125](#)
 - addressdecoder_bg1, [125](#)
 - mbistarchitect commands, [143](#)
 - port isolation, [170](#)
 - write enable mask, [164](#)

— V —

- Verilog
 - example memory model specparam, [105](#)

— W —

- Wait period, [134](#)
- Wait statements, [90](#)
- wen, [198](#)
- Write cycle
 - basic timing and events, [93](#)
 - definition, [92](#)
- Write enable mask algorithm, [164](#)
- Write enables
 - wen, [198](#)
- Write operations, [154](#)
- Write_cycle, [65](#)
- write_cycle example, [71](#)
- write_enable_map

- creating, [55](#)

— Y —

- Yield improvement
 - using BISA, [207](#)

Third-Party Information

For information about third-party software included with this release of Tessent products, refer to the [*Third-Party Software for Tessent Products*](#).



End-User License Agreement

The latest version of the End-User License Agreement is available on-line at:
www.mentor.com/eula

IMPORTANT INFORMATION

USE OF ALL SOFTWARE IS SUBJECT TO LICENSE RESTRICTIONS. CAREFULLY READ THIS LICENSE AGREEMENT BEFORE USING THE PRODUCTS. USE OF SOFTWARE INDICATES CUSTOMER'S COMPLETE AND UNCONDITIONAL ACCEPTANCE OF THE TERMS AND CONDITIONS SET FORTH IN THIS AGREEMENT. ANY ADDITIONAL OR DIFFERENT PURCHASE ORDER TERMS AND CONDITIONS SHALL NOT APPLY.

END-USER LICENSE AGREEMENT ("Agreement")

This is a legal agreement concerning the use of Software (as defined in Section 2) and hardware (collectively "Products") between the company acquiring the Products ("Customer"), and the Mentor Graphics entity that issued the corresponding quotation or, if no quotation was issued, the applicable local Mentor Graphics entity ("Mentor Graphics"). Except for license agreements related to the subject matter of this license agreement which are physically signed by Customer and an authorized representative of Mentor Graphics, this Agreement and the applicable quotation contain the parties' entire understanding relating to the subject matter and supersede all prior or contemporaneous agreements. If Customer does not agree to these terms and conditions, promptly return or, in the case of Software received electronically, certify destruction of Software and all accompanying items within five days after receipt of Software and receive a full refund of any license fee paid.

1. ORDERS, FEES AND PAYMENT.

- 1.1. To the extent Customer (or if agreed by Mentor Graphics, Customer's appointed third party buying agent) places and Mentor Graphics accepts purchase orders pursuant to this Agreement (each an "Order"), each Order will constitute a contract between Customer and Mentor Graphics, which shall be governed solely and exclusively by the terms and conditions of this Agreement, any applicable addenda and the applicable quotation, whether or not those documents are referenced on the Order. Any additional or conflicting terms and conditions appearing on an Order or presented in any electronic portal or automated order management system, whether or not required to be electronically accepted, will not be effective unless agreed in writing and physically signed by an authorized representative of Customer and Mentor Graphics.
- 1.2. Amounts invoiced will be paid, in the currency specified on the applicable invoice, within 30 days from the date of such invoice. Any past due invoices will be subject to the imposition of interest charges in the amount of one and one-half percent per month or the applicable legal rate currently in effect, whichever is lower. Prices do not include freight, insurance, customs duties, taxes or other similar charges, which Mentor Graphics will state separately in the applicable invoice. Unless timely provided with a valid certificate of exemption or other evidence that items are not taxable, Mentor Graphics will invoice Customer for all applicable taxes including, but not limited to, VAT, GST, sales tax, consumption tax and service tax. Customer will make all payments free and clear of, and without reduction for, any withholding or other taxes; any such taxes imposed on payments by Customer hereunder will be Customer's sole responsibility. If Customer appoints a third party to place purchase orders and/or make payments on Customer's behalf, Customer shall be liable for payment under Orders placed by such third party in the event of default.
- 1.3. All Products are delivered FCA factory (Incoterms 2010), freight prepaid and invoiced to Customer, except Software delivered electronically, which shall be deemed delivered when made available to Customer for download. Mentor Graphics retains a security interest in all Products delivered under this Agreement, to secure payment of the purchase price of such Products, and Customer agrees to sign any documents that Mentor Graphics determines to be necessary or convenient for use in filing or perfecting such security interest. Mentor Graphics' delivery of Software by electronic means is subject to Customer's provision of both a primary and an alternate e-mail address.

2. **GRANT OF LICENSE.** The software installed, downloaded, or otherwise acquired by Customer under this Agreement, including any updates, modifications, revisions, copies, documentation and design data ("Software") are copyrighted, trade secret and confidential information of Mentor Graphics or its licensors, who maintain exclusive title to all Software and retain all rights not expressly granted by this Agreement. Mentor Graphics grants to Customer, subject to payment of applicable license fees, a nontransferable, nonexclusive license to use Software solely: (a) in machine-readable, object-code form (except as provided in Subsection 5.2); (b) for Customer's internal business purposes; (c) for the term of the license; and (d) on the computer hardware and at the site authorized by Mentor Graphics. A site is restricted to a one-half mile (800 meter) radius. Customer may have Software temporarily used by an employee for telecommuting purposes from locations other than a Customer office, such as the employee's residence, an airport or hotel, provided that such employee's primary place of employment is the site where the Software is authorized for use. Mentor Graphics' standard policies and programs, which vary depending on Software, license fees paid or services purchased, apply to the following: (a) relocation of Software; (b) use of Software, which may be limited, for example, to execution of a single session by a single user on the authorized hardware or for a restricted period of time (such limitations may be technically implemented through the use of authorization codes or similar devices); and (c) support services provided, including eligibility to receive telephone support, updates, modifications, and revisions. For the avoidance of doubt, if Customer provides any feedback or requests any change or enhancement to Products, whether in the course of receiving support or consulting services, evaluating Products, performing beta testing or otherwise, any inventions, product improvements, modifications or developments made by Mentor Graphics (at Mentor Graphics' sole discretion) will be the exclusive property of Mentor Graphics.
3. **ESC SOFTWARE.** If Customer purchases a license to use development or prototyping tools of Mentor Graphics' Embedded Software Channel ("ESC"), Mentor Graphics grants to Customer a nontransferable, nonexclusive license to reproduce and distribute executable

files created using ESC compilers, including the ESC run-time libraries distributed with ESC C and C++ compiler Software that are linked into a composite program as an integral part of Customer's compiled computer program, provided that Customer distributes these files only in conjunction with Customer's compiled computer program. Mentor Graphics does NOT grant Customer any right to duplicate, incorporate or embed copies of Mentor Graphics' real-time operating systems or other embedded software products into Customer's products or applications without first signing or otherwise agreeing to a separate agreement with Mentor Graphics for such purpose.

4. BETA CODE.

- 4.1. Portions or all of certain Software may contain code for experimental testing and evaluation (which may be either alpha or beta, collectively "Beta Code"), which may not be used without Mentor Graphics' explicit authorization. Upon Mentor Graphics' authorization, Mentor Graphics grants to Customer a temporary, nontransferable, nonexclusive license for experimental use to test and evaluate the Beta Code without charge for a limited period of time specified by Mentor Graphics. Mentor Graphics may choose, at its sole discretion, not to release Beta Code commercially in any form.
- 4.2. If Mentor Graphics authorizes Customer to use the Beta Code, Customer agrees to evaluate and test the Beta Code under normal conditions as directed by Mentor Graphics. Customer will contact Mentor Graphics periodically during Customer's use of the Beta Code to discuss any malfunctions or suggested improvements. Upon completion of Customer's evaluation and testing, Customer will send to Mentor Graphics a written evaluation of the Beta Code, including its strengths, weaknesses and recommended improvements.
- 4.3. Customer agrees to maintain Beta Code in confidence and shall restrict access to the Beta Code, including the methods and concepts utilized therein, solely to those employees and Customer location(s) authorized by Mentor Graphics to perform beta testing. Customer agrees that any written evaluations and all inventions, product improvements, modifications or developments that Mentor Graphics conceived or made during or subsequent to this Agreement, including those based partly or wholly on Customer's feedback, will be the exclusive property of Mentor Graphics. Mentor Graphics will have exclusive rights, title and interest in all such property. The provisions of this Subsection 4.3 shall survive termination of this Agreement.

5. RESTRICTIONS ON USE.

- 5.1. Customer may copy Software only as reasonably necessary to support the authorized use. Each copy must include all notices and legends embedded in Software and affixed to its medium and container as received from Mentor Graphics. All copies shall remain the property of Mentor Graphics or its licensors. Customer shall maintain a record of the number and primary location of all copies of Software, including copies merged with other software, and shall make those records available to Mentor Graphics upon request. Customer shall not make Products available in any form to any person other than Customer's employees and on-site contractors, excluding Mentor Graphics competitors, whose job performance requires access and who are under obligations of confidentiality. Customer shall take appropriate action to protect the confidentiality of Products and ensure that any person permitted access does not disclose or use Products except as permitted by this Agreement. Customer shall give Mentor Graphics written notice of any unauthorized disclosure or use of the Products as soon as Customer becomes aware of such unauthorized disclosure or use. Except as otherwise permitted for purposes of interoperability as specified by applicable and mandatory local law, Customer shall not reverse-assemble, reverse-compile, reverse-engineer or in any way derive any source code from Software. Log files, data files, rule files and script files generated by or for the Software (collectively "Files"), including without limitation files containing Standard Verification Rule Format ("SVRF") and Tcl Verification Format ("TVF") which are Mentor Graphics' trade secret and proprietary syntaxes for expressing process rules, constitute or include confidential information of Mentor Graphics. Customer may share Files with third parties, excluding Mentor Graphics competitors, provided that the confidentiality of such Files is protected by written agreement at least as well as Customer protects other information of a similar nature or importance, but in any case with at least reasonable care. Customer may use Files containing SVRF or TVF only with Mentor Graphics products. Under no circumstances shall Customer use Products or Files or allow their use for the purpose of developing, enhancing or marketing any product that is in any way competitive with Products, or disclose to any third party the results of, or information pertaining to, any benchmark.
- 5.2. If any Software or portions thereof are provided in source code form, Customer will use the source code only to correct software errors and enhance or modify the Software for the authorized use. Customer shall not disclose or permit disclosure of source code, in whole or in part, including any of its methods or concepts, to anyone except Customer's employees or on-site contractors, excluding Mentor Graphics competitors, with a need to know. Customer shall not copy or compile source code in any manner except to support this authorized use.
- 5.3. Customer may not assign this Agreement or the rights and duties under it, or relocate, sublicense, or otherwise transfer the Products, whether by operation of law or otherwise ("Attempted Transfer"), without Mentor Graphics' prior written consent and payment of Mentor Graphics' then-current applicable relocation and/or transfer fees. Any Attempted Transfer without Mentor Graphics' prior written consent shall be a material breach of this Agreement and may, at Mentor Graphics' option, result in the immediate termination of the Agreement and/or the licenses granted under this Agreement. The terms of this Agreement, including without limitation the licensing and assignment provisions, shall be binding upon Customer's permitted successors in interest and assigns.
- 5.4. The provisions of this Section 5 shall survive the termination of this Agreement.

6. **SUPPORT SERVICES.** To the extent Customer purchases support services, Mentor Graphics will provide Customer with updates and technical support for the Products, at the Customer site(s) for which support is purchased, in accordance with Mentor Graphics' then current End-User Support Terms located at <http://supportnet.mentor.com/supportterms>.

7. LIMITED WARRANTY.

- 7.1. Mentor Graphics warrants that during the warranty period its standard, generally supported Products, when properly installed, will substantially conform to the functional specifications set forth in the applicable user manual. Mentor Graphics does not warrant that Products will meet Customer's requirements or that operation of Products will be uninterrupted or error free. The

warranty period is 90 days starting on the 15th day after delivery or upon installation, whichever first occurs. Customer must notify Mentor Graphics in writing of any nonconformity within the warranty period. For the avoidance of doubt, this warranty applies only to the initial shipment of Software under an Order and does not renew or reset, for example, with the delivery of (a) Software updates or (b) authorization codes or alternate Software under a transaction involving Software re-mix. This warranty shall not be valid if Products have been subject to misuse, unauthorized modification, improper installation or Customer is not in compliance with this Agreement. MENTOR GRAPHICS' ENTIRE LIABILITY AND CUSTOMER'S EXCLUSIVE REMEDY SHALL BE, AT MENTOR GRAPHICS' OPTION, EITHER (A) REFUND OF THE PRICE PAID UPON RETURN OF THE PRODUCTS TO MENTOR GRAPHICS OR (B) MODIFICATION OR REPLACEMENT OF THE PRODUCTS THAT DO NOT MEET THIS LIMITED WARRANTY. MENTOR GRAPHICS MAKES NO WARRANTIES WITH RESPECT TO: (A) SERVICES; (B) PRODUCTS PROVIDED AT NO CHARGE; OR (C) BETA CODE; ALL OF WHICH ARE PROVIDED "AS IS."

7.2. THE WARRANTIES SET FORTH IN THIS SECTION 7 ARE EXCLUSIVE. NEITHER MENTOR GRAPHICS NOR ITS LICENSORS MAKE ANY OTHER WARRANTIES EXPRESS, IMPLIED OR STATUTORY, WITH RESPECT TO PRODUCTS PROVIDED UNDER THIS AGREEMENT. MENTOR GRAPHICS AND ITS LICENSORS SPECIFICALLY DISCLAIM ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY.

8. **LIMITATION OF LIABILITY.** EXCEPT WHERE THIS EXCLUSION OR RESTRICTION OF LIABILITY WOULD BE VOID OR INEFFECTIVE UNDER APPLICABLE LAW, IN NO EVENT SHALL MENTOR GRAPHICS OR ITS LICENSORS BE LIABLE FOR INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES (INCLUDING LOST PROFITS OR SAVINGS) WHETHER BASED ON CONTRACT, TORT OR ANY OTHER LEGAL THEORY, EVEN IF MENTOR GRAPHICS OR ITS LICENSORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. IN NO EVENT SHALL MENTOR GRAPHICS' OR ITS LICENSORS' LIABILITY UNDER THIS AGREEMENT EXCEED THE AMOUNT RECEIVED FROM CUSTOMER FOR THE HARDWARE, SOFTWARE LICENSE OR SERVICE GIVING RISE TO THE CLAIM. IN THE CASE WHERE NO AMOUNT WAS PAID, MENTOR GRAPHICS AND ITS LICENSORS SHALL HAVE NO LIABILITY FOR ANY DAMAGES WHATSOEVER. THE PROVISIONS OF THIS SECTION 8 SHALL SURVIVE THE TERMINATION OF THIS AGREEMENT.

9. **HAZARDOUS APPLICATIONS.** CUSTOMER ACKNOWLEDGES IT IS SOLELY RESPONSIBLE FOR TESTING ITS PRODUCTS USED IN APPLICATIONS WHERE THE FAILURE OR INACCURACY OF ITS PRODUCTS MIGHT RESULT IN DEATH OR PERSONAL INJURY ("HAZARDOUS APPLICATIONS"). EXCEPT TO THE EXTENT THIS EXCLUSION OR RESTRICTION OF LIABILITY WOULD BE VOID OR INEFFECTIVE UNDER APPLICABLE LAW, IN NO EVENT SHALL MENTOR GRAPHICS OR ITS LICENSORS BE LIABLE FOR ANY DAMAGES RESULTING FROM OR IN CONNECTION WITH THE USE OF MENTOR GRAPHICS PRODUCTS IN OR FOR HAZARDOUS APPLICATIONS. THE PROVISIONS OF THIS SECTION 9 SHALL SURVIVE THE TERMINATION OF THIS AGREEMENT.

10. **INDEMNIFICATION.** CUSTOMER AGREES TO INDEMNIFY AND HOLD HARMLESS MENTOR GRAPHICS AND ITS LICENSORS FROM ANY CLAIMS, LOSS, COST, DAMAGE, EXPENSE OR LIABILITY, INCLUDING ATTORNEYS' FEES, ARISING OUT OF OR IN CONNECTION WITH THE USE OF MENTOR GRAPHICS PRODUCTS IN OR FOR HAZARDOUS APPLICATIONS. THE PROVISIONS OF THIS SECTION 10 SHALL SURVIVE THE TERMINATION OF THIS AGREEMENT.

11. INFRINGEMENT.

11.1. Mentor Graphics will defend or settle, at its option and expense, any action brought against Customer in the United States, Canada, Japan, or member state of the European Union which alleges that any standard, generally supported Product acquired by Customer hereunder infringes a patent or copyright or misappropriates a trade secret in such jurisdiction. Mentor Graphics will pay costs and damages finally awarded against Customer that are attributable to such action. Customer understands and agrees that as conditions to Mentor Graphics' obligations under this section Customer must: (a) notify Mentor Graphics promptly in writing of the action; (b) provide Mentor Graphics all reasonable information and assistance to settle or defend the action; and (c) grant Mentor Graphics sole authority and control of the defense or settlement of the action.

11.2. If a claim is made under Subsection 11.1 Mentor Graphics may, at its option and expense: (a) replace or modify the Product so that it becomes noninfringing; (b) procure for Customer the right to continue using the Product; or (c) require the return of the Product and refund to Customer any purchase price or license fee paid, less a reasonable allowance for use.

11.3. Mentor Graphics has no liability to Customer if the action is based upon: (a) the combination of Software or hardware with any product not furnished by Mentor Graphics; (b) the modification of the Product other than by Mentor Graphics; (c) the use of other than a current unaltered release of Software; (d) the use of the Product as part of an infringing process; (e) a product that Customer makes, uses, or sells; (f) any Beta Code or Product provided at no charge; (g) any software provided by Mentor Graphics' licensors who do not provide such indemnification to Mentor Graphics' customers; or (h) infringement by Customer that is deemed willful. In the case of (h), Customer shall reimburse Mentor Graphics for its reasonable attorney fees and other costs related to the action.

11.4. THIS SECTION 11 IS SUBJECT TO SECTION 8 ABOVE AND STATES THE ENTIRE LIABILITY OF MENTOR GRAPHICS AND ITS LICENSORS, AND CUSTOMER'S SOLE AND EXCLUSIVE REMEDY, FOR DEFENSE, SETTLEMENT AND DAMAGES, WITH RESPECT TO ANY ALLEGED PATENT OR COPYRIGHT INFRINGEMENT OR TRADE SECRET MISAPPROPRIATION BY ANY PRODUCT PROVIDED UNDER THIS AGREEMENT.

12. TERMINATION AND EFFECT OF TERMINATION.

12.1. If a Software license was provided for limited term use, such license will automatically terminate at the end of the authorized term. Mentor Graphics may terminate this Agreement and/or any license granted under this Agreement immediately upon written notice if Customer: (a) exceeds the scope of the license or otherwise fails to comply with the licensing or confidentiality provisions of this Agreement, or (b) becomes insolvent, files a bankruptcy petition, institutes proceedings for liquidation or winding up or enters into an agreement to assign its assets for the benefit of creditors. For any other material breach of any

provision of this Agreement, Mentor Graphics may terminate this Agreement and/or any license granted under this Agreement upon 30 days written notice if Customer fails to cure the breach within the 30 day notice period. Termination of this Agreement or any license granted hereunder will not affect Customer's obligation to pay for Products shipped or licenses granted prior to the termination, which amounts shall be payable immediately upon the date of termination.

- 12.2. Upon termination of this Agreement, the rights and obligations of the parties shall cease except as expressly set forth in this Agreement. Upon termination, Customer shall ensure that all use of the affected Products ceases, and shall return hardware and either return to Mentor Graphics or destroy Software in Customer's possession, including all copies and documentation, and certify in writing to Mentor Graphics within ten business days of the termination date that Customer no longer possesses any of the affected Products or copies of Software in any form.
13. **EXPORT.** The Products provided hereunder are subject to regulation by local laws and United States ("U.S.") government agencies, which prohibit export, re-export or diversion of certain products, information about the products, and direct or indirect products thereof, to certain countries and certain persons. Customer agrees that it will not export or re-export Products in any manner without first obtaining all necessary approval from appropriate local and U.S. government agencies. If Customer wishes to disclose any information to Mentor Graphics that is subject to any U.S. or other applicable export restrictions, including without limitation the U.S. International Traffic in Arms Regulations (ITAR) or special controls under the Export Administration Regulations (EAR), Customer will notify Mentor Graphics personnel, in advance of each instance of disclosure, that such information is subject to such export restrictions.
14. **U.S. GOVERNMENT LICENSE RIGHTS.** Software was developed entirely at private expense. The parties agree that all Software is commercial computer software within the meaning of the applicable acquisition regulations. Accordingly, pursuant to U.S. FAR 48 CFR 12.212 and DFAR 48 CFR 227.7202, use, duplication and disclosure of the Software by or for the U.S. government or a U.S. government subcontractor is subject solely to the terms and conditions set forth in this Agreement, which shall supersede any conflicting terms or conditions in any government order document, except for provisions which are contrary to applicable mandatory federal laws.
15. **THIRD PARTY BENEFICIARY.** Mentor Graphics Corporation, Mentor Graphics (Ireland) Limited, Microsoft Corporation and other licensors may be third party beneficiaries of this Agreement with the right to enforce the obligations set forth herein.
16. **REVIEW OF LICENSE USAGE.** Customer will monitor the access to and use of Software. With prior written notice and during Customer's normal business hours, Mentor Graphics may engage an internationally recognized accounting firm to review Customer's software monitoring system and records deemed relevant by the internationally recognized accounting firm to confirm Customer's compliance with the terms of this Agreement or U.S. or other local export laws. Such review may include FlexNet (or successor product) report log files that Customer shall capture and provide at Mentor Graphics' request. Customer shall make records available in electronic format and shall fully cooperate with data gathering to support the license review. Mentor Graphics shall bear the expense of any such review unless a material non-compliance is revealed. Mentor Graphics shall treat as confidential information all information gained as a result of any request or review and shall only use or disclose such information as required by law or to enforce its rights under this Agreement. The provisions of this Section 16 shall survive the termination of this Agreement.
17. **CONTROLLING LAW, JURISDICTION AND DISPUTE RESOLUTION.** The owners of certain Mentor Graphics intellectual property licensed under this Agreement are located in Ireland and the U.S. To promote consistency around the world, disputes shall be resolved as follows: excluding conflict of laws rules, this Agreement shall be governed by and construed under the laws of the State of Oregon, U.S., if Customer is located in North or South America, and the laws of Ireland if Customer is located outside of North or South America. All disputes arising out of or in relation to this Agreement shall be submitted to the exclusive jurisdiction of the courts of Portland, Oregon when the laws of Oregon apply, or Dublin, Ireland when the laws of Ireland apply. Notwithstanding the foregoing, all disputes in Asia arising out of or in relation to this Agreement shall be resolved by arbitration in Singapore before a single arbitrator to be appointed by the chairman of the Singapore International Arbitration Centre ("SIAC") to be conducted in the English language, in accordance with the Arbitration Rules of the SIAC in effect at the time of the dispute, which rules are deemed to be incorporated by reference in this section. Nothing in this section shall restrict Mentor Graphics' right to bring an action (including for example a motion for injunctive relief) against Customer in the jurisdiction where Customer's place of business is located. The United Nations Convention on Contracts for the International Sale of Goods does not apply to this Agreement.
18. **SEVERABILITY.** If any provision of this Agreement is held by a court of competent jurisdiction to be void, invalid, unenforceable or illegal, such provision shall be severed from this Agreement and the remaining provisions will remain in full force and effect.
19. **MISCELLANEOUS.** This Agreement contains the parties' entire understanding relating to its subject matter and supersedes all prior or contemporaneous agreements. Some Software may contain code distributed under a third party license agreement that may provide additional rights to Customer. Please see the applicable Software documentation for details. This Agreement may only be modified in writing, signed by an authorized representative of each party. Waiver of terms or excuse of breach must be in writing and shall not constitute subsequent consent, waiver or excuse.